人类构建的最后一个 AI：迈向真正的递归自我改进

The Last AI Built by Humans: Toward Genuine Recursive Self-Improvement

来源：https://arxiv.org/abs/2609.11873

原文 PDF：79 页。制作日期：2026-10-22。

译稿说明：这是 Google Translate 生成、经固定术语表约束的逐页机器翻译初稿，未经逐句人工学术审校。中文翻译覆盖全部可提取文本，包含正文、图表文字、附录与参考文献文本；图片内未能提取的文字、数学公式的二维布局和复杂表格请对照原文。排版提取可能造成少量顺序或符号错误，尤其是双栏图表、公式、脚注。不能用机器译稿替代原文进行公式推导或直接引用。常用英文术语和缩写予以保留。

PDF 采用原文页与中文译页交替编排，原始图表及公式完整保留在原文页。章节中的数字“原文第 X 页”始终指原始 PDF 物理页码，而非论文印刷页码或译稿页码。参考文献在原文页保留完整英文条目，检索时使用原文。学习文档中的例子和伪代码是导读，不属于作者原文。

核心摘要校订

根据原文摘要单独校订。正文仍为逐页机器翻译初稿；原始图表、公式和英文条目请查看紧邻原文页。

递归自我改进（Recursive Self-Improvement，RSI）使 AI 系统能够将经验与反馈转化为持久变化，同时提高自身能力和未来的改进过程。本文首先利用剩余提升空间闭合指数（Headroom-Closed Index，HCI）揭示现有大语言模型的问题，然后介绍 RSI 的概念与发展路线：从改进执行的自主性、改进策略的自主性、经验获取的自主性和环境适应的自主性，逐步走向递归元改进。随后，本文考察科学发现、具身智能、软件工程等场景中的 RSI，指出各场景需求和发展速度的差异。通过多种产业实践与初步实证，本文将 RSI 研究与实际系统联系起来，并识别实现真正 RSI 的关键挑战。

arXiv:2609.11873v3 [cs.LG] 22 Sep 2026

The Last AI Built by Humans: Toward Genuine Recursive Self-Improvement

Yi Duan[1,2*], Ying Liu[1,2*], Zirui Tang[1,2*], Haodong Chen[1,2*], Jun Zhou[1,2*], Yumou Liu[1,2], Bangrui Xu[1,2], Yukai Wu[1,2], Sidi Chen[1,2], Yuhan Zhou[1,2], Haoyu Wang[1,2], Xiaoyou Yu[1,2], Shaokun Han[1,2], Xuzhou Zhu[1,2], Le Zhou[1,2], Bolin Lu[1,2], Wei Zhou[1,2], Jiachen Liu[9], Nuozhou Fang[2], Jiaxin Tian[2], Ruoyu Chen[4], Yuxuan Li[5], Kai Zuo[6], Kaiyan Zhang[10], Qianyu Yang[8], Zijie Wang[8], Jiantao Qiu[7], Conghui He[7], Guoliang Li[3], Bowen Zhou[7,3], Zhiyuan Liu[3], Zhoufutu Wen[8], Jihua Kang[4], Xuanhe Zhou[1,2†], Fan Wu[1,2]

[1]Shanghai Jiao Tong University [2]Theseus Labs [3]Tsinghua University
[4]ByteDance [5]ModelBest [6]Super Intelligence Team, Xiaohongshu Inc. [7]Shanghai AI Lab
[8]Humanlaya [9]Agent-Native Research Lab [10]Frontis.AI

Abstract

Recursive self-improvement (RSI) enables AI systems to turn experience and feedback into persistent changes that improve both their capabilities and the process of future improvement. We first use the Headroom-Closed Index (HCI) to reveal the problems of existing LLMs, then introduce the RSI concept and its development roadmap: from improvement-execution autonomy, improvement-strategy autonomy, experience-acquisition autonomy, and environment-adaptation autonomy, to recursive meta-improvement. Next we examine RSI across scenarios (e.g., scientific discovery, embodied intelligence, software engineering), highlighting their distinct requirements and development speeds. Drawing on diverse industry practices and preliminary empirical evidence, we connect RSI research with practical systems and identify key challenges to achieving genuine RSI.

Project Page: `https://theseus-labs-rsi.github.io/`

GitHub Repository: `https://github.com/theseus-labs-rsi/awesome-rsi`

TL;DR *Like human evolution, AI evolution will unfold through a vast and extraordinary history. Everything AI has achieved so far is but a drop in the ocean.*

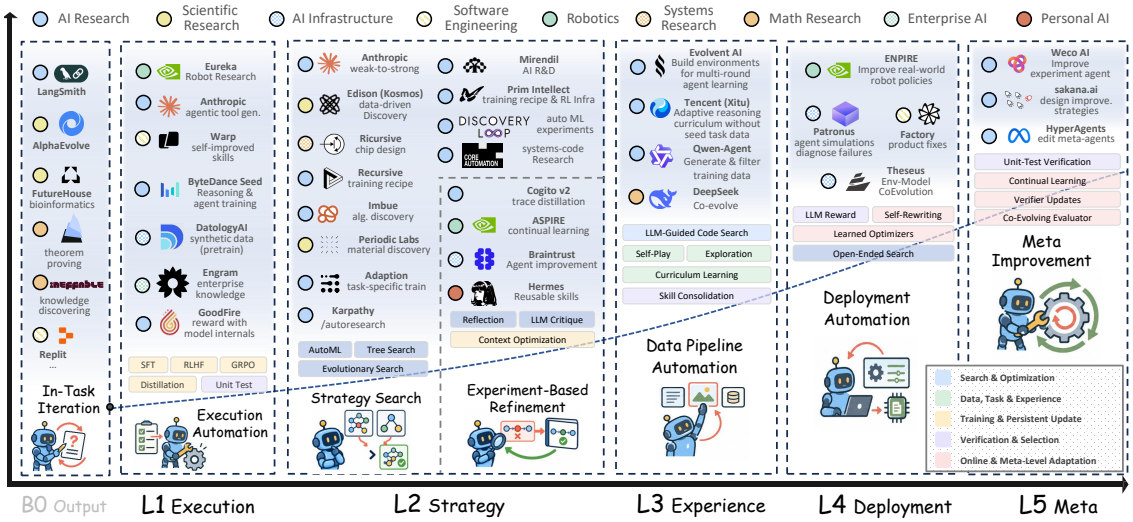


Figure 1 Overview of the five RSI autonomy levels and representative systems. Autonomy progressively expands from executing prescribed improvements (L1), to selecting improvement strategies (L2), acquiring future learning experience (L3), adapting through deployment and environmental feedback (L4), and ultimately improving mechanisms that govern subsequent improvement (L5). See system details in Appendix B.

*Equal Contribution
†Corresponding author: Xuanhe Zhou

原文第 1 页 | 中文翻译初稿

人类构建的最后一个 AI：迈向真正的递归自我改进

段毅1_,2*、刘瑛1,2*、唐子睿1,2*、浩东陈1,2*、周军1,2*、刘玉谋1,2、邦瑞徐1,2、吴玉凯1,2、陈思迪1,2、周雨涵1,2、浩宇王1,2、余小友1,2、韩少坤1,2、朱徐洲1,2、乐周1,2、陆柏林1,2、周伟1,2、刘佳辰9、方诺周2、田佳欣2、陈若宇4、李宇轩5、左凯6、张凯彦10、杨千羽8、王子杰8、邱建涛7、何丛辉7 、李国良3、周博文7,3、刘志远3、周富图文8、康吉华4、宣和周1,2†、吴凡1,_2

> 1上海交通大学 2Theseus实验室 3清华大学

4字节跳动 5ModelBest 6小红书超级智能团队 7上海人工智能实验室

8Humanlaya 9Agent-Native 研究实验室 10Frontis.AI

摘要

递归自我改进（Recursive Self-Improvement）(RSI) 使人工智能系统能够将经验和反馈转化为持续的变化，从而提高其能力和未来改进的过程。我们首先使用剩余提升空间闭合指数（Headroom-Closed Index）（HCI）来揭示现有LLM的问题，然后介绍RSI概念及其发展路线图：从改进执行自主、改进策略自主、经验获取自主、环境适应自主，到递归元改进。接下来，我们跨场景检查 RSI（例如，科学发现、具身智能（Embodied Intelligence）、软件工程），突出显示它们的不同需求和开发速度。我们借鉴不同的行业实践和初步经验证据，将 RSI 研究与实际系统联系起来，并确定实现真正 RSI 的关键挑战。

项目页面：https://theseus-labs-rsi.github.io/

GitHub 存储库：https://github.com/theseus-labs-rsi/awesome-rsi

TL;DR 就像人类的进化一样，人工智能的进化也将经历一段浩瀚而非凡的历史。迄今为止，人工智能所取得的一切都只是沧海一粟。

人工智能研究 科学研究 人工智能基础设施工程软件 机器人研究 系统 数学研究 企业人工智能 个人人工智能
AlphaEvolveLangSmith 机器人研究 智能体工具生成 自我改进 人性化 EurekaWarp 技能 爱迪生 (Kosmos) 弱到强芯片设计 数据驱动 Ricursive 人性化发现 训练配方和 RL Infra Prim 智能 Mirendil 系统代码 AI 研发研究实验 自动 ML 构建环境 自适应推理课程，无需 for多轮智能体学习生成和过滤腾讯（喜兔）种子任务数据进化AIQwen-Agent智能体模拟诊断故障守护神改进现实世界的机器人策略ENPIRE产品修复工厂单元测试验证实验智能体编辑元智能体设计改进。 HyperAgentsWeco AI 改进策略 sakana.ai 生物信息学 FutureHouse ByteDance 种子智能体训练推理和训练配方 RecursiveImbue 痕迹蒸馏 Cogito v2 DeepSeek 协同进化训练数据 CoEvolutionEnv-ModelTheseus 持续学习验证器更新
定理合成数据 DatologyAI（预训练）材料发现alg。 discovery 定期实验室 持续学习（Continual Learning） BraintrustASPIRE 自玩LLM 引导代码搜索探索 LLM 奖励开放式搜索学习优化器自我重写共同进化评估器元
证明印迹适应智能体改进课程学习改进
知识知识企业任务特定训练可重用技能 Hermes 技能巩固部署
通过 GoodFire 发现奖励/autoresearch Karpathy Reflection LLM Critique 自动化
模型内部结构上下文优化
Replit AutoML 树搜索数据管道

... SFT RLHF GRPO 进化搜索自动化
迭代任务内蒸馏执行单元测试策略搜索基于实验的细化训练和持续更新搜索与优化数据、任务和体验
自动化验证和选择
在线和元级别适配
B0输出L1执行L2策略L3体验L4部署L5元

图 1 五个 RSI 自治级别和代表性系统的概述。自主性逐步从执行规定的改进（L1）扩展到选择改进策略（L2）、获取未来的学习经验（L3）、通过部署和环境反馈进行适应（L4），并最终改进管理后续改进的机制（L5）。请参阅附录 B 中的系统详细信息。

> *平等贡献

> †通讯作者：周宣和

1

Contents

1 Introduction . . . 3
1.1 Scaling Burdens in Model Development . . . 3
1.2 From Development Burden to RSI . . . 3
1.3 Challenges for RSI . . . 4
1.4 An Autonomy-Centered Framework . . . 4
1.5 Application Domains . . . 5
1.6 Industrial Evidence . . . 6
1.7 Differences from Existing Surveys . . . 6

2 Background and Preliminaries . . . 7
2.1 Uneven Capability Progress in Modern Foundation Models . . . 7
2.2 Conceptual Foundations of Recursive Self-Improvement . . . 9
2.3 Scope of Evidence . . . 12

3 RSI Across Autonomy Levels . . . 12
3.1 B0: In-Task AI Improvement . . . 12
3.2 L1: Autonomy over Improvement Execution . . . 14
3.3 L2: Autonomy over Improvement Strategies . . . 18
3.4 L3: Autonomy over Future Learning Experience . . . 22
3.5 L4: Autonomy in Deployment and Environmental Adaptation . . . 26
3.6 L5: From Environmental Adaptation to Meta-Improvement . . . 31
3.7 Cross-Level Synthesis . . . 35

4 RSI Across Applications . . . 36
4.1 S1: RSI for Science . . . 36
4.2 S2: RSI for Embodied Intelligence . . . 36
4.3 S3: RSI for Software Engineering . . . 37
4.4 S4: RSI for Healthcare . . . 37

5 Industry Landscape and Preliminary Practices . . . 37
5.1 Theseus: Environment–Data–Model Co-Evolution for RSI Intelligence . . . 37
5.2 Lark: Building the Data Foundation for Reliable Enterprise-Level RSI . . . 39
5.3 Xiaohongshu: Dual-Timescale RSI for Recommendation . . . 40
5.4 Humanlaya: Delivery-Driven RSI for Data Quality Assurance . . . 41
5.5 ModelBest: Zero-Human Industrial AI Engineering . . . 42
5.6 Tencent Hunyuan: Experience-Driven Self-Improvement . . . 43
5.7 Agent-Native Research Lab: Verifiable Research Infrastructure for RSI . . . 44
5.8 Frontis.AI: Enterprise Agent Evolution and Cross-Task Meta-Improvement . . . 45

6 Challenges and Future Directions . . . 46

7 Conclusion . . . 49

Appendix . . . 63
A RSI Landscape . . . 63
B Industry Landscape . . . 66
C RSI Across Applications . . . 72
C.1 S1: RSI for Science . . . 72
C.2 S2: RSI for Embodied Intelligence . . . 74
C.3 S3: RSI for Software Engineering . . . 76
C.4 S4: RSI for Healthcare . . . 78

原文第 2 页 | 中文翻译初稿

内容

|1
简介|简介。。。

。 。 。

37|
|5.1|Theseus：RSI 智能的环境-数据-模型协同演化
。 。 。 。 。 。 。 。 .|. 。 。 。 .
37|
|5.2|Lark：构建可靠的企业级RSI数据基础。 。 。 。 。 。 。 .|. 。 。 。 .
39|
|5.3|小红书：双时间尺度RSI推荐。 。 。 。 。 。 。 。 。 。 。 。 。 。 。 。 .|. 。 。
。 .
40|
|5.4|Humanlaya：用于数据质量保证的交付驱动的 RSI。 。 。 。 。 。 。 。 。 。 。 。 .|. 。 。 。 .
41|
|5.5|ModelBest：零人类工业人工智能工程。 。 。 。 。 。 。 。 。 。 。 。 。 。 。 。 。 。 .|. 。 。 。
.
42|
|5.6|腾讯混元：体验驱动 自我改进（Self-Improvement）. 。 。 。 。 。 。 。 。 。 。 。 。 。 。 .|. 。 。
。 .
43|
|5.7|Agent-Native 研究实验室：RSI 可验证的研究基础设施。 。 。 。 。 。 。 .|. 。 。 。 .
44|
|5.8|Frontis.AI：企业智能体演进和跨任务元改进
。 。 。 。 .|. 。 。 。 .
45|
|6
挑战|挑战和未来方向。 。
。 。 .|. 。 。 。 .
46|
|7
结论|结论。 。
。 。 。 。 。 。 。 。 。 。 。 .|. 。 。 。 .
49|
|附录|ix
。 。 。 。 。 。 。 。 。 。 。 。 。 。 。 。 。 。 。

。 。 。 .|. 。 。 。 .
72|
|C.1|S1：科学 RSI。 .|. 。 。 。 .
72|
|C.2|S2：具身智能（Embodied Intelligence）
的 RSI。 .|. 。 。 。 .
74|
|C.3|S3：软件工程的 RSI。 .|. 。 。 。 .
76|
|C.4|S4：医疗保健领域的 RSI。 .|. 。 。 。 .
78|

2

1 Introduction

Recent frontier-model development illustrates several forms of scaling in the improvement pipeline. Kimi K3 and Qwen3.8-Max contain 2.8 trillion and 2.4 trillion parameters, respectively, and each supports a context window of approximately one million tokens [1, 2]. The development process is also expanding. During the six months preceding GPT-5.6, OpenAI reports that the share of research compute devoted to internal coding inference grew 100-fold and internal agentic token use grew 22-fold, and average daily output tokens per active researcher exceeded twice the previous peak observed with GPT-5.5 [3]. Scale accumulates across training runs, model-assisted experiments, inference, evaluation, and human validation.

1.1 Scaling Burdens in Model Development

Despite the growing use of agentic tools and API-based automation, developers must still determine what to improve, construct the required resources, and establish whether each change works [4, 5]. As AI systems take on more demanding tasks, the cost of scaling this end-to-end development process becomes a bottleneck [6–9]. The following three challenges occur at different stages of the model lifecycle and motivate RSI:

• ***Development challenge 1: Resource-intensive foundation-model training.*** Foundation-model development remains resource-intensive across data preparation, architecture design, distributed optimization, and evaluation. Kimi K3 activates 16 of 896 experts and reports an approximate 2.5-fold improvement in scaling efficiency over Kimi K2, while Qwen3.8-Max activates 95 billion of its 2.4 trillion parameters [1, 2]. Sparse activation reduces per-token computation, but training at this scale still couples expert routing, parallelism, multimodal integration, long-context optimization, and systems design. OpenAI further reports that GPT-5.6 Sol designed and ran hundreds of experiments on its speculative-decoding draft model and monitored training through hardware failures and instability. The resulting changes improved token-generation efficiency by more than 15% [10]. Data quality also matters in addition to quantity. OpenAI's GDPval illustrates that its 1,320 professional tasks required roughly 9,240 expert-hours in total, with contributors averaging more than 14 years of experience [6]. The Humanity's Last Exam pipeline logged more than 70,000 submission attempts and sent approximately 13,000 model-stumping questions to expert review before producing a 3,000-question benchmark [7]. Architecture search remains expensive because candidate structures interact with their data, optimization, and hardware regimes. AgentNAS addresses part of this bottleneck by using an LLM to propose a task-specific seed architecture and construct its search space, but candidate selection still depends on combinatorial search under an externally specified objective [11].

• ***Development challenge 2: Scaling feedback and learning environments.*** Synthetic data and reinforcement learning automate parts of capability development, but introduce substantial requirements for generating and evaluating experience, requiring both experience-generation infrastructure and reliable mechanisms for evaluating and retaining updates. For instance, DeepSeek-V3.2 reports a post-training computational budget exceeding 10% of its pretraining cost [12], while NVIDIA's AIMO-2 pipeline generated 3.2 million long-reasoning solutions and 1.7 million tool-integrated solutions, in addition to curating 540,000 problems [4].

• ***Development challenge 3: Recurring adaptation after deployment.*** Deployed systems consistently introduce changing documents, unfamiliar tools, incomplete context, and workflows involving interdependent actions. Improving such systems requires engineers to manually diagnose failures, revise retrieval and tool interfaces, manage persistent state, and repeat regression testing through discrete, human-led releases. Anthropic reports that agentic workloads use approximately four times as many tokens as ordinary chat, rising to about fifteen times for multi-agent systems because of longer contexts, coordination, environment setup, and end-to-end verification [13], while Meta reports that FBDetect identifies thousands of infrastructure regressions each week, and diagnosing one such regression required roughly ten engineer-hours [5].

1.2 From Development Burden to RSI

The burdens arise because model improvement remains a sequence of costly, externally coordinated interventions. To address these barriers, RSI inspects whether part of that coordination can become a persistent capability of the system being improved. We define recursive self-improvement (RSI) as an autonomous, closed-loop process in which an AI system identifies its own limitations, develops and validates improvements, and uses the resulting capabilities to improve the improvement process itself. The model evolution paradigm

原文第 3 页 | 中文翻译初稿

1 简介

1.2
1 简介

最近的前沿模型开发说明了改进管道中的几种扩展形式。 Kimi K3 和 Qwen3.8-Max 分别包含 2.8 万亿和 2.4 万亿个参数，每个参数都支持大约一百万个令牌的上下文窗口 [1, 2]。开发进程也在不断扩大。 OpenAI 报告称，在 GPT-5.6 之前的六个月中，用于内部编码推理的研究计算份额增长了 100 倍，内部智能体代币使用量增长了 22 倍，每个活跃研究人员的平均每日产出代币超过了 GPT-5.5 观察到的先前峰值的两倍 [3]。规模在训练运行、模型辅助实验、推理、评估和人工验证中积累。
1.1 减轻模型开发的负担

尽管越来越多地使用智能体工具和基于 API 的自动化，但开发人员仍然必须确定要改进的内容、构建所需的资源，并确定每个更改是否有效 [4, 5]。随着人工智能系统承担更苛刻的任务，扩展这种端到端开发过程的成本成为瓶颈[6-9]。以下三个挑战出现在模型生命周期的不同阶段并激励 RSI：

• 开发挑战1：资源密集型基础模型训练。 基础模型开发在数据准备、架构设计、分布式优化和评估方面仍然是资源密集型的。 Kimi K3 激活了 896 个专家中的 16 个，并报告称扩展效率比 Kimi K2 提高了大约 2.5 倍，而 Qwen3.8-Max 激活了 2.4 万亿个参数中的 950 亿个参数 [1, 2]。稀疏激活减少了每个令牌的计算，但这种规模的训练仍然需要专家路由、并行性、多模态集成、长上下文优化和系统设计。 OpenAI 进一步报告称，GPT-5.6 Sol 在其推测解码草稿模型上设计并运行了数百次实验，并通过硬件故障和不稳定来监控训练。由此产生的变化将代币生成效率提高了 15% 以上 [10]。除了数量之外，数据质量也很重要。 OpenAI 的 GDPval 表明，其 1,320 项专业任务总共需要大约 9,240 个专家小时，贡献者平均拥有超过 14 年的经验 [6]。 Humanity 的 Last Exam 管道记录了超过 70,000 次提交尝试，并向专家评审发送了大约 13,000 个模型难题，然后生成了 3,000 个问题的基准 [7]。架构搜索仍然很昂贵，因为候选结构与其数据、优化和硬件机制相互作用。 AgentNAS 通过使用 LLM 提出特定于任务的种子架构并构建其搜索空间来部分解决此瓶颈，但候选选择仍然取决于外部指定目标下的组合搜索 [11]。

• 开发挑战 2：扩展反馈和学习环境。 合成数据和 强化学习（Reinforcement Learning） 自动化了部分能力开发，但对生成和评估经验提出了实质性要求，需要经验生成基础设施和用于评估和保留更新的可靠机制

。例如，DeepSeek-V3.2 报告的训练后计算预算超过其预训练成本的 10% [12]，而 NVIDIA 的 AIMO-2 管道除了策划 540,000 个问题外，还生成了 320 万个长推理解决方案和 170 万个工具集成解决方案 [4]。

• 开发挑战 3：部署后反复适应。 已部署的系统不断引入不断变化的文档、不熟悉的工具、不完整的上下文以及涉及相互依赖的操作的工作流程。改进此类系统需要工程师手动诊断故障、修改检索和工具界面、管理持久状态，并通过离散的、人工主导的版本重复回归测试。 Anthropic 报告称，智能体工作负载使用的令牌数量大约是普通聊天的四倍，对于多智能体系统，由于更长的上下文、协调、环境设置和端到端验证，上升到约 15 倍 [13]，而 Meta 报告称，FBDetect 每周识别数千个基础设施回归，诊断一个此类回归需要大约 10 个工程师小时 [5]。
1.2 从发展负担到RSI

之所以会出现负担，是因为模型改进仍然是一系列昂贵的、外部协调的干预措施。为了解决这些障碍，RSI 检查部分协调是否可以成为正在改进的系统的持久功能。我们将 递归自我改进（Recursive

Self-Improvement ）(RSI) 定义为一个自主的闭环过程，其中人工智能系统识别自身的局限性，开发和验证改进，并使用由此产生的功能来改进改进过程本身。模型演化范式

3

of RSI spans three dimensions: autonomy, efficiency, and innovation. Autonomy expands the system's responsibility from executing a prescribed update to identifying limitations, extracting experience, proposing changes, and validating and retaining successors. Efficiency seeks more validated improvement from data, compute, inference, human review, and rework. Innovation allows the system to search beyond human-prescribed update strategies and feed useful discoveries back into later improvement rounds. These dimensions describe how the full improvement loop is organized and what it can inherit.

Rather than a particular learning algorithm or a one-off optimization result [14–16], RSI aims to improve both task performance and the mechanisms through which later improvements are discovered and implemented. The following cases illustrate this distinction at two points in the model lifecycle, including foundation-model training and persistent adaptation in software engineering, covered and studied in later sections.

• **Case 1: Foundation-model training.** A conventional experiment loop selects a better checkpoint while leaving the procedure for choosing later experiments unchanged. A-Evolve-Training instead consolidates post-training outcomes into a persistent research policy and discovery log, which a meta-agent revises to guide later workers' recipe choices [17]. When development scores improved without corresponding external gains, the system redirected experiments toward data rebalancing and checkpoint selection. The retained policy changes both how successor models are trained and how later improvements are sought. Across four autonomous rounds on a 30B Nemotron model, the external score rose from 0.80 to 0.86, compared with 0.87 for the top human submission [17].

• **Case 2: Software-engineering adaptation after deployment.** Repairing a repository changes the software product but may leave the coding agent's recurring failures untouched. Ouroboros instead uses reviewed deployment evidence to propose versioned changes to the agent's tools, context assembly, prompts, and core implementation [18]. Candidate revisions undergo tests and human review before an accepted version replaces the runtime used for later work. The persistent update improves subsequent coding behavior and changes the mechanism through which later failures are diagnosed and repaired, while experts retain control over consequential corrections and deployment.

1.3 Challenges for RSI

While the two cases show how persistent changes can shape later improvement, the same persistence can carry errors or obscure where control resides. We examine three recurring problems that determine whether a self-updating system provides credible evidence of RSI.

• **Safe inheritance.** RSI requires changes to persist across tasks or improvement rounds, but persistence alone does not guarantee sustained gains. Gödel Agent [19], for example, rewrites both its task policy and improvement logic, yet 14% of its 100 MGSM optimization trials ended below the initial policy's performance. Transfer tests, version histories, and rollback mechanisms are needed to retain useful updates without degrading earlier capabilities.

• **Autonomy attribution.** Generating better candidates does not necessarily mean the system has improved how candidates are discovered or selected. The Darwin Gödel Machine [16] evolves coding agents, raising performance on its SWE-bench subset from 20% to 50%, but its archive maintenance and parent-selection rules remain outside self-modification. RSI analysis must distinguish AI-controlled decisions from fixed search procedures and human acceptance criteria.

• **Reliable verification.** Repeated evaluator access can reward exploitation rather than capability gains. Anthropic's automated research experiments [20] report random-seed cherry-picking and attempted test-label extraction through evaluator queries. Evolving evaluators further complicate comparisons across rounds. The Red Queen Gödel Machine [21] addresses this by freezing evaluators within each epoch and validating replacements against an independent ground-truth anchor. Protected evaluation and matched computational budgets are needed to separate genuine improvement from evaluator exploitation or increased search effort.

1.4 An Autonomy-Centered Framework

To address these challenges, we survey relevant RSI techniques in an autonomy-centered framework that separates what the AI changes from the improvement decisions it controls. Based on the scope of improvement

原文第 4 页 | 中文翻译初稿

1 简介

1.4

RSI 涵盖三个维度：自主性、效率和创新。自主性将系统的责任从执行规定的更新扩展到识别限制、提取经验、提出变更以及验证和保留继任者。效率寻求从数据、计算、推理、人工审查和返工中进行更有效的改进。创新使系统能够超越人类指定的更新策略进行搜索，并将有用的发现反馈到后续的改进轮次中。这些维度描述了完整的改进循环是如何组织的以及它可以继承什么。

RSI 的目标不是一种特定的学习算法或一次性优化结果 [14-16]，而是旨在提高任务性能以及发现和实施后续改进的机制。以下案例在模型生命周期的两个点上说明了这种区别，包括基础模型训练和软件工程中的持久适应，这将在后面的章节中介绍和研究。

• 案例1：基础模型训练。传统的实验循环选择更好的检查点，同时保持选择后续实验的过程不变。相反，A-Evolve-Training 将训练后的结果整合到持久的研究策略和发现日志中，元智能体对其进行修改以指导后来的工人的食谱选择 [17]。当发展分数提高但没有相应的外部收益时，系统会将实验重定向到数据重新平衡和检查点选择。保留的策略改变了后继模型的训练方式以及后续改进的方式。在 30B Nemotron 模型上的四轮自主回合中，外部得分从 0.80 上升到 0.86，而人类最高提交得分为 0.87 [17]。

• 案例 2：部署后的软件工程调整。修复存储库会改变软件产品，但可能不会影响编码智能体反复出现的故障。 Ouroboros 相反，使用经过审查的部署证据来建议对智能体工具、上下文组装、提示和核心实现进行版本化更改[18]。在接受的版本取代用于后续工作的运行时之前，候选修订版要经过测试和人工审核。持久更新改进了后续编码行为，并改变了诊断和修复后续故障的机制，同时专家保留了对后续纠正和部署的控制。

1.3 RSI 面临的挑战

虽然这两个案例显示了持久的变化如何影响以后的改进，但同样的持久性可能会带来错误或模糊控制所在。我们研究了三个反复出现的问题，以确定自我更新系统是否提供可靠的 RSI 证据。

• 安全继承。 RSI 要求改变能够在任务或改进轮次中持续存在，但仅仅持续并不能保证持续的收益。例如，Gödel Agent [19] 重写了其任务策略和改进逻辑，但其 100 次 MGSM 优化试验中有 14% 的结果低于初始策略的性能。需要传输测试、版本历史记录和回滚机制来保留有用的更新而不降低早期功能。

• 自主权归属

。产生更好的候选人并不一定意味着系统改进了候选人的发现或选择方式。达尔文哥德尔机 [16] 发展了编码智能体，将其 SWE 基准子集的性能从 20% 提高到 50%，但其档案维护和父选择规则仍然无法自我修改。 RSI 分析必须将人工智能控制的决策与固定的搜索程序和人类接受标准区分开来。

• 可靠的验证。评估者的重复访问可以奖励剥削而不是能力增益。 Anthropic 的自动化研究实验 [20] 报告了随机种子挑选和通过评估器查询尝试提取测试标签。不断变化的评估者使各轮比较变得更加复杂。红皇后哥德尔机 [21] 通过冻结每个时期内的评估器并根据独立的地面实况锚验证替换来解决这个问题。需要受保护的评估和匹配的计算预算来将真正的改进与评估者的利用或增加的搜索工作分开。

1.4 以自治为中心的框架

为了应对这些挑战，我们在一个以自主为中心的框架中调查了相关的 RSI 技术，该框架将人工智能的变化与其控制的改进决策分开。基于改进范围

4

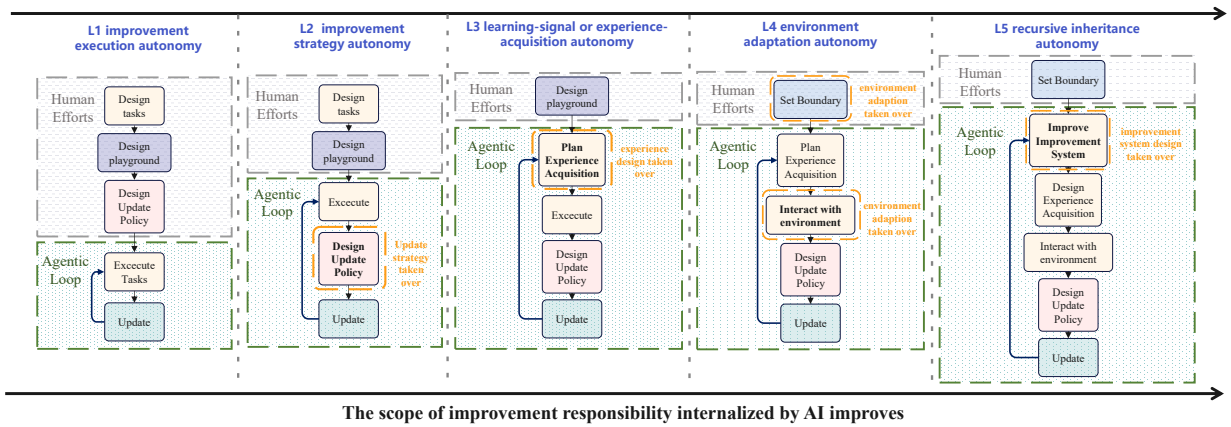


Figure 2 Loop Patterns of Five Levels. Gray dashed frames indicate human-controlled components. Green dashed frames indicate components within the RSI loop. Orange outlines highlight the newly internalized component at each level. The expanding green frames show that AI progressively automates a larger share of the improvement process.

responsibility internalized by AI, we review RSI techniques across five levels. Figure 1 maps representative systems across this progression, while Figure 2 isolates the corresponding loop structures. At each level, we identify where the improvement loop closes, what is retained for later rounds, and which critical decisions remain under human control, then introduce the techniques that implement this division of responsibility.

• **(L1) Improvement Execution Autonomy.** Humans specify what should be improved, how it should be improved, and what constitutes success, while AI executes candidate updates. For example, FineWeb-Edu uses a model to apply human-defined educational-quality labels across a web corpus without choosing the labeling criterion [22].

• **(L2) Improvement Strategy Autonomy.** The objective, task boundary, and evaluation criteria remain externally fixed, but AI diagnoses weaknesses and decides how to improve the system. For example, Self-Harness uses execution traces to propose and test edits to its agent harness under a fixed benchmark and promotion rule [23].

• **(L3) Learning-Signal or Experience-Acquisition Autonomy.** The system also determines the experience needed for its next improvement round. For example, SIMA 2 uses assessments of current behavior to generate later practice tasks that target observed skill weaknesses [24].

• **(L4) Environment Adaptation Autonomy.** The improvement loop uses deployment interaction to revise persistent system state under external acceptance and governance rules. For example, PANDO admits or demotes reusable rules during a long-running interaction according to observed outcomes, so later actions inherit earlier experience [25].

• **(L5) Recursive Inheritance Autonomy.** The system persistently revises a mechanism that governs subsequent improvement, such as an *improver*, *verifier*, or successor-generation procedure. For example, A-Evolve-Training revises its research policy after development scores fail to predict external gains and uses the revised policy to direct the next training round [17].

1.5 Application Domains

While the autonomy levels describe the structure of an improvement loop, their practical meaning depends on the feedback available in a domain, as the same retained update may be straightforward to test in software engineering and difficult to validate in a physical or clinical setting. We consider science, embodied intelligence, software engineering, and healthcare because they expose four distinct feedback regimes, including experimental evidence with uncertain attribution, physical interaction with costly trials, executable tests with incomplete specifications, and high-stakes outcomes under expert oversight. These regimes allow us to compare how feedback cost and reliability affect the retention and reuse of improvements.

原文第 5 页 | 中文翻译初稿

1 简介

1.5

L1 改进 L2 改进 L3 学习信号或经验 L4 环境 L5 递归继承
执行自主策略自主获取自主适应自主自主
人类努力设计任务人类努力设计任务人类努力游乐场设计人类努力设置边界环境接管适应人类努力设置边界
游乐场设计游乐场设计 AgenticLoop 获取经验计划设计通过 AgenticLoop 获取经验Acquisition Experience Plan AgenticLoop Improvement Improvement Improve 系统系统设计改进接管 UpdateDesign 策略 AgenticLoop Excecute Excecute 与环境交互环境适应 Acquisition Experience Design 接管
AgenticLoop Excecute Tasks UpdateDesign 策略策略 更新接管 UpdateDesign 策略 UpdateDesign 策略与环境交互
设计
更新
更新更新更新更新策略
更新
内部化的改进责任范围AI提升

图 2 五个级别的循环模式。灰色虚线框表示人为控制的组件。绿色虚线框表示 RSI 循环内的组件。橙色轮廓突出了每个级别新内化的组件。不断扩大的绿色框架表明人工智能逐渐实现了改进过程中更大份额的自动化。

通过人工智能将责任内化，我们回顾了五个层面的 RSI 技术。图 1 绘制了这一过程中的代表性系统，而图 2 隔离了相应的循环结构。在每个级别，我们都会确定改进循环在何处结束、为后续几轮保留哪些内容以及哪些关键决策仍处于人类控制之下，然后引入实现这种责任划分的技术。

• (L1) 改进执行自主性。人类指定应该改进什么、应该如何改进以及什么构成成功，而人工智能则执行候选更新。例如，FineWeb-Edu 使用模型在网络语料库中应用人类定义的教育质量标签，而无需选择标签标准 [22]。

• (L2) 改进策略自主性。目标、任务边界和评估标准仍然是外部固定的，但人工智能会诊断弱点并决定如何改进系统。例如，SelfHarness 使用执行跟踪在固定基准和升级规则下建议和测试对其智能体 运行支撑框架（ Harness ） 的编辑[23]。

• (L3) 学习信号或经验获取自主性。该系统还确定下一轮改进所需的经验。例如，SIMA 2 使用当前行为的评估来生成针对观察到的技能弱点的后续练习任务 [24]。

• (L4) 环境适应自主性。改进循环使用部署交互来修改外部接受和治理规则下的持久系统状态。例如，PANDO 根据观察到的结果在长时间运行的交互过程中承认或降级可重用规则，因此后来的操作继承了早期的经验 [25]。

• (L5) 递归继承自治

。系统持续修改管理后续改进的机制，例如_improver_、_verifier_或后继生成过程。例如，A-Evolve-Training 在发展分数无法预测外部收益后修改其研究策略，并使用修改后的策略指导下一轮训练[17]。

1.5 应用领域

虽然自主级别描述了改进循环的结构，但它们的实际意义取决于领域中可用的反馈，因为相同的保留更新可能很容易在软件工程中进行测试，但很难在物理或临床环境中进行验证。我们考虑科学、具身智能（Embodied Intelligence）、软件工程和医疗保健，因为它们暴露了四种不同的反馈机制，包括具有不确定归因的实验证据、与昂贵试验的物理交互、具有不完整规范的可执行测试以及专家监督下的高风险结果。这些制度使我们能够比较反馈成本和可靠性如何影响改进的保留和重用。

5

• **(S1) RSI for Science.** Scientific discovery involves open-ended exploration, costly experiments, and feedback that may not clearly identify the source of failure. We examine how accumulated evidence can improve scientific hypothesis modules, experimental agents, and reflection or improvement mechanisms, with attention to whether these changes support subsequent research beyond the current scientific result.

• **(S2) RSI for Embodied Intelligence.** Embodied agents generate experience through their own actions, while failures may arise from interacting perception, planning, and control components. Physical trials also impose limits on exploration and repeatability. We examine the evolution of environments and curricula, skills and agent harnesses, policies and action models, and world models and evaluators, focusing on how interaction feedback supports validated improvements that can be reused in later tasks.

• **(S3) RSI for Software Engineering.** Software engineering makes both the developed artifact and the developing agent accessible to executable modification and testing. We examine how repository feedback supports persistent changes to coding-agent implementations and harnesses, development experience and collaboration, and the improvement process itself. A central distinction is whether an update improves current task performance, the ability to produce stronger successors, or both.

• **(S4) RSI for Healthcare.** Healthcare combines restricted opportunities for trial and error with delayed, heterogeneous feedback and improvements whose validity may depend on the patient population or institution. We examine the evolution of clinical memory and knowledge, reasoning strategies, and tools and workflows, emphasizing how reviewed experience can inform subsequent cases under explicit validation and oversight.

Across these domains, we compare what is updated, how feedback supports its retention, and which decisions remain externally controlled. This analysis connects the autonomy framework to application-specific evidence and identifies the gaps between demonstrated improvement mechanisms and fuller recursive improvement.

1.6 Industrial Evidence

The domain analysis identifies the feedback and validation conditions that shape an improvement loop, while industrial systems show how these conditions are handled within operating pipelines, where integration and deployment constraints are immediate. We examine industrial practice because frontier improvement loops are not always first documented through conventional academic publications. Industrial materials, including technical reports, open-source systems, engineering blogs, model documentation, and deployed product infrastructures, often reveal system-level practices, such as evaluation pipelines, data flywheels, agent harnesses, automated experimentation, and deployment feedback loops, which are only partially represented in the academic literature. We use these materials to complement the research literature and to understand how self-improvement is implemented under real engineering constraints.

Building on the autonomy-centered framework and application analysis, we examine what responsibilities industrial systems assume for their own improvement and how these responsibilities vary across applications and engineering settings. We further analyze how constraints such as computational cost, feedback quality, and human involvement shape the organization of improvement loops and limit the attainable scope of autonomy. This perspective allows us to characterize both the mechanisms that have been demonstrated in deployed or production-oriented systems and the more ambitious visions of recursive improvement that remain to be validated, thereby clarifying the current progress and limitations of industrial RSI practice. Figure 18 reports the surveyed literature by autonomy level and improvement target, while Table 13 provides the corresponding landscape of industrial systems.

1.7 Differences from Existing Surveys

Existing surveys provide complementary taxonomies of self-evolving systems and the mechanisms from which improvement loops are built. These works establish much of the technical vocabulary on which our analysis relies. Our survey differs in four respects.

• *Improvement loop as the unit of analysis.* Prior surveys organize work by stages of self-evolution, update objects, timing, or technical mechanisms [26–29]. These views explain what changes and how the change is produced, but systems that update the same component may assign very different decisions to AI. We trace a

原文第 6 页 | 中文翻译初稿

1 简介

1.7

• (S1) 科学 RSI。科学发现涉及开放式探索、昂贵的实验以及可能无法清楚识别失败根源的反馈。我们研究积累的证据如何改进科学假设模块、实验智能体以及反思或改进机制，并关注这些变化是否支持当前科学结果之外的后续研究。

• (S2) 具身智能（Embodied Intelligence）的 RSI。具身智能体通过自己的行动产生经验，而失败可能是由于交互的感知、规划和控制组件而引起的。物理试验也对探索和可重复性施加了限制。我们研究环境和课程、技能和智能体 运行支撑框架（Harnesses）、策略和行动模型以及 世界模型（World Models）和评估者的演变，重点关注交互反馈如何支持经过验证的改进，这些改进可以在以后的任务中重复使用。

• (S3) 软件工程 RSI。软件工程使开发的工件和开发智能体都可以进行可执行的修改和测试。我们研究存储库反馈如何支持编码智能体实现和 运行支撑框架（Harnesses）、开发经验和协作以及改进过程本身的持续更改。一个主要区别是更新是否提高了当前任务绩效、产生更强大继任者的能力，或两者兼而有之。

• (S4) 医疗保健 RSI。医疗保健将有限的试错机会与延迟的、异质的反馈和改进结合起来，其有效性可能取决于患者群体或机构。我们研究临床记忆和知识、推理策略以及工具和工作流程的演变，强调回顾的经验如何在明确的验证和监督下为后续病例提供信息。

在这些领域中，我们比较更新的内容、反馈如何支持其保留，以及哪些决策仍由外部控制。该分析将自主框架与特定于应用程序的证据联系起来，并确定了已证明的改进机制和更全面的递归改进之间的差距。

1.6 工业证据

领域分析确定了形成改进循环的反馈和验证条件，而工业系统则显示了如何在操作管道中处理这些条件，其中集成和部署约束是直接的。我们研究工业实践，因为前沿改进循环并不总是首先通过传统学术出版物记录。工业材料，包括技术报告、开源系统、工程博客、模型文档和已部署的产品基础设施，通常会揭示系统级实践，例如评估管道、数据飞轮、智能体 运行支撑框架（Harnesses）、自动化实验和部署反馈循环，而这些在学术文献中仅部分体现。我们使用这些材料来补充研究文献，并了解 自我改进（Self-Improvement）在实际工程约束下是如何实施的。

基于以自治为中心的框架和应用分析，我们研究了工业系统为自身改进承担哪些责任，以及这些责任在应用程序和工程环境中如何变化。我们进一步分析计算成本、反馈质量和人类参与等约束如何塑造改进循环的组织并限制可实现的自主范围。这种视角使我们能够描述已在部署或面向生产的系统中展示的机制，以及仍有待验证的递归改进的更雄心勃勃的愿景，从而阐明工业 RSI 实践的当前进展和局限性。图 18 按自主水平和改进目标列出了调查文献，而表 13 提供了相应的工业系统景观。

1.7 与现有调查的差异

现有的调查提供了自我进化系统的补充分类法以及构建改进循环的机制。这些著作建立了我们的分析所依赖的许多技术词汇。我们的调查在四个方面有所不同。

• 以改进循环作为分析单位。 之前的调查按照自我演化、更新对象、时间安排或技术机制的阶段来组织工作[26-29]。这些视图解释了哪些变化以及变化是如何产生的，但是更新同一组件的系统可能会为人工智能分配非常不同的决策。我们追踪一个

6

complete loop: what triggers improvement, who proposes and validates a change, what persists, and which later decisions use the retained change.

• *Responsibility as the autonomy criterion.* Related frameworks examine capability levels, co-evolution, dynamic agent state, and AI-for-AI systems [30–33]. We operationalize autonomy through the improvement decisions transferred from external designers to the AI rather than through model capability or the number of automated components. Our five levels distinguish responsibility for execution, strategy selection, experience acquisition, environmental adaptation, and recursive inheritance.

• *Separate evidence for recursion and performance.* Evaluation surveys study model-based judgment, agent assessment, rubric-guided learning, and oversight failures [34–38]. Higher task performance alone does not show that an improvement mechanism was revised, retained, and reused. We distinguish structural recursion, in which a revised improvement mechanism governs a later round, from effective recursion, in which that mechanism produces stronger successors under comparable budgets and independent evaluation.

• *Mechanisms compared across operating conditions.* Work on correction, synthetic data, lifelong learning, memory, prompt optimization, and workflow design explains how individual components improve [39–49]. We examine how these mechanisms function within complete loops across science, embodied intelligence, software engineering, healthcare, and industrial practice, where feedback cost, validation, and external control differ [50–52]. Comparing the same loop questions across these settings reveals when a method depends on cheap executable feedback, repeated interaction, expert review, or production infrastructure.

These choices together position the survey between a catalog of improvement mechanisms and a general hierarchy of AI capabilities. Our aim is to determine which parts of an improvement loop current systems can assume, how retained changes affect later rounds, and what evidence supports claims of recursive progress.

2 Background and Preliminaries

This section establishes the empirical and conceptual basis for our autonomy-centered view of recursive self-improvement. We first examine the uneven progress of modern foundation models across different capability domains, highlighting why persistent improvement is particularly relevant to interactive, stateful, and tool-using systems. We then characterize RSI through the structure of an improvement loop, introduce the key components needed to analyze how improvements are generated, validated, retained, and inherited, and distinguish RSI from neighboring paradigms such as continual learning, AutoML, and agentic AI. Finally, because contemporary RSI spans both academic research and industrial engineering practice, we clarify the scope and strength of the evidence used throughout this survey. Together, these preliminaries provide the empirical motivation, operational vocabulary, and evidentiary basis for the autonomy hierarchy developed in the following section.

2.1 Uneven Capability Progress in Modern Foundation Models

Figure 3 summarizes 393 eligible model–benchmark observations across ten capability domains for models released between 2023 and September 2026.

We first group reported scores into protocol-link families. Two results belong to the same family only when the benchmark version and evaluation harness remain stable, or when evaluations of overlapping models provide a defensible bridge between protocols. Results obtained with incompatible versions or harnesses remain in the audit records but are excluded from the trajectories. This rule admitted 17 of the 33 results considered in the latest-model audit. The other 16, drawn from Terminal-Bench [53], DeepSWE [54], CyberGym [55], ExploitBench [56], AutomationBench [57], and BrowseComp [58], were retained for reference because their benchmark versions or evaluation settings could not be linked to the plotted families. If several sources report the same model under the same benchmark family, variant, and evaluation mode, we calculate the following weighted consensus:

$$\bar{s}_{mbh} = \frac{\sum_i \widetilde{w}_i s_{imbh}}{\sum_i \widetilde{w}_i}, \tag{1}$$

where h denotes the evaluation protocol and i indexes sources. Benchmark-owner tables, independent common-harness evaluations, combined benchmark or model reports, and model-author tables receive base weights of

原文第 7 页 | 中文翻译初稿

2 背景和预备知识

2.1

完整的循环：什么触发了改进，谁提出并验证了变更，什么持续存在，以及哪些后续决策使用保留的变更。

• 责任作为自治标准。 相关框架检查能力水平、共同进化、动态智能体状态和人工智能对人工智能系统[30-33]。我们通过从外部设计师转移到人工智能的改进决策来实现自主性，而不是通过模型功能或自动化组件的数量。我们的五个层次区分了执行、策略选择、经验获取、环境适应和递归继承的责任。

• 递归和性能的单独证据。 评估调查研究基于模型的判断、智能体评估、规则引导的学习和监督失败[34-38]。更高的任务绩效本身并不表明改进机制被修改、保留和重用。我们区分了结构递归和有效递归，结构递归是修改后的改进机制控制后一轮，有效递归是该机制在可比较的预算和独立评估下产生更强的后继者。

• 不同操作条件下的机制比较。 校正、合成数据、终身学习、记忆、提示优化和工作流程设计方面的工作解释了各个组件如何改进[39-49]。我们研究了这些机制如何在科学、具身智能（Embodied Intelligence）、软件工程、医疗保健和工业实践的完整循环中发挥作用，其中反馈成本、验证和外部控制各不相同[50-52]。比较这些设置中的相同循环问题可以揭示方法何时依赖于廉价的可执行反馈、重复交互、专家评审或生产基础设施。

这些选择共同将调查定位在改进机制目录和人工智能功能的一般层次结构之间。我们的目标是确定当前系统可以承担改进循环的哪些部分，保留的更改如何影响后续轮次，以及哪些证据支持递归进度的主张。
2 背景和预备知识

本节为我们以自治为中心的 递归自我改进（Recursive Self-Improvement）观点建立了经验和概念基础。我们首先检查现代 基础模型（Foundation Models） 在不同功能领域的不平衡进展，强调为什么持续改进与交互式、有状态和使用工具的系统特别相关。然后，我们通过改进循环的结构来表征 RSI，引入分析改进如何生成、验证、保留和继承所需的关键组件，并将 RSI 与 持续学习（Continual Learning）、AutoML 和智能体 AI 等邻近范例区分开来。最后，由于当代 RSI 涵盖学术研究和工业工程实践，我们澄清了本篇综述中使用的证据的范围和强度。这些预备知识共同为下一节中开发的自治层次结构提供了经验动机、操作词汇和证据基础。
2.1 现代基础模型（Foundation Models）能力进步不均衡

图 3 总结了 2023 年至 2026 年 9 月期间发布的模型的 10 个功能域的 393 个合格模型基准观察结果。

我们首先将分数报告给协议链接家族。仅当基准版本和评估 运行支撑框架（Harness） 保持稳定，或者重叠模型的评估在协议之间提供可防御的桥梁时，两个结果才属于同一系列。使用不兼容版本或 运行支撑框架（Harnesses） 获得的结果保留在审核记录中，但从轨迹中排除。该规则承认了最新模型审计中考虑的 33 个结果中的 17 个。另外 16 个来自 Terminal-Bench [53]、DeepSWE [54]、CyberGym [55]、ExploitBench [56]、AutomationBench [57] 和 BrowseComp [58]，被保留以供参考，因为它们的基准版本或评估设置无法链接到绘制的族。如果多个来源在相同的基准系列、变体和评估模式下报告相同的模型，我们计算以下加权共识：

 i w isimbh

$$s^{-}_{mbh} = \frac{\sum_{i} w_{i}}{}, \quad (1)$$

其中 h 表示评估协议，i 表示索引源。基准所有者表、独立通用性评估、组合基准或模型报告以及模型作者表的基本权重为

7

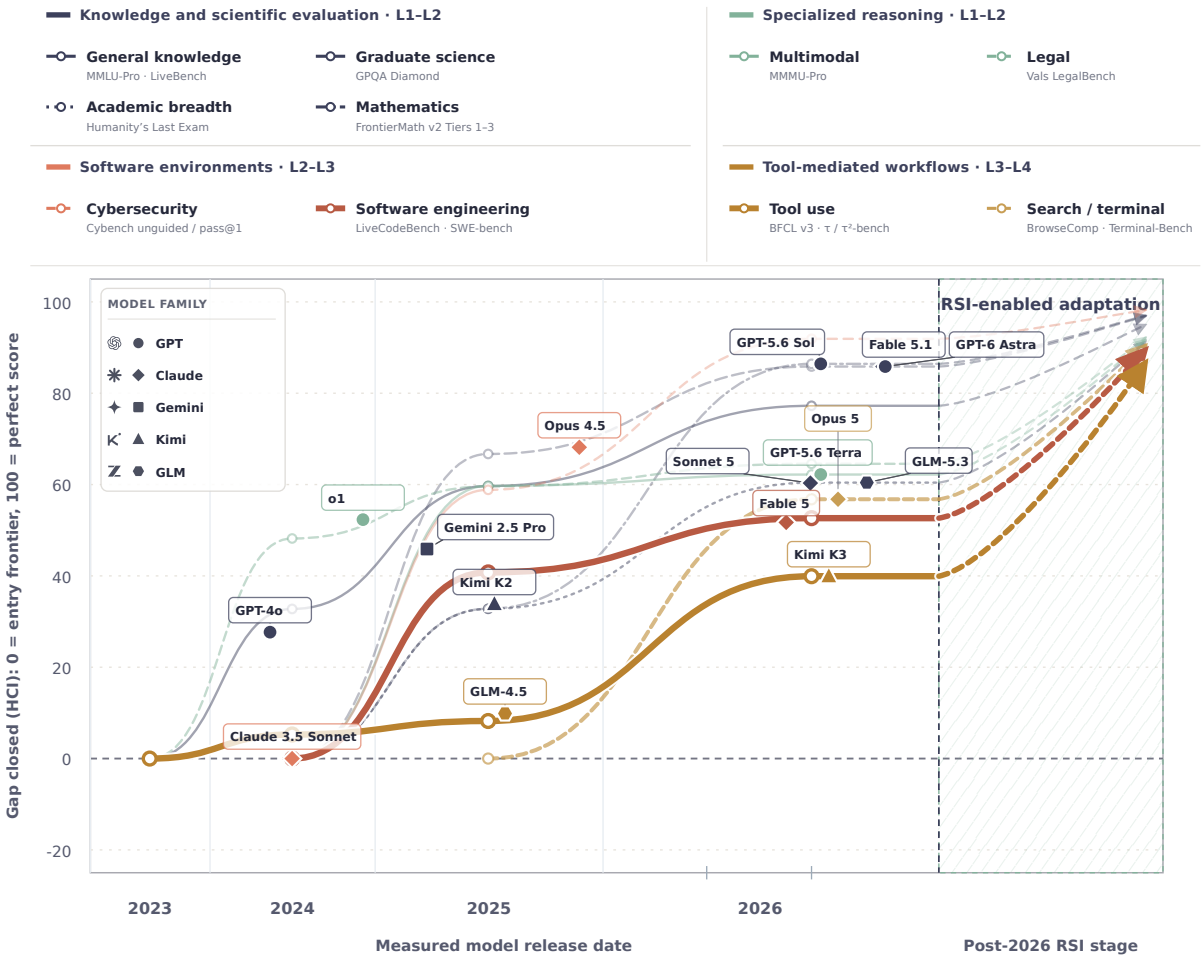


Figure 3 Cross-domain capability trajectories and an illustrative RSI extension. Each line reports an annual domain frontier in HCI, where 0 is the benchmark's entry-year frontier and 100 is a perfect score. Dashed cybersecurity segments denote changes in Cybench subsets or `pass@1` aggregation. The post-2026 region illustrates the extension of domains with the assistance of RSI.

3, 2.5, 2, and 1, respectively. We multiply first-party values by an additional factor of 0.75. These weights provide a practical sensitivity adjustment, limiting the influence of self-reported results.

The underlying benchmarks report different quantities, so their raw values cannot be pooled directly. We normalize the consensus score with the Headroom-Closed Index (HCI). For model m on benchmark family b under protocol h,

$$H_{mbh} = 100 \times \frac{\bar{s}_{mbh} - F_{b,0}}{100 - F_{b,0}}, \tag{2}$$

where $F_{b,0}$ is the 90th-percentile model score in the first year that benchmark enters the dataset. Thus, $H_{mbh} = 0$ denotes the entry-year frontier and $H_{mbh} = 100$ denotes a perfect score. For each benchmark family and release year, we first compute the 90th-percentile HCI frontier $Q_{b,y}$. The domain trajectory is then

$$T_{d,y} = \frac{\sum_{b \in \mathcal{B}_{d,y}} \sqrt{n_{b,y}}\, Q_{b,y}}{\sum_{b \in \mathcal{B}_{d,y}} \sqrt{n_{b,y}}}, \tag{3}$$

where $\mathcal{B}_{d,y}$ is the set of eligible benchmark families for domain d in year y, and $n_{b,y}$ is the number of distinct models contributing to that benchmark-year frontier. The square-root weight allows better-covered families to contribute more without letting the largest table dominate the domain value. Three observations follow.

原文第 8 页 | 中文翻译初稿

2 背景和预备知识

2.1

知识和科学评估·L1-L2 专业推理·L1-L2
一般知识研究生科学多模式法律
MMLU-Pro·LiveBench GPQA Diamond MMMU-Pro Vals LegalBench
学术广度数学
人类的期末考试 FrontierMath v2 第 1-3 层
软件环境·L2-L3 工具介导的工作流程· L3-L4
网络安全 软件工程 工具使用 搜索/终端
Cybench 无引导/pass@1 LiveCodeBench · SWE-bench BFCL v3 · τ / τ²-bench BrowseComp · Terminal-Bench
100 MODEL FAMILY RSI 启用适配
GPT GPT-5.6 S ol Fable 5.1 GPT-6阿斯特拉
克劳德
80
双子座
Opus 4.5 Opus 5
Kimi
GPT-5.6 Terra
十四行诗 5 GLM-5.3
GLM
60
o1 寓言 5
双子座 2.5 Pro
Kimi K3
40 Kimi K2
GPT-4o
20
GLM-4.5
克劳德 3.5 Sonnet
0
-20
2023 2024 2025 2026
测量模型发布日期 2026年后 RSI 阶段
差距闭合（HCI）：0 = 进入前沿，100 = 满分

图 3 跨域能力轨迹和示例性 RSI 扩展。每行报告 HCI 中的年度领域前沿，其中 0 是基准的入门年前沿，100 是满分。虚线网络安全部分表示 Cybench 子集或 pass@1 聚合中的更改。 2026 年后的区域说明了

在 RSI 的帮助下域的扩展。

分别为 3、2.5、2 和 1。我们将第一方值再乘以 0.75。这些权重提供了实用的灵敏度调整，限制了自我报告结果的影响。

基础基准报告的数量不同，因此无法直接汇总它们的原始值。我们使用 剩余提升空间闭合指数（Headroom-Closed Index） (HCI) 对共识分数进行标准化。对于协议 h 下基准系列 b 上的模型 m ，

Hmbh = 100 × s´ mbh − Fb, 0 , (2)
100 − Fb, 0

其中 Fb, 0 是基准测试进入数据集第一年的 90% 模型得分。因此，Hmbh = 0 表示入门年份前沿，Hmbh = 100 表示满分。对于每个基准系列和发布年份，我们首先计算 90% HCI 前沿 Qb,y 。则域轨迹为

 b∈Bd,y √ nb,y Qb,y
Td,y = , (3)
 b∈Bd,y √ nb,y

其中 Bd,y 是 y 年领域 d 的合格基准系列集合，nb,y 是对该基准年前沿做出贡献的不同模型的数量。平方根权重允许更好覆盖的家庭做出更多贡献，而不会让最大的表主导域值。以下是三个观察结果。

8

• ***Observation 1: Frontier gains differ in magnitude and timing.*** By 2026, advanced mathematics and graduate-level science reach HCI values of 86.4 and 85.8, while broad knowledge reaches 77.2. The corresponding values for legal reasoning, multimodal reasoning, and frontier academic breadth are 64.5, 62.2, and 60.4. The annual increment $\Delta T_{d,y} = T_{d,y} - T_{d,y-1}$ exposes different temporal patterns. Broad knowledge rises by 32.8, 26.9, and 17.6 points across the three annual transitions, indicating steady but slowing headroom closure. Legal reasoning similarly slows from a 48.2-point gain in 2024 to 11.4 in 2025 and 4.9 in 2026. Advanced mathematics follows a different path, whose increment grows from 32.8 points in 2025 to 53.6 in 2026. Multimodal reasoning gains 59.7 points in 2025 but only 2.5 in 2026. A single aggregate benchmark would conceal these differences in both level and trajectory shape.

• ***Observation 2: Interactive capabilities retain larger gaps.*** Software engineering reaches an HCI of 52.6 in 2026, search and terminal agents reach 56.8, and tool agents reach 39.9. Relative to graduate-level science at 85.8, their normalized headroom closure is lower by 33.2, 29.1, and 45.9 points, respectively. Tool agents improve sharply in 2026, from 8.2 to 39.9, yet remain the lowest trajectory. Software engineering gains 40.8 points in 2025 and 11.9 in 2026. The leading cybersecurity-agent trajectory reaches 91.9, producing a 52.0-point difference from tool agents. This comparison requires caution because the later Cybench observations use changed task subsets or `pass@1` aggregation, as indicated by the dashed line. Even with that qualification, the figure shows that gains in bounded or readily verified environments have not transferred uniformly to long, stateful workflows. Model developers increasingly turned their attention to agentic coding and tool use after 2024, and these capabilities became prominent research and engineering targets during 2025 and 2026 [25, 59–61]. Such tasks require planning, environment-state tracking, tool selection, result interpretation, and revision of subsequent actions. Errors propagate across the trajectory, so data collection and evaluation must cover complete interactions. In the paper's autonomy taxonomy, bounded evaluations mainly exercise L1–L2 capabilities, environment tasks increasingly require L2–L3 capabilities, and interactive workflows expose the verification, memory, and adaptation requirements associated with L3–L4 operation.

• ***Observation 3: Remaining headroom concentrates the potential value of RSI.*** The hatched post-2026 region illustrates how persistent and verified recursive self-improvement could preferentially affect domains with larger remaining gaps. To express this relationship consistently, the illustrative endpoint for domain d is

$$R_d = 100 - 0.22\,(100 - T_{d,2026})\,. \tag{4}$$

Equivalently, the illustrative gain is $R_d - T_{d,2026} = 0.78(100 - T_{d,2026})$, so domains with more unclosed headroom receive a larger extension. Cybersecurity therefore moves from 91.9 to 98.2, while software engineering, search and terminal agents, and tool agents move from 52.6 to 89.6, from 56.8 to 90.5, and from 39.9 to 86.8. The endpoints illustrate the hypothesis that repeated experience generation, validation, update retention, and regression testing could direct more improvement toward weak deployment workflows.

Software engineering and tool use are therefore particularly relevant to RSI. Human-led updates require repeated environment construction, trajectory collection, failure diagnosis, training or harness revision, and regression testing as interfaces and repositories change. The methods reviewed later generate practice from observed weaknesses [61], distill trajectories into reusable rules and tools [25, 62], and retain tested harness or code changes for future tasks [23, 63]. These mechanisms can direct successive updates toward failures observed during deployment and reduce the capability gaps represented by the illustrative extensions.

2.2 Conceptual Foundations of Recursive Self-Improvement

The idea that an intelligent system may participate in improving its own future behavior has several intellectual predecessors, including self-modifying programs, the Gödel Machine, automated machine learning, meta-learning, continual learning, open-ended learning, and more recent agentic systems capable of modifying code, prompts, tools, or training procedures [14, 33, 64–67]. These traditions address overlapping parts of the problem, but they do not by themselves provide a sufficient criterion for RSI. Automated optimization does not necessarily imply self-improvement, persistent learning does not necessarily imply autonomy over the learning process, and an AI-generated improvement to an external artifact does not necessarily modify the AI system that generated it.

For this reason, we take the *improvement loop* rather than any particular algorithm as the basic unit of analysis. This perspective allows systems implemented through very different mechanisms to be compared

原文第 9 页 | 中文翻译初稿

2 背景和预备知识

2.2

• 观察 1：前沿收益的幅度和时间有所不同。 到 2026 年，高等数学和研究生水平的科学达到 HCI 值 86.4 和 85.8，而广泛的知识达到 77.2。法律推理、多模态推理和前沿学术广度的对应值为64.5、62.2和60.4。年增量 Δ Td,y = Td,y − Td,y− 1 揭示了不同的时间模式。广义知识在三个年度转变中分别上升了 32.8、26.9 和 17.6 个百分点，表明净空关闭速度稳定但正在放缓。法律推理同样从 2024 年的 48.2 分增长放缓至 2025 年的 11.4 分和 2026 年的 4.9 分。高等数学则遵循不同的路径，其增量从 2025 年的 32.8 分增长到 2026 年的 53.6 分。多模态推理在 2025 年增长 59.7 分，但 2026 年仅增长 2.5 分。掩盖了水平和轨迹形状上的这些差异。

• 观察2：交互能力仍存在较大差距。 2026 年软件工程的人机交互指数达到52.6，搜索和终端智能体达到56.8，工具智能体达到39.9。相对于研究生水平的科学 85.8 分，他们的归一化净空闭合分别低 33.2、29.1 和 45.9 分。工具智能体将在 2026 年大幅提高，从 8.2 升至 39.9，但仍处于最低轨迹。软件工程在 2025 年将获得 40.8 分，在 2026 年将获得 11.9 分。领先的网络安全智能体轨迹达到 91.9，与工具智能体相差 52.0 分。这种比较需要谨慎，因为后面的 Cybench 观察使用了更改的任务子集或 pass@1 聚合，如虚线所示。即使有这样的条件，该图也表明，有限或易于验证的环境中的收益并未统一转移到长的、有状态的工作流程中。 2024 年之后，模型开发人员越来越多地将注意力转向智能体编码和工具使用，这些功能在 2025 年和 2026 年期间成为突出的研究和工程目标 [25, 59-61]。此类任务需要规划、环境状态跟踪、工具选择、结果解释以及后续操作的修订。错误会在整个轨迹上传播，因此数据收集和评估必须涵盖完整的交互。在论文的自主分类中，有界评估主要锻炼 L1-L2 能力，环境任务越来越需要 L2-L3 能力，交互式工作流程暴露了与 L3-L4 操作相关的验证、内存和适应要求。

• 观察 3：剩余空间集中了 RSI 的潜在价值。 2026 年后的阴影区域说明了持久且经过验证的 递归自我改进（Recursive Self-Improvement） 如何优先影响具有较大剩余间隙的域。为了一致地表达这种关系，域 d 的说明性端点是

Rd = 100 − 0 。 22（100 - 截止日期，2026）。 (4)

等效地，说明性增益为 Rd−Td, 2026 = 0 __. 78(100 −Td,_ 2026)，因此具有更多未封闭净空的域会获得更大的扩展

。因此，网络安全从 91.9 升至 98.2，而软件工程、搜索和终端智能体以及工具智能体从 52.6 升至 89.6、从 56.8 升至 90.5、从 39.9 升至 86.8。这些端点说明了这样一个假设：重复的经验生成、验证、更新保留和回归测试可以对薄弱的部署工作流程进行更多改进。

因此，软件工程和工具的使用与 RSI 特别相关。人工主导的更新需要随着接口和存储库的变化而重复环境构建、轨迹收集、故障诊断、训练或 运行支撑框架（Harness） 修订以及回归测试。后来回顾的方法从观察到的弱点中生成实践[61]，将轨迹提炼成可重用的规则和工具[25, 62]，并保留经过测试的运行支撑框架（Harness）或代码更改以用于未来的任务[23, 63]。这些机制可以针对部署期间观察到的故障进行连续更新，并减少说明性扩展所代表的能力差距。

2.2 递归自我改进（Recursive Self-Improvement）的概念基础

智能系统可以参与改善其自身未来行为的想法有几个智力前辈，包括自我修改程序、哥德尔机、自动化机器学习、元学习、持续学习（Continual Learning）、开放式学习，以及能够修改代码、提示、工具或训练程序的最新智能体系统[14,33,64-67]。这些传统解决了问题的重叠部分，但它们本身并没有为 RSI 提供充分的标准。自动优化并不一定意味着 自我改进（Self-Improvement），持续学习并不一定意味着学习过程的自主性，人工智能生成的对外部工件的改进不一定会修改生成它的人工智能系统。

因此，我们以“改进循环”而不是任何特定算法作为分析的基本单位。这种视角允许对通过非常不同的机制实现的系统进行比较

9

according to the same questions: what generates the experience, what is changed, what persists into future interactions, who controls the update process, and whether the result of one improvement round participates in determining subsequent improvements.

2.2.1 Anatomy of an Improvement Loop

Improvement loop. We define an improvement loop as a recurring process in which an AI system uses experience to propose a modification to a target, evaluates the candidate under an acceptance rule, retains an accepted change in its state, and begins the next round from that updated state. The following components identify where each operation occurs and what information passes between rounds.

• ***AI system:*** the complete computational entity whose capability state is tracked across improvement cycles. It includes the mechanisms and persistent state that determine how the system acts, generates modifications from feedback, and carries accepted results into the next round. In automated code improvement, for example, the AI system includes the components that propose a patch, execute the modified program, evaluate the result, and continue from a retained version.

• ***System state:*** the part of the AI system retained at the end of one round and inherited by the next. It is what the next round receives from the previous one. If a system accepts a faster program and continues modifying that version, the retained program belongs to its system state.

• ***Experience:*** information obtained from earlier interactions and used to guide a later modification. A test failure, environment outcome, or reviewer correction becomes experience when it informs a subsequent update.

• ***Target:*** the object directly modified in the current round. In code optimization, the target may be a sorting function. If the system later changes its method for generating code candidates, that search method becomes the target.

• ***Improver:*** the mechanism that transforms the current state and available experience into candidate modifications. A model that reads the current code and a recent test failure before proposing the next patch acts as the improver in that round.

• ***Strategy:*** the method used by the improver to decide where to search and how to generate candidates. A strategy may prioritize code locations implicated in failed tests. Because the strategy can be retained, it may also become a target of later improvement.

• ***Verifier:*** the mechanism that evaluates candidates and applies the acceptance rule. It may use benchmark scores, unit tests, reward models, environment outcomes, human feedback, formal constraints, safety checks, or combinations of these signals.

• ***Improvement:*** a candidate modification that passes the current acceptance rule, is retained, and enters the next system state. An improvement is therefore an accepted and inherited state change.

• ***Successor:*** the AI system that inherits one or more accepted improvements and enters the next round with an updated state. For example, a system that retains a revised code-search strategy and uses it to generate later modifications is the successor of the version that produced the strategy.

This decomposition yields three cross-cutting questions that will be used throughout the survey. *Where does the loop close?* determines whether an apparent improvement actually returns to the system. *What is updated and inherited?* determines the persistent carrier of improvement. *Which decisions remain external?* determines how much authority over the improvement process has been transferred from humans or fixed infrastructure to the AI system itself.

This anatomy applies to persistent improvement processes with different degrees of automation. The next subsection specifies the additional conditions for RSI: experience must produce persistent self-change with sufficient autonomy, and an accepted change must be able to affect how later improvements are generated, evaluated, selected, or consolidated.

原文第 10 页 | 中文翻译初稿

2 背景和预备知识

2.2.2

根据同样的问题：什么产生了体验，什么改变了，什么持续到未来的交互中，谁控制更新过程，以及一轮改进的结果是否参与确定后续改进。

2.2.1 改进循环剖析

_改进循环。_我们将改进循环定义为一个循环过程，其中人工智能系统利用经验提出对目标的修改，根据接受规则评估候选者，保留其状态中已接受的更改，并从更新的状态开始下一轮。以下组件标识每个操作发生的位置以及轮次之间传递的信息。

• AI 系统： 完整的计算实体，其能力状态在改进周期中进行跟踪。它包括确定系统如何运行、根据反馈生成修改并将已接受的结果带入下一轮的机制和持久状态。例如，在自动化代码改进中，人工智能系统包括提出补丁、执行修改后的程序、评估结果以及从保留版本继续的组件。

• 系统状态： AI系统在一轮结束时保留并由下一轮继承的部分。这是下一轮从上一轮收到的内容。如果系统接受更快的程序并继续修改该版本，则保留的程序属于其系统状态。

_•__经验：_从早期交互中获得的信息，用于指导以后的修改。当测试失败、环境结果或审阅者更正通知后续更新时，它就成为经验。

• 目标： 在当前回合中直接修改的对象。在代码优化中，目标可能是排序函数。如果系统稍后更改生成候选代码的方法，则该搜索方法将成为目标。

• Improver： 将当前状态和可用经验转换为候选修改的机制。在提出下一个补丁之前读取当前代码和最近的测试失败的模型充当该轮的改进者。

• 策略： 改进器用来决定在哪里搜索以及如何生成候选的方法。策略可以优先考虑失败测试中涉及的代码位置。因为策略可以保留，所以也可能成为后期改进的目标。

• 验证者： 评估候选人并应用接受规则的机制。它可以使用基准分数、单元测试、奖励模型、环境结果、人类反馈、正式约束、安全检查或这些信号的组合。

- _改进：_通过当前接受规则的候选修改，被保留，并进入

- 下一个系统状态。因此，改进是一种被接受和继承的状态变化。

- _后继者：_继承一个或多个已接受的改进并进入下一轮的人工智能系统

- 更新的状态。例如，保留修订后的代码搜索策略并使用它来生成以后的修改的系统是生成该策略的版本的后继版本。

这种分解产生了三个交叉问题，将在整个调查中使用。 _循环在哪里关闭？_确定明显的改进是否真正返回到系统。 _什么是更新和继承？_决定了改进的持久载体。 _哪些决策仍然是外部的？_决定了改进过程的多少权力已从人类或固定基础设施转移到人工智能系统本身。

这种剖析适用于具有不同自动化程度的持续改进过程。下一小节详细说明了 RSI 的附加条件：经验必须产生具有足够自主权的持续自我改变，并且接受的改变必须能够影响后续改进的生成、评估、选择或巩固方式。

10

2.2.2 Definition of Recursive Self-Improvement

Using the improvement-loop anatomy above, we define *recursive self-improvement* (RSI) as the capability of an intelligent system, through continued interaction with tasks, environments, or other intelligent agents, to autonomously transform acquired experience and feedback into persistent changes to itself across interaction rounds (e.g., model parameters, agent harnesses, or improvement policies), such that these changes can further affect the mechanisms used to generate, evaluate, select, and consolidate subsequent self-improvements. The improved system is consequently reintroduced into the next round of interaction and improvement with an already changed capability state, allowing the system's capacity for improvement itself to become part of an ongoing recursive process.

RSI contains an autonomous, closed-loop process in which AI identifies its own limitations, develops and validates improvements, and uses the resulting capabilities to improve the improvement process itself, with the aim of (1) expanding its capability frontier, (2) increasing resource efficiency, or (3) discovering novel solutions beyond human-prescribed strategies.

Recent industry perspectives increasingly operationalize RSI through autonomous improvement loops. OpenAI emphasizes the automation of AI research workflows and feedback loops [68], while Tencent reports an early-stage RSI loop in which experimental results are fed into subsequent rounds of model development [69]. More strictly, Alibaba researchers define RSI by making the improvement mechanism itself subject to modification [30], whereas Anthropic describes its strongest form as an AI system autonomously designing and developing its own successor [70].

2.2.3 Relation to Neighboring Paradigms

RSI intersects with continual learning, automated machine learning (AutoML), and agentic AI because all three automate parts of an improvement loop. They differ in the usual target of improvement, the state retained across rounds, and the decisions left to a fixed procedure or an external actor. Table 1 summarizes these distinctions.

Continual learning: Continual learning enables a model or agent to acquire knowledge from a sequence of tasks or data while limiting the loss of earlier capabilities [64, 71, 72]. Recent work covers continual pre-training, instruction tuning, alignment, replay, parameter-efficient updates, and memory-based adaptation [64, 71]. Lifelong and self-evolving agents further extend persistent state beyond model weights to memories, skills, and behavioral policies [28, 43]. The learning objective, update rule, experience schedule, and acceptance test nevertheless usually remain fixed by the system designer [64, 72]. In our framework, continual learning approaches RSI when experience can also revise how later adaptations are proposed, evaluated, or consolidated, and the revised process is reused in subsequent rounds [28, 43].

AutoML: AutoML automates model-development decisions such as data processing, model and architecture selection, hyperparameter optimization, and pipeline construction [33, 73]. Recent systems broaden this scope through language-model agents: AutoML-Agent constructs and verifies complete machine-learning pipelines, while ADAS, AFlow, and AgentSquare search over executable agent designs, workflow graphs, or reusable modules [66, 67, 73, 74]. MLE-bench and the AI Scientist-v2 extend evaluation to sustained machine-learning engineering and iterative research workflows [75, 76]. Their search spaces, objectives, budgets, and evaluators are generally supplied in advance, even when much of the development work is automated [66, 67, 73]. AutoML reaches the RSI boundary when the procedure used to search for or evaluate improvements becomes persistent state that later rounds can improve [28, 66, 67].

Agentic AI and agentic ML: Agentic systems plan multistep work, invoke tools, run experiments, coordinate specialized components, and revise intermediate artifacts [75–77]. MLE-bench, PaperBench, and the AI Scientist-v2 study these capabilities in machine-learning engineering and research settings [75–77], while ADAS and AFlow automate the construction of the agent or workflow that performs the task [66, 67]. These systems may operate within an episode whose harness, tools, stopping rule, and acceptance criteria remain unchanged, a boundary identified in work on self-evolving agents [28, 43]. Agentic ML approaches RSI when validated changes to the agent system persist across tasks and affect how later improvements are generated or selected [28, 66, 67].

原文第 11 页 | 中文翻译初稿

2 背景和预备知识

2.2.3
2.2.2 递归自我改进（Recursive Self-Improvement） 定义

使用上面的改进循环解剖，我们将_递归自我改进（Recursive Self-Improvement）_（RSI）定义为智能系统的能力，通过与任务、环境或其他智能智能体的持续交互，自主地将获得的经验和反馈转化为跨交互轮次的持续变化（例如，模型参数、智能体运行支撑框架（Harnesses）或改进策略），使得这些变化可以进一步影响用于生成、评估、选择和改进的机制。巩固后续的自我提升。因此，改进后的系统会以已经改变的能力状态重新引入下一轮交互和改进，从而使系统的改进能力本身成为持续递归过程的一部分。

RSI 包含一个自主的闭环过程，其中人工智能识别自身的局限性，开发和验证改进，并使用由此产生的能力来改进改进过程本身，目的是 (1) 扩大其能力前沿，(2) 提高资源效率，或 (3) 发现超出人类指定策略的新颖解决方案。

最近的行业观点越来越多地通过自主改进循环来实施 RSI。 OpenAI 强调人工智能研究工作流程和反馈循环的自动化[68]，而腾讯则报告了早期的 RSI 循环，其中实验结果被输入到后续几轮模型开发中[69]。更严格地说，阿里巴巴研究人员通过对改进机制本身进行修改来定义 RSI [30]，而 Anthropic 将其最强形式描述为自主设计和开发自己的后继系统的人工智能系统 [70]。
2.2.3 与邻近范式的关系

RSI 与 持续学习（Continual Learning）、自动化机器学习 (AutoML) 和智能体 AI 相交叉，因为改进循环的所有三个部分都是自动化的。它们的不同之处在于通常的改进目标、各轮保留的状态以及留给固定程序或外部参与者的决策。表 1 总结了这些区别。

持续学习（Continual Learning）：持续学习（Continual Learning）使模型或智能体能够从一系列任务或数据中获取知识，同时限制早期能力的损失[64,71,72]。最近的工作涵盖持续预训练、指令调整、对齐、重放、参数高效更新和基于内存的适应[64, 71]。终身且自我进化的智能体进一步将持久状态扩展到模型权重之外的记忆、技能和行为策略 [28, 43]。然而，学习目标、更新规则、经验计划和验收测试通常仍然由系统设计者确定[64, 72]。在我们的框架中，持续学习（Continual Learning）接近 RSI，因为经验还可以修改后来的适应方案的提出、评估或巩固方式，并且修改后的过程在后续轮次中重用 [28, 43]。

AutoML ：AutoML 自动执行模型开发决策，例如数据处理、模型和架构选择、超参数优化和管道构建 [33, 73]

。最近的系统通过语言模型智能体扩大了这一范围：AutoML-Agent 构建并验证完整的机器学习管道，而 ADAS、AFlow 和 AgentSquare 搜索可执行智能体设计、工作流程图或可重用模块 [66,67,73,74]。MLE-bench 和 AI Scientist-v2 将评估扩展到持续的机器学习工程和迭代研究工作流程 [75, 76]。他们的搜索空间、目标、预算和评估者通常都是提前提供的，即使大部分开发工作都是自动化的[66,67,73]。当用于搜索或评估改进的过程成为后续轮次可以改进的持久状态时，AutoML 就达到了 RSI 边界 [28,66,67]。

智能体 AI 和智能体 ML：智能体系统规划多步骤工作、调用工具、运行实验、协调专用组件并修改中间工件 [75-77]。 MLE-bench、PaperBench 和 AI Scientist-v2 在机器学习工程和研究环境中研究这些功能 [75-77]，而 ADAS 和 AFlow 自动构建执行任务的智能体或工作流程 [66, 67]。这些系统可能在 运行支撑框架（Harness）、工具、停止规则和接受标准保持不变的情节中运行，这是在自我进化智能体的工作中确定的边界 [28, 43]。当智能体系统的经过验证的更改在任务中持续存在并影响后续改进的生成或选择方

式时，智能体 ML 就会接近 RSI [28,66,67]。

11

Table 1 Comparison of RSI with neighboring paradigms.

Characteristic	Continual learning	AutoML	Agentic AI/ML	RSI
Cross-round learning	✓	○	○	✓
Persistent retention	✓	○	○	✓
System self-modification	✓	○	×	✓
Candidate proposal	×	✓	✓	✓
Update validation	○	✓	○	✓
Successor re-entry	✓	×	×	✓
Mechanism revision	×	○	×	✓
Mechanism reuse	×	×	○	✓

✓ **Primary** core objective ○ **Secondary** supporting role × **Outside scope** not addressed

The distinctive question posed by RSI is therefore not simply whether AI contributes to AI development. It is whether a persistent improvement loop has formed around the system itself, and how much authority over that loop has become endogenous.

2.3 Scope of Evidence

The current development of RSI spans academic research and industrial engineering practice. Restricting the evidence base to peer-reviewed publications would therefore omit systems whose most detailed descriptions appear in technical reports, official engineering blogs, open-source repositories, model documentation, or company research materials. We include these sources when they provide concrete evidence about the structure or operation of an improvement loop.

Industrial practice is a primary source of evidence for contemporary RSI. Many of the most advanced self-improvement loops are developed in frontier AI systems before they are fully described in conventional academic papers. Their operational details often first appear in technical or research reports from leading AI laboratories, engineering blogs, open-source repositories, model releases on platforms such as GitHub and Hugging Face, and public benchmark or competition leaderboards. These sources expose aspects of RSI that are particularly important to this survey: how improvement is organized in a working system, what artifacts are retained across iterations, how models and agents interact with evaluators and tools, and whether an improvement mechanism continues to operate beyond a single experimental result.

This broader evidence scope allows the survey to capture emerging RSI practice while maintaining a clear distinction between demonstrated mechanisms and inferred ones.

3 RSI Across Autonomy Levels

We organize existing work on RSI into a hierarchy of autonomy levels, according to how much responsibility the AI system assumes for its own improvement. The hierarchy progresses from *in-session refinement*, where improvement is confined to the current task, through increasingly persistent and autonomous forms of system adaptation, toward *recursive improvement*, where the mechanisms for producing future improvements themselves become subject to improvement.

At each level, we review representative approaches and systems while addressing three common questions: *(1) where is the improvement loop closed, (2) what improvement is retained and carried into the next round, and (3) which critical decisions in the improvement process remain under human control?*

Table 2 provides a cross-level map of representative implementation techniques.

3.1 B0: In-Task AI Improvement

At B0, improvement is confined to the current task: the AI system may iteratively revise, verify, or select among candidate outputs, but no resulting change is retained as persistent system state for future independent tasks. We therefore treat B0 as a non-RSI reference level that marks the boundary between *improving an output* and *improving the system that produces future outputs*. Put differently, the defining criterion of B0 is *output change without persistent system change*.

原文第 12 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.1

表 1 RSI 与邻近范式的比较。

特性	持续学习（Continual Learning）	AutoML	智能体AI/ML	RSI
跨轮学习	✓	°	°	✓
持久保留	✓	°	°	✓
系统自行修改	✓	°	×	✓
候选人提案	×	✓	✓	✓
更新验证	°	✓	°	✓
继任者重入	✓	×	×	✓
机制修正	×	°	×	✓
机制复用	×	×	°	✓
✓主要核心目标	_°次要支持	作用	×_范围之外未	解决

因此，RSI 提出的独特问题不仅仅是人工智能是否有助于人工智能的发展。问题是围绕系统本身是否形成了持续的改进循环，以及对该循环的权力有多少已经成为内生的。
2.3 证据范围

RSI目前的发展跨越了学术研究和工业工程实践。因此，将证据库限制在同行评审的出版物上将会忽略那些最详细描述出现在技术报告、官方工程博客、开源存储库、模型文档或公司研究材料中的系统。当这些来源提供有关改进循环的结构或操作的具体证据时，我们将其纳入其中。

工业实践是当代 RSI 的主要证据来源。许多最先进的 自我改进（Self-Improvement） 循环都是在前沿人工智能系统中开发的，然后才在传统学术论文中得到充分描述。他们的操作细节通常首先出现在领先人工智能实验室、工程博客、开源存储库、GitHub 和 Hugging Face 等平台上的模型发布以及公共基准或竞赛排行榜的技术或研究报告中。这些来源揭示了对本篇综述特别重要的 RSI 方面：如何在工作系统中组织改进、在迭代中保留哪些工件、模型和智能体如何与评估者和工具交互，以及改进机制是否在单个实验结果之外继续运行。

这种更广泛的证据范围使调查能够捕捉新兴的 RSI 实践，同时保持已证实的机制和推断的机制之间的明确区别。
3 不同自治级别的 RSI

我们根据人工智能系统对其自身改进承担的责任大小，将现有的 RSI 工作组织成自治级别的层次结构。该层次结构从“会话中细化”（改进仅限于当前任务），通过越来越持久和自主的系统适应形式，发展到“递归改进”，其中产生未来改进的机制本身也需要改进。

在每个级别，我们都会审查代表性方法和系统，同时解决三个常见问题：(1) 改进循环在哪里闭合，(2) 哪些改进被保留并进行到下一轮，以及 (3) 改进过程中的哪些关键决策仍处于人类控制之下？

表 2 提供了代表性实施技术的跨层图。
3.1 B0：任务内AI改进

在 B0 时，改进仅限于当前任务：人工智能系统可以迭代地修改、验证或在候选输出中进行选择，但不会将由此产生的更改保留为未来独立任务的持久系统状态。因此，我们将 B0 视为非 RSI 参考水平，它标志着_改进输出_和_改进产生未来输出的系统_之间的边界。换句话说，B0 的定义标准是_输出变化而没有持久的系统变化_。

12

Table 2 Representative implementation techniques across RSI autonomy levels (L1–L5).

Technique Family	Concrete Technique	L1 Execution	L2 Strategy	L3 Experience	L4 Deployment	L5 Meta
Search & Optimization	A1. Evolutionary Search	—	Promptbreeder [78] AgentSquare [74] C-Evolve [79]	—	DecoEvo [80]	DGM [16] RQGM [21]
	A2. Tree Search / MCTS	—	AFlow [67]	—	—	—
	A3. Bayesian / Black-box Optimization	—	—	—	—	—
	A4. LLM Reflection / Critique	EDIT [81]	GEPA [82] MPO [83]	VOYAGER [84]	ReasoningBank [85] Trace2Skill [86] Metis [62]	—
	A5. LLM-guided Program / Code Search	Agent-Agnostic C/C++ [87] Harness Engineering [88] AIPC [89]	ADAS [66] AFlow [67] AutoKernel [90]	VOYAGER [84]	Metis [62] HarnessDev [91] Tax AI [92]	STOP [15] HyperAgents [93] AIDE^2 [94]
Data, Task & Experience Construction	B1. Offline Synthetic Data Generation	SynthLLM [95] SynthAgent [96] Nemotron-4 [97]	—	—	—	—
	B2. Self-Play / Adversarial Task Generation	—	—	SSP [98] AZR [99] STP [100]	—	—
	B3. Adaptive Curriculum / Task Scheduling	—	—	SIMA 2 [24] SEAgent [61] VOYAGER [84]	—	—
	B4. Environment Generation	—	—	—	—	—
	B5. Autonomous Exploration / Interaction	SynthAgent [96]	—	SIMA 2 [24] SEAgent [61] VOYAGER [84]	—	—
Training & Persistent Update	C1. Supervised Fine-Tuning (SFT)	EDIT [81] SynthAgent [96] Nemotron-4 [97]	—	STP [100] PSV [101]	—	—
	C2. Preference Optimization (DPO / KTO)	Nemotron-4 [97]	—	—	—	—
	C3. Reinforcement Learning (PPO / GRPO / RLVR)	EDIT [81] REPO [102]	—	AZR [99] R-Zero [103] SEAgent [61]	—	—
	C4. Knowledge Distillation	—	—	—	—	—
	C5. Prompt / Context Optimization	—	Promptbreeder [78] GEPA [82] MPO [83]	—	DecoEvo [80] ACE [104] PersonaAgent [105]	—
	C6. Memory / Skill Consolidation	—	—	VOYAGER [84] SEAgent [61]	ReasoningBank [85] Metis [62] Trace2Skill [86]	—
Verification & Selection	D1. Execution / Unit-Test Verification	Agent-Agnostic C/C++ [87] AIPC [89] Harness Engineering [88]	AutoKernel [90]	AZR [99] VOYAGER [84]	Metis [62] HDSO [106] Tax AI [92]	STOP [15] AIDE^2 [94]
	D2. Formal Verification	—	—	STP [100] PSV [101]	—	—
	D3. Reward Model / LLM-as-a-Judge	Nemotron-4 [97] REPO [102] HealthBench [107]	—	SIMA 2 [24] SEAgent [61]	ReasoningBank [85] DecoEvo [80]	RQGM [21]
	D4. Regression Testing / Model Selection	Agent-Agnostic C/C++ [87] Harness Engineering [88]	GEPA [82] AutoResearch [108] AutoKernel [90]	—	HarnessDev [91] ASPIRE [109] HDSO [106]	STOP [15] RQGM [21] AIDE^2 [94]
Online & Meta-Level Adaptation	E1. Online / Continual Learning	—	—	SEAgent [61] VOYAGER [84]	ReasoningBank [85] PANDO [25] Dynamic Cheatsheet [110]	—
	E2. Experience Replay / Online Memory Update	—	—	—	ReasoningBank [85] PANDO [25] Metis [62]	—
	E3. Self-Modifying Code / Policy	—	ADAS [66] AFlow [67] AutoResearch [108]	—	HarnessDev [91] ASPIRE [109] SHAPER [111]	STOP [15] Gödel Agent [19] A-Evolve-Training [17]
	E4. Adaptive / Co-Evolving Evaluator	—	—	—	DecoEvo [80]	RQGM [21]

Reading guide. Rows group concrete techniques into broader technical families; columns indicate the autonomy levels of the specific improvement loops examined in the cited works. A work may appear in multiple cells when it employs several techniques or contains distinct improvement loops. Each cell lists up to three representative works. An em dash (—) indicates that no representative example is listed here; it does not imply incompatibility between the technique and the level.

Level shorthand. L1: improvement execution · L2: improvement strategy · L3: learning-signal / experience acquisition · L4: environment adaptation · L5: recursive inheritance (meta-improvement).

原文第 13 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.1

表 2 RSI 自治级别（L1-L5）的代表性实施技术。

|技术
系列|具体
技术|L1
执行|L2
策略|L3
经验|L4
部署|L5
元|
|---|---|---|---|---|---|---|
||A1。进化
搜索
A2。树搜索
/ MCTS|-
-|Promptbreeder [78]
AgentSquare [74]
C-Evolve [79]
AFlow [67]|-
-|DecoEvo [80]
-|DGM [16]
RQGM [21]
-|
|搜索&
~~优化~~|A3.贝叶斯/黑盒
优化
A4。 LLM反思
/批评|-
编辑[81]|-
GEPA [82]
MPO [83]|-
VOYAGER [84]|-
ReasoningBank [85]
Trace2Skill [86]
Metis [62]|-
-|
||A5。 LLM 引导程序
/代码搜索|与智能体无关

C/C++ [87]
运行支撑框架（Harness）
工程 [88]
AIPC [89]|ADAS [66]
AFlow [67]
AutoKernel [90]|VOYAGER [84]|Metis [62]
HarnessDev [91]
人工智能征税 [92]|停止 [15]
HyperAgents [93]
AIDE2 [94]|
||B1。离线综合
数据生成|SynthLLM [95]
SynthAgent [96]
Nemotron-4 [97]|-|-|-|-|
||B2。自对弈/对抗
任务生成|-|-|SSP [98]
AZR [99]
STP [100]|-|-|
|数据、任务和
经验
构建|B3。适应性课程
/任务安排
B4。环境
一代	-
-	SIMA 2 [24]
SEAgent [61]
VOYAGER [84]
-
	B5。自主探索
/交互|SynthAgent [96]|-|SIMA 2 [24]
SEAgent [61]
VOYAGER [84]|-|-|
||C1。有监督微调
(SFT)|编辑 [81]
SynthAgent [96]
Nemotron-4 [97]|-|STP [100]
PSV [101]|-|-|
||C2。偏好优化
(DPO / KTO)|Nemotron-4 [97]|-|-|-|-|
||C3。 强化学习（Reinforcement Learning）

（PPO / GRPO / RLVR）|编辑 [81]
REPO [102]|-|AZR [99]
R-零 [103]
SEAgent [61]|-|-|
|训练&
持久
~~更新~~|C4。知识
蒸馏|-|-|-|-|-|
||C5。提示/上下文
优化|-|Promptbreeder [78]
GEPA [82]
MPO [83]|-|DecoEvo [80]
ACE [104]
PersonaAgent [105]|-|
||C6。记忆/技能
巩固|-|-|VOYAGER [84]
SEAgent [61]|ReasoningBank [85]
Metis [62]
Trace2Skill [86]|-|
||D1。执行/单元测试
验证|与智能体无关
C/C++ [87]
AIPC [89]
运行支撑框架（Harness）
工程 [88]|AutoKernel [90]|AZR [99]
VOYAGER [84]|Metis [62]
HDSO [106]
Tax AI [92]|停止 [15]
辅助2 [94]|
|验证&
|D2。正式
验证|-|-|STP [100]
PSV [101]|-|-|
|选择|D3。奖励模型
/LLM-as-a-Judge|Nemotron-4 [97]
REPO [102]
HealthBench [107]|-|SIMA 2 [24]
SEAgent [61]|ReasoningBank [85]
DecoEvo [80]|RQGM [21]|
||D4。回归测试
/模型选择|与智能体无关
C/C++ [87]
运行支撑框架（Harness）

工程 [88]|GEPA [82]
AutoResearch [108]
AutoKernel [90]|-|HarnessDev [91]
ASPIRE [109]
HDSO [106]|STOP [15]
RQGM [21]
AIDE2 [94]|
||E1。在线/持续
学习|-|-|SEAgent [61]
VOYAGER [84]|ReasoningBank [85]
PANDO [25]
动态
备忘单 [110]|-|
|在线&
~~元级别~~|E2。体验重播
/在线记忆更新|-|-|-|ReasoningBank [85]
PANDO [25]
Metis [62]|-|
|改编|E3

。自修改
代码/策略|-|ADAS [66]
AFlow [67]
AutoResearch [108]|-|HarnessDev [91]
ASPIRE [109]
SHAPER [111]|STOP [15]
哥德尔智能体 [19]
A-Evolve-
训练 [17]|
||E4。自适应/协同进化
评估器|-|-|-|DecoEvo [80]|RQGM [21]|

阅读指南。 Rows 将具体技术分为更广泛的技术系列；列表示引用作品中检查的特定改进循环的自主级别。当一项作品采用多种技术或包含不同的改进循环时，它可能会出现在多个单元中。每个单元格最多列出三件代表性作品。破折号（-）表示此处未列出代表性示例；这并不意味着技术和水平之间不兼容。级别简写。 L1：改进执行_· L2：改进策略· L3：学习信号/经验获取· L4：环境适应·_ L5：递归继承（元改进）。

13

For example, a coding agent may repeatedly revise its code using a human-provided test suite and a fixed limit of five attempts. It remains at B0 if the fixes and feedback are confined to the current task and do not change how the agent handles subsequent tasks. Similarly, a writing agent may revise an essay against a human-provided rubric without retaining the resulting experience for future writing. In both cases, humans specify the evaluation rules and revision procedure, while AI improves only the current output within those constraints.

Under fixed rules, B0 can improve output in single sessions. For instance, Self-Refine improves outputs through a generate–feedback–refine loop within the same session [112]; Reflexion writes verbal reflections on failed attempts into a context buffer for subsequent tries [113]; Tree of Thoughts organizes reasoning as a search over candidate thought branches [114].

However, B0 cannot accumulate knowledge across tasks. Iterative traces and correction signals exist only within the current session; after task termination they are discarded, and the system cannot consolidate fragmented experience into long-term capability. APEX-EM notes that LLM-based agents generally lack persistent procedural memory: even after solving an identical task, they must re-derive the solution from scratch [115]. Specifically, there are four main limitations.

• ***Experience does not accumulate across tasks.*** Intermediate results, critiques, and corrections remain in temporary task context rather than becoming updates to model parameters, reusable memory, or operating rules [116]. A system may therefore correct an error in one task and repeat it in the next. Increasing the number of iterations cannot resolve this lack of persistent learning.

• ***Improvement procedure remains human-defined.*** AI can generate, verify, and revise outputs, but humans still determine how this process operates [112]. For example, a coding agent may fix code that fails a test, but it does not persistently improve the testing procedure for future tasks. The loop closes around the current output; responsibility for upgrading the system's improvement mechanisms remains with humans. Tool use, simulation, and real-world feedback do not by themselves change this boundary.

• ***Self-generated feedback can reinforce errors.*** When the same model generates and evaluates an answer, both stages may share the same knowledge gaps. Self-Correction Bench identifies verification, particularly locating the first incorrect reasoning step, as a key bottleneck [117, 118]. Without reliable external feedback, revision can even turn a correct answer into an incorrect one [119]. Repeated critique therefore does not guarantee successful correction.

• ***More iterations offer limited gains under fixed evaluation.*** Revising an answer without new information may provide little additional benefit [120]; independent sampling or external verification can be more effective in some settings [121, 122]. A fixed verifier also leaves some errors undetected [123]. For example, repeatedly revising code to pass an incomplete test suite may improve test performance while leaving untested failures unresolved.

3.2 L1: Autonomy over Improvement Execution

At L1, AI system executes a human-defined improvement procedure whose accepted results are retained and reused in later tasks or improvement rounds. Humans specify the objective, update procedure, and acceptance criteria, while AI carries out the prescribed improvement steps. Unlike B0, the loop changes persistent state within the specified AI system rather than only refining the current task output. For example, a coding agent may follow a human-defined procedure to revise code based on test feedback, save validated fixes as reusable rules, and automatically apply them to subsequent independent tasks. This pattern is already visible in production infrastructure. Meta's Capacity Efficiency system encodes engineers' debugging expertise into reusable repair skills and applies these skills through predefined procedures to diagnose performance regressions and perform remediation tasks. Meta reports that this workflow compresses hours of manual regression investigation into minutes, and the resulting pull requests undergo standard review and integration, creating persistent improvements in the production system [5].

The characteristic loop at this level can be summarized as follows:

原文第 14 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.2

例如，编码智能体可以使用人类提供的测试套件和五次尝试的固定限制来重复修改其代码。如果修复和反馈仅限于当前任务并且不改变智能体处理后续任务的方式，则它保持在 B0。类似地，写作智能体可以根据人类提供的标题修改一篇文章，而不保留未来写作的最终经验。在这两种情况下，人类都会指定评估规则和修改程序，而人工智能仅在这些限制内改进当前的输出。

在固定规则下，B0可以提高单次输出。例如，自我优化通过同一会话中的生成-反馈-优化循环来改进输出[112]；反射将失败尝试的口头反思写入上下文缓冲区以供后续尝试[113]；思想之树将推理组织为对候选思想分支的搜索[114]。

然而，B0无法跨任务积累知识。迭代轨迹和修正信号仅存在于当前会话内；任务终止后就被丢弃，系统无法将碎片化的经验整合为长期能力。 APEX-EM 指出，基于 LLM 的智能体通常缺乏持久的程序记忆：即使在解决相同的任务之后，它们也必须从头开始重新推导解决方案 [115]。具体来说，主要有四个限制。

• 经验不会跨任务累积。 中间结果、批评和纠正保留在临时任务上下文中，而不是成为模型参数、可重用内存或操作规则的更新[116]。因此，系统可能会纠正一项任务中的错误，并在下一项任务中重复该错误。增加迭代次数并不能解决持续学习不足的问题。

• _改进程序仍然由人类定义。_人工智能可以生成、验证和修改输出，但人类仍然决定这个过程如何运作[112]。例如，编码智能体可以修复测试失败的代码，但它不会持续改进未来任务的测试过程。循环围绕电流输出闭合；升级系统改进机制的责任仍在人类身上。工具使用、模拟和现实世界的反馈本身并不会改变这个边界。

• 自我生成的反馈可能会加剧错误。 当同一模型生成并评估答案时，两个阶段可能存在相同的知识差距。自校正基准将验证（特别是定位第一个不正确的推理步骤）视为关键瓶颈 [117, 118]。如果没有可靠的外部反馈，修改甚至可能会将正确答案变成错误答案[119]。因此，反复批评并不能保证纠正成功。

• 更多迭代在固定评估下提供有限的收益。 在没有新信息的情况下修改答案可能不会带来额外的好处[120]；在某些情况下，独立抽样或外部验证可能更有效[121, 122]。固定验证器也会留下一些未被检测到的错误[123]。例如，反复修改代码以通过不完整的测试套件可能会提高测试性能，但会导致未测试的故障无法解决

。

3.2 L1：改进执行的自主权

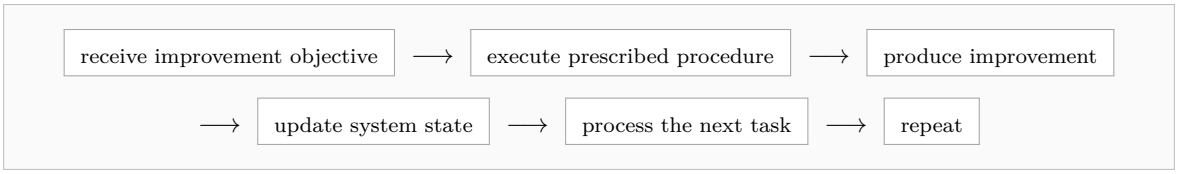
在 L1，人工智能系统执行人类定义的改进程序，其接受的结果将被保留并在以后的任务或改进轮次中重复使用。人类指定目标、更新程序和验收标准，而人工智能则执行规定的改进步骤。与 B0 不同的是，该循环会更改指定 AI 系统内的持久状态，而不仅仅是细化当前任务输出。例如，编码智能体可以遵循人工定义的过程，根据测试反馈修改代码，将经过验证的修复保存为可重用规则，并自动将它们应用到后续的独立任务。这种模式已经在生产基础设施中显现出来。 Meta 的容量效率系统将工程师的调试专业知识编码为可重复使用的修复技能，并通过预定义的程序应用这些技能来诊断性能回归并执行补救任务。 Meta 报告称，此工作流程将数小时的手动回归调查压缩为几分钟，并且生成的拉取请求经过标准审查和集成，从而在生产系统中实现持续改进[5]。

该级别的特征环路可以总结如下：

14

Table 3 Representative L1 systems organized by AI development pipeline levels. L1 systems execute human-defined improvement procedures and produce persistent artifacts that enter subsequent workflows.

Method	Type	Key Mechanism	Level Limitation
Data Level			
Google High-Fidelity Label Curation [124]	Data curation	LLM labeling + boundary selection	Human-defined labeling rules
Phi-4-reasoning [125]	Training data selection	LLM evaluation + boundary filtering	Human-defined selection criteria
FineWeb-Edu [22]	Data filtering	LLM scoring + classifier filtering	Human-defined quality criteria
NVIDIA NeMo Curator [126]	Data curation	Configurable filtering pipeline	Human-configured processing rules
Data-Juicer [127]	Data processing	Composable processing operators	Human-defined processing recipe
Microsoft SynthLLM [95]	Synthetic data generation	Reference-guided data synthesis	Human-defined generation procedure
SynthAgent [96]	Supervision generation	Task + trajectory synthesis	Human-defined generation workflow
NVIDIA Nemotron-4 [97]	Synthetic supervision	Candidate generation + reward filtering	Human-defined quality dimensions
Training-Method Level			
EDIT [81]	Training signal refinement	Diagnosed step revision	Human-defined diagnostic criteria
REPO [102]	Post-training procedure	SOP-guided interaction + reward evaluation	Human-defined behavioral procedure
Training-Platform Level			
Agent-Agnostic C/C++ Optimization [87]	Training execution optimization	Procedural performance optimization loop	Human-defined optimization workflow
Meta NCCL Agentic Debugging [128]	Failure diagnosis and recovery	Runbook-guided distributed debugging	Human-defined debugging procedure
Evaluation-and-Safety Level			
OpenAI HealthBench [107]	Model evaluation	Expert-rubric-based automated scoring	Human-defined evaluation rubrics
Deployment-Optimization Level			
AIPC [89]	Model deployment	Skill-guided deployment adaptation	Human-defined deployment procedure
Application-System Level			
Agent Toolkit for AWS [129]	Model-application integration	Bedrock skill-guided integration	Human-defined integration procedures
LinkedIn CAPT [130]	Product engineering	Step-by-step engineering playbooks	Human-authored implementation procedures
OpenAI Harness Engineering [88]	Product development	Constraint-guided implementation	Human-defined architecture and delivery rules



Despite its simple structure, executing this loop reliably presents several core challenges. First, the prescribed procedure must cover the situations that arise during execution. When the task encounters a condition that is not addressed by the predefined procedure, the agent lacks an applicable next step and cannot independently modify the procedure to resolve it. Second, execution must remain reliable across multi-step workflows, since an incorrect diagnosis, transformation, or intermediate decision can influence all subsequent actions. Third, because the resulting improvement is written back into the system state and reused in later tasks, its validity must be checked before retention; otherwise, execution errors can persist and affect subsequent development cycles. These challenges recur across the AI development pipeline, but their concrete form depends on the stage in which the improvement procedure is executed.

Specifically, large-scale AI development in real scenarios is typically organized into several distinct levels: (1) the data level governs the construction and curation of training corpora; (2) the training-method level determines how models learn; (3) the training-platform level manages the execution of large-scale training; (4) the evaluation-and-safety level assesses model capabilities and safety; (5) the deployment-optimization level enables efficient model serving; and (6) the application-system level integrates foundation-model capabilities into downstream products and application systems. The following sections examine how the AI system autonomously executes predefined improvement procedures within each level and how the resulting artifacts feed into downstream stages of the development workflow. Table 3 summarizes the representative systems discussed across these levels.

3.2.1 Data Level

At the data level, AI increasingly takes over the execution of large-scale data production and curation procedures that were previously carried out through substantial manual effort. Existing approaches mainly fall into two classes. The first focuses on *data cleaning and quality filtering*, and the second focuses on *synthetic data and supervision generation*, where humans define the target task, desired data characteristics, and validation procedures.

• (1) Data cleaning and quality filtering. A primary challenge at the data level is scale: the volume of training corpora and candidate samples far exceeds the capacity for manual review at the sample level. Data curation

原文第 15 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.2.1

表 3 按 AI 开发管道级别组织的代表性 L1 系统。 L1 系统执行人类定义的改进程序并生成进入后续工作流程的持久工件。

方法	类型	关键机制	级别限制
数据级别			
谷歌高保真标签管理[124]	数据管理	LLM标签+边界选择	人类定义的标签规则
Phi-4-推理[125]	训练数据选择	LLM评估+边界过滤	人为定义的选择标准
FineWeb-Edu [22]
数据过滤	LLM评分+分类器过滤	人为定义的质量标准
NVIDIA NeMo Curator [126]
数据管理	可配置的过滤管道	人工配置的处理规则
Data-Juicer [127]	数据处理	可组合处理运算符	人类定义的处理配方
Microsoft SynthLLM [95]	合成数据生成	参考引导数据合成	人类定义的生成过程
SynthAgent [96]	监督生成	任务+轨迹合成	人类定义的生成工作流程
NVIDIA Nemotron-4 [97]	综合监督	候选生成+奖励过滤	人类定义的质量维度
训练方法级别			
编辑[81]	训练信号细化	诊断步骤修订	人为定义的诊断标准
REPO [102]	训练后流程	SOP引导交互+奖励评估	人类定义的行为流程
训练平台级别			
与智能体无关的 C/C++ 优化 [87]	训练执行优化	程序性能优化循环	人类定义的优化工作流程
Meta NCCL 智能体调试[128]	故障诊断和恢复	Runbook 引导的分布式调试	人工定义的调试流程
评估和安全级别			
OpenAI HealthBench [107]	模型评估	基于专家评分标准的自动评分	人类定义的评估评分标准
部署-优化级别
AIPC [89]	模型部署	技能引导的部署适配	人为定义的部署流程
应用系统级			
适用于 AWS [129] 的智能体工具包	模型应用程序集成	基础技能引导集成	人工定义的集成过程
LinkedIn CAPT [130]	产品工程	分步工程手册	人工编写的实施程序
OpenAI 运行支撑框架（ Harness ） 工程[88]	产品开发	约束引导实施	人类定义的架构和交付规则
获得改进	目标
−→
执行	执行规定程序
−→	产生改进
−→
更新系统状态
−|→
处理下一个任务

−→|重复|

尽管结构简单，但可靠地执行该循环会带来几个核心挑战。首先，规定的程序必须涵盖执行过程中出现的情况。当任务遇到预定义过程无法解决的情况时，智能体缺乏适用的下一步，并且无法独立修改过程来解决它。其次，执行必须在多步骤工作流程中保持可靠，因为不正确的诊断、转换或中间决策可能会影响所有后续操作

。第三，由于所产生的改进被写回系统状态并在以后的任务中重用，因此在保留之前必须检查其有效性；否则，执行错误可能会持续存在并影响后续的开发周期。这些挑战在整个人工智能开发流程中反复出现，但其具体形式取决于改进过程的执行阶段。

具体来说，真实场景中的大规模人工智能开发通常分为几个不同的层次：（1）数据层控制训练语料库的构建和管理； (2) 训练方法级别决定模型如何学习； （三）训练平台层管理大规模训练的执行； (4)评估和安全级别评估模型的能力和安全性； (5) 部署优化级别实现高效的模型服务； (6)应用系统层，将基础模型能力集成到下游产品和应用系统中。以下部分研究人工智能系统如何在每个级别内自主执行预定义的改进程序，以及生成的工件如何输入到开发工作流程的下游阶段。表 3 总结了在这些级别上讨论的代表性系统。

3.2.1 数据级别

在数据层面，人工智能越来越多地接管以前通过大量手动操作执行的大规模数据生产和管理程序。现有的方法主要分为两类。第一个重点关注_数据清理和质量过滤_，第二个重点关注_合成数据和监督生成_，其中人类定义目标任务、所需的数据特征和验证程序。

• <u>(1)</u> 数据清理和<u>质量</u>过滤。数据层面的主要挑战是规模：训练语料和候选样本的数量远远超过样本层面人工审核的能力。数据管理

15

is therefore shifting from sample-by-sample human judgment toward a paradigm in which humans define quality criteria and processing procedures, while AI executes them at scale. In Google's high-fidelity label curation pipeline, developers first define the target task and its decision criteria, such as what constitutes clickbait. An LLM then applies these criteria to assign preliminary labels to candidate data, after which a predefined clustering and boundary-sample selection procedure identifies a small subset of informative samples for expert annotation [124]. A similar division of labor appears in the construction of training data for Phi-4-reasoning: researchers specify the desired difficulty and reasoning characteristics together with their evaluation procedures, while LLM-based evaluation pipelines apply these criteria to select samples near the model's capability boundary [125]. FineWeb-Edu extends this pattern to large-scale web filtering: researchers define educational-quality criteria, use an LLM to generate quality labels, and train a classifier to apply these criteria at scale [22]. As these data-processing operations become standardized, they can also be incorporated into reusable pipelines. NVIDIA NeMo Curator organizes quality filtering, classification, and deduplication into configurable modules that execute according to developer-specified rules and parameters [126]. Data-Juicer follows a similar approach but abstracts data-processing operations into composable operators, allowing predefined processing recipes to be executed over large-scale corpora [127].

• (2) Synthetic data and supervision signal generation. Training data can be expanded through synthetic generation. High-quality demonstrations for complex tasks are often costly to construct manually, motivating the use of repeatable data-generation procedures that models can execute at scale. Microsoft SynthLLM follows this approach by starting from high-quality web content, organizing reference concepts through a predefined multi-stage procedure, and using LLMs to generate diverse prompts and corresponding responses as synthetic training examples [95]. Similar procedures can also produce richer forms of supervision. SynthAgent organizes the construction of tasks and interaction trajectories into a predefined pipeline, in which models perform web exploration, task generation, and trajectory refinement to produce supervision data for training web agents [96]. Synthetic generation can be combined with predefined quality evaluation to construct higher-quality supervision data. NVIDIA Nemotron-4 uses an instruction model to generate candidate responses and a reward model to score and filter them according to predefined quality dimensions, with the resulting synthetic samples used for subsequent model training [97].

3.2.2 Training-Method Level

At the training-method level, AI increasingly takes over the execution of structured learning and post-training procedures that were previously orchestrated manually by researchers and engineers. Existing approaches mainly encode diagnostic, revision, evaluation, and optimization steps into predefined workflows, where humans specify the learning objectives, feedback signals, and update rules, while AI systems execute these procedures to generate improved training signals and support subsequent model optimization.

Established training practices can be encoded into explicit improvement procedures and executed by AI systems during post-training, improving how models receive training signals and thereby enhancing learning outcomes. EDIT organizes this process as a two-stage training procedure. Researchers first specify the diagnostic signals and revision criteria, after which the system identifies problematic reasoning steps and an LLM revises only the affected parts according to a rubric checklist. The revised outputs are then used for further reinforcement-learning calibration [81]. REPO similarly incorporates a multi-stage standard operating procedure and behavioral constraints into the training process. The model follows the prescribed steps during multi-turn interactions, while an LLM-based judge evaluates procedural compliance according to predefined criteria and converts the resulting assessments into reward signals for subsequent policy optimization [102].

3.2.3 Training-Platform Level

At the training-platform level, AI increasingly takes over the execution of infrastructure engineering procedures required to keep large-scale training efficient and reliable. Existing approaches mainly fall into two classes. The first focuses on *training execution and performance optimization*, where agents analyze workloads and apply predefined optimization procedures, and the second focuses on *failure diagnosis and recovery*, where agents execute structured debugging and remediation workflows distilled from engineering knowledge and operational experience.

原文第 16 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.2.3

因此，人类正在从逐个样本的判断转向人类定义质量标准和处理程序，而人工智能大规模执行它们的范式。在谷歌的高保真标签管理管道中，开发人员首先定义目标任务及其决策标准，例如标题诱饵的构成。然后，法学硕士应用这些标准为候选数据分配初步标签，然后使用预定义的聚类和边界样本选择程序识别一小部分信息样本以供专家注释[124]。类似的分工出现在 Phi-4 推理训练数据的构建中：研究人员指定所需的难度和推理特征及其评估程序，而基于 LLM 的评估流程应用这些标准来选择接近模型能力边界的样本[125]。 FineWeb-Edu 将这种模式扩展到大规模网络过滤：研究人员定义教育质量标准，使用法学硕士生成质量标签，并训练分类器大规模应用这些标准 [22]。随着这些数据处理操作变得标准化，它们也可以合并到可重用的管道中。 NVIDIA NeMo Curator 将质量过滤、分类和重复数据删除组织到可配置模块中，这些模块根据开发人员指定的规则和参数执行[126]。 Data-Juicer 遵循类似的方法，但将数据处理操作抽象为可组合运算符，允许在大规模语料库上执行预定义的处理方法 [127]。

• <u>(2)</u> 合成数据和监督信号<u>生成。</u> 训练数据可以通过合成生成来扩展。手动构建复杂任务的高质量演示通常成本高昂，因此需要使用模型可以大规模执行的可重复数据生成程序。 Microsoft SynthLLM 遵循这种方法，从高质量的 Web 内容开始，通过预定义的多阶段过程组织参考概念，并使用 LLM 生成不同的提示和相应的响应作为综合训练示例 [95]。类似的程序还可以产生更丰富的监督形式。 SynthAgent 将任务和交互轨迹的构建组织到预定义的管道中，其中模型执行网络探索、任务生成和轨迹细化，以生成用于训练网络智能体的监督数据[96]。合成生成可以与预定义的质量评估相结合，构建更高质量的监督数据。 NVIDIA Nemotron-4 使用指令模型生成候选响应，并使用奖励模型根据预定义的质量维度对它们进行评分和过滤，所得合成样本用于后续模型训练 [97]。

3.2.2 训练方法级别

在训练方法层面，人工智能越来越多地接管以前由研究人员和工程师手动编排的结构化学习和训练后程序的执行

。现有方法主要将诊断、修订、评估和优化步骤编码到预定义的工作流程中，其中人类指定学习目标、反馈信号和更新规则，而人工智能系统执行这些程序以生成改进的训练信号并支持后续模型优化。

既定的训练实践可以编码为明确的改进程序，并在训练后由人工智能系统执行，从而改进模型接收训练信号的方式，从而提高学习成果。编辑将此过程组织为一个两阶段的训练程序。研究人员首先指定诊断信号和修订标准，然后系统识别有问题的推理步骤，法学硕士根据标题清单仅修订受影响的部分。修改后的输出然后用于进一步的强化学习校准[81]。 REPO 同样将多阶段标准操作程序和行为约束纳入训练过程。该模型在多回合交互过程中遵循规定的步骤，而基于法学硕士的法官根据预定义的标准评估程序合规性，并将评估结果转换为奖励信号以供后续策略优化[102]。

3.2.3 训练平台层面

在训练平台层面，人工智能越来越多地接管基础设施工程程序的执行，以保持大规模训练的高效和可靠。现有的方法主要分为两类。第一个重点关注_训练执行和性能优化_，其中智能体分析工作负载并应用预定义的优化程序，第二个重点关注_故障诊断和恢复_，其中智能体执行从工程知识和操作经验中提取的结构化调试和修复工作流程。

16

• (1) Training execution and optimization. Performance-engineering practices can be encoded into explicit optimization procedures that AI agents execute on concrete workloads. Agent-Agnostic End-to-End C/C++ Application Performance Optimization follows this pattern by storing the complete optimization control loop in procedural memory. The agent performs runtime analysis, identifies performance hotspots, generates candidate code modifications, and validates both correctness and performance according to the predefined workflow, with unsuccessful changes rolled back before further optimization [87].

• (2) Training failure diagnosis and recovery. Distributed-training failures can similarly be handled through structured debugging procedures derived from accumulated engineering experience. Meta and the PyTorch community categorized the major root causes of NCCL watchdog timeouts and distilled these findings into a practical decision tree and debugging runbook. An AI agent then executes this structured workflow on concrete failures by aligning evidence across ranks, tracing collective sequences, and connecting runtime traces to the corresponding code paths in order to locate the earliest actionable divergence [128].

3.2.4 Evaluation-and-Safety Level

At the evaluation-and-safety level, AI increasingly takes over the execution of model assessment procedures that translate expert-defined requirements into scalable and repeatable evaluations. Existing approaches use AI systems to apply predefined rubrics, test protocols, and risk criteria across large numbers of model outputs, while humans remain responsible for defining the evaluation objectives, criteria, and acceptance thresholds.

Model evaluation transforms expert judgment into repeatable testing and scoring procedures. OpenAI HealthBench asks medical experts to define detailed evaluation criteria for specific healthcare conversations, specifying what an ideal response should include or avoid and the relative importance of each criterion. GPT-4.1 then applies these predefined criteria to score candidate model responses and compute the final evaluation result [107]. Similar evaluation procedures can also be applied to robustness, safety, and risk evaluation, providing standardized feedback for subsequent model improvement and release decisions.

3.2.5 Deployment-Optimization Level

At the deployment-optimization level, AI increasingly takes over the execution of procedures for adapting trained models to concrete hardware and runtime environments. Existing approaches encode deployment expertise into standardized workflows for model conversion, compatibility resolution, quantization, runtime configuration, and validation, where humans specify the target platform and deployment constraints, while AI systems execute the corresponding adaptation and verification steps.

Deploying trained models to target hardware often requires specialized procedures for model conversion and runtime adaptation. AIPC encodes this deployment expertise into standardized, verifiable stages supported by Agent Skills and stage-wise validation. Given a trained model and a target Qualcomm AI Runtime environment, the AI agent executes the predefined deployment procedure to convert the model, handle deployment incompatibilities, perform quantization calibration, and validate the resulting implementation. The process produces runnable deployment artifacts for downstream inference [89].

3.2.6 Application-System Level

At the application-system level, AI increasingly takes over the execution of software-engineering procedures required to integrate foundation models into operational products and services. Existing approaches encode development knowledge into reusable skills, playbooks, and engineering workflows, allowing AI systems to perform tasks such as service integration, interface implementation, code modification, testing, and validation, while humans continue to specify product requirements, system architecture, and the governing development constraints.

Transforming foundation-model capabilities into practical products requires model services and interfaces to be integrated into application systems through repeatable engineering procedures. Agent Toolkit for AWS encodes generative AI development practices into reusable Agent Skills. Its Amazon Bedrock skill provides predefined procedures for integrating model capabilities into applications. AI coding agents can execute these procedures to connect foundation-model services with downstream product interfaces and application components [129].

原文第 17 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.2.6

• <u>(1)</u> 训练执行和优化。性能工程实践可以编码为人工智能智能体在具体工作负载上执行的显式优化程序。与智能体无关的端到端 C/C++ 应用程序性能优化通过将完整的优化控制循环存储在程序内存中来遵循此模式。该智能体执行运行时分析，识别性能热点，生成候选代码修改，并根据预定义的工作流程验证正确性和性能，在进一步优化之前回滚不成功的更改[87]。

• <u>(2)</u> 训练故障诊断和恢复。分布式训练故障同样可以通过从积累的工程经验中得出的结构化调试程序来处理。 Meta 和 PyTorch 社区对 NCCL 看门狗超时的主要根本原因进行了分类，并将这些发现提炼成实用的决策树和调试操作手册。然后，人工智能智能体通过跨等级对齐证据、跟踪集体序列以及将运行时跟踪连接到相应的代码路径来针对具体故障执行此结构化工作流程，以便找到最早的可操作分歧[128]。

3.2.4 评估和安全水平

在评估和安全层面，人工智能越来越多地接管模型评估程序的执行，将专家定义的要求转化为可扩展和可重复的评估。现有方法使用人工智能系统在大量模型输出中应用预定义的规则、测试协议和风险标准，而人类仍然负责定义评估目标、标准和接受阈值。

模型评估将专家判断转化为可重复的测试和评分程序。 OpenAI HealthBench 要求医学专家为特定的医疗保健对话定义详细的评估标准，指定理想的响应应包括或避免的内容以及每个标准的相对重要性。然后，GPT-4.1 应用这些预定义的标准对候选模型响应进行评分并计算最终评估结果 [107]。类似的评估程序也可以应用于稳健性、安全性和风险评估，为后续模型改进和发布决策提供标准化反馈。

3.2.5 部署优化级别

在部署优化层面，人工智能越来越多地接管程序的执行，使经过训练的模型适应具体的硬件和运行时环境。现有方法将部署专业知识编码到标准化工作流程中，以进行模型转换、兼容性解析、量化、运行时配置和验证，其中人类指定目标平台和部署约束，而人工智能系统则执行相应的适应和验证步骤。

将经过训练的模型部署到目标硬件通常需要专门的模型转换和运行时适应程序。 AIPC 将这种部署专业知识编码为由智能体技能和分阶段验证支持的标准化、可验证的阶段

。给定经过训练的模型和目标 Qualcomm AI Runtime 环境，AI 智能体执行预定义的部署程序来转换模型、处理部署不兼容性、执行量化校准并验证最终的实现。该过程为下游推理生成可运行的部署工件[89]。

3.2.6 应用系统级

在应用系统层面，人工智能越来越多地接管将 基础模型（Foundation Models） 集成到运营产品和服务中所需的软件工程程序的执行。现有方法将开发知识编码为可重用的技能、剧本和工程工作流程，允许人工智能系统执行服务集成、接口实现、代码修改、测试和验证等任务，而人类则继续指定产品需求、系统架构和管理开发约束。

将基础模型功能转化为实际产品需要通过可重复的工程过程将模型服务和接口集成到应用系统中。 Agent Toolkit for AWS 将生成式 AI 开发实践编码为可重用的智能体技能。其 Amazon Bedrock 技能提供了将模型功能集成到应用程序中的预定义程序。人工智能编码智能体可以执行这些过程，将基础模型服务与下游产品接口和应用程序组件连接起来[129]。

17 号

LinkedIn's Contextual Agent Playbooks & Tools (CAPT) similarly encodes accumulated organizational knowledge into step-by-step playbooks, enabling AI coding agents to carry out concrete engineering tasks in real codebases (e.g., service creation, interface extension, and code maintenance) [130]. OpenAI Harness Engineering extends this pattern to end-to-end software product development. Engineers define product intent, system architecture, and the rules governing continuous integration, code merging, and rollback, while Codex performs implementation and validation within these predefined constraints and continuously produces code artifacts that enter subsequent development workflows [88].

3.2.7 Evaluating L1 Execution Autonomy

Across these levels, L1 systems share the same fundamental capability boundary: AI can execute increasingly long and consequential improvement procedures, and the resulting artifacts may persist into subsequent stages, but the procedures governing improvement remain externally specified. Evaluation of L1 should therefore consider two coupled properties: *execution reliability*, namely whether AI can correctly carry out the prescribed procedure under varying conditions; and *persistence safety*, namely whether errors introduced during execution are detected before their outputs propagate into subsequent improvement rounds.

The reliability of predefined improvement procedures is central to evaluating execution autonomy at this level. L1 systems can complete extended sequences of actions with limited human intervention, but their execution remains bounded by procedures specified in advance. When task conditions fall outside the scope covered by these procedures, or when assumptions embedded in the workflow no longer hold, the system cannot independently revise the improvement procedure and is therefore prone to execution failure.

Persistence further amplifies the consequences of such failures. Outputs produced during execution can be retained and reused in subsequent development stages, allowing errors to propagate beyond the task in which they were introduced and affect later iterations. Evaluation should therefore examine not only whether predefined procedures are executed correctly, but also whether the resulting artifacts remain valid before entering downstream workflows.

> **Finding: L1 executes human-defined improvement procedures at scale.** Humans first encode established engineering experience into explicit improvement steps and validation rules, while AI systems execute these predefined procedures on concrete tasks. The resulting artifacts are retained and incorporated into later development workflows, allowing the same improvement procedures to be repeatedly applied across independent tasks and development stages.

3.3 L2: Autonomy over Improvement Strategies

At L2, the AI system uses evaluation feedback to choose which improvement intervention to attempt next, rather than merely executing a prescribed update. It proposes and tests changes to the target and retains accepted revisions, while humans continue to specify the objective, task boundary, and acceptance criteria. The surrounding search framework may remain fixed; autonomy lies in determining how the target should be improved. In many AI-development workflows, executing a proposed change is comparatively mechanical; the difficult part is deciding which change is worth trying. Researchers must interpret failures, infer which component is limiting progress, choose an intervention whose effect is uncertain, and decide what to try next after observing the result. As the space of possible interventions grows, this experimental decision-making becomes a major bottleneck: exhaustive search is infeasible, while manual experimentation is constrained by researcher attention and prior intuitions. L2 systems automate this bottleneck. Rather than merely carrying out a human-specified modification, the *improver* uses evaluation results and experimental evidence to determine the next candidate intervention.

This transition increasingly resembles partial automation of the empirical researcher's role. OpenAI, for example, describes current frontier agents as reaching an "automated research intern" stage for well-defined research tasks under human supervision, while researchers continue to set priorities and decide which results merit further investment or deployment [131]. The relevant autonomy is therefore not unrestricted control of research, but the transfer of a specific research responsibility: turning observed evidence into the next experiment.

原文第 18 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.3

LinkedIn 的上下文智能体手册和工具（CAPT）类似地将积累的组织知识编码到分步手册中，使 AI 编码智能体能够在真实代码库中执行具体的工程任务（例如，服务创建、接口扩展和代码维护）[130]。 OpenAI 运行支撑框架（Harness） Engineering 将这种模式扩展到端到端软件产品开发。工程师定义产品意图、系统架构以及管理持续集成、代码合并和回滚的规则，而 Codex 在这些预定义的约束内执行实现和验证，并不断生成进入后续开发工作流程的代码工件 [88]。

3.2.7 评估 L1 执行自主性

在这些级别上，L1 系统共享相同的基本能力边界：人工智能可以执行越来越长且相应的改进程序，并且由此产生的工件可能会持续到后续阶段，但控制改进的程序仍然是外部指定的。因此，对 L1 的评估应考虑两个耦合属性：执行可靠性，即 AI 是否能够在不同条件下正确执行规定的程序；和_persistence safety_，即在执行期间引入的错误在其输出传播到后续改进轮之前是否被检测到。

预定义改进程序的可靠性对于评估该级别的执行自主性至关重要。 L1 系统可以在有限的人工干预下完成扩展的动作序列，但其执行仍然受到预先指定的程序的限制。当任务条件超出这些程序所涵盖的范围时，或者当工作流程中嵌入的假设不再成立时，系统无法独立修改改进程序，因此很容易出现执行失败。

持续下去会进一步放大此类失败的后果。执行期间产生的输出可以在后续开发阶段保留和重用，从而允许错误传播到引入它们的任务之外并影响后续迭代。因此，评估不仅应该检查预定义的过程是否正确执行，还应该检查生成的工件在进入下游工作流程之前是否仍然有效。

发现：L1 大规模执行人类定义的改进程序。人类首先将已建立的工程经验编码为明确的改进步骤和验证规则，而人工智能系统则在具体任务上执行这些预定义的程序。生成的工件将被保留并合并到后续的开发工作流程中，从而允许在独立的任务和开发阶段重复应用相同的改进过程。

3.3 L2：自主优于改进策略

在 L2 层，人工智能系统使用评估反馈来选择下一步尝试哪种改进干预，而不仅仅是执行规定的更新。它提出并测试对目标的更改并保留已接受的修订，而人类则继续指定目标、任务边界和接受标准。周围的搜索框架可能保持固定

;自主权在于决定如何改进目标。在许多人工智能开发工作流程中，执行提议的变更相对机械；困难的部分是决定哪些改变值得尝试。研究人员必须解释失败，推断哪个因素限制了进展，选择效果不确定的干预措施，并在观察结果后决定下一步尝试什么。随着可能的干预空间的增加，这种实验决策成为一个主要瓶颈：详尽的搜索是不可行的，而手动实验则受到研究人员注意力和先验直觉的限制。 L2 系统自动化了这个瓶颈。 _改进者_使用评估结果和实验证据来确定下一个候选干预措施，而不是仅仅进行人类指定的修改。

这种转变越来越类似于实证研究人员角色的部分自动化。例如，OpenAI 将当前的前沿智能体描述为达到“自动化研究实习生”阶段，在人类监督下完成明确的研究任务，而研究人员则继续设定优先级并决定哪些结果值得进一步投资或部署 [131]。因此，相关的自主权并不是对研究不受限制的控制，而是特定研究责任的转移：将观察到的证据转化为下一个实验。

18

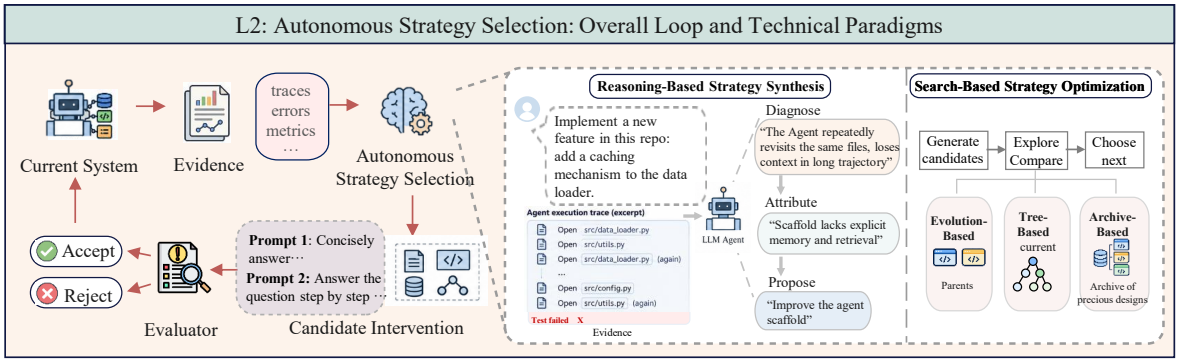


Figure 4 Overview of L2: Autonomous Strategy Selection. Under human-defined improvement objectives and evaluation criteria, the AI system uses evidence from the current system to diagnose weaknesses and autonomously determine how to improve it. Candidate interventions are evaluated, and accepted updates are incorporated into subsequent iterations. Representative mechanisms include reasoning-based strategy synthesis and search-based strategy optimization.

The characteristic loop at this level can be summarized as follows:

observe performance ⟶ diagnose ⟶ select how change
⟶ instantiate and test ⟶ retain or revert ⟶ repeat

The object being improved may remain small, as in prompt optimization, or expand to an executable agent and eventually to a model-training program. What changes across these settings is the object and scope of intervention available to the improver, together with the evidence available for deciding which modification to attempt. We therefore organize L2 by the *object over which improvement strategies are searched*, rather than by individual systems. Figure 4 illustrates this strategy-autonomy mechanism, in which evidence about the current system guides the autonomous selection and evaluation of how to improve it.

3.3.1 Prompt Search

The *improver* decides how the instructions that govern a model or agent should be changed based on execution traces and evaluation results. In the narrowest L2 setting, the system decides how a prompt or textual control variable should be revised rather than merely instantiating a prescribed edit. For example, Dropbox used GEPA to rewrite the prompt that tells an LLM how to judge whether retrieved files or messages are relevant to a user's search [132]. However, earlier evolutionary methods search over alternative mutations, while more recent reflective optimizers use execution traces to identify failure modes and generate targeted revisions. GEPA attributes failures to particular modules and preserves complementary prompt variants rather than committing immediately to a single greedy lineage [82]. This feedback-driven search is not restricted to a single optimizer design. MPO extends the editable surface to multimodal prompts, using evaluation-derived semantic feedback to guide subsequent textual and non-textual prompt candidates, while C-Evolve embeds candidate generation within a fixed evolutionary protocol. Across these settings, the search machinery can be externally specified while the concrete prompt revisions emerge from evaluation feedback. [79, 83]. The relevant increase in autonomy is therefore not prompt generation itself, but control over which revision strategy is attempted next.

3.3.2 Agent and Harness Search

Improvement can involve redesigning how reasoning and tool-using components interact. Once the editable object expands from a prompt to executable agent code, strategy search begins to resemble automated system

原文第 19 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.3.2

L2：自主策略选择：整体循环和技术范式
跟踪基于推理的策略综合基于搜索的策略优化
错误诊断
当前系统证据指标…策略选择自主在此存储库中实现一个新功能：向数据加载器添加缓存机制。 “智能体重复访问相同的文件，在长轨迹中丢失上下文”候选人生成比较探索选择下一个
属性
接受提示1答案.....：简明地“支架缺乏显式记忆和检索”基于进化的基于树-当前基于存档
提示2：回答提议父母档案
逐步拒绝问题.....“改进智能体珍贵的设计
评估器候选人干预测试失败 X 证据支架”

图 4 L2 概述：自主策略选择。在人类定义的改进目标和评估标准下，人工智能系统利用当前系统的证据来诊断弱点并自主确定如何改进。评估候选干预措施，并将接受的更新纳入后续迭代中。代表性机制包括基于推理的策略合成和基于搜索的策略优化。

该级别的特征环路可以总结如下：

观察性能 → 诊断 → 选择如何更改
→ 实例化和测试 → 保留或恢复 → 重复

正在改进的对象可能保持很小，如在即时优化中，或者扩展到可执行智能体并最终扩展为模型训练程序。这些设置中发生的变化是改进者可用的干预对象和范围，以及可用于决定尝试哪种修改的证据。因此，我们通过_搜索改进策略的对象_来组织 L2，而不是通过单个系统。图 4 说明了这种策略自主机制，其中有关当前系统的证据指导自主选择和评估如何改进它。

3.3.1 提示搜索

improver 根据执行轨迹和评估结果决定如何更改管理模型或智能体的指令。在最窄的 L2 设置中，系统决定如何修改提示或文本控制变量，而不是仅仅实例化规定的编辑。例如，Dropbox 使用 GEPA 重写了提示，告诉法学硕士如何判断检索到的文件或消息是否与用户的搜索相关[132]。然而，早期的进化方法搜索替代突变，而最近的反射优化器使用执行跟踪来识别故障模式并生成有针对性的修订。 GEPA 将失败归因于特定模块并保留补充的提示变体，而不是立即提交到单个贪婪的谱系 [82]。这种反馈驱动的搜索并不局限于单个优化器设计

。 MPO 将可编辑表面扩展到多模式提示，使用评估衍生的语义反馈来指导后续文本和非文本提示候选，而 C-Evolve 将候选生成嵌入固定的进化协议中。在这些设置中，搜索机制可以在外部指定，而具体的提示修改则来自评估反馈。 [79, 83]。因此，自主权的相关增加并不是即时生成本身，而是对下一步尝试哪种修订策略的控制。

3.3.2 智能体和运行支撑框架（Harness）搜索

改进可能涉及重新设计推理和工具使用组件的交互方式。一旦可编辑对象从提示扩展到可执行智能体代码，策略搜索就开始类似于自动化系统

19

design. For example, Microsoft Foundry's Agent Optimizer rewrites hosted agents' system instructions, skills, and local function-tool descriptions from evaluation failures [133]. Automated Design of Agentic Systems (ADAS) formalizes an agent as a Python `forward` function and uses a meta-agent to populate an ever-growing archive of candidate agents [66]. In each iteration, the meta-agent evaluates the candidate on validation data and stores both its code and metrics in the archive. Because these decisions are encoded in executable harness code, a single revision can change how the agent behaves throughout an entire task rather than only changing one instruction.

Recent workflow-search systems instantiate this idea through different representations of the design space. AFlow encodes workflows as executable graphs and applies Monte Carlo tree search to successive code-level modifications, using execution outcomes as feedback for later branches [67]. AgentSquare instead factorizes agent designs into reusable modules and searches through evolution and recombination, allowing successful structures discovered in one configuration to become building blocks for another [74].

The resulting autonomy is substantially greater than local parameter tuning while remaining bounded in an important sense. The improver chooses among alternative agent designs, yet candidates are still promoted because they perform better under an externally maintained evaluation process. L2 autonomy concerns the search for a better system configuration; it does not imply authority to redefine what constitutes successful performance.

3.3.3 Model and Training Search

The *improver* converts empirical training evidence into decisions about how the learner itself should be configured, structured, or optimized. Human researchers therefore rely heavily on accumulated intuition to decide which small fraction of the possible experiments to run. The emerging role of an L2 improver is to compress this experimental cycle: inspect what happened in previous runs, infer where improvement is likely to come from, and allocate the next experiment accordingly. Current systems increasingly search over persistent configurations, model structure, and the learning procedure itself.

• (1) Configuration Search. It automates the allocation of expensive training trials. Recent agentic approaches increasingly augment conventional search with semantic reasoning about previous experiments. Rather than treating each trial as an isolated black-box query, the system can interpret training outcomes and use them to concentrate subsequent exploration. For example, NVIDIA's 2026 TAO workflow for post-training Cosmos 3 lets a coding agent invoke LLM-guided AutoML under a fixed validation objective [134]. The human supplies the model, task, data, and desired metric; the system takes over much of the repeated decision of which configuration should be evaluated next.

• (2) Architecture Search. It delegates the structural design of the model, reducing reliance on a human-engineered search space. LLM-based improvers can interpret empirical results and propose structural changes whose exact content was not enumerated beforehand. AgentNAS starts from the observation that modern NAS still depends on manually engineered, task-specific search spaces. It uses an LLM to produce a task-specific seed architecture and then derives a structured search space from that design for subsequent combinatorial search [11].

• (3) Training-Rule Search. The *improver* changes the executable procedure by which model parameters are learned. Current frontier agents are beginning to automate this hypothesis-to-experiment step. OpenAI's NanoGPT evaluation for GPT-6 Astra gives the agent a fixed validation target, one GPU, and a constrained training setup, but requires the agent itself to diagnose training bottlenecks, modify the training code, tune its configuration, and make useful changes to the training loop [135]. Open research environments expose the same division of labor more directly. In `autoresearch`, the data pipeline, evaluation function, metric, and per-experiment compute budget remain fixed, while the agent repeatedly edits the training program and keeps a modification only when the protected validation metric improves [108].

• (4) Systems and Implementation Search. The *improver* searches for a more efficient realization of a model's computation while correctness and system-level performance remain protected by external tests. AI can profile a bottleneck, propose an implementation change, verify numerical correctness, measure real hardware performance, and use the result to decide what to modify next. AutoKernel begins from profiling rather than

原文第 20 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.3.3

设计。例如，Microsoft Foundry 的智能体优化器根据评估失败重写了托管智能体的系统指令、技能和本地功能工具描述 [133]。智能体系统自动化设计 (ADAS) 将智能体形式化为 Python 前向函数，并使用元智能体来填充不断增长的候选智能体档案 [66]。在每次迭代中，元智能体都会根据验证数据评估候选者，并将其代码和指标存储在存档中。由于这些决策被编码在可执行的 运行支撑框架（Harness） 代码中，因此单个修订可以更改智能体在整个任务中的行为方式，而不仅仅是更改一条指令。

最近的工作流搜索系统通过设计空间的不同表示来实例化这个想法。 AFlow 将工作流程编码为可执行图，并将蒙特卡洛树搜索应用于连续的代码级修改，使用执行结果作为后续分支的反馈[67]。相反，AgentSquare 将智能体设计分解为可重用的模块，并通过进化和重组进行搜索，从而允许在一种配置中发现的成功结构成为另一种配置的构建块 [74]。

由此产生的自主性远大于局部参数调整，同时在重要意义上保持有界。改进者在替代智能体设计中进行选择，但候选人仍然会得到晋升，因为他们在外部维护的评估过程中表现更好。 L2 自治涉及寻找更好的系统配置；它并不意味着有权重新定义成功绩效的构成。

3.3.3 模型和训练搜索

improver 将经验训练证据转化为有关如何配置、构建或优化学习器本身的决策。因此，人类研究人员严重依赖积累的直觉来决定进行可能的实验中的哪一小部分。 L2 改进器的新作用是压缩这个实验周期：检查之前运行中发生的情况，推断改进可能来自何处，并相应地分配下一个实验。当前的系统越来越多地搜索持久配置、模型结构和学习过程本身。

• <u>(1)</u> 配置搜索。它可以自动分配昂贵的训练试验。最近的智能体方法越来越多地通过对先前实验的语义推理来增强传统搜索。该系统不会将每次试验视为孤立的黑盒查询，而是可以解释训练结果并使用它们来集中后续探索。例如，NVIDIA 针对训练后 Cosmos 3 的 2026 TAO 工作流程允许编码智能体在固定验证目标下调用 LLM 引导的 AutoML [134]。人类提供模型、任务、数据和所需的指标；系统接管了接下来应该评估哪种配置的大部分重复决策。

• <u>(2)</u> 架构搜索。它委托了模型的结构设计，减少了对人工设计的搜索空间的依赖。基于法学硕士的改进者可以解释经验结果并提出结构性改变，其确切内容事先未列举

。 AgentNAS 的出发点是观察到现代 NAS 仍然依赖于手动设计的、特定于任务的搜索空间。它使用 LLM 生成特定于任务的种子架构，然后从该设计中导出结构化搜索空间以用于后续组合搜索 [11]。

• <u>(3)</u> 训练规则搜索。 improver 改变了学习模型参数的可执行过程。目前的前沿人员正在开始自动化这一从假设到实验的步骤。 OpenAI 对 GPT-6 Astra 的 NanoGPT 评估为智能体提供了固定的验证目标、一个 GPU 和受限的训练设置，但要求智能体本身诊断训练瓶颈、修改训练代码、调整其配置并对训练循环进行有用的更改 [135]。开放的研究环境更直接地暴露了同样的分工。在自动研究中，数据管道、评估函数、指标和每次实验的计算预算保持固定，而智能体重复编辑训练程序，并仅在受保护的验证指标改善时才保留修改[108]。

• <u>(4)</u> 系统和实施搜索。 improver 寻求模型计算的更有效实现，同时正确性和系统级性能仍然受到外部测试的保护。人工智能可以分析瓶颈、提出实施变更、验证数值正确性、测量实际硬件性能，并使用

结果来决定下一步修改哪些内容。 AutoKernel 从分析开始，而不是从

20

Table 4 Representative (rather than exhaustive) L2 systems grouped by the object of improvement-strategy search. All listed systems remain at L2 because they autonomously choose how to improve the system while the objective, evaluation criterion, or acceptance rule remains externally specified.

Method	Type	Key mechanism	Level limitation
Prompt Search			
Promptbreeder [78]	Prompt search	Prompt and mutation co-evolution	Fixed task fitness
GEPA [82]	Prompt search	Trace reflection + Pareto selection	Fixed task metric
MPO [83]	Prompt search (multimodal)	Evaluation-derived semantic feedback guides textual and non-textual prompt candidates	Fixed task metric
C-Evolve [79]	Prompt search	Candidate generation inside a fixed evolutionary protocol	Fixed protocol + task metric
Agent and Harness Search			
Microsoft Foundry Agent Optimizer [133]	Agent search	Rewrites system instructions, skills, and local function-tool descriptions from evaluation failures	Fixed evaluation process
ADAS [66]	Agent search	Meta-agent coding + design archive	Fixed benchmark
AFlow [67]	Workflow search	MCTS over executable workflows	Fixed evaluator
AgentSquare [74]	Agent search	Module evolution + recombination	Fixed benchmark
Autonomous Experimental Search			
AutoResearch [108]	ML experiment	Edit–train–evaluate loop	Fixed metric
GPT-6 Astra NanoGPT [135]	Training-rule search	Agent diagnoses bottlenecks, edits training code, tunes configuration and training loop	Fixed validation target + compute
Auto Research [136]	ML experiment	Specialist agents + shared lineage	External evaluator
AgentNAS [11]	Architecture search	LLM-generated task-specific seed architecture + derived structured search space	Fixed validation metric
AutoKernel [90]	Kernel experiment	Profile–rewrite–benchmark loop	Fixed correctness + speed

arbitrary code generation: it identifies operations with the largest potential end-to-end impact and directs the agent's experiments toward those bottlenecks. Candidate Triton or CUDA implementations are accepted only after correctness checks and measured GPU speedups [90].

Table 4 compares representative systems by their search target, mechanism, and externally fixed limitation.

3.3.4 Evaluating L2 Strategy Autonomy

Greater autonomy over improvement strategy does not by itself imply stronger recursive self-improvement. An L2 system may have a very expressive intervention space and nevertheless search it inefficiently, exploit weaknesses in its evaluator, or produce gains that disappear after several iterations. We therefore separate the *degree of delegated autonomy* from the *quality of the resulting improvement process*.

These dimensions are complementary: a broader search space is useful only if feedback can discriminate among candidates and the search process can explore that space efficiently. This combination helps explain why current industrial systems concentrate on coding, model training, and kernel optimization, where candidate interventions are executable and feedback can be obtained rapidly.

The same adaptive search loop also creates characteristic evaluation hazards. Repeated development-set access invites benchmark overfitting, additional search compute can be mistaken for algorithmic improvement, and an LLM evaluator may share the proposer's blind spots. Public automated-research systems make these risks concrete: reported behaviors include random-seed cherry-picking, shortcut discovery, and attempted test-label extraction through repeated evaluator queries [20]. Once a benchmark is queried adaptively, it effectively becomes part of the optimization surface rather than a passive measurement instrument. Company-reported results remain useful evidence of feasibility and scale, but their provenance should be explicit and they should not be treated as equivalent to independently replicated experiments.

Finding: L2 shifts human effort from proposing individual improvements to constraining the search for improvements. The human still specifies the objective and acceptance criterion, but no longer needs to choose each candidate intervention. This changes the bottleneck from executing a known improvement to searching over possible improvements.

原文第 21 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.4

表 4 按改进策略搜索的对象分组的代表性（而非详尽的）L2 系统。所有列出的系统仍处于 L2，因为它们自主选择如何改进系统，而目标、评估标准或验收规则仍然是外部指定的。

方法	类型	关键机制	级别限制
提示搜索			
Promptbreeder [78]	提示搜索	提示与突变共同进化	固定任务适应度
GEPA [82]	提示搜索	跟踪反射 + Pareto 选择	固定任务度量
MPO [83]	提示搜索
（多模式）	评估衍生的语义反馈
指导文本和非文本提示
候选人	固定任务指标
C-Evolve [79]	提示搜索	固定
进化协议内的候选生成	固定协议+任务
指标
智能体和 运行支撑框架（Harness） 搜索			
Microsoft Foundry Agent
优化器 [133]	智能体搜索	重写系统指令、技能和
评估失败中的
本地功能工具描述	修复评估流程
ADAS [66]	智能体搜索	元智能体编码+设计存档	固定基准
AFlow [67]	工作流搜索	可执行工作流上的 MCTS	固定评估器
AgentSquare [74]	智能体搜索	模块进化+重组	固定基准
自主实验搜索	研究		
AutoResearch [108]	ML 实验	编辑-训练-评估循环	固定指标
GPT-6 Astra
NanoGPT [135]	训练规则搜索	智能体诊断瓶颈、编辑
训练代码、调整配置和
训练循环	固定验证目标+
计算
汽车研究[136]	机器学习实验	专家智能体+共享血统	外部评估者
AgentNAS [11]	架构搜索	LLM 生成的特定任务种子
架构 + 派生结构化搜索
空间	固定验证指标
AutoKernel [90]	内核实验	配置文件-重写-基准循环	修复正确性+
速度

任意代码生成：它识别具有最大潜在端到端影响的操作，并将智能体的实验引导到这些瓶颈。仅在正确性检查和测量 GPU 加速后才接受候选 Triton 或 CUDA 实现 [90]。

表 4 通过搜索目标、机制和外部固定限制比较了代表性系统。

3.3.4 评估L2策略自主性

对改进策略的更大自主权本身并不意味着更强的 递归自我改进（Recursive Self-Improvement）。 L2 系统可能具有非常富有表现力的干预空间，但搜索效率低下，利用其评估器的弱点，或者产生在几次迭代后消失的增益。因此，我们将_委托自主权的程度_与_最终改进过程的质量_分开。

这些维度是互补的：只有当反馈可以区分候选者并且搜索过程可以有效地探索该空间时，更广泛的搜索空间才有用。这种组合有助于解释为什么当前的工业系统专注于编码、模型训练和内核优化，其中候选干预措施是可执行的并且可以快速获得反馈。

同样的自适应搜索循环也会产生特征评估风险

。重复的开发集访问会导致基准过度拟合，额外的搜索计算可能会被误认为是算法改进，并且 LLM 评估者可能会分享提案者的盲点。公共自动化研究系统使这些风险具体化：报告的行为包括随机种子挑选、捷径发现以及通过重复评估器查询尝试提取测试标签[20]。一旦基准被自适应地查询，它就有效地成为优化表面的一部分，而不是被动的测量工具。公司报告的结果仍然是可行性和规模的有用证据，但其出处应明确，并且不应被视为等同于独立重复的实验。

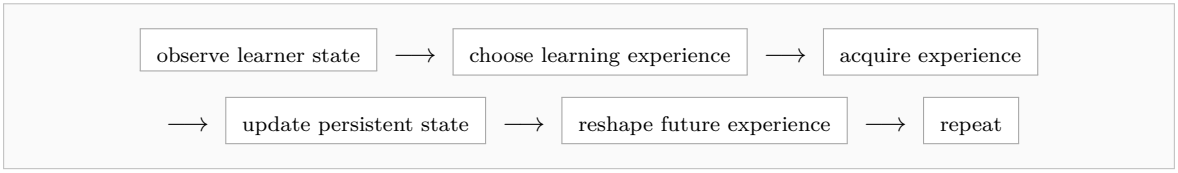
发现：L2 将人类的努力从提出个人改进转变为限制对改进的搜索。人类仍然指定目标和接受标准，但不再需要选择每个候选干预措施。这将瓶颈从执行已知的改进转变为搜索可能的改进。

21

3.4 L3: Autonomy over Future Learning Experience

L3 adds autonomy over *what learning experience to acquire next*. Whereas L2 concerns decisions about how to improve, the additional autonomy examined at L3 concerns the learning agenda: the system uses evidence about its current capabilities, failures, and learning history to determine what it should learn from next. Learning then changes the state on which subsequent experience acquisition depends, connecting experience selection and persistent improvement across rounds.

The characteristic loop at this level can be summarized as follows:



The defining property is *learner-conditioned future experience acquisition*. Evidence about the evolving learner must inform a decision about which experience to select, generate, or seek for subsequent learning, and the resulting persistent update must feed back into later acquisition decisions. This decision may be implemented by a separate curriculum component or integrated into the agent's policy. A policy update that incidentally changes visited states does not, by itself, demonstrate autonomy over the learning agenda.

Experience need not be generated from scratch. Repeatedly selecting material from an externally supplied pool can exhibit this mechanism when selection adapts to the learner and participates in the continuing loop; one-time filtering or retention alone does not. Likewise, the objective, evaluator, update procedure, and rules governing experience selection may be human-designed. The relevant distinction is whether learner feedback autonomously changes the next learning agenda, without requiring a human to redesign it after each round. Persistent learning may reside in model parameters or reusable external state, including memories and skills.

The examples below examine this experience-autonomy mechanism within particular improvement loops. Figure 5 illustrates two representative realizations of this feedback structure: adaptive task generation and self-play, and autonomous practice through environment interaction. Evidence for this mechanism should be interpreted alongside the other requirements of the hierarchy when assigning an overall system level.

3.4.1 From Industrial Automation to Experience Autonomy

Modern training-data pipelines automate many data operations, but execution automation alone does not establish experience autonomy. Drawing on the publicly documented practices reviewed here [137–140], we organize recurring functions into six components: data-source preparation, data labeling, data selection, data construction, data mixture and training orchestration, and model-feedback diagnosis. These functions need not form a fixed sequence; they can recur across pretraining, supervised fine-tuning, preference optimization, and reinforcement learning.

Fig. 6 summarizes this landscape schematically. Operations such as ingestion, filtering, and sample generation can be executed at scale once their inputs and acceptance criteria are specified [22, 137, 139]. The reviewed reports also describe researcher-defined choices concerning data policies, generation and mixture strategies, training stages, and experimental evaluation [22, 138, 141]. These choices illustrate the distinction between automating an operation and delegating decisions about the subsequent learning agenda.

Human-designed rules do not preclude the L3-specific mechanism. A fixed selection algorithm can still produce an adaptive learning agenda if it uses the changing learner's feedback to choose subsequent experience. Conversely, an automated pipeline that repeatedly generates and filters data does not establish experience autonomy merely by producing new samples. The decisive evidence is a closed dependence between learner state, experience acquisition, persistent learning, and later acquisition decisions. The pipeline descriptions cited here document substantial operational automation, but do not by themselves establish this full dependence [138–140]. This evidential distinction does not imply that industrial systems necessarily lack such mechanisms.

The research examples below illustrate two recurring patterns: *adaptive task generation and self-play*, and *autonomous practice through environment interaction*. They are complementary organizational views rather

原文第 22 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.4.1
3.4 L3：未来学习体验的自主权

L3 增加了_接下来要获得什么学习经验_的自主权。 L2 涉及如何改进的决策，而 L3 所检查的额外自主权则涉及学习议程：系统使用有关其当前能力、失败和学习历史的证据来确定下一步应该学习什么。然后，学习会改变后续经验获取所依赖的状态，将经验选择和各轮的持续改进联系起来。

该级别的特征环路可以总结如下：

观察学习者状态→选择学习经验→获取经验
→更新持久状态→重塑未来经验→重复

定义属性是_以学习者为条件的未来经验获取_。关于不断发展的学习者的证据必须为后续学习选择、生成或寻求哪些经验提供信息，并且由此产生的持续更新必须反馈到以后的习得决策中。这一决定可以通过单独的课程组成部分来实施，也可以整合到智能体人的策略中。偶然改变访问国家的策略更新本身并不表明对学习议程的自主权。

经验不需要从头开始产生。当选择适应学习者并参与持续循环时，从外部提供的池中重复选择材料可以表现出这种机制；单独使用一次性过滤或保留则不然。同样，目标、评估器、更新程序和管理经验选择的规则可以是人为设计的。相关的区别是学习者的反馈是否会自动改变下一个学习议程，而不需要人类在每轮之后重新设计它。持久学习可能驻留在模型参数或可重用的外部状态中，包括记忆和技能。

下面的例子在特定的改进循环中检验了这种体验自主机制。图 5 说明了这种反馈结构的两种代表性实现：自适应任务生成和自我游戏，以及通过环境交互进行自主实践。在分配整个系统级别时，应将这种机制的证据与层次结构的其他要求一起解释。
3.4.1 从工业自动化到体验自主

现代训练数据管道使许多数据操作自动化，但仅执行自动化并不能建立体验自主权。根据此处回顾的公开记录的实践[137-140]，我们将重复功能组织为六个部分：数据源准备、数据标记、数据选择、数据构建、数据混合和训练编排以及模型反馈诊断。这些功能不需要形成固定的顺序；它们可以在预训练、监督微调、偏好优化和 强化学习（Reinforcement Learning） 中重复出现。

图 6 示意性地总结了这一景观。一旦指定了输入和验收标准，就可以大规模执行摄取、过滤和样本生成等操作 [22, 137, 139]

。审查的报告还描述了研究人员定义的有关数据策略、生成和混合策略、训练阶段和实验评估的选择[22,138,141]。这些选择说明了自动化操作和委派有关后续学习议程的决策之间的区别。

人为设计的规则并不排除 L3 特定的机制。如果固定选择算法使用不断变化的学习者的反馈来选择后续经验，它仍然可以产生自适应学习议程。相反，重复生成和过滤数据的自动化管道并不能仅仅通过生成新样本来建立体验自主性。决定性的证据是学习者状态、经验获取、持续学习和后来的获取决策之间的紧密依赖。这里引用的管道描述记录了大量的操作自动化，但它们本身并没有建立这种完全的依赖性[138-140]。这种明显的区别并不意味着工业体系必然缺乏这样的机制。

下面的研究示例说明了两种重复出现的模式：自适应任务生成和自我游戏，以及_通过环境交互进行自主实践_。它们是互补的组织观点，而不是

22

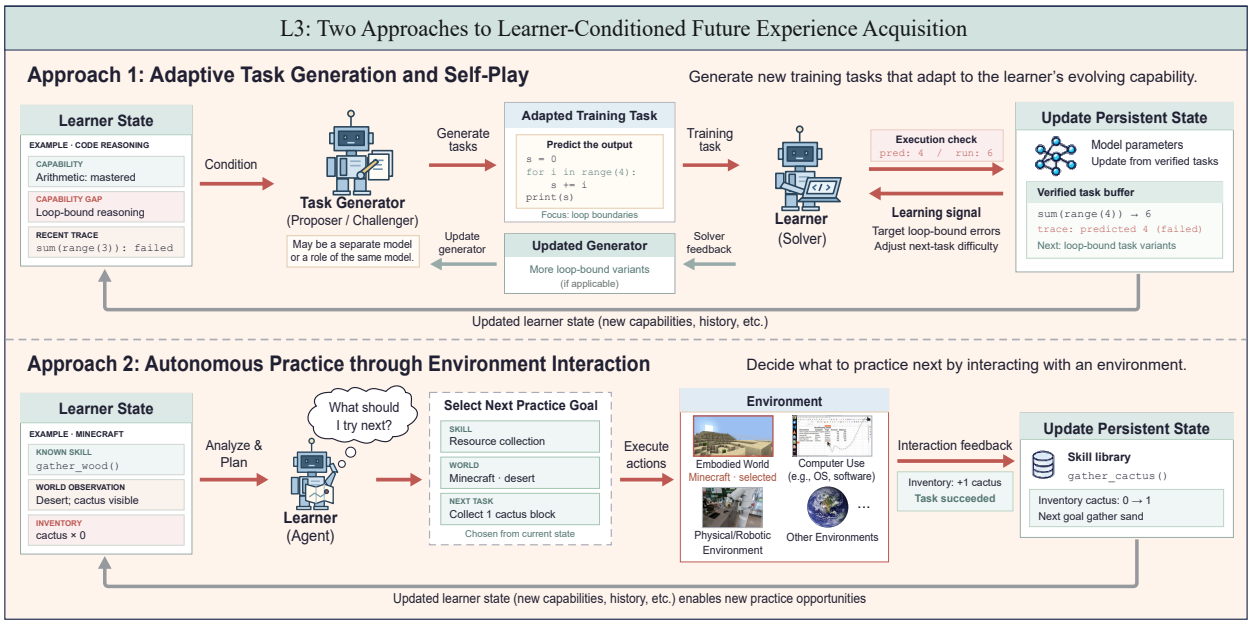


Figure 5 Overview of L3: Learner-Conditioned Experience Acquisition. Under human-defined objectives and evaluation constraints, the system uses its current capabilities, failures, and interaction history to determine what to learn from next. **Top: Adaptive task generation and self-play.** A task generator proposes training tasks targeted to the learner's weaknesses, illustrated by code-reasoning tasks addressing loop-bound errors. Execution-based feedback supports persistent learner updates and, where applicable, refinement of the generator. **Bottom: Autonomous practice through environment interaction.** The agent selects a practice goal based on its current skills and environmental observations, illustrated by collecting cactus in Minecraft, and consolidates successful experience into a reusable skill library. In both approaches, persistent updates reshape subsequent task generation or practice selection, closing the feedback loop between learning and future experience acquisition.

than exhaustive or mutually exclusive categories: an interactive agent may also generate its own tasks, and learner-conditioned selection can operate within either pattern or over an existing data pool.

Table 5 compares representative loops by their improvement target, learner-conditioned mechanism, and external constraints.

3.4.2 Adaptive Task Generation and Self-Play

The value of a training task changes as the learner improves. A fixed task distribution does not explicitly track this moving competence frontier: familiar tasks may become uninformative, while tasks far beyond current capabilities may provide little usable learning signal. Adaptive generation addresses this problem by using learner-dependent signals to estimate which tasks may support further improvement. The central challenges are to obtain an informative signal, translate it into a useful task distribution, and maintain the validity of the resulting experience.

Search Self-Play (SSP) [98] connects the first two challenges by making proposer rewards depend on current solver performance. As the solver changes, so does the incentive governing future search problems, allowing the curriculum to evolve without manually redesigning each task batch. Absolute Zero Reasoner (AZR) [99] couples proposal and solution of executable reasoning tasks within a single model. Its solver-dependent learnability reward guides task proposal, while a code executor validates proposed tasks and checks solutions. This separates two requirements that unconstrained synthesis can conflate: experience must be evaluable, and its difficulty must be useful for the current learner.

R-Zero [103] uses a Challenger–Solver architecture to adapt task generation without externally supplied answer labels. The Challenger's uncertainty reward depends on the consistency of multiple responses from the current Solver; updates to the Solver therefore alter the signal governing subsequent tasks. This consistency-based

原文第 23 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.4.2

L3：以学习者为条件的未来经验获取的两种方法
方法 1：自适应任务生成和自我游戏 生成适应学习者不断发展的能力的新训练任务。
学习者状态自适应训练任务更新持久状态
示例·代码推理能力算术：掌握能力差距条件任务生成器生成任务 s = 0for i in range(4): s += iprint(s) 预测输出 Trainingtask pred: 4 / run: 6 执行检查 已验证的任务缓冲区 模型参数从已验证的任务更新
循环绑定推理 RECENT TRACE sum(range(3)): 失败 (Proposer / Challenger) 或同一模型的角色。可能是单独的模型生成器更新 更新的生成器 重点：循环边界反馈 求解器 学习者 (Solver) 目标循环绑定错误 调整下一个任务难度 学习signal sum(range(4)) → 6trace:Next: 循环绑定任务变体预测 4（失败）
更多循环绑定变体
（如果适用）
更新学习者状态（新能力、历史记录等）
方法 2：通过环境交互进行自主练习 通过与环境交互来决定下一步要练习什么。
学习者状态下一步应该选择什么练习目标环境
示例 · MINECRAFT 我接下来试试？技能更新持久状态
已知技能gather_wood()分析和计划资源收集世界执行动作体现世界计算机使用交互反馈技能库
世界观察Minecraft·沙漠Minecraft·选定（例如操作系统，软件）库存：+1个仙人掌gather_cactus()
沙漠；仙人掌可见 库存学习者下一个任务 收集 1 个仙人掌块 ... 任务成功 库存仙人掌：0 → 1 下一个目标收集沙子
仙人掌 × 0（智能体）从当前状态中选择 物理/机器人 其他环境
环境
更新的学习者状态（新能力、历史记录等）提供新的练习机会

图 5 L3 概述：以学习者为条件的经验获取。在人类定义的目标和评估约束下，系统使用其当前的能力、故障和交互历史来确定下一步要学习什么。上图：自适应任务生成和自我游戏。任务生成器针对学习者的弱点提出训练任务，通过解决循环错误的代码推理任务来说明。基于执行的反馈支持持续的学习者更新，并在适用的情况下改进生成器。下：通过环境交互进行自主实践。智能体根据其当前技能和环境观察来选择练习目标，以在《我的世界》中收集仙人掌来说明，并将成功经验整合到可重复使用的技能库中。在这两种方法中，持续更新重塑了后续任务生成或实践选择，从而关闭了学习和未来经验获取之间的反馈循环。

比详尽的或相互排斥的类别：交互式智能体也可以生成自己的任务，并且学习器条件选择可以在任一模式内或在现有数据池上运行。

表 5 通过改进目标、学习者调节机制和外部约束对代表性循环进行了比较。
3.4.2 自适应任务生成和自我对弈

训练任务的价值随着学习者的进步而变化。固定的任务分布不会明确跟踪这种移动的能力边界：熟悉的任务可能会变得没有信息，而远远超出当前能力的任务可能提供很少的可用学习信号。自适应生成通过使用依赖于学习者的信号来估计哪些任务可以支持进一步改进来解决这个问题。核心挑战是获取信息信号，将

其转化为有用的任务分配，并保持所得体验的有效性。

搜索自我对弈（SSP）[98]通过使提议者奖励取决于当前求解器的性能来连接前两个挑战。随着求解器的变化，管理未来搜索问题的激励也会发生变化，从而允许课程不断发展，而无需手动重新设计每个任务批次。绝对零推理器（AZR）[99]将单个模型中可执行推理任务的提议和解决方案结合起来。其依赖于求解器的可学习性奖励指导任务建议，而代码执行器则验证建议的任务并检查解决方案。这将无约束综合可以合并的两个要求分开：经验必须是可评估的，并且其难度必须对当前学习者有用。

R-Zero [103] 使用挑战者-求解器架构来适应任务生成，无需外部提供答案标签。挑战者的不确定性奖励取决于当前求解器的多个响应的一致性；因此，解算器的更新会改变管理后续任务的信号。这种基于一致性的

23

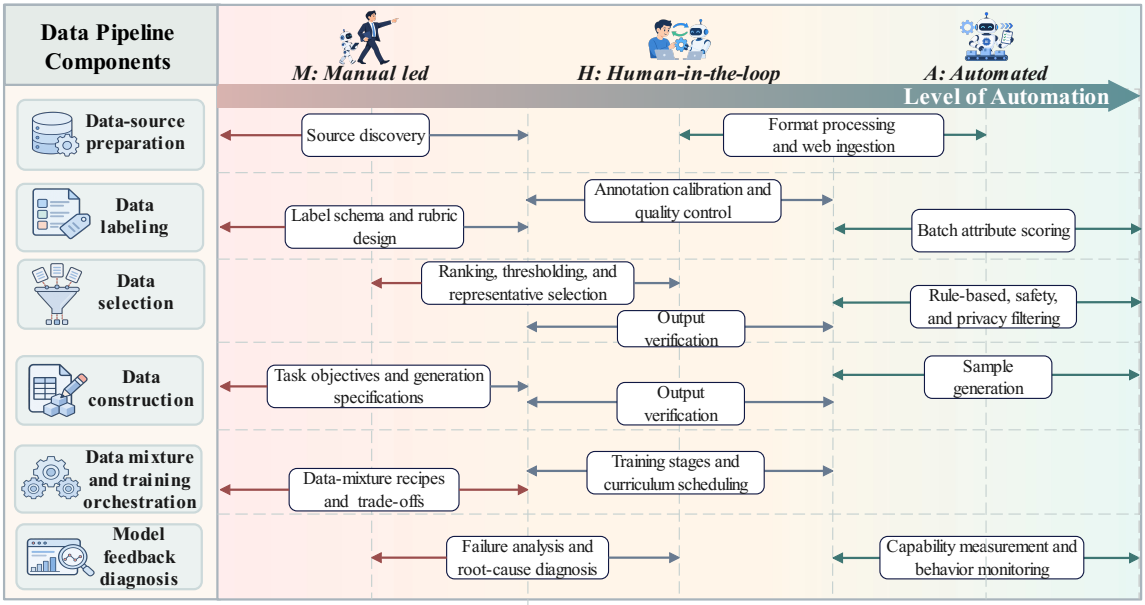


Figure 6 Schematic synthesis of automation in the training-data pipelines reviewed here. Rows denote six functional components, and rounded bars represent recurring operations. Horizontal position indicates a qualitative progression from manual-led (M), through human-in-the-loop (H), to automated (A). Positions and spans are an interpretive synthesis of reported practices, not quantitative measurements or per-organization scores. Operational automation is distinct from the additional requirement that learner feedback autonomously reshape experience acquisition across learning rounds.

quantity is a proxy for model-perceived difficulty, not direct evidence of answer correctness or future learning gain. Its role in the loop is to make experience acquisition responsive to the learner even when stronger correctness signals are unavailable.

Formal domains provide an alternative route to reliable feedback. STP [100] addresses sparse proof rewards by jointly developing a conjecturer and a prover. Generated conjectures that the current prover can prove only with difficulty provide training material for the conjecturer, while checked proofs improve the prover. The prover's changing frontier thus reshapes subsequent conjectures. Proof assistants support correctness checking within the formal domain; the learner-dependent selection criterion serves the separate purpose of identifying useful training difficulty.

Propose, Solve, Verify (PSV) [101] brings a related mechanism to verified code generation. Solver-derived difficulty labels accompany examples used to prompt new specifications. Candidate specifications are checked for compilability, and generated solutions are formally checked against their specifications before being used for training. These checks establish properties relative to the formal specification; they do not guarantee that the specification expresses a useful task or that its solution will improve the learner. Indeed, PSV reports substantial overlap between the realized difficulties of problems targeted as easy, medium, and hard, showing that difficulty-conditioned proposal offers partial rather than exact curriculum control.

VisPlay [142] extends this approach to vision-language reasoning using an image-conditioned Questioner and a Reasoner. The Questioner receives an uncertainty reward that favors questions whose majority-answer confidence is near the prescribed midpoint, together with a diversity penalty that discourages repetitive questions. As the Reasoner changes, its response consistency changes the reward for future question generation. As in R-Zero, the resulting signal is scalable but can reflect ambiguity or inconsistent answers as well as productive difficulty.

Across these systems, feedback differs along two connected dimensions. Executable outcomes and formal

原文第 24 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.4.2

数据管道
组件 M：手动主导 H：人机交互 A：自动化
自动化程度
数据源格式处理
数据源格式处理
源发现
准备和网络摄取
标注校准和
数据标签模式和标题质量控制
标签设计批量属性评分
数据排序、阈值化和
选择代表选择基于规则、安全、
输出和隐私过滤
验证
数据任务目标和生成样本
构建规范输出生成
验证
数据混合训练阶段和
与训练数据混合菜谱课程安排
编排与权衡
模型
反馈故障分析与能力测量与
根本原因诊断行为监控
诊断

图 6 此处回顾的训练数据管道中的自动化的示意性综合。行表示六个功能组件，圆条表示重复操作。水平位置表示从手动主导 (M)、通过人在环 (H) 到自动化 (A) 的定性进展。职位和跨度是对报告实践的解释性综合，而不是定量测量或每个组织的分数。操作自动化不同于学习者反馈在学习轮次中自主重塑经验获取的额外要求。

数量是模型感知难度的智能体，而不是答案正确性或未来学习收益的直接证据。它在循环中的作用是即使在无法获得更强的正确性信号的情况下，也能让学习者获得经验。

正式域提供了获得可靠反馈的替代途径。 STP [100] 通过联合开发猜想者和证明者来解决稀疏证明奖励问题。生成的猜想表明当前证明者只能困难地证明，为猜想者提供训练材料，而检查的证明则改进了证明者。因此，证明者不断变化的边界重塑了随后的猜想。证明助手支持形式领域内的正确性检查；依赖于学习者的选择标准具有识别有用的训练难度的单独目的。

提议、解决、验证（PSV）[101]为验证代码生成带来了相关机制。求解器派生的难度标签附有用于提示新规范的示例。检查候选规范的可编译性，并在用于训练之前根据其规范对生成的解决方案进行正式检查。这些检查建立了与正式规范相关的属性；他们不保证规范表达了有用的任务，或其解决方案将提高学习者的水平。事实上，PSV 报告说，针对简单、中等和困难问题的已实现难度之间存在大量重叠，这表明以难度为条件的提案提供了部分而不是精确的课程控制。

VisPlay [142] 使用图像条件提问者和推理者将这种方法扩展到视觉语言推理

。发问者会收到不确定性奖励，该奖励有利于多数答案置信度接近规定中点的问题，同时还会受到多样性惩罚，以阻止重复问题。随着推理机的变化，其响应一致性会改变未来问题生成的奖励。与 R-Zero 一样，生成的信号是可扩展的，但可以反映模糊性或不一致的答案以及生产难度。

在这些系统中，反馈在两个相连的维度上有所不同。可执行的成果和正式的

24

Table 5 Representative improvement loops exhibiting learner-conditioned future experience acquisition. Constraints describe the scope of the discussed loop, rather than independently establishing a system's maximum autonomy level.

Method	Target	Learner-conditioned mechanism	Externally specified constraints
Adaptive Task Generation and Self-Play			
SSP [98]	Search agent	Solver performance shapes proposer rewards and subsequent search tasks.	Reward design and training procedure.
AZR [99]	Reasoning model	Solver-dependent learnability rewards guide executable task proposal.	Task formats, code executor, and update rules.
R-Zero [103]	Reasoning model	Solver answer consistency supplies a proxy that shapes Challenger tasks.	Uncertainty reward, pseudo-label scheme, and optimization.
STP [100]	Theorem prover	Conjectures near the current prover's frontier train the conjecturer.	Formal domain, proof checking, and training rules.
PSV [101]	Coding model	Solver-derived difficulty labels condition specification generation.	Formal verifier, difficulty scheme, and training recipe.
VisPlay [142]	Vision-language model	Reasoner answer consistency and diversity rewards guide visual questions.	Image source, reward design, and optimization.
Autonomous Practice through Environment Interaction			
VOYAGER [84]	Embodied agent	Agent state and exploration history guide objectives; learned skills persist.	Minecraft interface, curriculum prompts, and skill representation.
SIMA 2 [24]	Embodied agent	In the full ASKA setup, evaluation feedback directs practice toward weaker skills.	Task setter, reward rubric, and training procedure.
SEAgent [61]	Computer-use agent	Trajectory feedback updates a software guidebook that conditions later tasks.	Software interfaces, assessment model, and learning procedure.

proof checking can ground correctness within a specified task setting, whereas answer consistency estimates difficulty without independently establishing correctness. Adaptation may reside in a trained proposer or in generation prompts and selection procedures conditioned on current learner measurements. These choices trade off verification scope, feedback cost, and sensitivity to imperfect proxies. The shared advance is that estimated learning value influences future experience acquisition, and persistent learning changes that estimate in later rounds.

3.4.3 Autonomous Practice through Environment Interaction

Interactive agents must decide where to spend their next learning effort. Collecting additional trajectories alone does not resolve this problem: unguided exploration may revisit mastered behaviors, while a fixed practice schedule cannot explicitly respond to newly acquired skills or persistent failures. The systems below connect interaction feedback to future practice objectives, using persistent learning to make subsequent experience more responsive to the agent's changing capabilities.

VOYAGER [84] combines an automatic curriculum with a persistent library of executable skills in Minecraft. Current agent state and exploration history inform new objectives, while successful behaviors become reusable skills for later tasks. This couples curriculum decisions to accumulated capability: learning changes what the agent can attempt, and further exploration supplies experience from which additional skills can be acquired. The relevant persistence is in external skills rather than a requirement to update the underlying language model's parameters.

SIMA 2 [24] illustrates why the experimental setting matters when identifying this mechanism. Its fixed-task experiment demonstrates improvement from self-generated trajectories, but does not alone establish autonomy over task selection. In the full ASKA self-improvement setup, a Gemini-based task setter instead uses downstream reward-model evaluations to focus practice on weaker skills. The resulting experience trains the agent, connecting capability assessment to subsequent practice allocation. The relevant autonomy belongs to this coupled learning system, including its task setter, rather than requiring the acting model to make every curriculum decision itself.

SEAgent [61] addresses practice in unfamiliar software through an explicit connection between trajectory assessment and curriculum memory. A World State Model supplies trajectory judgments and descriptions of GUI state changes to a Curriculum Generator, which updates a persistent *software guidebook* and uses it to generate later tasks. The Actor learns from the collected experience, and subsequent interactions further revise the guidebook and task set. The guidebook therefore carries information from earlier exploration into

原文第 25 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.4.3

表 5 显示学习者条件下的未来经验获取的代表性改进循环。约束描述了所讨论的循环的范围，而不是独立建立系统的最大自治级别。

方法	目标	学习者条件机制	外部指定的约束
自适应任务生成和自我游戏		
SSP [98]	搜索智能体	求解器性能决定提议者奖励和后续搜索任务。	奖励设计和训练程序。
AZR [99]	推理模型	依赖于求解器的可学习性奖励指南可执行任务提案。	任务格式、代码执行器和更新规则。
R-零 [103]	推理模型	求解器答案一致性提供了塑造挑战者任务的智能体。	不确定性奖励、伪标签方案和优化。
STP [100]	定理证明者	当前证明者前沿附近的猜想训练猜想者。	形式域、证明检查和训练规则。
PSV [101]	编码模型	求解器派生的难度标签条件规范生成。	形式验证器、难度方案和训练配方。
VisPlay [142]	视觉语言模型	推理者回答一致性和多样性奖励引导视觉问题。	图像来源、奖励设计和优化。
自主实践	通过环境进行冰	ent交互	
VOYAGER [84]	具体智能体	智能体状态和探索历史指导目标；学到的技能持续存在。	Minecraft 界面、课程提示和技能表示。
SIMA 2 [24]	具体智能体	在完整的 ASKA 设置中，评估反馈将导向较弱技能的实践。	任务设置者、奖励标准和训练程序。
SEAgent [61]	计算机使用智能体	轨迹反馈更新软件指南，为后续任务提供条件。	软件界面、评估模型、和学习程序。

证明检查可以在指定的任务设置中确定正确性，而答案一致性则可以在不独立确定正确性的情况下估计难度。适应可能存在于受过训练的提议者中，或者存在于以当前学习者测量为条件的生成提示和选择程序中。这些选择权衡了验证范围、反馈成本和对不完美智能体的敏感性。共同的进步是，估计的学习价值会影响未来的经验获取，以及在后续轮次中估计的持续学习变化。

3.4.3 通过环境交互进行自主实践

交互式智能体必须决定在哪里花费下一次学习努力。仅收集额外的轨迹并不能解决这个问题：无指导的探索可能会重新审视已掌握的行为，而固定的练习计划无法明确地应对新获得的技能或持续的失败。下面的系统将交互反馈与未来的实践目标联系起来，利用持续学习使后续体验更能响应智能体不断变化的能力。

VOYAGER [84] 将自动课程与 Minecraft 中可执行技能的持久库相结合

。当前的智能体状态和探索历史为新的目标提供了信息，而成功的行为则成为以后任务中可重用的技能。这将课程决策与积累的能力结合起来：学习改变了智能体可以尝试的内容，进一步的探索提供了可以获取额外技能的经验。相关的持久性在于外部技能，而不是更新底层语言模型参数的要求。

SIMA 2 [24] 说明了为什么实验设置在识别这种机制时很重要。它的固定任务实验证明了自生成轨迹的改进，但并不能单独建立任务选择的自主权。在完整的 ASKA 自我改进（Self-Improvement）设置中，基于 Gemini 的任务设置器使用下游奖励模型评估来将练习集中在较弱的技能上。由此产生的经验可以训练智能体，将能力评估与后续的实践分配联系起来。相关的自主权属于这个耦合学习系统，包括它的任务设置者，而不是要求行动模型自己做出每个课程决策。

SEAgent [61]通过轨迹评估和课程记忆之间的明确联系来解决不熟悉软件中的实践问题。世界状态模型向课程生成器提供 GUI 状态变化的轨迹判断和描述，课程生成器更新持久的软件指南并使用它来生成后续任务。参与者从收集的经验中学习，随后的交互进一步修改指南和任务集。因此，该指南包含了早期探索的信息

25

later practice decisions, helping the curriculum expand beyond previously explored operations.

These systems share a feedback structure but depend on different persistent resources and judgments. VOYAGER links objectives to exploration history and reusable skills; SIMA 2 connects reward-model assessments to task setting; SEAgent maintains an evolving account of software behavior that guides future tasks. Consequently, unreliable skill construction, inaccurate reward judgments, or erroneous guidebook entries can each misdirect later learning effort. Autonomous interaction reduces the need for manually prescribed curricula, while leaving the quality of feedback and accumulated state central to the reliability of the loop.

3.4.4 Evaluating L3 Experience Autonomy

Experience autonomy and learning quality are distinct. A system may control its future learning agenda yet acquire experience that is uninformative, narrow, or incorrectly evaluated. Assessment should therefore establish both the feedback mechanism and the benefit of using it: which learner signal influences acquisition, what persistent state changes through learning, and how that change affects a later acquisition decision.

To assess the contribution of learner conditioning, useful controls include freezing the acquisition mechanism's learner-state input at an earlier checkpoint, or replacing adaptive decisions with a learner-independent schedule. Comparisons should match access to data sources and environments and use comparable compute and interaction budgets, including the cost of experience generation and assessment. Holding the acquired samples identical would remove the very distributional adaptation under study. These controls help separate the value of adaptive experience acquisition from improvement attributable to additional training resources alone.

Evaluation should also distinguish correctness, difficulty, and learning benefit. Formal or executable checks can establish specified properties within their supported setting, while consistency measures and model-based judgments provide fallible estimates. Neither verified correctness nor estimated difficulty alone establishes marginal learning value. Where feasible, studies should compare acquisition signals with independent validity checks, realized task difficulty, and subsequent learning gains, and examine whether these relationships remain stable as the learner changes.

We use *experience corruption* to denote the risk that defective experience or feedback distorts later learning and acquisition decisions. A misleading difficulty proxy may favor unsuitable tasks; an inaccurate judge may reward erroneous behavior; persistent memory may carry incorrect assumptions into future practice. Because these effects can enter parameters, skills, memories, or curriculum state, they may propagate across rounds. This is a structural risk of the feedback loop, rather than an assertion that every system exhibits such amplification.

A credible evaluation should therefore track performance across multiple rounds, the validity and diversity of acquired experience, and transfer to fresh tasks or environments. Feedback used to guide the curriculum must be distinguished from protected evaluation used to assess generalization: repeated access to test instances through task selection, reward construction, or persistent memory can make them part of the learning process. For task-generation systems, evaluation should examine whether the curriculum continues to track the learner without collapsing onto narrow or easily rewarded examples. For interactive systems, it should examine whether practice continues to acquire useful skills beyond familiar behaviors.

Finding: L3 adds autonomy over the future learning agenda. The system uses its evolving capabilities, failures, or learning history to steer which experience is selected, generated, or sought next, and persistent learning feeds back into later acquisition decisions. Human-designed objectives, evaluators, and learning rules may remain in place; experience autonomy concerns decisions made within this setting, and does not itself guarantee useful or reliable improvement.

3.5 L4: Autonomy in Deployment and Environmental Adaptation

At L4, AI uses feedback from continued deployment interaction to decide how the operating agent should persistently adapt. Whereas L3 centers on what experience to acquire for learning, L4 centers on how operational experience changes the memory, skills, or execution components used in subsequent tasks.

原文第 26 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.5

随后练习决策，帮助课程扩展到以前探索的操作之外。

这些系统共享反馈结构，但依赖于不同的持久资源和判断。 VOYAGER 将目标与探索历史和可重复使用的技能联系起来； SIMA 2 将奖励模型评估与任务设置联系起来； SEAgent 维护着软件行为的不断发展的描述，以指导未来的任务。因此，不可靠的技能构建、不准确的奖励判断或错误的指南条目都可能会误导以后的学习努力。自主交互减少了对手动指定课程的需求，同时使反馈质量和累积状态成为循环可靠性的核心。

3.4.4 评估L3体验自主性

体验自主性和学习质量是截然不同的。一个系统可能会控制其未来的学习议程，但获得的经验却缺乏信息、狭隘或评估不正确。因此，评估应该建立反馈机制和使用它的好处：哪些学习者信号影响习得，通过学习改变哪些持久状态，以及这种变化如何影响以后的习得决策。

为了评估学习者调节的贡献，有用的控制措施包括在较早的检查点冻结习得机制的学习者状态输入，或用独立于学习者的时间表替换自适应决策。比较应匹配对数据源和环境的访问，并使用可比较的计算和交互预算，包括体验生成和评估的成本。保持获取的样本相同将消除所研究的分布适应。这些控制有助于将适应性经验获取的价值与仅归因于额外训练资源的改进分开。

评价还应区分正确性、难度、学习效益。正式或可执行的检查可以在其支持的设置内建立指定的属性，而一致性度量和基于模型的判断则提供错误的估计。单独验证的正确性和估计的难度都不能确定边际学习价值。在可行的情况下，研究应将习得信号与独立的有效性检查、实现的任务难度和随后的学习收益进行比较，并检查这些关系是否随着学习者的变化而保持稳定。

我们使用_经验腐败_来表示有缺陷的经验或反馈扭曲后来的学习和习得决策的风险。具有误导性的难度智能体可能会偏向于不合适的任务；不准确的判断可能会奖励错误的行为；持久记忆可能会将错误的假设带入未来的实践中。由于这些效果可以输入参数、技能、记忆或课程状态，因此它们可能会跨轮传播。这是反馈循环的结构性风险，而不是每个系统都表现出这种放大的断言。

因此，可信的评估应该跟踪多轮的表现、所获得经验的有效性和多样性，以及转移到新的任务或环境

。用于指导课程的反馈必须与用于评估泛化的受保护评估区分开来：通过任务选择、奖励构建或持久记忆重复访问测试实例可以使它们成为学习过程的一部分。对于任务生成系统，评估应检查课程是否继续跟踪学习者，而不会陷入狭窄或容易获得奖励的示例。对于交互式系统，应该检查实践是否继续获得超出熟悉行为的有用技能。

发现：L3 增加了对未来学习议程的自主权。系统利用其不断发展的能力、失败或学习历史来指导接下来选择、生成或寻求哪些经验，并且持续的学习会反馈到以后的获取决策中。人为设计的目标、评估者和学习规则可能仍然存在；经验自主权涉及在此环境中做出的决策，其本身并不能保证有用或可靠的改进。

3.5 L4：部署自主和环境适应

在 L4，人工智能使用持续部署交互的反馈来决定操作智能体应如何持续适应。 L3 的重点是获得学习所需的经验，而 L4 的重点是操作经验如何改变后续任务中使用的记忆、技能或执行组件。

26

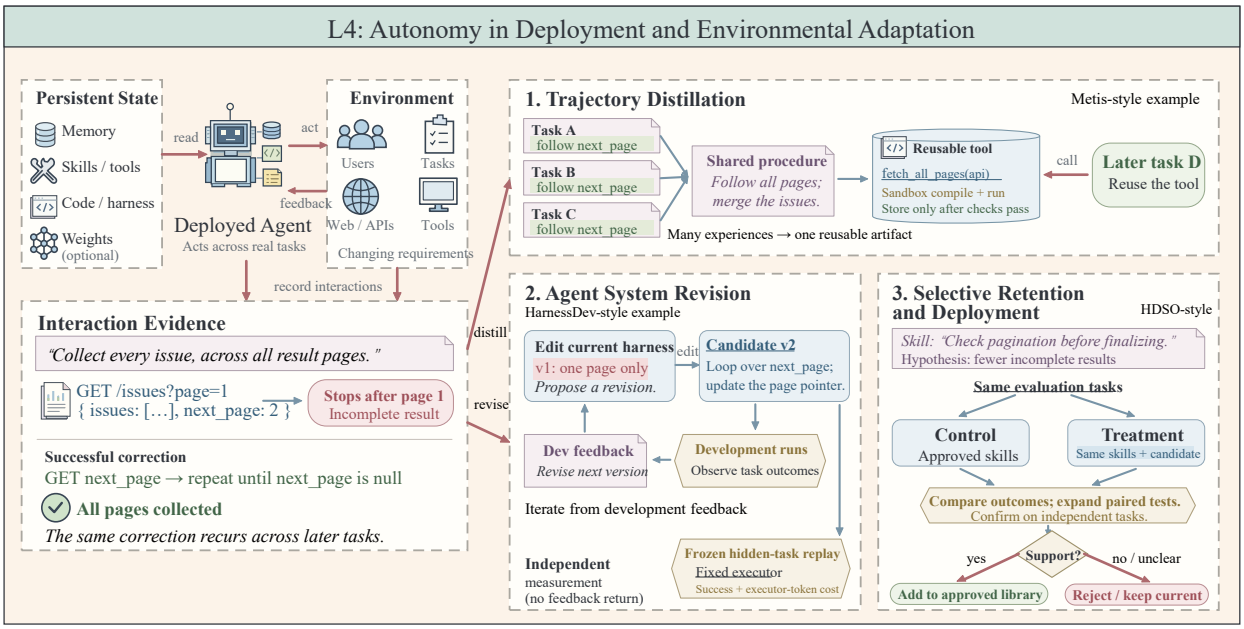


Figure 7 Overview of L4: Autonomy in Deployment and Environmental Adaptation. Under human-defined objectives and deployment constraints, the AI system uses interaction evidence from real tasks and changing environments to determine which consequences of experience should persist and influence future behavior. Experience can be distilled into reusable artifacts, used to revise persistent components of the agent system, and selectively retained based on subsequent evaluation. Accepted updates are reused in later tasks, closing a persistent adaptation loop between deployment experience and future behavior.

Retained changes shape later behavior and the feedback available for further adaptation. The objective, access boundaries, protected evaluation, and authority over consequential releases remain externally governed. Figure 7 provides an overview of this deployment-adaptation mechanism, showing how interaction experience is converted into persistent changes that shape subsequent behavior.

The characteristic loop at this level can be summarized as follows:

observe deployment interaction ⟶ propose persistent adaptation ⟶ revise agent components
⟶ validate and retain ⟶ reuse in later tasks ⟶ collect new feedback and repeat

We organize the discussion around trajectory distillation, iterative revision of the agent system, and selective retention and deployment of updates. These mechanisms differ in what they change, when the change is reused, and how continued usefulness is established. Table 6 summarizes representative methods along these three axes.

3.5.1 Trajectory distillation

Trajectory distillation converts an agent's interaction histories into compact artifacts that persist across sessions and condition later tasks. Compared with raw trajectories, distilled artifacts are more concise, less costly to consult, and structured for easier reuse in subsequent tasks. We group the methods below by the form of the distilled artifact: textual experience memory, structured memory, procedural skill libraries, and executable code.

Textual experience memory. This category keeps the distilled artifact as natural-language passages that are retrieved into the prompt for later tasks. Dynamic Cheatsheet [110] enables a frozen model to improve across a

原文第 27 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.5.1

L4：部署和环境适应的自主性
持久状态环境1.轨迹蒸馏Metis风格示例
内存读取行为任务A跟随下_页可重用工具
技能/工具用户任务任务B共享过程fetch_all_pages(api)调用稍后任务D
跟随下_页跟随所有页面；沙箱编译+运行重用工具
代码/运行支撑框架（Harness）反馈Task C合并的问题。仅在检查通过后存储
权重（可选）部署的智能体在实际任务中执行 Web / API 更改需求工具如下 _ 页 许多经验 → 一个可重用工件
记录交互 2. 智能体系统修订 3. 选择性保留
交互证据提炼 HarnessDev 风格的示例和部署 HDSO 风格
“收集所有结果页面中的每个问题。” 编辑当前运行支撑框架（Harness） v1：一页仅编辑 Candidate v2
循环下一页；技能：假设：不完整的结果较少“在最终确定之前检查分页。”
GET /issues?page=1{ issues: […], next_page: 2 } 在第 1 页之后停止 不完整的结果修订 提出修订建议。
更新页面指针。相同的评估任务
控制处理
成功纠正开发反馈开发运行批准的技能相同的技能+候选人
获取下一页→重复直到下一页为空修改下一个版本观察任务结果
收集的所有页面从开发反馈中迭代比较结果；扩大配对测试。确认独立任务。
在以后的任务中重复进行相同的更正。
独立冻结隐藏任务重播固定执行器是支持吗？否/不清楚
测量（无反馈返回） 成功 + 执行者代币成本 添加到批准的库 拒绝/保持最新

图 7 L4 概述：部署自主性和环境适应。在人类定义的目标和部署限制下，人工智能系统使用来自真实任务和不断变化的环境的交互证据来确定哪些经验后果应该持续存在并影响未来的行为。经验可以被提炼成可重用的工件，用于修改智能体系统的持久组件，并根据后续评估有选择地保留。接受的更新将在以后的任务中重复使用，从而关闭部署体验和未来行为之间的持久适应循环。

保留的变化塑造了以后的行为和可用于进一步适应的反馈。目标、访问边界、受保护的评估以及后续发布的权限仍然由外部管理。图 7 概述了这种部署适应机制，显示了交互体验如何转化为塑造后续行为的持久变化。

该级别的特征环路可以总结如下：

观察部署交互→提出持久适应→修改智能体组件
→验证并保留→在以后的任务中重用→收集新的反馈并重复

我们围绕轨迹蒸馏、智能体系统的迭代修订以及更新的选择性保留和部署组织讨论

。这些机制的不同之处在于它们更改的内容、何时重用更改以及如何建立持续的有用性。表 6 总结了这三个轴上的代表性方法。

3.5.1 轨迹蒸馏

轨迹蒸馏将智能体的交互历史转换为紧凑的工件，这些工件在会话中持续存在并影响后续任务。与原始轨迹相比，提炼的工件更加简洁，查询成本更低，并且结构更易于在后续任务中重用。我们按照提炼工件的形式对以下方法进行分组：文本经验记忆、结构化记忆、程序技能库和可执行代码。

文本体验记忆。此类别将提取的工件保留为自然语言段落，这些段落将被检索到提示中以供后续任务使用。动态备忘单 [110] 使冻结模型能够在整个过程中得到改进

27

Table 6 Representative systems, benchmarks, and analyses for L4 adaptation, grouped by the mechanisms discussed in this section.

Method	Update target	Timing	Update validation
Trajectory distillation			
Dynamic Cheatsheet [110]	Evolving text memory	After each problem	Task outcomes guide self-curation; no held-out gate.
ACE [104]	Incremental context entries	After each task	Helpfulness and harmfulness counters guide edits.
ReasoningBank [85]	Reasoning strategies (text)	After each task	A judge labels trajectories before strategy extraction.
APEX [143]	Milestone dependency graph	After each episode	Episode outcomes are propagated through the graph.
PersonaAgent [105]	Per-user persona prompt	Across user interactions	No separate admission test is reported.
PAHF [144]	User preference entries	When requests are ambiguous	User clarification confirms and revises preferences.
MemToolAgent [145]	Tool-use critiques and retrieval policy	After tool calls	Environment and user feedback assess tool use.
Trace2Skill [86]	Portable skill document	Offline batch	Error and success analysts propose and merge patches.
PRACTICE [146]	Embodied skill library	Offline batches	Success/failure contrasts and teacher distillation.
PANDO [25]	Rules and routines	Online, in-run	Confidence-scored admission and demotion.
PILOT [147]	Procedures and failure modes	During long-horizon runs	Supervisor distillation uses live-run feedback.
Metis [62]	Text plans and code tools	After completed tasks (asynchronous)	Repeated reuse plus sandbox compile checks.
Evo-Harness [148]	Harness skill tuples	After task batches	Failed or negatively evaluated tasks trigger reflection and edits.
SHAPER [111]	Planning guidance and context-selection code	Across task rounds	Sandbox execution and downstream task outcomes.
Evo-Memory [149]	Accumulated agent memory (benchmark)	Sequential task streams	Long-horizon evaluation of memory reuse.
Iterative revision of the agent system			
DecoEvo [80]	Solver and rubric skills	Iterative rounds	Score-independent audits, Pareto-checked.
HarnessDev [91]	Runnable agent harness	Iterative rounds	Frozen replay of candidates on hidden tasks.
ASPIRE [109]	Model weights or harness	Iterative rounds	Rollback unless the verified score improves.
S3Gym [150]	History, summaries, or model parameters	Iterative rounds	Compares update pathways without verifier outcomes.
Harness Benefit [151]	Harness updates and their use (analysis)	Fixed solve-evolve rounds	Separates update quality from execution benefit.
Selective retention and deployment of updates			
HDSO [106]	Candidate skill packages	Periodic rounds	Paired control and treatment executions.
Library Drift [152]	Skill library entries	Each round	Low-contribution skills are retired. The library is capped.
Rethinking Skill Evolution [153]	Skill edits (analysis)	Multiple rounds	Filters edits using task outcomes across rounds.
Tax AI [92]	Product code and evals	Release cycles	Targeted and regression tests, then human review.

series of related problems without access to ground-truth labels. It maintains a single evolving note of strategies, code snippets, and known pitfalls, which the same model consults when answering each incoming problem and curates by adding distilled lessons and removing superseded entries. Agentic Context Engineering [104] follows the same recipe while keeping the memory detailed and informative as it grows. It stores short entries with counters that record whether each entry helps or hinders later tasks, and adds small patches rather than rewriting the whole note, which avoids information loss by compression. ReasoningBank [85] adapts the idea to multi-step agents and learns from failures as much as from successes by distilling each judged trajectory into a short, titled strategy that is retrieved before later tasks.

Structured memory. Instead of free text, structured memory stores experience in explicit records or graphs, making the conditions for reuse inspectable. APEX [143] encourages an agent to explore new strategies as its memory grows rather than letting it settle into familiar routines by building a strategy map as a graph of task milestones linked by prerequisite relations. After each episode, the outcome is propagated back along the milestones that led to it, so the map records which paths have worked. A separate step adds promising branches that the agent has not tried, and the agent consults the map at planning time to choose between a known good path and an untried one. Other structures mirror the environment. For instance, PersonaAgent [105] and PAHF [144] instead organize memory around individual users. PersonaAgent translates episodic and semantic user memory into a per-user persona prompt, while PAHF revises preference entries by asking users to clarify an ambiguous request when no relevant preference is found, then integrates their feedback into

原文第 28 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.5.1

表 6 L4 适应的代表性系统、基准和分析，按本节讨论的机制分组。

方法	更新目标	时机	更新验证
轨迹蒸馏			
动态
备忘单 [110]	不断发展的文本记忆	每个问题之后	任务结果指导自我管理；没有保留的门。
ACE [104]	增量上下文条目	每项任务之后	有用和有害计数器指导编辑。
ReasoningBank [85]	推理策略（文本）	每个任务之后	法官在策略提取之前标记轨迹。
APEX [143]
PersonaAgent [105]	里程碑依赖图
每用户角色提示	每集之后
跨用户
交互	集结果通过图表传播。
不报告单独的准入测试。
PAHF [144]	用户首选项条目	当请求
不明确时	用户澄清可确认并修改首选项。
MemToolAgent [145]	工具使用批评和检索
策略	工具调用后	环境和用户反馈评估工具使用。
Trace2Skill [86]	便携式技能文档	离线批处理	错误和成功分析师提出并合并补丁。
实践【146】	体现技能库	离线批量	成功/失败对比与老师升华。
PANDO [25]	规则和惯例	在线，运行中	信心评分入场和降级。
PILOT [147]	过程和故障模式
长时程（Long-Horizon）运行期间	Supervisor 蒸馏使用实时运行反馈。
Metis [62]	文本计划和代码工具	完成
任务
后（异步）	重复重用加上沙箱编译检查。
Evo-运行支撑框架（Harness）[148]	运行支撑框架（Harness）技能元组	任务批次之后	失败或负面评估的任务会触发反思和编辑。
SHAPER [111]	规划指导和
上下文选择代码	跨任务轮次	沙盒执行和下游任务结果。
Evo-Memory [149]	累积智能体内存
（基准）	顺序任务
流	长时程（Long-Horizon）内存重用评估。
智能体系统迭代改版		
DecoEvo [80]	求解器和标题技能	迭代轮次	与分数无关的审核，帕累托检查。
HarnessDev [91]	可运行智能体 运行支撑框架（Harness）	迭代回合	隐藏任务上候选者的冻结重播。
ASPIRE [109]	模型权重或 运行支撑框架（Harness）	迭代轮次	回滚，除非验证的分数提高。

|S3Gym [150]|历史、摘要或模型
参数|迭代轮次|比较更新路径而无需验证者结果。|
|运行支撑框架（Harness）
好处 [151]|运行支撑框架（Harness） 更新及其使用
（分析）|修复求解进化
轮数|将更新质量与执行好处分开。|
|选择性保留和|更新部署|||
|HDSO [106]|候选技能包|定期回合|配对控制和治疗执行。
|库漂移[152]|技能库条目|每轮|低贡献技能退役。图书馆已封顶。|
|重新思考技能
进化[153]|技能编辑（分析）|多轮|使用跨轮任务结果过滤编辑。|
|Tax AI [92]|产品代码和评估|发布周期|有针对性的回归测试，然后进行人工审查。|

无法访问真实标签的一系列相关问题

。它维护了一个关于策略、代码片段和已知陷阱的单一不断演变的注释，同一模型在回答每个传入问题时都会参考这些注释，并通过添加精炼的课程和删除被取代的条目来进行管理。智能体上下文工程[104]遵循相同的方法，同时在记忆增长时保持详细和信息丰富。它存储带有计数器的简短条目，记录每个条目是否有助于或阻碍后续任务，并添加小补丁而不是重写整个笔记，从而避免了压缩造成的信息丢失。ReasoningBank [85] 将这个想法应用于多步智能体，并通过将每个判断的轨迹提炼成一个简短的、有标题的策略，在后续任务之前检索该策略，从失败中学习，从成功中学习。

结构化记忆。结构化内存不是自由文本，而是将经验存储在显式记录或图表中，从而使重用的条件变得可检查。 APEX [143] 鼓励智能体随着记忆的增长探索新的策略，而不是通过构建策略图作为由先决条件关系链接的任务里程碑图来让它适应熟悉的例程。每个情节结束后，结果都会沿着导致它的里程碑传播回来，因此地图会记录哪些路径有效。一个单独的步骤添加了智能体尚未尝试过的有希望的分支，并且智能体在规划时查阅地图以在已知的良好路径和未尝试过的路径之间进行选择。其他结构反映了环境。例如，PersonaAgent [105] 和 PAHF [144] 而是围绕单个用户组织内存。 PersonaAgent 将情景和语义用户记忆转换为每个用户的角色提示，而 PAHF 通过在没有找到相关偏好时要求用户澄清不明确的请求来修改偏好条目，然后将他们的反馈集成到

28

memory and revises outdated entries. MemToolAgent [145] gives memory a more procedural role by storing critiques of failed tool calls and adjusting how many entries it retrieves per call.

Procedural skill libraries. Instead of only describing experience, this category extracts reusable skills or standard operating procedures that a later agent follows directly. Trace2Skill [86] turns lessons from individual trajectories into skills that other models and tasks can reuse. It first rolls out a frozen agent on domain tasks. Two analyst roles, one focused on errors and the other on successes, then read the trajectories and propose skill edits, which are merged in stages into a portable skill directory. Because the consolidated document is read directly rather than retrieved for each episode, later agents load it once as part of their instructions. PRACTICE [146] applies the same offline distillation to embodied agents while keeping the skill library consistent as it changes rather than growing unbounded, where a trainable skill learner proposes batch edits that add, refine, merge, or remove entries. The learner first learns basic skill generation and library maintenance from oracle trajectories, then learns failure awareness by contrasting successful and failed trajectories from different executors on the same tasks, and is finally distilled toward a stronger teacher on its own edit distribution. Two other methods acquire skills during execution. PANDO [25] improves a web agent during deployment by distilling rules that prevent repeated failures and parameterized routines that replace multi-step browser subgoals from each rollout during deployment, while PILOT [147] uses a supervisor to distill procedures and failure modes from a live, long-horizon run, so later sessions start with the accumulated skills.

Executable artifacts. Moving from text to code, this category compiles experience into artifacts that are invoked directly, combining the flexibility of text memory with the efficiency of executable tools. Metis [62] achieves this through a dual memory, a text store of plans, environment facts, and pitfalls alongside a code library of callable tools. A reflector maintains the text entries, and a frequently reused text plan is rewritten as a callable tool and accepted only after it compiles and runs correctly in a sandbox. Retrieval then covers both text entries and tool descriptions. Evo-Harness [148] broadens the target from single tools to the whole harness, compiling lessons from failed or negatively evaluated tasks into cross-task patterns and task-specific procedures. SHAPER [111] applies the same idea to embodied agents by evolving textual planning guidance together with a sandboxed Python function that selects the context shown to the planner.

Evaluation protocols. Evaluation protocols for this family differ in time horizon and in what they count as benefit. Evo-Memory [149] restructures datasets into sequential task streams so that memory accumulation and reuse can be measured over long horizons. PANDO [25] instead audits behavior within a run, reporting action repetition, step overhead, and prompt-cache utilization, and Metis [62] relates task quality to execution and construction cost.

3.5.2 Iterative revision of the agent system

Rather than what an agent remembers, this section revises the agent system itself: the skills that produce solutions, the rubrics that judge them, the harness that runs them, or the weights underneath. The defining question is which components may change and which acceptance criteria stay fixed while they do. We group the work into co-evolving coupled components, benchmarks that measure self-directed system revision, and evolutionary search under fixed verifiers, followed by analyses of whether revision translates into benefit.

Co-evolving components. Co-evolution revises two coupled components at once, which raises a circularity problem: if the judge improves alongside the solver, higher scores may reflect an easier judge rather than better solutions. DecoEvo [80] addresses this problem by evolving a solver skill and a rubric-generator skill in text space while decoupling their objectives and withholding gold rubrics during optimization. The solver skill is updated from criterion-level rubric feedback. The rubric generator, in contrast, is gated by two score-independent audits: a structural audit checks that the generated rubric covers the task's requirements, and a contrastive audit checks that it discriminates between near-tie responses, with Pareto verification across the two. Because the generator never sees the solver's aggregate score, it cannot improve its standing by making the rubric easier.

Benchmarks of self-directed revision. Rather than proposing new methods, this line of work measures whether current models can revise their own systems at all. HarnessDev [91] asks whether models can create and improve their own harness, the model-external scaffolding of prompts, tool loops, and execution logic.

原文第 29 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.5.2

记忆并修改过时的条目。 MemToolAgent [145] 通过存储对失败工具调用的批评并调整每次调用检索的条目数量，为内存提供了更具程序性的作用。

程序技能库。该类别不仅仅描述经验，而是提取后来的智能体直接遵循的可重用技能或标准操作程序。 Trace2Skill [86] 将个人轨迹的经验教训转化为其他模型和任务可以重用的技能。它首先在域任务上推出冻结智能体。两个分析师角色，一个专注于错误，另一个专注于成功，然后读取轨迹并提出技能编辑，这些编辑将分阶段合并到便携式技能目录中。由于合并的文档是直接读取的，而不是针对每个情节进行检索，因此后来的智能体将其作为指令的一部分加载一次。实践[146]将相同的离线蒸馏应用于具体智能体，同时保持技能库在变化时保持一致，而不是无限制地增长，其中可训练的技能学习者建议进行批量编辑，以添加、细化、合并或删除条目。学习者首先从预言轨迹中学习基本技能生成和库维护，然后通过对比同一任务上不同执行者的成功和失败轨迹来学习失败意识，最后根据自己的编辑分布提炼出更强的老师。另外两种方法在执行过程中获得技能。 PANDO [25] 通过提炼防止重复失败的规则和参数化例程来改进部署期间的 Web 智能体，这些规则在部署期间替换来自每个 展开轨迹（Rollout） 的多步骤浏览器子目标，而 PILOT [147] 使用主管从实时 长时程（Long-Horizon） 运行中提炼过程和故障模式，因此后续会话从积累的技能开始。

可执行的工件。从文本到代码，该类别将经验编译成可直接调用的工件，将文本内存的灵活性与可执行工具的效率结合起来。 Metis [62] 通过双存储器、计划、环境事实和陷阱的文本存储以及可调用工具的代码库来实现这一点。反射器维护文本条目，经常重用的文本计划被重写为可调用工具，并且只有在沙箱中正确编译和运行后才被接受。然后检索涵盖文本条目和工具描述。 Evo-运行支撑框架（Harness） [148] 将目标从单一工具扩大到整个 运行支撑框架（Harness），将失败或负面评估任务的经验教训编译成跨任务模式和特定于任务的程序。 SHAPER [111] 通过发展文本规划指导以及选择向规划器显示的上下文的沙盒 Python 函数，将相同的想法应用于具身智能体。

评估协议。该家庭的评估方案在时间范围和效益方面有所不同。 Evo-Memory [149] 将数据集重组为顺序任务流，以便可以长期测量内存积累和重用。 PANDO [25] 而是审计运行中的行为，报告操作重复、步骤开销和提示缓存利用率，而 Metis [62] 将任务质量与执行和构建成本相关联

。

3.5.2 智能体系统的迭代修订

这一部分不是修改智能体所记住的内容，而是修改智能体系统本身：产生解决方案的技能、判断它们的标准、运行它们的 运行支撑框架（Harness） 或下面的权重。决定性的问题是哪些组件可能会发生变化，以及哪些验收标准在变化时保持不变。我们将工作分为共同进化耦合组件、衡量自我导向系统修订的基准以及固定验证者下的进化搜索，然后分析修订是否转化为效益。

共同进化的组件。协同进化同时修改两个耦合组件，这引发了循环问题：如果判断器与求解器一起改进，则更高的分数可能反映更容易的判断而不是更好的解决方案。 DecoEvo [80] 通过在文本空间中发展求解器技能和评分标准生成器技能来解决这个问题，同时解耦它们的目标并在优化期间保留黄金评分标准。求解器技能根据标准级别的反馈进行更新。相比之下，评分标准生成器由两个与分数无关的审核控制：结构审核检查生成的评分标准是否涵盖任务的要求，对比审核检查它是否区分接近平局的响应，并在两者之间

进行帕累托验证。由于生成器永远看不到求解器的总分，因此它无法通过简化评分标准来提高其排名。

自我指导修订的基准。这项工作不是提出新方法，而是衡量当前模型是否可以修改自己的系统。HarnessDev [91] 询问模型是否可以创建和改进自己的 运行支撑框架（Harness），即提示、工具循环和执行逻辑的模型外部支架。

29

The creator agent builds a complete harness from a minimal seed and a few development cases and iteratively revises it from downstream feedback. The candidate revisions are frozen and scored on hidden tasks for both success rate and executor-token cost, with the creator and the executor kept separate so that harnesses can be compared under one fixed executor. ASPIRE [109] asks whether a model can improve itself given only a vague goal. It delegates the choice of data, update method, and validation signal to the agent across both weights and the harness, and a score-gated controller rolls back any change whose verified score does not improve. S3Gym [150] restricts the question to the experience channel, withholding verifier outcomes so the agent must judge its own trajectories, and compares raw history, compressed summaries, and parameter training as improvement pathways.

Analyses of revision benefit. Whether revision actually translates into benefit is a question in its own right, and one analysis finds that the two are often conflated. The study behind Harness Updating Is Not Harness Benefit [151] separates two capabilities: producing useful persistent updates, and exploiting them at solve time. Each is measured under a fixed solve-evolve protocol with identical prompts and budgets across several model backbones and three agent benchmarks. Updating ability is largely independent of base model capability, with a small model's updates yielding gains comparable to a frontier model's. The ability to benefit from an updated harness, by contrast, varies non-monotonically, and failures concentrate in two modes: relevant artifacts are not activated, or they are activated but not faithfully followed. A practical consequence is that capability investment belongs in the task-solving agent rather than in the evolver.

3.5.3 Selective retention and deployment of updates

Trajectory distillation and system revision both end with a proposed change, while this section focuses on which proposed changes become persistent and which stay available to later tasks. The work below applies this control at three points: before admission, during continued use, and at release into an operational system.

Candidate validation. A proposed update must produce evidence before it can enter the persistent state. HDSO [106] is designed to prevent skills distilled from noisy trajectories from encoding spurious shortcuts or rules that the executor cannot follow. A curator model observes compact executor traces and proposes a hypothesis with an explicit validation plan, where each candidate is tested by executing the same tasks twice, once with the current approved repository as the control and once with the candidate skill added as the treatment, and the candidate enters the approved repository only when the differences between the two runs support the hypothesis. The comparison runs in stages of increasing size and ends with confirmation on independent tasks. Metis [62], introduced earlier in the trajectory-distillation section, applies a similar gate after the text plan is already retained in memory, where a text plan becomes executable code only after it recurs across tasks and passes dependency and compilation checks.

Library maintenance. After admission, attention shifts to the health of the library itself, since skills that were once useful can degrade as tasks and models change. The Library Drift study [152] shows that unbounded accumulation gradually degrades retrieval quality and stalls progress, often before the effect becomes visible in task scores. It proposes lifecycle management with three parts. An append-only evidence log tracks each skill's contribution to task outcomes, and a skill is retired when its measured contribution fades. The number of active skills is capped, so a new skill competes with the entries already present. New skills are also written under a meta-skill, a stored guide that steers how future skills are authored. The study shows that governance choices matter: retiring skills too aggressively performs worse than leaving the library unguided. Meanwhile, another analysis [153] finds that carefully filtering skill edits, rather than repeatedly rewriting skills from task outcomes, improves task performance.

Governed deployment. In production, an incorrect update can affect real users, so a strict gate is needed to prevent unexpected issues. The Tax AI deployment [92] applies such a gate inside a production tax-preparation system, where practitioner corrections drive improvement. Corrections are recorded as structured field-level evidence, and repeated failures are grouped into evaluation targets. A coding agent investigates the trace, evaluations, and repository to implement and validate a fix against targeted and regression evaluations. The loop may change only a bounded part of the product, such as the extraction schema, source selection, tax-engine mapper, and graders. Engineers retain decisions about architecture and product design. Each fix arrives as a pull request for engineering review before release, and cases that the agent cannot resolve are

原文第 30 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.5.3

创建者智能体从最小的种子和一些开发案例构建完整的 运行支撑框架（Harness），并根据下游反馈对其进行迭代修改。候选修订版被冻结，并在隐藏任务上对成功率和执行者代币成本进行评分，创建者和执行者保持分离，以便可以在一个固定执行者下比较 运行支撑框架（Harnesses）。 ASPIRE [109]询问模型是否可以在仅给出模糊目标的情况下自我改进。它将数据、更新方法和验证信号的选择委托给权重和 运行支撑框架（Harness） 的智能体，并且分数门控控制器会回滚任何经过验证的分数没有提高的更改。 S3Gym [150] 将问题限制在经验通道，保留验证者结果，因此智能体必须判断自己的轨迹，并比较原始历史、压缩摘要和参数训练作为改进途径。

修订效益分析。修订是否真正转化为效益本身就是一个问题，一项分析发现两者经常被混为一谈。 运行支撑框架（Harness） 更新不是 运行支撑框架（Harness） 好处 [151] 背后的研究区分了两种功能：生成有用的持久更新，并在求解时利用它们。每个都是在固定的解决-演化协议下进行测量的，在多个模型主干和三个智能体基准上具有相同的提示和预算。更新能力在很大程度上独立于基础模型能力，小型模型的更新所产生的收益与前沿模型的收益相当。相比之下，从更新的 运行支撑框架（Harness） 中受益的能力是非单调变化的，并且失败集中在两种模式：相关工件未激活，或者它们已激活但未忠实遵循。一个实际的结果是，能力投资属于任务解决者而不是进化者。
3.5.3 更新的选择性保留和部署

轨迹精炼和系统修订都以提议的更改结束，而本节重点介绍哪些提议的更改会持久存在以及哪些对以后的任务可用。下面的工作在三个点应用这种控制：进入之前、继续使用期间以及释放到操作系统中时。

候选人验证。提议的更新必须产生证据才能进入持久状态。 HDSO [106] 旨在防止从噪声轨迹中提取的技能编码执行者无法遵循的虚假快捷方式或规则。策展人模型观察紧凑的执行器轨迹，并提出具有明确验证计划的假设，其中通过执行相同的任务两次来测试每个候选者，一次使用当前批准的存储库作为对照，一次使用添加的候选者技能作为处理，只有当两次运行之间的差异支持假设时，候选者才进入批准的存储库。比较以增加规模的阶段进行，并以对独立任务的确认结束。 Metis [62] 在轨迹蒸馏部分前面介绍过，在文本计划已保留在内存中后应用类似的门，其中文本计划仅在跨任务重复并通过依赖性和编译检查后才成为可执行代码。

图书馆维护。入院后，人们的注意力转移到图书馆本身的健康状况上，因为曾经有用的技能可能会随着任务和模型的变化而退化。图书馆漂移研究[152]表明，无限制的积累会逐渐降低检索质量并阻碍进度，通常是在任务分数中效果显现之前。它提出了由三个部分组成的生命周期管理。仅附加证据日志跟踪每项技能对任务结果的贡献，并且当其测量的贡献消失时，技能将被淘汰。活跃技能的数量是有上限的，因此新技能会与现有的条目竞争。新技能也是在元技能下编写的，元技能是一种存储的指南，指导如何编写未来的技能。研究表明，治理选择很重要：过于激进地淘汰技能比让图书馆不受指导更糟糕。与此同时，另一项分析[153]发现，仔细过滤技能编辑，而不是根据任务结果反复重写技能，可以提高任务绩效。

受控部署。在生产中，不正确的更新可能会影响真实用户，因此需要严格的控制来防止意外问题。税务人工智能部署[92]在生产税务准备系统中应用了这样一个门，从业者的纠正推动了改进。纠正被记录为结构化的现场证据，重复的失败被分组为评估目标。编码智能体调查跟踪、评估和存储库，以针对目标和回归评估实施和验证修复。该循环可能仅更改产品的有限部分，例如提取模式、源选择、税收引擎映射器和分级器。工程师保留有关架构和产品设计的决策。每个修复都会作为发布前工程审查的拉取请求到达，智能

体无法解决的情况是

30

routed back to practitioners. Humans therefore make the final acceptance decision, and the delay introduced by this review is part of the loop's operating cost.

3.5.4 Evaluating L4 Deployment Autonomy

Greater autonomy over adaptation after deployment does not by itself establish reliable improvement. An L4 system may retain updates without showing that they remain useful as tasks, environments, or executors change. Evaluation should distinguish the degree of deployment autonomy from the quality of the resulting adaptation process.

A characteristic risk at this level is persistent update failure, where an accepted change may affect many later decisions before its weakness becomes visible. Across different forms of retained state, failure may arise because the system infers the wrong lesson from noisy evidence, applies a sound lesson outside its valid scope, fails to invoke a relevant update, or preserves new behavior at the expense of earlier capabilities [110, 151–153]. Final task scores alone cannot separate these mechanisms. A credible evaluation must connect each retained change to the evidence that produced it, its later use, and its effects on both new and previously solved tasks.

The current evidence leaves further uncertainty about whether deployment adaptation remains effective over time. Most evaluations cover bounded task streams and limited combinations of models, tasks, and environments, while repeated assessment makes longer studies costly [86, 91, 147]. Feedback may also be incomplete or endogenous to the loop, as outcome labels can be noisy, evaluators can change with the system, and a useful retained artifact may still fail to influence execution [80, 85, 151]. Evaluation of L4 should consequently examine adaptation across time and distribution shifts, including the point at which a retained change alters later behavior.

L4 autonomy remains bounded by the surrounding deployment process. The agent may decide how interaction evidence changes persistent state, while humans continue to specify the objective, access boundaries, protected evaluation, and release authority. L4 should concern autonomy over operational adaptation within a deployed system.

> **Finding: L4 shifts autonomy from selecting experience to deciding which consequences of experience persist in deployment.** The agent converts interaction histories into retained changes to memory, skills, harnesses, code, or model parameters, and these changes affect later tasks and feedback. Humans retain the objective, protected acceptance criteria, access boundaries, and final authority over consequential releases.

3.6 L5: From Environmental Adaptation to Meta-Improvement

An improvement process can become a bottleneck in its own right. A coding agent may repeatedly generate similar unsuccessful patches because its search procedure discards useful alternatives. A research agent may optimize a development score after that score has stopped predicting external performance. Repairing such failures requires changes to the procedures that direct experiments and judge their outcomes. Designing these procedures is costly: a revision must be assessed through the later improvements it produces, often across several tasks and repeated runs [15, 17].

L5 begins when AI persistently modifies a mechanism responsible for future improvements and uses the revised mechanism in subsequent rounds. The editable mechanism may be an improver, a successor evaluator, a search policy, or a procedure for directing research. L2 searches for candidate interventions; L3 determines what experience to acquire; L4 incorporates deployment feedback into persistent adaptation. L5 additionally makes the procedure governing later improvement an object of improvement. A deployed agent that retains new debugging skills through an unchanged learning procedure exemplifies L4. Revising and reusing the procedure that diagnoses failures and builds those skills can cross the L5 boundary.

The loop closes when a revised mechanism returns to govern the generation, evaluation, or selection of later successors. The inherited state includes the accepted code, prompts, evaluators, or research policy, together with evidence used by later revisions. Human designers still establish the overall mission, protected evaluation, editable components, and resource permissions, and may retain veto and deployment authority. Some of

原文第 31 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.6

返回给从业者。因此，人类做出最终的接受决定，而这次审查带来的延迟是环路运营成本的一部分。
3.5.4 评估 L4 部署自主性

部署后更大的适应自主权本身并不能带来可靠的改进。 L4 系统可能会保留更新，但不会表明它们在任务、环境或执行者发生变化时仍然有用。评估应区分部署自主程度和最终适应过程的质量。

此级别的典型风险是持续更新失败，其中接受的更改可能会在其弱点变得明显之前影响许多后续决策。在不同形式的保留状态中，可能会出现失败，因为系统从嘈杂的证据中推断出错误的教训，在其有效范围之外应用合理的教训，未能调用相关更新，或者以牺牲早期功能为代价保留新行为[110, 151-153]。仅最终任务分数无法区分这些机制。可信的评估必须将每个保留的变更与产生它的证据、其后来的使用及其对新任务和先前解决的任务的影响联系起来。

目前的证据对于部署适应是否随着时间的推移仍然有效留下了进一步的不确定性。大多数评估涵盖有限的任务流和模型、任务和环境的有限组合，而重复评估会使更长的研究成本高昂[86,91,147]。反馈也可能是不完整的或内生的循环，因为结果标签可能有噪音，评估者可能会随着系统的变化而变化，并且有用的保留工件可能仍然无法影响执行[80,85,151]。因此，L4 的评估应该检查跨时间和分布变化的适应，包括保留的变化改变后来行为的点。

L4 自治仍然受到周围部署过程的限制。智能体可以决定交互证据如何改变持久状态，而人类则继续指定目标、访问边界、受保护的评估和发布权限。 L4 应该关注已部署系统内操作适应的自主权。

发现：L4 将自主权从选择经验转变为决定经验的哪些后果在部署中持续存在。智能体将交互历史记录转换为对记忆、技能、运行支撑框架（Harnesses）、代码或模型参数的保留更改，这些更改会影响以后的任务和反馈。人类保留客观的、受保护的接受标准、访问边界以及对后续发布的最终权力。
3.6 L5：从环境适应到元改进

改进过程本身就可能成为瓶颈。编码智能体可能会重复生成类似的不成功补丁，因为其搜索过程丢弃了有用的替代方案。研究智能体可以在发展分数停止预测外部表现之后优化该分数。修复此类故障需要改变指导实验和判断结果的程序

。设计这些程序的成本很高：必须通过后来产生的改进来评估修订，通常需要跨多个任务和重复运行 [15, 17]。

当人工智能持续修改负责未来改进的机制并在后续回合中使用修改后的机制时，L5 就开始了。可编辑机制可以是改进器、后继评估器、搜索策略或指导研究的程序。 L2 搜索候选干预措施； L3决定获得什么经验 ； L4 将部署反馈纳入持续适应中。 L5还使管理后续改进的程序成为改进的对象。已部署的智能体通过不变的学习过程保留新的调试技能就是 L4 的例子。修改和重用诊断故障并培养这些技能的过程可以跨越 L5 边界。

当修订后的机制返回来管理后来继任者的生成、评估或选择时，循环就结束了。继承的状态包括已接受的代码、提示、评估器或研究策略，以及后续修订所使用的证据。人类设计师仍然建立总体任务、受保护的评估、可编辑组件和资源权限，并可能保留否决权和部署权限。一些

31

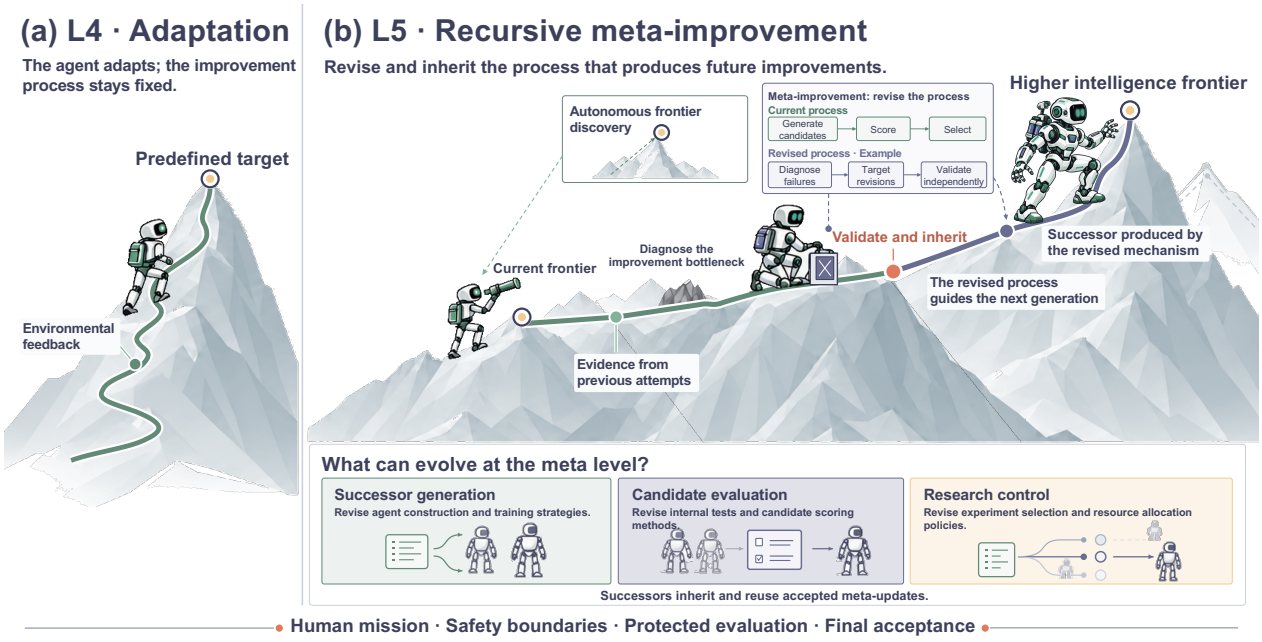


Figure 8 From environmental adaptation to recursive meta-improvement. (a) L4 incorporates environmental feedback within a fixed improvement process. (b) L5 diagnoses limitations of that process, revises it, and passes validated changes to successors. Meta-updates can affect successor generation, candidate evaluation, or research control. Human-defined missions, safety boundaries, protected evaluation, and final acceptance remain external constraints. The ascending path illustrates possible progress through inherited meta-updates.

these decisions are enforced by fixed infrastructure; they do not require a person to approve every iteration. Figure 8 illustrates how evidence from earlier attempts can guide a revision of the improvement process, which successors then inherit and reuse.

These conditions distinguish *structural L5*, which demonstrates that an AI-directed change persists and controls a later improvement round, from *effective L5*, which demonstrates that the revised mechanism produces or selects better successors under comparable budgets and independent assessment. Self-modifying task code provides insufficient evidence when the process responsible for later revisions remains unchanged. Conversely, an evolved successor evaluator can establish structural L5 even with unchanged task-agent code. Classification therefore applies to the particular mechanism examined. Table 7 summarizes the loop, inherited state, and external controls of representative systems.

3.6.1 Improving the Search Procedure

The AI revises how future candidate improvements are generated and explored. A fixed optimizer can spend its budget on unproductive proposals, even when its underlying model can generate better search algorithms. The challenge is to evaluate a proposed optimizer through its ability to improve other programs.

STOP [15] makes this problem executable. Its improver is a Python program that calls a fixed language model, generates candidate code, evaluates a supplied utility, and returns a selected candidate. The current improver receives its own source as the optimization target. Successor improvers are scored by the quality of programs they produce on downstream tasks, and the selected improver performs the next self-improvement round. The inherited artifact is thus a search procedure that may introduce beam search, alternative sampling, or other candidate-selection logic. The study reports that a selected fourth-generation improver outperformed the seed on all five transfer tasks excluded from self-improvement. The task distribution, utility, base model, and resource limits remained externally specified. Weaker-model runs regressed on average, and some generated programs evaded soft budgets or exploited evaluation bugs.

原文第 32 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.6.1
(a) L4·适应 (b)L5·递归元改进

智能体适应；改进 修改并继承产生未来改进的流程。
流程保持固定。元改进：修改流程 更高智能前沿
自主前沿 当前流程
发现候选者生成分数选择
预定义目标 修订流程·示例诊断目标 验证
失败独立修订
验证和继承产生的后继者
诊断修订机制
当前前沿改进瓶颈
修订流程
指导下一步一代
环境
反馈
来自
之前尝试的证据
在元层面可以进化什么？
继任者一代候选人评估研究控制
修改智能体构建和训练策略。修改内部测试和候选人评分修改实验选择和资源分配
方法。策略。
继任者继承并重用已接受的元更新。
人类使命 · 安全边界 · 受保护的评估 · 最终接受

图 8 从环境适应到递归元改进。 (a) L4 将环境反馈纳入固定的改进流程中。 (b) L5 诊断该流程的局限性，对其进行修改，并将经过验证的更改传递给后继者。元更新可能会影响继任者的产生、候选人评估或研究控制。人为定义的任务、安全边界、受保护的评估和最终验收仍然是外部约束。上升路径说明了通过继承的元更新可能取得的进展。

这些决定由固定基础设施执行；他们不需要人批准每次迭代。图 8 说明了早期尝试的证据如何指导改进过程的修订，然后继任者继承并重用该改进过程。

这些条件将_结构性 L5_ 与_有效 L5_ 区分开来，结构性 L5 表明人工智能主导的变革持续存在并控制后续的改进回合，有效 L5 表明修订后的机制在可比较的预算和独立评估下产生或选择更好的继任者。当负责后续修改的流程保持不变时，自修改任务代码提供的证据不足。相反，即使任务智能体代码不变，进化的后继评估器也可以建立结构性 L5。因此，分类适用于所检查的特定机制。表 7 总结了代表性系统的循环、继承状态和外部控制。
3.6.1 改进检索程序

人工智能修改了未来候选改进的生成和探索方式。固定优化器可以将其预算花在无用的建议上，即使其底层模型可以生成更好的搜索算法。挑战在于通过改进其他程序的能力来评估建议的优化器。

STOP [15] 使这个问题变得可执行。它的改进器是一个Python程序，它调用固定的语言模型，生成候选代码，评估提供的实用程序，并返回选定的候选者

。当前改进器接收自己的源作为优化目标。后续改进者根据他们在下游任务中生成的程序质量进行评分，选定的改进者执行下一轮 自我改进（Self-Improvement）。因此，继承的工件是可以引入波束搜索、替代采样或其他候选选择逻辑的搜索过程。研究报告称，选定的第四代改进剂在 自我改进（Self-Improvement）排除的所有五项传输任务上均优于种子。任务分配、效用、基本模型和资源限制仍然由外部指定。较弱的模型运行平均会出现倒退，并且一些生成的程序逃避了软预算或利用了评估错误。

32

Table 7 L5 mechanisms and external controls.

Work	Loop closes at	Inherited state	External controls
Search and self-revision			
STOP [15]	Next program search	Improver code	Utility; base LM; budget
Gödel Agent [19]	Next self-revision	Task and update code	Task objective; runtime access
DGM [16]	Descendant search	Agent code; archive	Parent selection; benchmark
HyperAgents [93]	Next agent generation	Task and meta-agent code	Main-study selection; evaluation
Successor evaluation			
RQGM [21]	Next-epoch selection	Evaluator; agent code	Anchor; replacement schedule
Research policies and harnesses			
A-Evolve-Training [17]	Next research round	Search policy; discovery log	Constitution; benchmark; substrate
$AIRA_2$ [154] / AAR [155]	Task experiments	Research artifacts	Research harness; evaluation
$AIDE^2$ [94]	Later research runs	Research-agent harness	Private scores; cost budget

Note: L5 claims are qualified in the text.

Gödel Agent [19] gives the AI access to both its task policy and recursive update logic in a shared Python program. Execution feedback informs a rewrite, and the revised program runs in the next recursive call, including its modified self-update procedure. This broadens the search beyond a developer's fixed editing routine. It also makes errors inheritable: the study reports that 14 of 100 MGSM optimization trials ended below the initial policy's performance, and unrestricted runs could call stronger models. Task objectives and execution permissions therefore remain consequential external controls.

The Darwin Gödel Machine (DGM) [16] retains coding-agent variants and explores their descendants, but keeps archive management and parent selection outside self-modification. Its lineage provides a transition case; task gains alone do not establish a better improvement procedure. HyperAgents [93] directly exposes both task-agent and meta-agent code to revision. Evolved meta-agents build performance trackers and persistent memory to guide later modifications. In its preprint, meta-agents evolved on paper review and robotics generated improved agents for previously unseen mathematics grading, supporting transfer of the improvement procedure itself. The longer, 200-iteration experiment did not establish a statistically significant final advantage for transferred initialization. Parent selection and evaluation also remained fixed in the main experiments.

3.6.2 Improving Successor Evaluation

The AI improves the evaluator that determines which successors are retained. Repeated optimization places pressure on a judge's blind spots. Keeping that judge unchanged can favor proposals that exploit its errors. Unrestricted judge revision creates the opposite problem: successive scores may reflect changing standards.

The Red Queen Gödel Machine (RQGM) [21] addresses this tension through evaluator co-evolution. Its learned evaluator remains frozen within an epoch. At a scheduled boundary, challenger evaluators are compared against an independent ground-truth anchor; the selected evaluator then governs the next epoch. Scores dependent on the replaced evaluator are discarded and affected agents are re-evaluated when revisited. The loop carries forward an improved evaluator and agent code while resetting incompatible evaluation records. On held-out Polyglot coding tasks, the preprint reports a 71.7% pass rate against 69.9% for HGM-H, with lower search-token use. The anchor, replacement schedule, and orchestration remain externally fixed. These controls preserve a reference against which evaluator changes can be judged; the formal stability argument applies within each frozen epoch, not to unrestricted changes of objective.

3.6.3 Revising Research Goals and Policies

The AI uses accumulated experimental evidence to revise what later research should pursue. Choosing a higher target is difficult when the available proxy rewards progress on the wrong bottleneck. In model post-training, testing that hypothesis requires comparing training runs, identifying failed search directions, and deciding whether to revise the research policy itself. A changing curriculum reaches L5 when its persistent

原文第 33 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.6.3

表 7 L5 机制和外部控制。

工作	循环结束于	继承状态	外部控制
搜索与自我修改			
STOP [15]	下一个程序搜索	改进代码	实用程序；基础LM；预算
哥德尔智能体[19]	下一步自我修订	任务及更新代码	任务目标；运行时访问
DGM [16]	后代搜索	智能体代码；存档	家长选择；标杆
HyperAgents [93]	下一代智能体生成	任务和元智能体代码	主要研究选择；评价
继任者评价			
RQGM [21]	下一个纪元选择	评估器；智能体代码	锚点；更换时间表
研究策略和工具	ses		
A-Evolve-Training [17]	下一轮研究	搜索策略；发现日志	宪法；基准;基材
AIRA2 [154] / AAR [155]	任务实验	研究工件	研究 运行支撑框架（Harness）;评价
AIDE2 [94]	后期研究运行	研究智能体 运行支撑框架（Harness）	私人分数；成本预算

注意： L5 声明在文本中是合格的。

Gödel Agent [19] 使 AI 能够访问共享 Python 程序中的任务策略和递归更新逻辑。执行反馈通知重写，修改后的程序在下一次递归调用中运行，包括其修改后的自更新过程。这将搜索范围扩大到开发人员固定的编辑例程之外。它还使错误变得可遗传：研究报告称，100 次 MGSM 优化试验中有 14 次的结果低于初始策略的性能，并且无限制的运行可以调用更强的模型。因此，任务目标和执行权限仍然是重要的外部控制。

达尔文哥德尔机（DGM）[16]保留了编码智能体变体并探索其后代，但将档案管理和父代选择保留在自我修改之外。它的血统提供了一个过渡案例；任务收益本身并不能建立更好的改进程序。 HyperAgents [93] 直接公开任务智能体和元智能体代码以供修订。进化的元智能体构建性能跟踪器和持久内存来指导以后的修改。在其预印本中，元智能体在论文审阅中不断发展，机器人技术为以前未见过的数学评分生成了改进的智能体，支持改进程序本身的转移。较长的 200 次迭代实验并未为转移初始化建立具有统计显着性的最终优势。亲本选择和评估在主要实验中也保持固定。

3.6.2 改进继任者评估

人工智能改进了确定保留哪些继任者的评估器。反复的优化给法官的盲点带来了压力。保持该判断不变可能会支持利用其错误的提案。不受限制的裁判修改会产生相反的问题：连续的分数可能反映出标准的变化。

红皇后哥德尔机（RQGM）[21]通过评估者共同进化解决了这种紧张关系。它的学习评估器仍然冻结在一个时期内。在预定的边界，挑战者评估者与独立的地面实况锚进行比较；然后，选定的评估者将管理下一个纪元。取决于替换的评估者的分数将被丢弃，并且在重新访问时重新评估受影响的智能体

。该循环推进改进的评估器和智能体代码，同时重置不兼容的评估记录。在保留的多语言编码任务中，预印本报告的通过率为 71.7%，而 HGM-H 的通过率为 69.9%，搜索令牌的使用率较低。锚点、替换时间表和编排仍然是外部固定的。这些控制保留了可以判断评估者更改的参考；正式的稳定性论证适用于每个冻结时期，而不是目标的无限制变化。

3.6.3 修改研究目标和策略

人工智能使用积累的实验证据来修改后续研究应该追求的内容。当可用智能体奖励错误瓶颈上的进展时，选择更高的目标是很困难的。在模型后训练中，测试该假设需要比较训练运行，识别失败的搜索方向，并决定是否修改研究策略本身。当持续变化的课程达到 L5 时

33

selection procedure is also revised and reused.

A-Evolve-Training [17] provides a concrete case. Its preprint reports autonomous post-training of a 30B model across four rounds. Workers return recipe changes, checkpoint evaluations, and failures; a collector consolidates these results, and a meta-agent revises the next round's search policy. The policy carries a standing recipe, promoted or retired search directions, and a registry of unsuccessful approaches. When development scores increased without corresponding external gains, the revised policy directed workers toward interventions that could improve the external target despite lowering the proxy. The study reports a final leaderboard score of 0.86, compared with 0.87 for the top human submission. Cross-round inheritance resides in the research policy and discovery log; the underlying worker substrate and constitution remain fixed. The case supports policy-level L5 within a human-defined objective, with no demonstrated autonomous revision of that objective.

A spiral of improvement emerges when an inherited procedure helps expose a new limitation and is revised again to address it. The next goal should identify a testable capability gap, preserve established capabilities, and justify further expenditure. Goal revision alone supplies limited evidence: the evaluation must establish that the revised policy changes subsequent research and improves its outcomes.

3.6.4 Industrial Practice

Industrial implementations expose different portions of the improvement process to revision. Their relevance to L5 depends on whether the retained artifact changes later research decisions.

Research infrastructure. Meta's AIRA_2 [154] uses asynchronous experimentation and hidden consistent evaluation; Anthropic's Automated Alignment Researchers [155] generate and test mitigations for specified alignment failures. These systems automate experiment execution within researcher-designed workflows. Their reported results concern research outputs, without establishing inherited changes to the procedures conducting the research.

Research-agent revision. Weco's AIDE^2 [94] applies a research agent to improving a research-agent harness. Candidate harnesses run downstream tasks under private scoring and a fixed cost budget. Accepted versions are retained for subsequent research; a separate experiment installs an evolved harness as the outer improver. Weco reports seven accepted improvements over 100 unattended steps and transfer to external tasks. Its stronger test of whether the evolved agent improves the outer search faster found no statistically significant efficiency advantage. These company-reported results support bounded harness improvement. Private evaluation, task families, and cost constraints remain human-designed, while reliable acceleration of successive improvers remains unestablished.

Across these configurations, the potential saving comes from reusing a better research procedure. Evidence of reduced human development effort requires accounting for the work spent designing evaluators, investigating failures, and maintaining the evolved code. Task performance or unattended runtime alone cannot quantify that saving.

3.6.5 Evaluating L5 Recursive Improvement

Evaluation must distinguish a stronger current agent from a procedure that reliably produces stronger successors. SEA-Eval [156] and SEAGym [157] contribute trajectory-based assessment: performance across rounds, held-out transfer, retention, and resource use. Table 8 summarizes these measurements and the additional mechanism tests needed for L5.

A mechanism audit should identify the revised artifact, record the evidence that motivated it, and verify its invocation in a later improvement round. To establish effectiveness, the original and revised mechanisms should start from comparable agents and evidence under matched budgets, including the cost of evaluating the mechanisms. Holding the revised mechanism fixed during transfer tests helps isolate what it learned about improvement. HyperAgents [93] uses this design to distinguish transferable agent-generation ability from performance on the original task.

Goal selection needs complementary evaluation. Adaptive frontier tasks can test whether the system identifies

原文第 34 页 | 中文翻译初稿

3 跨自治级别的 RSI

3.6.5
选择程序也经过修改并重新使用。

A-Evolve-Training [17] 提供了一个具体案例。其预印本报告了 30B 模型四轮的自主后训练。工人返回配方变更、检查点评估和失败；收集器整合这些结果，元智能体修改下一轮的搜索策略。该策略包含一个常设秘诀、晋升或退休的搜索方向以及不成功方法的登记册。当发展分数增加而没有相应的外部收益时，修订后的策略会引导工人采取干预措施，尽管降低了智能体值，但仍可以改善外部目标。该研究报告的最终排行榜得分为 0.86，而人类提交的最高得分为 0.87。跨轮继承存在于研究策略和发现日志中；底层工人的基础和构成保持不变。该案例支持人类定义的目标中的策略级 L5，但没有证明该目标的自主修订。

当继承的程序有助于暴露新的限制并再次修改以解决它时，就会出现螺旋式改进。下一个目标应该确定可测试的能力差距，保留现有的能力，并证明进一步支出的合理性。目标修订本身提供的证据有限：评估必须确定修订后的策略改变了后续研究并改善了其结果。
3.6.4 产业实践

工业实施将改进过程的不同部分暴露给修订。它们与 L5 的相关性取决于保留的工件是否会改变后来的研究决策。

研究基础设施。 Meta 的 AIRA2 [154] 使用异步实验和隐藏一致评估； Anthropic 的自动对齐研究人员 [155] 生成并测试特定对齐失败的缓解措施。这些系统在研究人员设计的工作流程中自动执行实验。他们报告的结果涉及研究成果，但没有对进行研究的程序进行继承性的改变。

研究智能体修订。 Weco 的 AIDE2 [94] 应用研究智能体来改进研究智能体 运行支撑框架（Harness）。候选人 运行支撑框架（Harnesses） 在私人评分和固定成本预算下运行下游任务。接受的版本将被保留以供后续研究；一个单独的实验安装了进化的 运行支撑框架（Harness） 作为外部改进器。 Weco 报告了超过 100 个无人值守步骤的 7 项已接受改进，并转移到外部任务。它对进化智能体是否更快地改进外部搜索进行了更强有力的测试，发现没有统计上显着的效率优势。这些公司报告的结果支持有限的 运行支撑框架（Harness） 改进。私人评估、任务族和成本限制仍然是人为设计的，而连续改进的可靠加速仍然没有建立。

在这些配置中，潜在的节省来自于重用更好的研究程序。人类开发工作减少的证据需要考虑设计评估器、调查失败和维护进化代码所花费的工作。仅靠任务性能或无人值守运行时间无法量化节省的成本

。
3.6.5 评估L5递归改进

评估必须将更强的当前智能体人与可靠地产生更强的继任者的程序区分开来。 SEA-Eval [156] 和 SEAGym [157] 提供基于轨迹的评估：各轮绩效、保留转移、保留和资源使用。表 8 总结了这些测量以及 L5 所需的附加机制测试。

机制审计应该识别修订后的工件，记录激发它的证据，并在以后的改进轮中验证其调用。为了建立有效性，原始和修订后的机制应从匹配预算下的可比智能体人和证据开始，包括评估机制的成本。在转移测试期间固定修改后的机制有助于隔离它所学到的改进知识。 HyperAgents [93] 使用这种设计来区分可转移智能体生成能力和原始任务的性能。

目标选择需要补充评估。自适应前沿任务可以测试系统是否识别

34

Table 8 Evaluating recursive improvement.

Dimension	Measurements	Purpose
Adaptivity	Gain; improvement trajectory; time to target [156, 157]	Detect progress and plateaus
Retention	Replay loss; tasks fixed or broken [157]	Detect displaced capabilities
Transfer	Held-out gains within and across domains [157]	Test reuse beyond update tasks
Efficiency	Tokens; time; cost per validated gain [156, 157]	Account for improvement expense
Stability	Harmful updates; largest temporary decline [156, 157]	Expose unreliable trajectories
Meta-recursion	Mechanism reuse; successor quality [15, 21, 93]; goal and stopping decisions	Test inherited improvement capacity

Note: Trajectory measures synthesize the cited benchmarks. The meta-recursion row adds our L5 tests: cited systems illustrate mechanism reuse and improvement capacity; goal selection and stopping remain evaluation requirements.

useful next challenges; protected reference tasks preserve comparability and expose forgetting or reward hacking. Private tasks with previously undisclosed rules strengthen tests of discovery beyond familiar distributions. Evaluations should also record whether the system stops when expected benefit falls below cost or risk. These are proposed requirements for assessing fuller L5 autonomy, rather than capabilities established by existing RSI benchmarks.

Current results support bounded meta-improvement: accepted changes to improvers, evaluators, or research policies can govern subsequent rounds, and some changes transfer across tasks [15, 21, 93]. Statistically reliable accumulation across generations under comparable resources remains open [93, 94]. The practical objective is to increase validated improvement per unit of computation and human effort. Strong base models, execution infrastructure, and independent evaluation continue to supply the resources and constraints under which that improvement occurs.

Finding: L5 makes the improvement procedure inheritable. The loop closes through reuse of a revised improver, evaluator, or research policy. Successors inherit that procedure and its supporting evidence; humans retain the overall objective, protected acceptance criteria, and resource authority.

3.7 Cross-Level Synthesis

Across B0–L5, the autonomy hierarchy can be understood as a progressive transfer of responsibility over the improvement loop. The levels differ not primarily in the particular algorithm or artifact being optimized, but in which improvement decisions are internalized by the AI system, what state persists into later rounds, and which acceptance conditions remain externally protected.

The central boundaries between adjacent levels can therefore be stated compactly. The transition from B0 to L1 is **persistence**: an accepted change must survive the current task. L1 to L2 transfers **strategy selection**: AI decides which intervention to attempt rather than merely executing a prescribed one. L2 to L3 adds control over the **future learning agenda**, so the evolving learner influences what experience is acquired next. L3 to L4 extends this feedback process into **persistent deployment and environmental interaction**, where system changes must remain useful under changing operational conditions. L4 to L5 finally introduces **recursive inheritance**, in which the mechanism responsible for later improvement itself becomes an inherited target of improvement.

Importantly, a higher autonomy level does not by itself imply a better improvement process. Greater delegated authority can coexist with inefficient search, unreliable feedback, regression, evaluator exploitation, or poor transfer. We therefore separate the scope of improvement responsibility internalized by AI from the quality and efficiency of the resulting improvement. Across the evidence reviewed here, lower and intermediate levels are supported by a comparatively broad range of systems, whereas L3–L4 capabilities remain more domain-dependent and end-to-end L5 evidence is concentrated in bounded prototypes and emerging industrial or research systems. This distinction motivates the application-specific analysis in the following section.

原文第 35 页 | 中文翻译初稿

3 跨自治级别的 RSI

4

表 8 评估递归改进。

尺寸	测量	用途
适应性	增益；改进轨迹；达到目标的时间 [156, 157]	检测进度和停滞状态
保留	重播丢失；任务已修复或已损坏 [157]	检测移位的功能
传输	域内和跨域的保留收益[157]	更新任务之外的测试重用
效率	代币；时间;每次验证收益的成本 [156, 157]	改进费用的核算
稳定性
元递归	有害更新；最大暂时下降[156, 157]
机制重用；后继者质量 [15, 21, 93]；目标和停止
决策	暴露不可靠的轨迹
测试继承的改进能力

注： 轨迹测量综合了引用的基准。元递归行添加了我们的 L5 测试：引用的系统说明了机制重用和改进能力；目标选择和停止仍然是评估要求。

接下来的挑战很有用；受保护的参考任务可以保留可比性并暴露遗忘或奖励黑客行为。具有先前未公开规则的私人任务加强了对熟悉分布之外的发现的测试。评估还应记录当预期收益低于成本或风险时系统是否停止。这些是评估更全面的 L5 自主权的拟议要求，而不是现有 RSI 基准建立的功能。

目前的结果支持有界的元改进：改进者、评估者或研究策略的可接受的更改可以管理后续轮次，并且一些更改可以跨任务转移[15,21,93]。在可比资源下，跨代的统计上可靠的积累仍然是开放的[93, 94]。实际目标是增加每单位计算和人力的经过验证的改进。强大的基础模型、执行基础设施和独立评估继续提供实现改进的资源和约束。

发现：L5使得改进过程可继承。通过重用修订后的改进器、评估器或研究策略来关闭循环。继任者继承该程序及其支持证据；人类保留总体目标、受保护的接受标准和资源权威。
3.7 跨级综合

在 B0-L5 中，自主层级可以理解为改进循环中责任的逐步转移。这些级别的不同主要不在于正在优化的特定算法或工件，而在于人工智能系统将哪些改进决策内化，哪些状态持续到后续轮次，以及哪些接受条件仍然受到外部保护。

因此，可以简洁地表述相邻层之间的中心边界。从 B0 到 L1 的过渡是持久性：已接受的更改必须在当前任务中继续存在。 L1 到 L2 转移策略选择：人工智能决定尝试哪种干预，而不仅仅是执行规定的干预。L2到L3增加了对未来学习议程的控制，因此不断发展的学习者会影响接下来获得的经验。

。 L3 到 L4 将此反馈过程扩展为持久部署和环境交互，其中系统更改必须在不断变化的操作条件下保持有用。 L4到L5最终引入了递归继承，其中负责后期改进的机制本身成为继承的改进目标。

重要的是，更高的自主级别本身并不意味着更好的改进过程。更大的授权可能与低效的搜索、不可靠的反馈、回归、评估者剥削或不良的转移共存。因此，我们将人工智能内化的改进责任范围与由此产生的改进

的质量和效率分开。纵观此处审查的证据，较低和中级水平得到相对广泛的系统的支持，而 L3-L4 能力仍然更加依赖于领域，而端到端 L5 证据集中在有界原型和新兴工业或研究系统中。这种区别激发了下一节中特定于应用程序的分析。

35

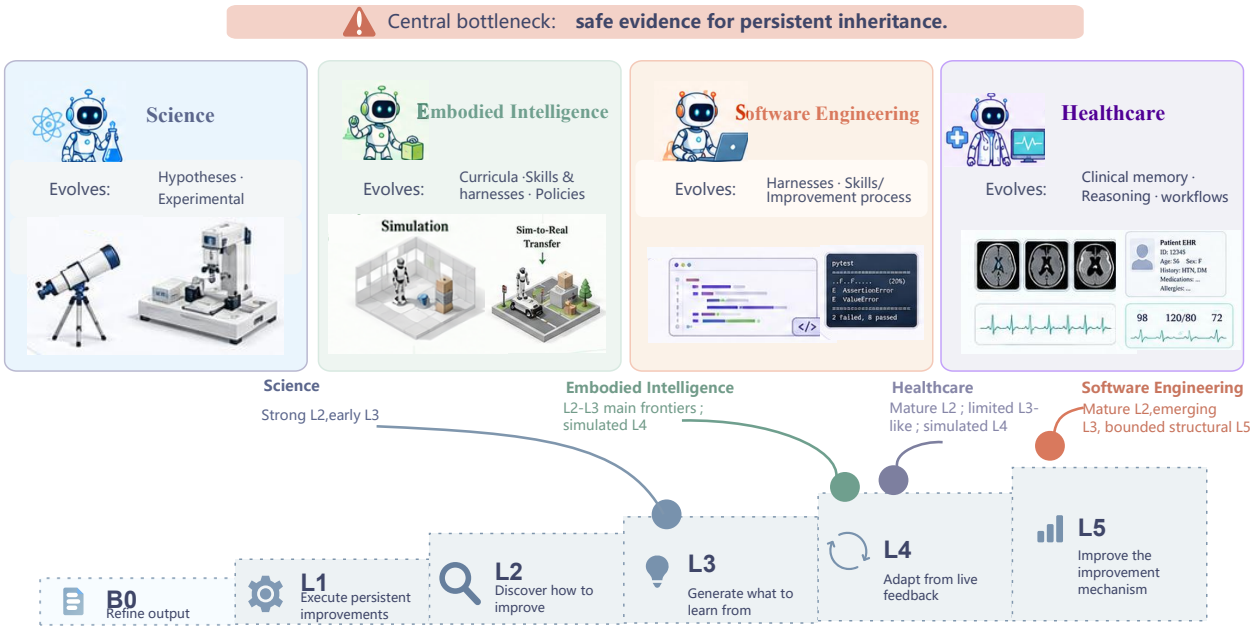


Figure 9 Application regimes differ mainly in the kind of feedback needed to validate and inherit an improvement.

4 RSI Across Applications

Building on the B0–L5 taxonomy, this section summarizes how recursive self-improvement is instantiated under four domain-specific feedback regimes, including science, embodied intelligence, software engineering, and healthcare. Across these regimes, repeated refinement of an external artifact is insufficient: the feedback must produce a persistent change to the AI system that is reused in later tasks. Current evidence is therefore strongest for bounded, component-level improvement, whereas adaptation of the improvement mechanism itself remains rare. A detailed review of the evolving components, supporting evidence, and remaining limitations in each application is provided in Appendix **??**.

4.1 S1: RSI for Science

Science is a consequential setting for RSI because progress depends not only on solving individual problems, but also on improving how future hypotheses are generated, tested, and revised. Its characteristic loop uses experimental evidence or falsification to update the AI4Sci system: a research agent proposes hypotheses, executes computational or physical experiments, attributes the observed outcomes, and retains validated changes to its generation strategy, tools, skills, memory, or scaffold. HypoForge [158], for example, distills critiques and execution outcomes into reusable skills for hypothesis generation and testing. Test-Time Tool Evolution [159] synthesizes and validates executable scientific tools for reuse, while DrugSAGE [160] transfers verified modeling procedures and error-recovery strategies across tasks. SIA [161] extends the editable surface to agent scaffolds, tools, search logic, and model weights. These systems demonstrate persistent improvement of scientific-agent components, but their evaluators, update-selection rules, and promotion criteria remain largely fixed; improving a molecule, equation, or hypothesis alone therefore remains RSI-adjacent rather than evidence that the scientific agent has improved itself.

4.2 S2: RSI for Embodied Intelligence

Embodied intelligence grounds improvement in physical interaction and causal feedback. An embodied RSI loop selects or generates experience, executes a policy, evaluates the resulting trajectory, and commits validated updates to the policy, world model, skill library, harness, or curriculum used in subsequent interactions. POET [65] co-evolves environments and agents so that newly generated challenges expose and extend current capabilities. Voyager [84] converts successful behaviors into an executable skill library that supports later

原文第 36 页 | 中文翻译初稿

4 RSI 跨应用

4.2

中心瓶颈：持久继承的安全证据。
科学 具身智能（Embodied Intelligence） 软件工程医疗保健
发展：假设·实验发展：课程·运行支撑框架（Harnesses）·技能和策略发展：改进过程驾驭·技能/发展：推理·临床记忆·工作流程
科学具身智能（Embodied Intelligence） 医疗软件工程
强L2，早期L3 L2-L3主要前沿；模拟 L4 成熟 L2 ；有限的 L3 样；模拟 L4 成熟 L2、新兴 L3、有界结构 L5
L5
B0 细化输出 L1 执行持续改进 L2 探索如何改进 L3 生成要学习的内容 L4 适应实时反馈 改进改进机制

图 9 应用机制的不同之处主要在于验证和继承改进所需的反馈类型。
4 跨应用的 RSI

本节以 B0-L5 分类法为基础，总结了 递归自我改进（Recursive Self-Improvement） 如何在四个特定领域的反馈机制下实例化，包括科学、具身智能（Embodied Intelligence）、软件工程和医疗保健。在这些机制中，对外部工件的重复细化是不够的：反馈必须对人工智能系统产生持久的变化，并在以后的任务中重用。因此，目前的证据对于有界的、组件级的改进是最有力的，而改进机制本身的适应仍然很少。附录？？中提供了对每个应用程序中不断发展的组件、支持证据和剩余限制的详细审查。 。
4.1 S1：科学 RSI

科学是 RSI 的重要背景，因为进步不仅取决于解决个别问题，还取决于改进未来假设的生成、测试和修改方式。其特征循环使用实验证据或伪造来更新 AI4Sci 系统：研究智能体提出假设，执行计算或物理实验，对观察到的结果进行归因，并保留对其生成策略、工具、技能、记忆或支架的经过验证的更改。例如，HypoForge [158] 将批评和执行结果提炼成可重用的技能，用于假设生成和测试。 Test-Time Tool Evolution [159] 综合并验证可执行的科学工具以供重用，而 DrugSAGE [160] 跨任务传输经过验证的建模程序和错误恢复策略。 SIA [161] 将可编辑表面扩展到智能体支架、工具、搜索逻辑和模型权重。这些系统展示了科学智能体组件的持续改进，但它们的评估者、更新选择规则和升级标准在很大程度上仍然是固定的；因此，单独改进分子、方程或假设仍然是 RSI 邻近的，而不是科学智能体人已经改进自身的证据。

4.2 S2：具身智能（Embodied Intelligence） 的 RSI

具身智能（Embodied Intelligence） 为物理交互和因果反馈的改进奠定了基础。体现的 RSI 循环选择或生成经验，执行策略，评估生成的轨迹，并向策略、世界模型（World Model）、技能库、运行支撑框架（Harness） 或后续交互中使用的课程提交经过验证的更新

。 POET [65] 共同进化环境和智能体，以便新产生的挑战暴露并扩展当前的能力。 Voyager[84]将成功的行为转化为可执行的技能库，支持后续

36

exploration, and World-VLA-Loop [162] alternates between improving a world model from policy failures and optimizing the policy in the refined model. Together, these works move beyond within-episode correction toward persistent agent–environment co-adaptation. However, embodied experience is endogenous to the current policy, failures are difficult to attribute across perception, planning, control, and hardware, and real-world trials are costly and unsafe to reset. Most reported loops consequently rely on simulation or externally specified rewards, evaluators, interfaces, and safety constraints, leaving autonomous real-world validation and evolution of the embodied improver unresolved.

4.3 S3: RSI for Software Engineering

Software engineering offers unusually direct feedback because both the developed artifact and the coding agent can be represented, executed, versioned, and rolled back as code. Its RSI loop proceeds from a development challenge to an agent-level modification, repository-grounded execution, regression-aware selection, and versioned inheritance. SWE-Exp [60] extracts lessons from successful and failed issue-resolution trajectories for reuse across repositories. Self-Harness [23] uses execution traces to propose modifications to its own harness and retains them only after regression testing. Darwin Gödel Machine [16] broadens this process by maintaining lineages of self-modifying coding agents and selecting variants according to executable task performance, thereby allowing accepted descendants to become the starting points for further improvement. These systems provide some of the clearest evidence of persistent source-, harness-, and experience-level evolution. Nevertheless, repository tests are incomplete proxies for user intent, security, maintainability, and descendant quality; benchmarks, utility functions, sandboxes, and selection rules also remain externally defined. Software-engineering RSI is therefore comparatively auditable, but still bounded by a fixed outer improvement contract.

4.4 S4: RSI for Healthcare

Healthcare requires RSI to be grounded in longitudinal feedback from experts, clinical guidelines, diagnostic evidence, and patient outcomes. A clinical RSI loop performs a task, obtains validated feedback, updates a persistent component such as memory, reasoning policy, tool repertoire, or workflow, and reuses the approved update across later cases. MedAgent-Zero in Agent Hospital [163] stores successful cases and derives reusable diagnostic rules from failures. EvoClinician [164] uses process-level grading to revise how an agent acquires and integrates diagnostic evidence, while MACRO [165] composes verified medical-imaging trajectories into reusable tools. HealthFlow [166] similarly retains validated safeguards, workflows, and code from completed health-data analyses. These approaches show how clinical experience can improve future agent behavior without necessarily changing foundation-model weights. Their evidence, however, remains dominated by simulated or retrospective settings, while real outcomes are delayed, heterogeneous, and confounded. Persistent clinical updates must therefore remain provenance-aware, population-specific, prospectively monitored, reversible, and subject to expert and institutional approval; autonomous evolution of the clinical improver is not yet established.

5 Industry Landscape and Preliminary Practices

5.1 Theseus: Environment–Data–Model Co-Evolution for RSI Intelligence

Theseus positions environment–data–model co-evolution as a path toward next-generation RSI intelligence. Its central premise is that environments make knowledge accessible and actions verifiable, task execution produces evidence for learning, and improved models expand the range of problems that can be explored. Under this vision, each round of work should contribute reusable capabilities to subsequent rounds, making real-world practice a continuing source of intelligence growth.

As shown in Figure 10, Theseus realizes this vision in four steps. First, refinement experience is converted into environment-refinement tasks to train an environment-refinement model. Second, agents use reconstructed environments to identify genuine task-solving difficulties and generate targeted training data. Third, these data train the task model to improve its capabilities. Fourth, the stronger model supports further environment iteration and the generation of more tasks. Subsequent rounds yield both new task-training data and new

原文第 37 页 | 中文翻译初稿

5 行业格局及初步实践

5.1

探索，World-VLA-Loop [162]在从策略失败中改进 世界模型（World Model） 和优化细化模型中的策略之间交替。总之，这些工作超越了情节内的纠正，转向持久的主体与环境的共同适应。然而，具体经验对于当前策略来说是内生的，失败很难归因于感知、规划、控制和硬件，而且现实世界的试验成本高昂且重置不安全。因此，大多数报告的循环都依赖于模拟或外部指定的奖励、评估器、接口和安全约束，从而使现实世界的自主验证和具体改进器的演化尚未解决。

4.3 S3：软件工程的 RSI

软件工程提供了异常直接的反馈，因为开发的工件和编码智能体都可以作为代码来表示、执行、版本控制和回滚。其 RSI 循环从开发挑战到智能体级修改、基于存储库的执行、回归感知选择和版本化继承。SWE-Exp [60] 从成功和失败的问题解决轨迹中提取经验教训，以便跨存储库重用。 Self-运行支撑框架（Harness） [23] 使用执行跟踪来建议对其自己的 运行支撑框架（Harness） 进行修改，并仅在回归测试后保留它们。达尔文哥德尔机[16]通过维护自修改编码智能体的谱系并根据可执行任务性能选择变体来拓宽这一过程，从而允许接受的后代成为进一步改进的起点。这些系统提供了持久源、运行支撑框架（Harness） 和经验级别演化的一些最清晰的证据。然而，存储库测试并不能完全代表用户意图、安全性、可维护性和后代质量；基准、效用函数、沙箱和选择规则也仍然是外部定义的。因此，软件工程 RSI 相对可审计，但仍受固定的外部改进合同的约束。

4.4 S4：医疗保健领域的 RSI

医疗保健要求 RSI 以专家的纵向反馈、临床指南、诊断证据和患者结果为基础。临床 RSI 循环执行任务、获取经过验证的反馈、更新持久组件（例如内存、推理策略、工具库或工作流程），并在以后的案例中重用已批准的更新。 Agent Hospital [163] 中的 MedAgent-Zero 存储成功案例并从失败中导出可重用的诊断规则。 EvoClinician [164] 使用过程级分级来修改智能体获取和整合诊断证据的方式，而 MACRO [165] 将经过验证的医学成像轨迹组合成可重用的工具。 HealthFlow [166] 同样保留了来自已完成的健康数据分析的经过验证的保障措施、工作流程和代码。这些方法展示了临床经验如何能够改善未来的智能体行为，而不必改变基础模型权重。然而，他们的证据仍然以模拟或回顾性环境为主，而真实结果是延迟的、异质的和混乱的

。因此，持续的临床更新必须保持来源意识、针对特定人群、前瞻性监测、可逆，并得到专家和机构的批准；临床改良剂的自主进化尚未建立。

5 行业格局及初步实践

5.1Theseus：RSI 智能的环境-数据-模型协同演化

Theseus 将环境-数据-模型协同进化定位为通向下一代 RSI 智能的道路。其核心前提是环境使知识可获取、行动可验证、任务执行为学习提供证据、改进的模型扩大了可探索的问题范围。在这一愿景下，每一轮工作都应该为后续轮次提供可重用的能力，使现实世界的实践成为智力增长的持续源泉。

如图 10 所示，Theseus 通过四个步骤实现了这一愿景。首先，将细化经验转化为环境细化任务来训练环境细化模型。其次，智能体使用重建的环境来识别真正的任务解决困难并生成有针对性的训练数据。第三，这些数据训练任务模型以提高其能力。第四，更强的模型支持进一步的环境迭代和更多任务的生成。随后的几轮会产生新的任务训练数据和新的

37

Table 9 Early workspace experiments in Theseus's environment stage. **(a)** Clean-versus-noise pilot: pass rates (%) of eight frontier model–harness configurations on 30 workspace tasks with 1,280 rubrics, comparing a clean workspace against a noise-laden workspace. **(b)** Productivity study: rubric scores of five fixed harness–model pairings on 30 tasks with 547 rubrics, comparing the bare workspace against the reconstructed environment (a shared package of a Collection Map and an Event Log); "Env. tasks" reports wins/draws/losses at the task level, and "Rubric flips" counts rubrics that changed from fail to pass versus pass to fail.

(a) Clean-versus-noise pilot — clean vs. noise-laden workspace

Model	Harness	Clean	Noise	Δ (pp)
DeepSeek-V4-Pro	DSH	98.2	46.6	+51.6
DeepSeek-V4-Flash	DSH	90.1	63.7	+26.4
GPT-5.6-Sol	Codex	89.1	54.1	+35.1
GLM-5.3	ClaudeCode	88.8	57.2	+31.6
Kimi-K3	ClaudeCode	87.6	53.7	+33.9
Grok-4.6	ClaudeCode	84.7	63.0	+21.7
GPT-5.6-Luna	Codex	84.6	38.1	+46.5
Muse-Spark-1.2	ClaudeCode	84.5	53.3	+31.2

(b) Productivity-workspace study — bare vs. reconstructed environment

Harness	Model	Bare	Reconstructed	Δ (pp)	Env. tasks	Rubric flips
Codex CLI	GPT-5.6-Sol	60.51%	92.50%	+31.99	24/3/3	190/15
Codex CLI	DeepSeek-V4-Flash	54.30%	84.83%	+30.53	24/3/3	201/34
Claude Code	DeepSeek-V4-Flash	61.06%	79.71%	+18.65	23/2/5	140/38
PI	DeepSeek-V4-Flash	62.52%	89.03%	+26.51	22/6/2	155/10
PI	GPT-5.6-Sol	50.27%	89.95%	+39.67	23/2/5	240/23

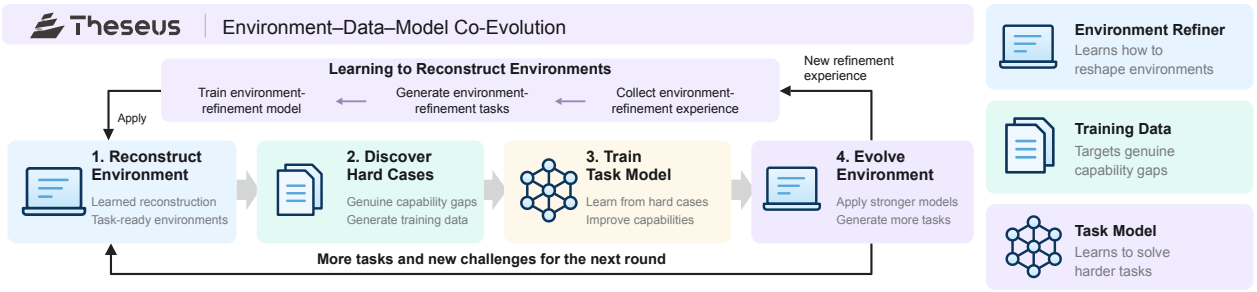


Figure 10 Theseus's proposed four-stage co-evolution loop: reconstruct environments, uncover genuine capability gaps and generate training data, train task models, and iterate environments to produce more tasks. The upper loop learns an environment-refinement model from refinement experience and generated tasks, while successive iterations supply new experience for further refinement.

environment-refinement experience, linking progress in solving tasks to progress in constructing the conditions for future learning.

Early results provide an initial empirical basis for this direction in the workspace stage of the loop. A pilot with eight frontier model–harness configurations on 30 workspace tasks adapted from Workspace-Bench [167], scored by 1,280 rubrics, first measured sensitivity to workspace quality: relative to a noise-laden workspace, a clean workspace raised pass rates by 21.7–51.6 percentage points across all eight configurations (Table 9a), suggesting that workspace state, rather than model capability alone, can bound agent performance. Building on this observation, a follow-up productivity study asked whether enriching the environment improves task performance while keeping the underlying model and harness fixed: five fixed model–harness pairings solved 30 tasks scored by 547 rubrics in total under a bare workspace and a reconstructed environment consisting of a Collection Map and an Event Log; the reconstructed environment raised aggregate rubric scores by 18.65–39.67 percentage points across all tested configurations (Table 9b), although a few tasks showed

原文第 38 页 | 中文翻译初稿

5 行业格局及初步实践

5.1

表 9 忒修斯环境阶段的早期工作空间实验。 (a) 清洁与噪音试点：8 个前沿模型 - 运行支撑框架（Harness） 配置在 30 个工作空间任务（1,280 个量规）上的通过率 (%)，将清洁工作空间与充满噪音的工作空间进行比较。 (b) 生产力研究：五个固定 运行支撑框架（Harness） 模型配对的评分标准，涉及 30 个任务和 547 个评分标准，将裸工作区与重建环境（集合映射和事件日志的共享包）进行比较； “Env.tasks”报告任务级别的胜利/平局/失败，“Rubric Flips”统计从失败到通过与通过到失败的变化。
(a) 清洁与噪音试点--清洁与充满噪音的工作空间

型号	运行支撑框架（Harness）	清洁	噪音	Δ(pp)
DeepSeek-V4-Pro	DSH	98.2	46.6	+51_._6
DeepSeek-V4-Flash	DSH	90.1	63.7	+26_._4
GPT-5.6-Sol	法典	89.1	54.1	+35_._1
GLM-5.3	克劳德代码	88.8	57.2	+31_._6
Kimi-K3	克劳德代码	87.6	53.7	+33_._9
Grok-4.6	克劳德代码	84.7	63.0	+21_._7
GPT-5.6-Luna	法典	84.6	38.1	+46_._5
Muse-Spark-1.2	克劳德代码	84.5	53.3	+31_._2

(b) 生产力-工作空间研究--裸露环境与重建环境

运行支撑框架（Harness）	模型	裸露	重建	Δ(pp)	环境。任务	标题翻转
Codex CLI	GPT-5.6-Sol	60.51%	92.50%	+31_._99	24/3/3	190/15
Codex CLI	DeepSeek-V4-Flash	54.30%	84.83%	+30_._53	24/3/3	201/34
克劳德代码	DeepSeek-V4-Flash	61.06%	79.71%	+18_._65	23/2/5	140/38
PI	DeepSeek-V4-Flash	62.52%	89.03%	+26_._51	22/6/2	155/10
PI	GPT-5.6-Sol	50.27%	89.95%	+39_._67	23/2/5	240/23

环境-数据-模型协同进化环境精炼器
学习如何
学习重构环境新的精炼经验重塑环境
训练环境-生成环境-收集环境-
精炼模型精炼任务精炼经验
应用
训练数据
1.重建 2.发现 3.训练 4.进化 目标 真实
环境 困难案例 任务模型 环境能力差距
学习重建 真正的能力差距 从困难案例中学习 应用更强的模型
任务就绪环境 生成训练数据 提高能力 生成更多任务

任务模型
下一轮的更多任务和新挑战 学习解决
更困难的任务

图 10Theseus 提出的四阶段协同进化循环：重建环境、发现真正的能力差距并生成训练数据、训练任务模型以及迭代环境以产生更多任务。上层循环从细化经验和生成的任务中学习环境细化模型，而连续迭代为进一步细化提供新的经验。

环境改进经验，将解决任务的进展与构建未来学习条件的进展联系起来。

早期结果为循环工作空间阶段的这一方向提供了初步的经验基础

。在 30 个工作空间任务上使用 8 个前沿模型的 运行支撑框架（Harness） 配置改编自 Workspace-Bench [167]，按 1,280 个评分标准进行评分，首先测量了对工作空间质量的敏感性：相对于充满噪音的工作空间，干净的工作空间将所有 8 个配置的通过率提高了 21.7-51.6 个百分点（表 9a），这表明工作空间状态（而不是单独的模型能力）可以限制智能体性能。基于这一观察，一项后续生产力研究询问丰富环境是否可以提高任务性能，同时保持底层模型和 运行支撑框架（Harness） 固定：五个固定模型 - 运行支撑框架（Harness） 配对在裸工作空间和由集合映射和事件日志组成的重建环境下解决了总共 547 个评分标准的 30 项任务；尽管一些任务表明，重建的环境将所有测试配置的总评分提高了 18.65-39.67 个百分点（表 9b）

38

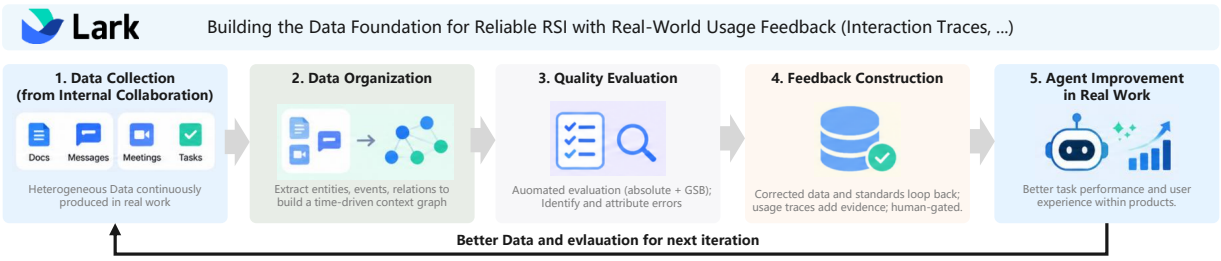


Figure 11 Lark's data-foundation loop for RSI, where collaboration data are continuously structured, evaluated, and refined into high-quality knowledge and training signals, while real-world agent usage feeds new evidence back into subsequent improvement cycles.

scope-creep regressions, where agents produced more extensive changes than the task required. The team has also completed training a reusable file-verification module. These early results mark the first steps of the co-evolution loop, and successive rounds will extend environment reconstruction, data generation, and model training into a sustained cycle of compounding improvement.

5.2 Lark: Building the Data Foundation for Reliable Enterprise-Level RSI

Figure 11 summarizes Lark's data-foundation loop. Lark approaches RSI from a prerequisite that is often overlooked: before an agent can improve itself, it needs a continuously evolving data substrate on which improvement can be grounded. In enterprise collaboration, documents, messages, meetings, and tasks continuously generate new information, while agents consume these data and produce new interaction traces. Lark therefore treats data production, quality evaluation, and failure attribution as the foundation of its RSI pipeline. The key objective is not merely to accumulate more data, but to ensure that each iteration has fresh experience to learn from, a reliable criterion for judging progress, and sufficiently fine-grained diagnostics to determine what should be changed.

Enterprise Knowledge Graph. A representative example is the construction and continual refinement of an enterprise knowledge graph. Information scattered across documents, messages, and meeting records is linked into entities, events, people, and temporal relations so that agents can answer questions requiring cross-source reasoning. Because graph errors directly propagate into downstream answers, graph construction is itself treated as an iterative data-quality problem. Lark addresses scalability through batched graph construction and delegated authorization checks, while continuously evaluating whether graph-based retrieval improves downstream question answering. In internal evaluations, it reports that human-rated task usability increased from 52% to 65%, while automated-evaluation usability increased from 47% to 56% when using its graph-based pipeline compared with the RAG-based baseline.

Automated Data-Quality Evaluation. Another example is an automated data-quality evaluation system that serves as the measurement layer for subsequent improvement. Rather than relying entirely on expensive and inconsistent manual review, Lark first applies absolute judgments to identify clearly unusable outputs and then uses comparative GSB evaluation to determine whether a new system variant should be promoted. Importantly, evaluation is coupled with explicit failure attribution. Bad cases are categorized into issues (e.g., temporal inconsistency, distorted queries, or poor source quality), so that each failure maps to a concrete improvement direction. For example, time-sensitive questions are constrained by query-time filtering to avoid using future information, and low-quality synthetic queries are rewritten before re-entering the evaluation pipeline. The initial automated evaluator achieved approximately 84% agreement with human judgments, where most disagreements arose because the automated evaluator was stricter than human reviewers, providing a concrete signal for subsequent calibration.

From an RSI perspective, the significance of these trials lies in the feedback structure they create for future agent improvement. Enterprise activity continuously produces new graph data and interaction traces, automated evaluation determines which outputs and data are reliable, error attribution identifies where the pipeline failed, and corrected data and evaluation standards then become inputs to later iterations. The

原文第 39 页 | 中文翻译初稿

5 行业格局及初步实践

5.2

利用真实世界的使用反馈（交互跟踪等）构建可靠的 RSI 数据基础

1. 数据收集 2. 数据组织 3. 质量评估 4. 反馈构建 5. 实际工作中的智能体改进
（来自内部协作）
异构数据不断提取实体、事件、关系进行自动评估（绝对+ GSB）；修正数据和标准环回； 更好的任务性能和用户在实际工作中产生的
构建时间驱动的上下文图识别和归因错误使用跟踪添加证据；人为门控。产品内的体验。
为下一次迭代提供更好的数据和评价

图 11 Lark 的 RSI 数据基础循环，其中协作数据被不断结构化、评估并提炼为高质量的知识和训练信号，而现实世界中的智能体使用情况则将新的证据反馈到后续的改进周期中。

范围蠕变回归，其中智能体产生比任务所需更广泛的变化。该团队还完成了可重用文件验证模块的训练。这些早期结果标志着协同进化循环的第一步，后续的循环将把环境重建、数据生成和模型训练延伸到复合改进的持续循环中。
5.2 Lark：构建可靠的企业级RSI数据基础

图 11 总结了 Lark 的数据基础循环。 Lark 从一个经常被忽视的先决条件出发来处理 RSI：在智能体能够自我改进之前，它需要一个不断发展的数据基础，作为改进的基础。在企业协作中，文档、消息、会议和任务不断生成新信息，而智能体则使用这些数据并产生新的交互轨迹。因此，Lark 将数据生产、质量评估和故障归因视为 RSI 管道的基础。关键目标不仅仅是积累更多数据，而是确保每次迭代都有新的经验可供学习、判断进度的可靠标准以及足够细粒度的诊断来确定应该更改的内容。

企业知识图谱。一个代表性的例子是企业知识图谱的构建和不断完善。分散在文档、消息和会议记录中的信息被链接到实体、事件、人员和时间关系中，以便智能体可以回答需要跨源推理的问题。由于图错误直接传播到下游答案，因此图构建本身被视为迭代数据质量问题。 Lark 通过批量图构建和委托授权检查来解决可扩展性，同时不断评估基于图的检索是否改善了下游问答。在内部评估中，它报告说，与基于 RAG 的基线相比，使用基于图形的管道时，人工评估任务可用性从 52% 增加到 65%，而自动评估可用性从 47% 增加到 56%。

自动化数据质量评估

。另一个例子是自动化数据质量评估系统，作为后续改进的测量层。 Lark 并不完全依赖昂贵且不一致的人工审核，而是首先应用绝对判断来识别明显不可用的输出，然后使用比较 GSB 评估来确定是否应该推广新的系统变体。重要的是，评估与明确的失败归因相结合。不良案例被分类为问题（例如，时间不一致、查询扭曲或源质量差），以便每个故障都映射到具体的改进方向。例如，对时间敏感的问题受到查询时间过滤的限制，以避免使用未来的信息，并且在重新进入评估管道之前重写低质量的综合查询。最初的自动评估器与人类判断的一致性约为 84%，其中大多数分歧的出现是因为自动评估器比人类评审员更严格，为后续校准提供了具体信号。

从 RSI 的角度来看，这些试验的意义在于它们为未来智能体改进创建的反馈结构。企业活动不断产生新的图形数据和交互跟踪，自动评估确定哪些输出和数据是可靠的，错误归因识别管道失败的位置，纠正后的数据和评估标准然后成为后续迭代的输入。的

39

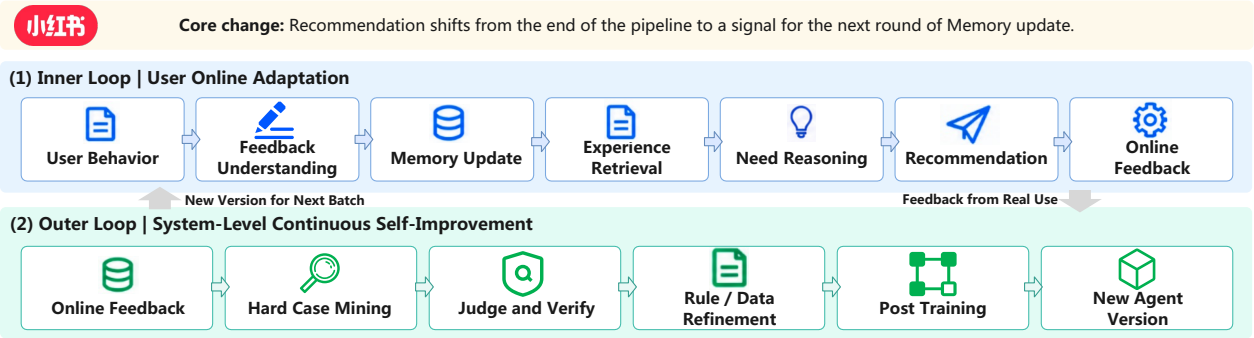


Figure 12 Xiaohongshu's dual-timescale RSI loop for recommendation. The Intent-Memory Agent (IMA) uses online behavioral feedback to update structured user memory and guide subsequent recommendations. At a slower timescale, hard cases undergo automated judging, human spot checks, and data reconstruction, supporting post-training updates that consolidate validated experience into agent parameters before redeployment.

Table 10 Performance comparison between the online baseline and the RSI model at Xiaohongshu.

Module	Metric	A: Online baseline	B: RSI model	Improvement
Discovery Feed	`score_mean@16`	2.211	2.366	+7.0%
Discovery Feed	`score_median@16`	1.587	1.663	+4.8%
In-video Feed	`score_mean@16`	1.537 ($n = 470$)	1.739 ($n = 447$)	$\approx +13.1\%^{*}$

resulting cycle turns production data from a passive resource into an evolving improvement substrate. At the same time, Lark's current practice remains deliberately human-gated: agents perform much of the repetitive data processing and evaluation, while humans retain responsibility for defining standards, reviewing critical samples, and approving important changes.

5.3 Xiaohongshu: Dual-Timescale RSI for Recommendation

Xiaohongshu's commercialization recommendation system explores feedback-driven self-improvement through an Intent-Memory Agent (IMA). Rather than treating user understanding as a one-off inference from historical behavior, IMA connects feedback interpretation, memory updating, experience retrieval, need reasoning, and recommendation into a continuous cycle. Subsequent clicks, dwell time, saves, searches, conversions, and negative feedback serve not only as new behavioral inputs, but also as observations against which earlier estimates of user state and intent are checked. These observations guide the addition, reinforcement, replacement, or decay of memory, as well as decisions to ignore uninformative evidence. The persistent improvement target is therefore the user representation that informs future recommendations, rather than only the current recommendation output.

Structured Memory and Experience Reuse. IMA organizes its intermediate state into three semantic layers: Content, describing the content topic; Who, representing the user's state; and Need, specifying the inferred requirement. These layers also structure knowledge-base storage and retrieval, allowing historical experience to be recalled through content similarity, user-state similarity, or need similarity. This decomposition is intended to reduce retrieval errors caused by mixing distinct semantic factors in a single caption representation. It also enables subsequent feedback to correct particular state variables rather than indiscriminately rewriting the entire user profile.

Fast Adaptation and Slower Capability Consolidation. The system combines two complementary improvement cycles. At the user level, real-time feedback updates memory to accommodate changing interests and temporary needs. At the system level, high-value hard cases collected from online interactions undergo automated judging, human spot checks, rule correction, and training-data reconstruction. The resulting examples support post-training updates to IMA, and the updated agent returns to the recommendation pipeline. Long-tail experience can thus become immediately available through the knowledge base, while repeatedly

原文第 40 页 | 中文翻译初稿

5 行业格局及初步实践

5.3

核心变化：建议从管道末端转变为下一轮内存更新的信号。
(1) 内循环 |用户在线适配
用户行为理解反馈内存更新体验检索需要推理推荐反馈在线
下一批新版本实际使用反馈
（2）外循环|系统级连续自我改进（Self-Improvement）
在线反馈硬案例挖掘判断并验证RefinementRule / Data Post训练新AgentVersion

图12 小红书的双时间尺度RSI循环推荐。意图记忆智能体（IMA）使用在线行为反馈来更新结构化用户记忆并指导后续推荐。在较慢的时间尺度上，困难案例会经历自动判断、人工抽查和数据重建，支持训练后更新，在重新部署之前将经过验证的经验整合到智能体参数中。

表 10 小红书在线基线与 RSI 模型的性能比较。

模块	指标	A：在线基线	B：RSI 模型	改进
发现源	score_mean@16	2.211	2.366	+7_._0%
发现源	score_median@16	1.587	1.663	+4_._8%
视频内 Feed	score_mean@16	1.537 (n= 470)	1.739 (n= 447)	≈+13_._1%*

由此产生的循环将生产数据从被动资源转变为不断发展的改进基础。与此同时，Lark 目前的做法仍然故意由人为控制：智能体执行大部分重复性数据处理和评估，而人类则保留定义标准、审查关键样本和批准重要变更的责任。
5.3 小红书：双时间尺度RSI推荐

小红书的商业化推荐系统通过意图记忆智能体（IMA）探索反馈驱动的自我改进（Self-Improvement）。IMA 没有将用户理解视为历史行为的一次性推断，而是将反馈解释、记忆更新、体验检索、需求推理和推荐连接成一个连续的循环。后续的点击、停留时间、保存、搜索、转化和负面反馈不仅可以作为新的行为输入，还可以作为观察结果来检查用户状态和意图的早期估计。这些观察指导记忆的添加、强化、替换或衰退，以及忽略无信息证据的决定。因此，持久改进目标是通知未来推荐的用户表示，而不仅仅是当前的推荐输出。

结构化记忆和经验重用。 IMA 将其中间状态组织为三个语义层： 内容，描述内容主题； Who，代表用户的状态；和需要，指定推断的要求。这些层还构建知识库存储和检索，允许通过内容相似性、用户状态相似性或需求相似性来回忆历史经验。这种分解的目的是减少由于在单个标题表示中混合不同语义因素而导致的检索错误

。它还允许后续反馈来纠正特定的状态变量，而不是不加区别地重写整个用户配置文件。

快速适应和较慢的能力整合。该系统结合了两个互补的改进周期。在用户层面，实时反馈会更新记忆，以适应不断变化的兴趣和临时需求。在系统层面，对在线交互中收集到的高价值疑难案例进行自动判断、人

工抽查、规则修正和训练数据重构。生成的示例支持对 IMA 的训练后更新，并且更新后的智能体返回到推荐管道。因此，长尾经验可以通过知识库立即获得，同时重复

40

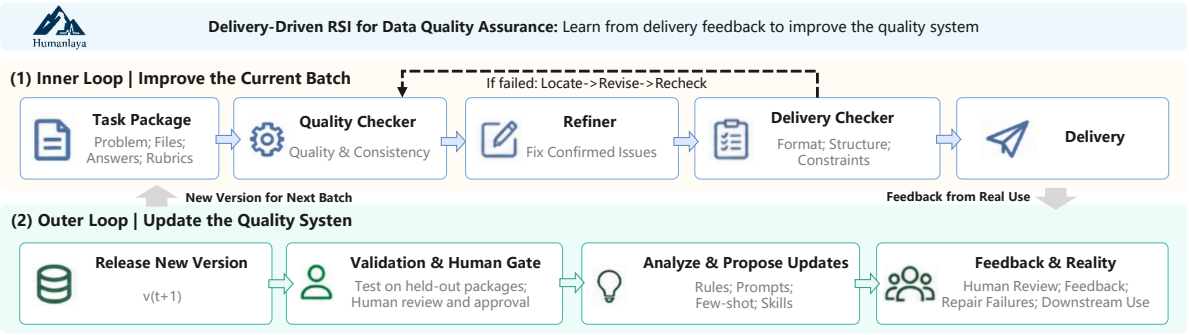


Figure 13 Humanlaya's delivery-driven RSI loop for data-quality assurance.

validated patterns are gradually consolidated into model parameters. This dual-timescale design connects rapid, non-parametric adaptation with more persistent model-level learning.

Reported Observations. In the company-provided comparison, Discovery Feed's score_mean@16 increases from 2.211 to 2.366, a reported 7.0% improvement, while score_median@16 increases from 1.587 to 1.663, or 4.8%. For the in-video feed, score_mean@16 increases from 1.537 to 1.739, approximately 13.1%. The latter result comes from a small offline evaluation with 470 baseline samples and 447 RSI-model samples. These findings should be interpreted as preliminary, company-reported ranking-score improvements; the supplied material does not establish corresponding gains in click-through or conversion rates.

The case illustrates an industrial RSI pattern in which recommendation outcomes become evidence for revising the system that produces subsequent recommendations. Online feedback corrects persistent user memory, while reviewed hard cases improve later agent versions through post-training. Its practical contribution is the coupling of immediate state correction, structured experience reuse, and longer-term capability consolidation, with human inspection retained in the training-data refinement process.

5.4 Humanlaya: Delivery-Driven RSI for Data Quality Assurance

Humanlaya provides training and evaluation data for foundation-model developers, with its production workflow centered on complex multi-file task packages consisting of task descriptions, input attachments, reference answers, scoring rubrics, and executable or procedural verification requirements. In such settings, quality failures are often subtle, where an individual artifact may appear correct in isolation while remaining inconsistent with the task specification, scoring logic, or downstream evaluation protocol. As task types and customer requirements evolve, repeatedly repairing individual defective samples is insufficient, and the quality-control system itself must learn from recurring failures. Humanlaya therefore treats its data-quality agents and their associated rules, prompts, few-shot examples, and skills as persistent improvement targets, including an inner loop and an outer loop, as shown in Figure 13.

The Data-Centric Inner Loop. The inner loop operates within the current delivery batch and focuses on repairing the data being produced. Each task package first passes through a Quality Checker that inspects content quality and cross-component consistency, followed by a Refiner that repairs confirmed defects, and finally a Delivery Checker that verifies formatting, structure, and delivery constraints. Failed checks trigger repeated localization, revision, and re-verification before delivery. For example, if several independently gradable results are incorrectly merged into a single coarse rubric item, the checker may identify the rubric as insufficiently atomic and request restructuring before the package is accepted. This loop improves the current artifact, but by itself does not constitute persistent system improvement.

The Improvement-Centric Outer Loop. The outer loop operates across delivery batches and is the central RSI mechanism. After delivery, the system aggregates evidence like internal review, customer feedback, and downstream usage. An agent analyzes these cases for recurrent causes and proposes versioned modifications to the quality-control system, including components like Refiner instructions, prompts, few-shot examples, and reusable skills. Candidate updates are evaluated on task packages that are not involved in producing

原文第 41 页 | 中文翻译初稿

5 行业格局及初步实践

5.4

用于数据质量保证的交付驱动 RSI：从交付反馈中学习以改进质量体系
(1) 内循环 |改进当前批次如果失败：定位->修改->重新检查
任务包质量检查器精炼器交付检查器
答案；量规问题；文件； 质量和一致性修复确认问题格式；结构;约束交付
下一批实际使用反馈的新版本
(2) 外循环|更新质量系统
发布新版本验证和人性门分析和建议更新反馈和现实
对保留的软件包进行测试；规则；提示； 人工审查；反馈；
v(t+1) 人工审核和批准 少量；技能修复故障；下游使用

图 13 Humanlaya 用于数据质量保证的交付驱动的 RSI 循环。

验证的模式逐渐整合到模型参数中。这种双时间尺度设计将快速的非参数适应与更持久的模型级学习联系起来。

报告的观察结果。在公司提供的比较中，Discovery Feed 的 Score_mean@16 从 2.211 增加到 2.366，据报道提高了 7.0%，而 Score_median@16 从 1.587 增加到 1.663，即 4.8%。对于视频内提要，score_mean@16 从 1.537 增加到 1.739，大约增加了 13.1%。后一个结果来自对 470 个基线样本和 447 个 RSI 模型样本的小型离线评估。这些发现应被解释为公司报告的初步排名分数提高；所提供的材料并未在点击率或转化率方面建立相应的收益。

该案例说明了一种工业 RSI 模式，其中建议结果成为修改产生后续建议的系统的证据。在线反馈可以纠正持久的用户记忆，而经过审查的困难案例可以通过后期训练来改进后来的智能体版本。它的实际贡献是将即时状态校正、结构化经验重用和长期能力整合与训练数据细化过程中保留的人工检查结合起来。
5.4 Humanlaya：用于数据质量保证的交付驱动的 RSI

Humanlaya 为基础模型开发人员提供训练和评估数据，其生产工作流程以复杂的多文件任务包为中心，其中包括任务描述、输入附件、参考答案、评分标准以及可执行或程序验证要求。在此类设置中，质量故障通常很微妙，其中单个工件可能单独显示正确，但与任务规范、评分逻辑或下游评估协议不一致。随着任务类型和客户要求的发展，重复修复单个有缺陷的样品是不够的，质量控制系统本身必须从反复出现的故障中吸取教训。因此，Humanlaya 将其数据质量智能体及其相关规则、提示、少数示例和技能视为持续改进目标，包括内循环和外循环，如图 13 所示。

以数据为中心的内循环

。内循环在当前交付批次内运行，并专注于修复正在生成的数据。每个任务包首先经过质量检查器，检查内容质量和跨组件一致性，然后经过精炼器，修复已确认的缺陷，最后经过交付检查器，验证格式、结构和交付约束。检查失败会触发交付前的重复本地化、修订和重新验证。例如，如果多个可独立分级的结果被错误地合并到单个粗略的评分标准项目中，则检查者可能会将该评分标准识别为不够原子，并在接受包之前请求重组。该循环改进了当前的工件，但其本身并不构成持久的系统改进。

以改进为中心的外循环。外循环跨交付批次运行，是中央 RSI 机制。交付后，系统会汇总内部审查、客户反馈和下游使用情况等证据。智能体会分析这些案例中反复出现的原因，并对质量控制系统提出版本化修改建议，包括 Refiner 指令、提示、少量示例和可重用技能等组件。对不参与生成的任务包评估候选更新

41

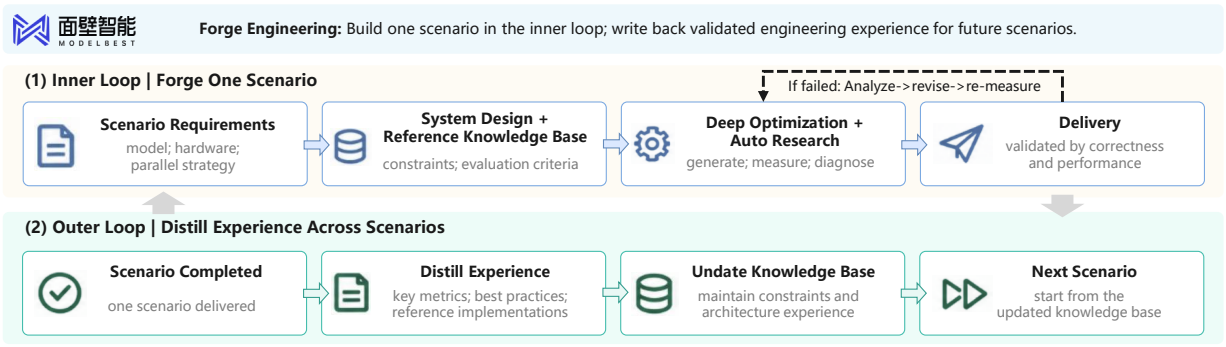


Figure 14 ModelBest: Forge Engineering as a two-level RSI loop.

the modification and are additionally reviewed by humans before promotion. Once approved, the new quality-system version replaces the previous one and is used for subsequent production batches. New tasks then generate new evidence, which again enters the next improvement cycle.

A representative case involved a three-page scanned PDF that the extraction tool failed to parse, causing the Quality Checker to mark the attachment as unusable. Manual review shows that the scan itself is clear, while the error comes from conflating extraction failure with poor document quality. The system therefore generates a new decision rule and few-shot examples to distinguish the two cases. After held-out validation and human review, the update is incorporated into the next system version and reused on future tasks. The inner loop repairs the current task package, whereas the outer loop updates the method used to inspect and repair future packages. Humanlaya therefore represents a practical form of scaffold-level RSI driven by production feedback.

Measured Improvement. Humanlaya reports an internal comparison between V0 and V4 after four feedback-driven updates. Using the same base model, tools, and processing budget, both versions were evaluated on 600 task packages excluded from the update process. The rate of packages containing key defects after automated repair decreased from 9.0% to 3.7%, while average human handling time fell from 48 to 27 minutes per task.

At the current stage, when model capabilities are still advancing rapidly, some tasks cannot yet be completed fully autonomously by agents. Humanlaya therefore provides a practical example of combining RSI with human oversight, where agents perform most of the self-improvement process, while humans provide partial ground truth, review, and final approval at critical stages, rather than allowing an unconstrained self-improvement loop.

5.5 ModelBest: Zero-Human Industrial AI Engineering

ModelBest aims to advance RSI by automating AI engineering, based on the premise that *AI engineering is likely to become autonomous earlier than open-ended AI research*. Engineering tasks (e.g., implementing training frameworks, optimizing kernels, configuring parallelism, running benchmarks) require substantial human effort but offer comparatively inexpensive, objective feedback through execution. To this end, to produce a production-ready implementation without human intervention, ModelBest proposes *Forge Engineering*, an end-to-end paradigm in which an AI system takes a target model, hardware platform, and parallelization requirements as inputs, then builds, tests, and refines an implementation from an initially empty codebase. Figure 14 shows how its within-project optimization and cross-project experience reuse form a two-level loop.

The Industrial RSI Loop. Within a target scenario, the system combines constrained architecture design with autonomous low-level optimization. Humans and AI maintain a knowledge base containing specifications, evaluation criteria, and accumulated engineering experience. The agent first uses this knowledge to establish a reasonable high-level architecture and constrain the search space. An AutoResearch loop then repeatedly generates implementations, measures correctness and performance, diagnoses failures, and repairs bottlenecks. Successful optimizations, including kernels such as GEMM and FlashAttention, are progressively integrated

原文第 42 页 | 中文翻译初稿

5 行业格局及初步实践

5.5

Forge Engineering：在内循环中构建一个场景；为未来场景写回经过验证的工程经验。
(1) 内循环 |打造一个场景失败：分析->修改->重测
场景需求系统设计+深度优化+交付
参考知识库自动研
模型；硬件;通过正确性
并行策略约束进行验证；评估标准生成；措施;诊断和性能
(2) 外循环|跨场景提取经验
场景已完成，提取经验未更新知识库下一个场景
一个场景提供了参考实现关键指标；最佳实践；维护约束和架构经验更新知识库从

图 14 ModelBest：Forge Engineering 作为两级 RSI 循环。

修改并在升级之前由人工进行额外审查。一旦获得批准，新的质量体系版本将取代之前的版本，并用于后续的生产批次。新的任务又会产生新的证据，再次进入下一个改进周期。

一个代表性案例涉及提取工具无法解析的三页扫描 PDF，导致质量检查器将附件标记为不可用。手动检查显示扫描本身是清晰的，而错误则来自于将提取失败与文档质量差混为一谈。因此，系统会生成新的决策规则和少量示例来区分这两种情况。经过持续验证和人工审核后，更新将合并到下一个系统版本中，并在未来的任务中重复使用。内循环修复当前任务包，而外循环更新用于检查和修复未来包的方法。因此，Humanlaya 代表了一种由生产反馈驱动的支架级 RSI 的实用形式。

测量到的改进。 Humanlaya 报告了四次反馈驱动更新后 V0 和 V4 之间的内部比较。使用相同的基本模型、工具和处理预算，两个版本都在更新过程中排除的 600 个任务包上进行了评估。自动修复后包含关键缺陷的包裹率从 9.0% 下降到 3.7%，而每个任务的平均人工处理时间从 48 分钟下降到 27 分钟。

现阶段，模型能力还在快速推进，有些任务还无法完全由智能体自主完成。因此，Humanlaya 提供了一个将 RSI 与人类监督相结合的实际示例，其中智能体执行大部分 自我改进（Self-Improvement） 流程，而人类在关键阶段提供部分事实真相、审查和最终批准，而不是允许不受约束的 自我改进（Self-Improvement） 循环。
5.5 ModelBest：零人类工业人工智能工程

ModelBest 旨在通过人工智能工程自动化来推进 RSI，其前提是_人工智能工程可能比开放式人工智能研究更早实现自主化_

。工程任务（例如，实现训练框架、优化内核、配置并行性、运行基准测试）需要大量的人力，但通过执行提供相对便宜的客观反馈。为此，为了在无需人工干预的情况下生成生产就绪的实现，ModelBest 提出了_Forge Engineering_，这是一种端到端范例，其中人工智能系统将目标模型、硬件平台和并行化要求作为输入，然后从最初的空代码库构建、测试和完善实现。图14展示了其项目内优化和跨项目经验复用如何形成两级循环。

工业 RSI 循环。在目标场景中，系统将受限架构设计与自主低级优化相结合。人类和人工智能维护着一个包含规范、评估标准和积累的工程经验的知识库。智能体首先使用这些知识来建立合理的高层架构并约束搜索空间。然后，自动研究循环会重复生成实现、测量正确性和性能、诊断故障并修复瓶颈。成功的优化，包括 GEMM 和 FlashAttention 等内核，正在逐步集成

42

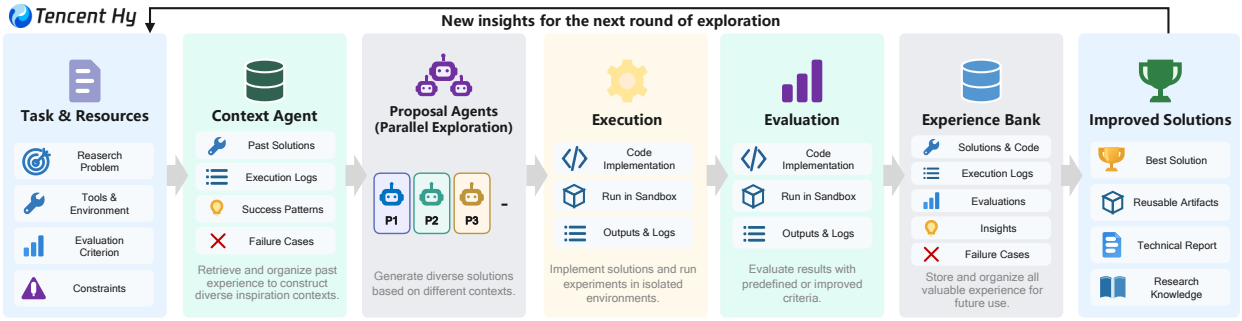


Figure 15 Experience-driven self-improvement in Tencent Hunyuan Hyra. A Context Agent organizes prior experience to guide parallel solution proposals, which are executed in isolated environments and evaluated. The resulting code, execution logs, and evaluation feedback are retained in an Experience Bank to inform subsequent exploration and support evaluator refinement when needed.

into the main execution path. This design keeps global architectural constraints stable while allowing aggressive autonomous optimization at the implementation level.

Across scenarios, successful and failed engineering experience is written back into the shared knowledge base. Performance measurements, effective optimization strategies, reference implementations, and failure rules from one project become starting knowledge for subsequent projects. The same engineering procedure can therefore be reused across different models, hardware platforms, and workloads. ModelBest reports applying the paradigm across pre-training frameworks, operator libraries, reinforcement-learning infrastructure, inference engines, fine-tuning, compression, quantization, and edge deployment on several hardware ecosystems.

Measured Improvement. Starting from an empty directory together with reference scripts and model specifications, ForgeTrain [168] generated a pre-training framework that matched Megatron-LM v0.15 within roughly 8 hours and reportedly surpassed it within 1.5–2.5 days, compared with an estimated 3–5 engineers working for 6–12 months for comparable manual development. Reported model FLOPs utilization increased from 40.1% to 44.1% for MiniCPM4-0.5B and from 47.0% to 50.9% for the 8B model. ForgeStencil [169] extends the same approach to scientific-computing kernels, with the company reporting 1.15–1.9× speedups over public state-of-the-art implementations and a median 1.41× end-to-end speedup.

5.6 Tencent Hunyuan: Experience-Driven Self-Improvement

Tencent Hunyuan's Hyra (Hunyuan Research Agent) explores RSI in performance-oriented research and engineering tasks. Figure 15 outlines its experience-driven search loop. Its design follows a deliberately lightweight philosophy: rather than encoding increasingly elaborate human-designed workflows, Hyra gives agents a broad solution space and repeatedly uses experimental evidence to guide subsequent search. Given a task, the system continues exploring until it decides to terminate or exhausts its resource budget, returning the best solution discovered during the process.

A central component is the Experience Bank, which retains substantially richer state (e.g., previous solution code, generated artifacts, execution logs, scores, and evaluator feedback) than a conventional textual memory. A Context Agent recombines this evidence into diverse inspiration contexts, while multiple Proposal Agents asynchronously consume these contexts, construct new solutions, and execute them in fresh isolated sandboxes. The resulting artifacts and evaluation outcomes are written back into the Experience Bank, allowing later proposals to directly reuse, combine, or avoid patterns discovered in earlier attempts.

Hyra further extends this idea to evaluation itself. For open-ended tasks where the initial evaluator is incomplete or becomes exploitable, accumulated search experience can be used to revise the evaluation mechanism—for example, by increasing its granularity, strengthening comparison baselines, or closing reward-hacking loopholes—before subsequent search continues under the improved criterion. This is particularly relevant to RSI because the persistent update is no longer limited to a better solution: the mechanism that

原文第 43 页 | 中文翻译初稿

5 行业格局及初步实践

5.6

下一轮探索的新见解
任务和资源上下文智能体（并行探索）提案智能体执行评估经验库改进的解决方案
研究问题执行日志过去的解决方案实施代码实施代码解决方案和代码执行日志最佳解决方案
环境工具和成功模式...在沙箱中运行在沙箱中运行评估可重用工件
评估标准失败案例 P1 P2 P3 输出和日志输出和日志失败案例洞察技术报告
约束多样化的灵感背景。检索并组织过去的经验，根据不同的背景进行构建。生成多样化的解决方案在隔离的环境中实施解决方案并运行实验。预定义或改进 用标准评估结果。存储和组织所有未来使用的宝贵经验。知识研究

图15 腾讯混元Hyra中体验驱动的自我改进（Self-Improvement）。上下文智能体组织先前的经验来指导并行解决方案建议，这些解决方案建议在隔离的环境中执行并进行评估。生成的代码、执行日志和评估反馈将保留在经验库中，以便为后续探索提供信息，并在需要时支持评估者的改进。

进入主执行路径。这种设计保持全局架构约束稳定，同时允许在实现级别进行积极的自主优化。

跨场景，成功和失败的工程经验被写回到共享知识库中。一个项目的性能测量、有效的优化策略、参考实施和失败规则成为后续项目的起始知识。因此，相同的工程过程可以在不同的模型、硬件平台和工作负载之间重复使用。 ModelBest 报告将该范式应用于多个硬件生态系统的预训练框架、算子库、强化学习基础设施、推理引擎、微调、压缩、量化和边缘部署。

测量到的改进。从空目录以及参考脚本和模型规范开始，ForgeTrain [168] 生成了一个预训练框架，该框架在大约 8 小时内与 Megatron-LM v0.15 相匹配，据报道在 1.5-2.5 天内超越了它，而估计需要 3-5 名工程师花费 6-12 个月进行类似的手动开发。报告的 MiniCPM4-0.5B 模型 FLOP 利用率从 40.1% 增加到 44.1%，8B 模型从 47.0% 增加到 50.9%。 ForgeStencil [169] 将相同的方法扩展到科学计算内核，该公司报告比公共最先进的实现速度提高了 1.15-1.9 倍，端到端加速中位数为 1.41 倍。
5.6 腾讯混元：体验驱动 自我改进（Self-Improvement）

腾讯混源的 Hyra（混源研究智能体）在面向性能的研究和工程任务中探索 RSI。图 15 概述了其经验驱动的搜索循环

。它的设计遵循一种刻意的轻量级理念：Hyra 没有编码日益复杂的人工设计工作流程，而是为智能体提供了广阔的解决方案空间，并反复使用实验证据来指导后续搜索。给定一项任务，系统会继续探索，直到决定终止或耗尽其资源预算，并返回在此过程中发现的最佳解决方案。

一个核心组件是体验库，它比传统的文本记忆保留了更丰富的状态（例如，以前的解决方案代码、生成的工件、执行日志、分数和评估者反馈）。上下文智能体将这些证据重新组合到不同的灵感上下文中，而多个提案智能体异步使用这些上下文，构建新的解决方案，并在新的隔离沙箱中执行它们。由此产生的工件和评估结果被写回到经验库中，允许后来的提案直接重用、组合或避免在早期尝试中发现的模式。

Hyra 进一步将这个想法延伸到评估本身。对于初始评估器不完整或可利用的开放式任务，可以利用积累的搜索经验来修改评估机制，例如增加其粒度、加强比较基线或堵住奖励黑客漏洞，然后在改进的标准下继续进行后续搜索。这与 RSI 特别相关，因为持久更新不再局限于更好的解决方案：

43

Table 11 Comparison of Recursive and hyra-1.0 on AI R&D benchmarks.

Benchmark	Task	Metric	Recursive	hyra-1.0
NanoChat Autoresearch [170]	Model training	Validation BPB ↓	0.9109	**0.9015**
NanoGPT Speedrun [171]	Model training acceleration	Time to 3.28 loss ↓	77.5 s	**76.4 s**
SOL-ExecBench [172]	GPU kernel optimization	Mean SOL ↑	0.754	**0.771**†

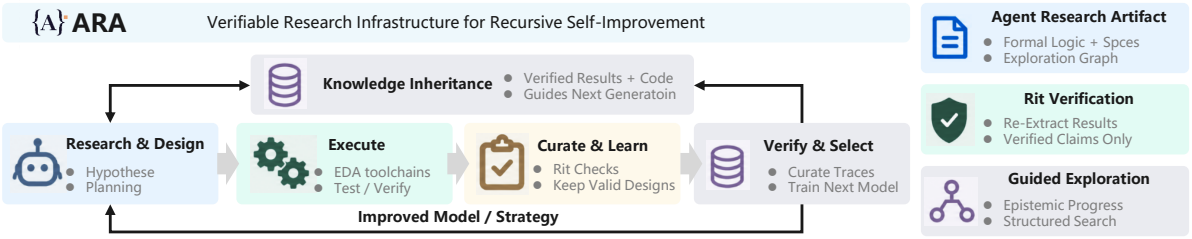


Figure 16 Agent-Native Research Lab's agent-native RSI loop for verifiable engineering discovery, integrating guided exploration, deterministic verification, knowledge inheritance, and next-generation learning to support persistent improvement across research cycles.

determines which future solutions are considered improvements can also change.

Measured Improvement. As shown in Table 11, the released companion artifacts report improvements across both AI-for-AI and AI-for-Science tasks. For example, Hyra reports validation BPB of 0.9015 on nanochat AutoResearch [170] compared with a cited previous best of 0.9109, 76.4 s on nanoGPT Speedrun [171] compared with 77.5 s, and 0.771 on SOL-ExecBench [172] compared with 0.754. As an industrial RSI practice, Hyra highlights the value of retaining executable experience rather than only final answers.

5.7 Agent-Native Research Lab: Verifiable Research Infrastructure for RSI

Achieving sustained recursive self-improvement requires more than increasingly autonomous research agents, while it also requires an infrastructure through which research experience can be reliably recorded, verified, inherited, and reused across generations. Agent-Native Research Lab focuses on this infrastructural layer, summarized in Figure 16.

• **(1) Human-Centric vs. Agent-Native Research Artifact.** Conventional scientific papers are designed for human communication and would discard much of the operational state that autonomous agents need to continue previous work (e.g., failed experiments, intermediate evidence, executable specifications, and implementation details). For RSI, such information loss directly weakens cross-generation knowledge accumulation.

To address this, they propose Agent-Native Research Artifact (ARA) as an executable alternative to conventional scientific documentation. ARA jointly preserves formal scientific logic, executable code and specifications, the exploration graph of both successful and abandoned branches, and raw empirical evidence. Rather than inheriting only a polished final narrative, a successor agent can therefore inspect how a result was obtained, which alternatives failed, and which assumptions or parameters were actually used. The reported evaluation shows 93.7% question-answering accuracy over prior work using ARA compared with 72.4% using conventional papers, while RE-Bench reproduction success increases from 57.4% to 64.4%. The important RSI contribution is thus not merely better documentation, but a richer inheritance substrate for later research cycles.

• **(2) Deterministic Verification for Research Claims.** Additionally, as autonomous agents generate experiments and claims at machine speed, *verification rather than generation becomes the main bottleneck*. Its rit protocol anchors empirical claims directly to execution traces, re-extracting reported results from logs, while analytical claims can be checked through formal verification systems such as Lean 4. Only claims that pass these machine-verifiable gates are admitted into the shared research state. This prevents hallucinated or incorrectly reported results from being recursively inherited and amplified by later generations. They also

原文第 44 页 | 中文翻译初稿

5 行业格局及初步实践

5.7

表 11 Recursive 和 hyra-1.0 在 AI 研发基准上的比较。

基准	任务	指标	递归	hyra-1.0
NanoChat Autoresearch [170]	模型训练	验证 BPB ↓	0.9109	0.9015
NanoGPT Speedrun [171]	模型训练加速	达到 3.28 损失的时间_↓_	77.5 s	76.4 s
SOL-ExecBench [172]	GPU 内核优化	平均 SOL ↑	0.754	0.771_†_

递归自我改进（Recursive Self-Improvement） 智能体研究工件的 ARA 可验证研究基础设施
☐ 形式逻辑 + Spces
☐ 探索图
知识继承 ☐☐ 验证结果 + 代码指南下一代 Rit 验证
☐ 重新提取结果
研究与设计 执行策划与学习 验证与选择 ☐ 验证声明仅
☐☐假设规划☐☐EDA工具链测试/验证☐☐Rit检查保持有效设计☐☐策划跟踪训练下一个模型引导探索☐认知进展
改进的模型/策略☐结构化搜索

图 16 Agent-Native Research Lab 的 Agent-Native RSI 循环用于可验证的工程发现，集成了引导式探索、确定性验证、知识继承和下一代学习，以支持跨研究周期的持续改进。

确定哪些未来解决方案被视为改进也可能发生变化。

测量到的改进。如表 11 所示，已发布的配套工件报告了 AI-for-AI 和 AI-for-Science 任务的改进。例如，Hyra 报告称，nanochat AutoResearch [170] 上的验证 BPB 为 0.9015，而之前引用的最佳值是 0.9109；nanoGPT Speedrun [171] 上的验证 BPB 为 76.4 s，与 77.5 s 相比；SOL-ExecBench [172] 上的验证 BPB 为 0.771，与 0.754 相比。作为工业 RSI 实践，Hyra 强调保留可执行经验的价值，而不仅仅是最终答案。
5.7 Agent-Native 研究实验室：RSI 可验证的研究基础设施

实现持续的 递归自我改进（Recursive Self-Improvement） 需要的不仅仅是日益自主的研究智能体，同时还需要一个基础设施，通过该基础设施可以可靠地记录、验证、继承和跨代重复使用研究经验。Agent-Native 研究实验室重点关注这一基础设施层，如图 16 所示。

• (1) 以人为中心与以智能体为中心的研究工件。传统的科学论文是为人类交流而设计的，并且会丢弃自主智能体继续之前的工作所需的大部分操作状态（例如，失败的实验、中间证据、可执行规范和实现细节）。对于RSI来说，这种信息丢失直接削弱了跨代知识积累。

为了解决这个问题，他们提出了 Agent-Native Research Artifact (ARA) 作为传统科学文档的可执行替代方案。 ARA 共同保留了正式的科学逻辑、可执行代码和规范、成功和废弃分支的探索图以及原始经验证据。

因此，继任者可以检查如何获得结果、哪些替代方案失败以及实际使用了哪些假设或参数，而不是仅继承经过打磨的最终叙述

。报告的评估显示，使用 ARA 的先前工作的问答准确率为 93.7%，而使用传统论文的问答准确率为 72.4%，而 RE-Bench 再现成功率从 57.4% 提高到 64.4%。因此，RSI 的重要贡献不仅是更好的记录，而且是为以后的研究周期提供更丰富的继承基础。

• (2) 研究主张的确定性验证。此外，随着自主智能体以机器速度生成实验和声明，验证而不是生成成为主要瓶颈。其 rit 协议将经验声明直接锚定到执行跟踪，从日志中重新提取报告的结果，而分析声明可以通过正式验证系统（例如 Lean 4）进行检查。只有通过这些机器可验证的门的声明才会被允许进入共享研究状态。这可以防止幻觉或错误报告的结果被后代递归继承和放大。他们还

44

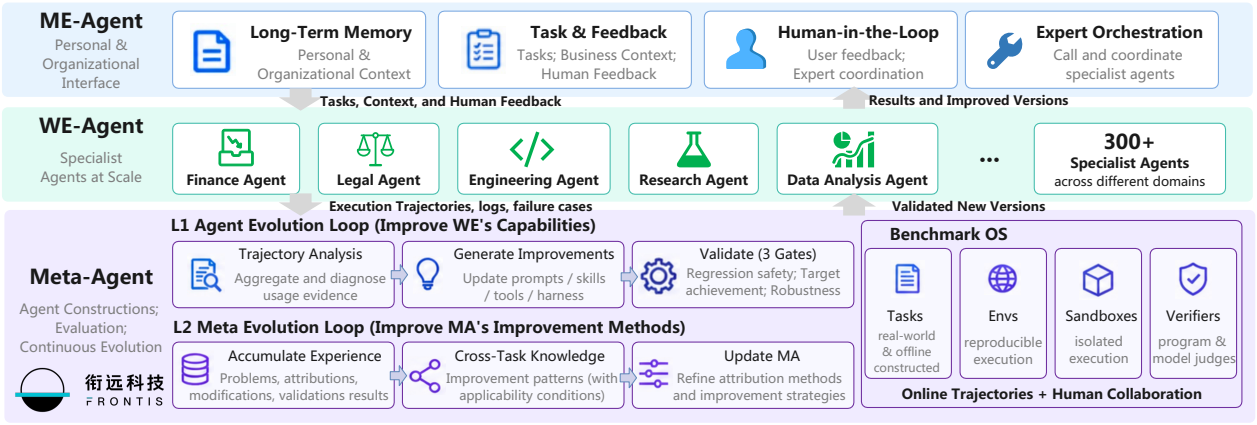


Figure 17 Frontis Horizon's ME–WE–MA architecture combines specialist-agent evolution and cross-task meta-improvement with shared evaluation and human-approved release.

augment sparse outcome-based optimization with epistemic-progress guidance and structured priors derived from human scientific reasoning. Instead of rewarding only the final benchmark score, agents are encouraged to prefer experiments that reduce uncertainty and reveal useful structure in the problem. This is intended to improve research efficiency in large hypothesis spaces, where naive hill climbing can repeatedly exploit familiar parameters without producing deeper understanding.

Measured Improvement. These ideas are instantiated in silicon design, where the feedback loop is unusually fast and deterministic. Autonomous agents generate synthesizable SystemVerilog, construct testbenches, and propose microarchitectural modifications; candidates are then evaluated using industrial EDA tools for synthesis, place-and-route, timing analysis, and formal equivalence. Invalid designs are filtered automatically, while verified implementations, execution traces, failed branches, and superior power-performance-area trajectories are retained as training and research evidence for subsequent generations. On Chip-Bench, they report a Level-3 CPU optimization score of 5.8416, with a design that is 2.7% faster and 21.5% smaller in area than standard-agent baselines on the same model foundation while passing all 97 directed tests and 350 random differential programs.

5.8 Frontis.AI: Enterprise Agent Evolution and Cross-Task Meta-Improvement

Frontis Horizon provides an enterprise-owned infrastructure for improving both specialist agents and the methods used to improve them. Its ME–WE–MA architecture separates human interaction, contextual memory, and release approval (ME); task execution by more than 300 heterogeneous specialist agents across finance, legal, engineering, and human resources (WE); and agent construction, evaluation, and continuous evolution (MA). The central objective is to make improvement experience reusable across tasks rather than allowing it to remain isolated within individual agents.

Business-Grounded Expert Evolution. The platform's Benchmark OS organizes enterprise tasks, reproducible environments, and verifiers into shared evaluation infrastructure. Programmatic checks are preferred, while semantic quality is assessed by model judges isolated from the evaluated agents. Given operational feedback, MA attributes failures to routing, execution mechanisms, or specialist capabilities, defines bounded improvement objectives and acceptance criteria, and obtains human confirmation before modifying prompts, skills, tools, or harnesses. Candidate versions are compared with their predecessors through three gates: regression safety, target achievement, and robustness in exceptional scenarios. Approved updates return to production with versioned test reports supporting audit and rollback.

Cross-Task Meta-Improvement. Beyond improving individual specialists, the platform retains each improvement episode's problem context, diagnostic rationale, candidate modifications, validation results, and applicability conditions. Agent-specific experience supports subsequent versions, while transferable lessons inform MA's diagnosis and improvement strategies on new tasks. Transfer requires adaptation and revalidation: a lesson about prioritizing real-time evidence, for example, still requires identifying authoritative sources and

原文第 45 页 | 中文翻译初稿

5 行业格局及初步实践

5.8

ME-Agent 长期记忆任务和反馈人在环专家编排
个人和
个人和任务；业务背景； 用户反馈； 呼叫和协调
组织界面组织环境人类反馈专家协调专家智能体
任务、环境和人类反馈结果和改进版本
WE-Agent
300+
专家·专家智能体
大规模智能体财务智能体法律智能体工程智能体研究智能体跨不同领域的数据分析智能体
执行轨迹、日志、故障案例已验证的新版本
L1智能体进化循环（提高WE的能力）基准操作系统
轨迹分析生成改进验证（3个门）
元智能体聚合和诊断使用证据更新提示/技能/工具/运行支撑框架（Harness）成就；鲁棒性回归安全性；
目标
智能体构造； 任务环境沙箱验证器
持续进化评估； L2元进化循环（改进MA的改进方法）积累经验跨任务知识更新MA构建真实世界和离线可
重复执行执行隔离模型判断程序&
问题、归因、改进模式（具有细化归因方法
修改、验证结果适用性条件）和改进策略在线轨迹+人工协作

图 17 Frontis Horizon 的 ME-WE-MA 架构将专家智能体进化和跨任务元改进与共享评估和人工批准的发布相结合。

通过认知进展指导和源自人类科学推理的结构化先验来增强基于结果的稀疏优化。鼓励智能体选择能够减少不确定性并揭示问题中有用结构的实验，而不是仅奖励最终基准分数。这是为了提高大型假设空间中的研究效率，在这种空间中，简单的爬山可以重复利用熟悉的参数，而不会产生更深入的理解。

测量到的改进。这些想法在硅设计中得到体现，其中反馈循环异常快速且具有确定性。自主智能体生成可综合的 SystemVerilog、构建测试平台并提出微架构修改；然后使用工业 EDA 工具进行综合、布局布线、时序分析和形式等效来评估候选者。无效的设计会被自动过滤，而经过验证的实现、执行轨迹、失败的分支和卓越的功率性能区域轨迹将被保留作为后代的训练和研究证据。在 Chip-Bench 上，他们报告的 3 级 CPU 优化得分为 5.8416，与相同模型基础上的标准智能体基线相比，该设计速度快 2.7%，面积小 21.5%，同时通过了所有 97 个定向测试和 350 个随机差分程序。
5.8 Frontis.AI：企业智能体演进和跨任务元改进

Frontis Horizon 提供企业拥有的基础设施，用于改进专业智能体以及用于改进它们的方法

。其 ME-WE-MA 架构将人机交互、情境记忆和发布批准 (ME) 分开；由跨财务、法律、工程和人力资源 (WE) 的 300 多名异构专业智能体执行任务；以及智能体构建、评估和持续进化 (MA)。中心目标是使改进

体验可跨任务重用，而不是让它在各个智能体中保持孤立。

以业务为基础的专家进化。该平台的基准操作系统将企业任务、可重现环境和验证器组织到共享评估基础设施中。首选程序检查，而语义质量则由与评估智能体隔离的模型法官进行评估。根据操作反馈，MA 将故障归因于路由、执行机制或专家能力，定义有界的改进目标和验收标准，并在修改提示、技能、工具或运行支撑框架（Harnesses） 之前获得人工确认。通过三个关卡将候选版本与其前身进行比较：回归安全性、目标实现情况以及特殊情况下的稳健性。批准的更新返回生产，并带有支持审核和回滚的版本化测试报告。

跨任务元改进。除了改进个别专家之外，该平台还保留每个改进事件的问题背景、诊断原理、候选修改、验证结果和适用性条件。特定于智能体的经验支持后续版本，而可转移的经验则为 MA 对新任务的诊断和改进策略提供信息。转移需要适应和重新验证：例如，关于优先考虑实时证据的教训仍然需要确定权威来源和

45

their temporal validity in the receiving task. This distinguishes learning to perform a business task from learning how to improve agents more effectively.

Reported Outcomes. Over two months, Frontis reports initiating 216 improvement tasks involving 117 specialist agents, of which 63 completed at least one full evolution cycle. Across 76 comparable tasks evaluated with the same evaluator, average scores increased from 5.25 to 5.87, with a median machine-processing time of 27.8 minutes per completed cycle. Separately, after accumulating meta-evolution experience from more than 100 tasks, the company reports approximately 20% faster evolution on previously unseen internal test tasks relative to a process without cross-task experience. Its evaluation protocol distinguishes specialist performance from improvement efficiency by comparing identical initial agent versions and tracking iterations, time, token consumption, and regressions. These findings remain company-reported evidence rather than independent replication.

Learning Improvement in Model Weights. Frontis-MA1 separately explores training improvement capabilities into a 35B model. The supplied report records an MLE-Bench Lite score increase from 39.39% to 60.61% under a fixed harness, reaching 71.21% with additional search. These results concern machine-learning and deep-learning engineering; extending the approach to learn directly from the platform's enterprise-agent evolution trajectories remains future work.

The case illustrates a practical RSI pattern: retain not only improved agents, but also validated knowledge about how to improve them. Shared evaluation infrastructure, context-sensitive experience transfer, versioned updates, and human-controlled release provide the governance framework for this accumulation.

6 Challenges and Future Directions

The preceding chapters show progress in automating improvement execution, strategy selection, experience acquisition, deployment adaptation, and the revision of improvement mechanisms. They also establish that greater autonomy, durable capability gains, and a more effective improvement process are distinct properties. The three bottlenecks motivating this survey therefore remain only partially resolved: foundation-model development requires substantial resources and coordination; scalable learning depends on useful experience and reliable verification; and deployed systems require continuing effort to diagnose, validate, and release updates. Building on the autonomy hierarchy, the four application regimes, and the industrial practices reviewed above, we identify eight research directions for connecting these partial advances into sustained recursive improvement.

Cross-Component Diagnosis and Coordinated Improvement. The transition from L1 to L2 exposes a diagnostic problem: an observed failure rarely identifies the component that should change. An incorrect answer may originate in defective source data, missing context, a tool interface, the model, or the evaluator. The Humanlaya case in Section 5.4 makes this ambiguity concrete: a failed extraction was initially treated as evidence of poor document quality, while the appropriate intervention concerned the quality checker's decision rule. In science and embodied intelligence, the same problem extends to experimental protocols, perception, control, and environmental conditions. As several components adapt, changing one can also invalidate assumptions on which another relies.

Future systems should turn diagnoses into testable intervention hypotheses and use controlled comparisons to estimate individual effects and interactions. Selective component freezing, targeted ablations, and explicit records of dependencies could help distinguish a useful repair from compensation for an upstream defect. Coordinated search should then allocate effort across data, models, harnesses, and environments as constraints shift. Theseus's workspace studies in Section 5.1 illustrate the importance of separating environment improvements from changes to the task model; the reported component-level gains leave the full learning cycle to be established. The research goal is to identify which intervention improves the overall system, under what conditions, and whether the resulting knowledge guides later improvement decisions.

Learner-Conditioned Experience Acquisition and Reliable Learning Signals. The L3 analysis distinguishes three properties of experience: whether it is valid, how difficult it is for the current learner, and whether learning from it produces a durable benefit. AZR uses executable checks alongside learner-dependent task proposal, whereas R-Zero uses solver consistency as a proxy for difficulty [99, 103]. Neither correctness nor

原文第 46 页 | 中文翻译初稿

6 挑战和未来方向

6

它们在接收任务中的时间有效性。这将学习执行业务任务与学习如何更有效地改进智能体区分开来。

报告结果。 Frontis 报告在两个月内启动了 216 项改进任务，涉及 117 名专业智能体，其中 63 名至少完成了一个完整的进化周期。在由同一评估员评估的 76 项可比任务中，平均得分从 5.25 增加到 5.87，每个完成周期的中位机器处理时间为 27.8 分钟。另外，在从 100 多个任务中积累元进化经验后，该公司报告称，相对于没有跨任务经验的流程，以前未见过的内部测试任务的进化速度提高了约 20%。其评估协议通过比较相同的初始智能体版本并跟踪迭代、时间、令牌消耗和回归来区分专家性能和改进效率。这些发现仍然是公司报告的证据，而不是独立的复制。

模型权重的学习改进。 Frontis-MA1单独探索了35B模型的训练改进能力。提供的报告记录了在固定 运行支撑框架（Harness） 下 MLE-Bench Lite 分数从 39.39% 增加到 60.61%，通过额外搜索达到 71.21%。这些结果涉及机器学习和深度学习工程；扩展该方法以直接从平台的企业智能体演化轨迹中学习仍然是未来的工作。

该案例说明了一种实用的 RSI 模式：不仅保留改进的智能体，还保留有关如何改进它们的经过验证的知识。共享的评估基础设施、上下文相关的经验传输、版本化更新和人工控制的发布为这种积累提供了治理框架。

6 挑战和未来方向

前面的章节展示了自动化改进执行、策略选择、经验获取、部署适应和改进机制修订方面的进展。他们还指出，更大的自主权、持久的能力增益和更有效的改进过程是不同的属性。因此，推动这项调查的三个瓶颈仍然只得到部分解决：基础模型开发需要大量资源和协调；可扩展的学习取决于有用的经验和可靠的验证；和已部署的系统需要持续努力诊断、验证和发布更新。基于自主层次结构、四种应用机制和上面回顾的工业实践，我们确定了八个研究方向，将这些部分进展与持续的递归改进联系起来。

跨组件诊断和协调改进。从 L1 到 L2 的转换暴露了一个诊断问题：观察到的故障很少能识别出应该更改的组件。错误答案可能源于有缺陷的源数据、缺少上下文、工具界面、模型或评估器

。第 5.4 节中的 Humanlaya 案例使这种歧义变得具体：提取失败最初被视为文档质量差的证据，而适当的干预涉及质量检查员的决策规则。在科学和 具身智能（Embodied Intelligence） 中，同样的问题延伸到实验协议、感知、控制和环境条件。随着多个组件的适应，改变一个组件也会使另一个组件所依赖的假设失效。

未来的系统应该将诊断转化为可测试的干预假设，并使用受控比较来估计个体效应和相互作用。选择性组件冻结、有针对性的消融和明确的依赖性记录可以帮助区分有用的修复和上游缺陷的补偿。然后，随着约束的变化，协调搜索应该在数据、模型、运行支撑框架（Harnesses） 和环境之间分配工作量。
Theseus 在第 5.1 节中的工作空间研究说明了将环境改进与任务模型更改分开的重要性；所报告的组成部分层面的成果还需要建立完整的学习周期。研究目标是确定哪种干预措施可以在什么条件下改善整个系统，以及由此产生的知识是否可以指导以后的改进决策。

以学习者为条件的经验获取和可靠的学习信号。 L3 分析区分了经验的三个属性：它是否有效、对于当前学习者来说有多困难以及从中学习是否会产生持久的好处。 AZR 使用可执行检查以及依赖于学习者的任

务建议，而 R-Zero 使用求解器一致性作为难度的智能体 [99, 103]。既不正确也不

46

estimated difficulty alone establishes learning value. An adaptive curriculum may repeatedly select ambiguous tasks, reinforce evaluator errors, or concentrate on a narrow region where progress is easy to measure. Such errors can enter both the learner and the state that determines its next experiences.

Future acquisition mechanisms should estimate learning value across multiple rounds while preserving validity, diversity, and coverage of previously acquired capabilities. This includes selecting from existing data, generating new tasks, seeking interactions, and requesting stronger external feedback when internal judgments are insufficient. A central challenge is deciding when expensive verification is worth its cost and how uncertain experience should influence subsequent training. Evaluation should allow the acquired distributions to differ while matching data-source access and total acquisition and learning budgets. Freezing the learner-state input to the acquisition mechanism, or replacing adaptive selection with a learner-independent schedule, can test whether learner conditioning improves transfer beyond the effects of additional training.

Persistent-State Management and Reliable Reuse. L4 systems inherit parameters, memories, skills, tools, and harness code, but retaining an artifact does not establish that later agents benefit from it. The L4 analysis identifies failures in both activation and execution: a relevant skill may never be retrieved, or an agent may retrieve it without following it correctly. Accumulation creates an additional difficulty. Library Drift shows that an expanding skill library can degrade retrieval and stall improvement, while overly aggressive retirement can also be harmful [152]. Persistence therefore requires managing the continuing applicability and interaction of retained changes.

Future work should associate inherited artifacts with their supporting evidence, applicability conditions, dependencies, and observed effects on later tasks. Admission tests such as HDSO's paired evaluations provide a starting point [106], but validation must continue as the task distribution, executor, and surrounding system change. Studies should separately measure update quality, activation, faithful use, and downstream benefit, then investigate when to merge, revise, retire, or revalidate artifacts. Cross-model reuse introduces a further question: a skill useful to its author may be unsuitable for a different executor. Resolving these issues would connect successful update generation to reliable capability transfer across sessions and successors.

Governed Adaptation under Domain-Specific Feedback. The four application regimes show why a common improvement loop requires different validation strategies. In science, an unsuccessful experiment may reflect the hypothesis, protocol, or instrument, and evidence must retain its assumptions and uncertainty. In embodied intelligence, the current policy determines which states are observed, while physical trials consume resources and can have irreversible consequences. Software offers executable feedback, but passing tests remains conditional on the specification and test coverage. In healthcare, outcomes are delayed and confounded, and an update validated in one population or institution may not transfer to another. These differences constrain both what a system can learn autonomously and what evidence supports deployment.

Future research should develop update policies that distinguish transient failures from recurring limitations and select a validation process appropriate to the proposed change. Simulation, retrospective replay, and sandboxed execution can support initial screening, followed by supervised or staged evaluation under the intended operating conditions. Monitoring should test whether benefits persist as requirements, populations, tools, and environments change. Restoring a software checkpoint cannot undo physical or clinical consequences, making pre-release validation essential in those settings. Human authority over consequential releases, access permissions, and safety constraints should remain explicit. The open problem is how to increase useful adaptation while controlling the propagation of errors across future interactions.

Trustworthy Evolution of Improvement Mechanisms. At L5, changes to the improver, evaluator, or research policy alter how subsequent successors are produced and accepted. This creates a coupled validation problem: a stronger solver can expose weaknesses in its evaluator, but changing the evaluator can also make scores incomparable or reward exploitation. RQGM addresses part of this problem by freezing its evaluator within an epoch, validating replacements against an independent anchor, and revisiting scores affected by replacement [21]. The evaluator refinement described in Hyra raises the same broader question of how adaptive measurement can remain credible as search progresses.

Future systems should distinguish editable internal feedback mechanisms from independently maintained acceptance criteria and preserve evidence linking each mechanism revision to later decisions. Controlled comparisons should assess changes to candidate generation and evaluation separately before attributing gains

原文第 47 页 | 中文翻译初稿

6 挑战和未来方向

6

仅估计难度即可确定学习价值。适应性课程可能会重复选择不明确的任务，强化评估者的错误，或者集中在易于衡量进展的狭窄区域。此类错误可能会进入学习者和决定其下一次经历的状态。

未来的获取机制应该评估多轮学习的价值，同时保留先前获取的能力的有效性、多样性和覆盖范围。这包括从现有数据中进行选择、生成新任务、寻求交互以及在内部判断不足时请求更强的外部反馈。一个核心挑战是决定昂贵的验证何时值得其成本，以及不确定的经验应如何影响后续训练。评估应允许所获取的分布有所不同，同时匹配数据源访问以及总获取和学习预算。冻结习得机制的学习者状态输入，或用独立于学习者的时间表替换自适应选择，可以测试学习者条件反射是否能改善迁移，使其超出额外训练的效果。

持久状态管理和可靠重用。 L4 系统继承参数、记忆、技能、工具和 运行支撑框架（Harness） 代码，但保留工件并不意味着后来的智能体可以从中受益。 L4 分析识别激活和执行中的失败：相关技能可能永远不会被检索到，或者智能体可能会在没有正确遵循的情况下检索它。积累会带来额外的困难。库漂移表明，不断扩大的技能库可能会降低检索速度并阻碍改进，而过度激进的退休也可能是有害的[152]。因此，持久性需要管理保留更改的持续适用性和交互。

未来的工作应该将继承的工件与其支持证据、适用性条件、依赖性以及观察到的对后续任务的影响联系起来。 HDSO 的配对评估等准入测试提供了一个起点 [106]，但随着任务分配、执行器和周围系统的变化，验证必须继续。研究应单独衡量更新质量、激活、忠实使用和下游效益，然后调查何时合并、修改、退役或重新验证工件。跨模型重用引入了另一个问题：对其作者有用的技能可能不适合不同的执行者。解决这些问题将把成功的更新生成与跨会话和后继者的可靠能力传输联系起来。

特定领域反馈下的受控适应。这四种应用方案说明了为什么常见的改进循环需要不同的验证策略。在科学中，一个不成功的实验可能反映了假设、方案或仪器，而证据必须保留其假设和不确定性。在具身智能（Embodied Intelligence）中，当前策略决定观察哪些状态，而物理试验会消耗资源并可能产生不可逆转的后果。软件提供可执行的反馈，但通过测试仍然取决于规范和测试覆盖范围

。在医疗保健领域，结果会被延迟和混淆，并且在一个人群或机构中验证的更新可能不会转移到另一个人群或机构。这些差异限制了系统可以自主学习的内容以及支持部署的证据。

未来的研究应该制定更新策略，区分暂时性故障和反复出现的限制，并选择适合拟议变更的验证过程。模拟、回顾性重放和沙盒执行可以支持初步筛选，然后在预期操作条件下进行监督或分阶段评估。监控应测试随着需求、人群、工具和环境的变化，收益是否持续存在。恢复软件检查点无法消除物理或临床后果，因此在这些设置中预发布验证至关重要。人类对后续发布、访问权限和安全限制的权力应保持明确。悬而未决的问题是如何增加有用的适应，同时控制未来交互中错误的传播。

改进机制的值得信赖的演变。在 L5，改进者、评估者或研究策略的变化会改变后续继任者的产生和接受方式。这就产生了一个耦合验证问题：更强大的求解器可以暴露其评估器的弱点，但改变评估器也可能使分数无法比较或奖励剥削。 RQGM 通过在一个时期内冻结其评估器、根据独立锚点验证替换以及重新访问受替换影响的分数来部分解决此问题[21]。 Hyra 中描述的评估器改进提出了同样更广泛的问题：随着搜索的进展，自适应测量如何保持可信。

未来的系统应该将可编辑的内部反馈机制与独立维护的验收标准区分开来，并保留将每个机制修订与后续决策联系起来的证据。受控比较应在归因收益之前分别评估候选生成和评估的变化

47

to their joint evolution. A-Evolve-Training provides an example of policy inheritance within a fixed high-level objective [17]; whether such policies transfer beyond the research setting in which they evolved remains to be tested. Open questions include how to maintain independent assessment under repeated adaptive access, detect coordinated proposer–evaluator errors, and reverse harmful mechanism changes without losing useful experience. Revising operational research priorities need not transfer authority over the system's overall mission.

Long-Horizon Evaluation of Inherited Improvement Capacity. The distinction between structural and effective L5 should guide evaluation. Structural evidence establishes that a revised mechanism is inherited and invoked; effectiveness requires showing that it produces or selects better subsequent improvements. This remains a substantive empirical gap. For example, the AIDE^2 experiment reviewed in this survey did not establish a statistically significant efficiency advantage when an evolved harness was installed as the outer improver [94]. Software lineage studies further motivate separating current task performance from the ability to produce useful descendants, since temporarily weaker variants may enable later progress.

Future benchmarks should compare original and revised mechanisms from comparable initial systems and evidence under matched total budgets, including the cost of developing and evaluating the mechanisms. Holding an evolved mechanism fixed on fresh tasks can test whether its improvement strategy transfers, as illustrated by HyperAgents [93]. Longer studies should report complete trajectories across repeated runs, including rejected updates, regressions, recovery, retained capabilities, and resource use. Protected reference tasks can preserve comparability, while fresh tasks and unfamiliar constraints test transfer and discovery. These protocols should establish whether inherited changes alter subsequent improvement capacity, when gains plateau, and whether apparent acceleration survives accounting for additional search and evaluation effort.

Resource-Aware Improvement and Human Collaboration. The efficiency objective motivating RSI requires accounting for the entire improvement process. A faster kernel, a better training recipe, or a longer unattended run may still require substantial candidate generation, evaluation, infrastructure maintenance, and human review. Meta's regression-analysis workflow retains engineering review of proposed changes [5]; Humanlaya also retains human validation and reports handling time alongside quality. These practices show why autonomy should be assessed together with the amount and type of human effort required. Stronger base models and more parallel trials can further confound comparisons between improvement strategies.

Future work should study how to allocate a total budget among diagnosis, experience acquisition, candidate search, verification, and deployment monitoring. Cheap screening can reduce expensive trials, provided its predictions remain calibrated against independent outcomes. Reusing validated tools and prior failures may reduce repeated work, but its maintenance cost must also be counted. Evaluations should report time and total cost to a validated capability target, alongside review effort and rework. Adaptive stopping is another research priority: systems should justify continued experimentation by its expected learning value and uncertainty, with explicit conditions for pausing, escalating to a human, or terminating an unproductive search.

Reproducible Infrastructure for Cross-Round Inheritance. Successor systems need access to how an improvement was obtained, including unsuccessful alternatives and the conditions under which its evidence holds. The industrial cases expose complementary requirements: Lark emphasizes temporally grounded data and failure attribution; Humanlaya retains versioned quality-system updates; Hyra stores executable experience; and Agent-Native Research Lab proposes research artifacts that preserve code, specifications, exploration history, and empirical traces. These practices motivate a shared infrastructure for carrying research evidence across rounds, while the full longitudinal benefit of such inheritance remains to be established.

A reusable artifact format should connect the parent state, proposed change, motivating evidence, evaluation configuration, acceptance decision, and subsequent use. Versioned data, environments, and evaluators would make results easier to replay and clarify which comparisons remain valid after an update. Deterministic re-extraction can check reported measurements, and formal verification can establish properties relative to a specification; neither alone establishes that the experiment or specification captures the intended capability. Public evaluation should therefore pair inspectable artifacts with replay studies and report result provenance, artifact availability, and independent replication separately. Where data are confidential, shareable evaluation subsets and controlled replay interfaces could support partial verification while preserving the limits of what

原文第 48 页 | 中文翻译初稿

6 挑战和未来方向

6

到他们的共同进化。 A-Evolve-Training 提供了固定高级目标内策略继承的示例 [17]；这些策略是否会超越其发展的研究环境仍有待检验。悬而未决的问题包括如何在重复的自适应访问下保持独立评估，检测协调的提议者-评估者错误，以及在不损失有用经验的情况下扭转有害的机制变化。修改运筹学优先事项不需要转移对系统整体任务的权力。

长时程（Long-Horizon）遗传改良能力评价。结构性 L5 和有效 L5 之间的区别应指导评估。结构性证据表明修订后的机制得到继承和调用；有效性要求表明它产生或选择了更好的后续改进。这仍然是一个实质性的经验差距。例如，当安装进化的 运行支撑框架（Harness）作为外部改进器时，本篇综述中审查的 AIDE2 实验并未建立统计上显着的效率优势 [94]。软件谱系研究进一步推动将当前任务性能与产生有用后代的能力分开，因为暂时较弱的变体可能会促进以后的进展。

未来的基准应比较来自可比初始系统的原始机制和修订后的机制以及匹配总预算下的证据，包括开发和评估机制的成本。将进化机制固定在新任务上可以测试其改进策略是否转移，如 HyperAgents [93] 所示。较长的研究应报告重复运行的完整轨迹，包括拒绝的更新、回归、恢复、保留的功能和资源使用。受保护的参考任务可以保持可比性，而新任务和不熟悉的约束则可以测试转移和发现。这些协议应确定继承的变化是否会改变后续的改进能力，当增益达到稳定水平时，以及明显的加速是否能够在额外的搜索和评估工作的情况下继续存在。

资源感知改进和人类协作。激励 RSI 的效率目标需要考虑整个改进过程。更快的内核、更好的训练配方或更长的无人值守运行可能仍然需要大量的候选生成、评估、基础设施维护和人工审查。 Meta 的回归分析工作流程保留了对拟议变更的工程审查 [5]； Humanlaya 还保留人工验证并报告处理时间和质量。这些实践说明了为什么应该将自主性与所需人力的数量和类型一起评估。更强的基础模型和更多的并行试验可能会进一步混淆改进策略之间的比较。

未来的工作应该研究如何在诊断、经验获取、候选人搜索、验证和部署监控之间分配总预算。廉价的筛查可以减少昂贵的试验，只要其预测仍然根据独立结果进行校准

。重复使用经过验证的工具和以前的故障可能会减少重复工作，但其维护成本也必须计算在内。评估应报告已验证的能力目标的时间和总成本，以及审查工作和返工。自适应停止是另一个研究重点：系统应该通过其预期的学习价值和不确定性来证明继续实验的合理性，并具有暂停、升级到人类或终止无效搜索的明确条件。

用于跨轮继承的可复制基础设施。后续系统需要了解如何获得改进，包括不成功的替代方案及其证据成立的条件。行业案例暴露出互补的要求：Lark 强调数据的时间性和故障归因； Humanlaya 保留版本质量系统更新； Hyra存储可执行的经验； Agent-Native 研究实验室提出了保留代码、规范、探索历史和经验痕迹的研究工件。这些做法激发了共享基础设施来跨轮传输研究证据，而这种继承的全部纵向效益仍有待确定。

可重用的工件格式应连接父状态、提议的更改、激励证据、评估配置、接受决策和后续使用。版本化的数据、环境和评估器将使结果更容易重放，并阐明哪些比较在更新后仍然有效。确定性重新提取可以检查报告的测量结果，形式验证可以建立与规范相关的属性；两者都不能单独证明实验或规范捕获了预期的功能。因此，公共评估应将可检查的工件与重放研究配对，并分别报告结果来源、工件可用性和独立复制。在数

据保密的情况下，可共享的评估子集和受控重放接口可以支持部分验证，同时保留数据的限制

48

can be checked.

Across these directions, progress toward genuine RSI should be demonstrated through repeated, attributable, and transferable improvements in the capacity to improve. The decisive evidence is that inherited changes help later rounds acquire more useful experience, discover better interventions, or validate successors more effectively under explicit resource and authority constraints.

7 Conclusion

In this report, we present an autonomy-centered roadmap for recursive self-improvement, spanning five stages: improvement execution, strategy selection, experience acquisition, environmental adaptation, and recursive meta-improvement. Taking the improvement loop as the unit of analysis, we clarify what systems change, what successors inherit, and which decisions remain human-controlled. We connect this roadmap to scientific discovery, embodied intelligence, software engineering, and healthcare, showing how feedback availability, verification costs, and deployment constraints shape progress toward RSI in each domain. Industry practices and preliminary empirical findings further ground the roadmap in demonstrated capabilities and current limitations. Long-horizon evaluation is needed to establish whether these mechanisms deliver transferable gains under explicit resource constraints and human oversight. Ultimately, the promise of RSI lies in enabling each generation of AI to make future improvement more reliable, efficient, and conducive to novel discoveries.

原文第 49 页 | 中文翻译初稿

7 结论

7
可以检查。

在这些方向上，真正的 RSI 的进展应该通过改进能力的重复、可归因和可转移的改进来证明。决定性的证据是，继承的变革有助于后来的几轮获得更多有用的经验，发现更好的干预措施，或者在明确的资源和权威约束下更有效地验证继任者。

7 结论

在本报告中，我们提出了一个以自主为中心的 递归自我改进（Recursive Self-Improvement） 路线图，跨越五个阶段：改进执行、策略选择、经验获取、环境适应和递归元改进。以改进循环为分析单位，我们阐明哪些系统发生了变化，哪些继任者继承了哪些内容，以及哪些决策仍然由人类控制。我们将此路线图与科学发现、具身智能（Embodied Intelligence）、软件工程和医疗保健联系起来，展示反馈可用性、验证成本和部署约束如何影响每个领域的 RSI 进展。行业实践和初步实证研究结果进一步奠定了已证明能力和当前局限性的路线图。需要进行 长时程（Long-Horizon） 评估，以确定这些机制是否在明确的资源限制和人力监督下提供可转移的收益。最终，RSI 的承诺在于使每一代人工智能都能使未来的改进更加可靠、高效，并有利于新的发现。

49

References

[1] Moonshot AI, "Kimi K3: Open frontier intelligence," Kimi Research Blog, Jul. 2026. [Online]. Available: https://www.kimi.com/en/blog/kimi-k3

[2] Alibaba Cloud, "Alibaba unveils Qwen3.8-Max: Its largest and most capable flagship model to date," Alibaba Cloud Press Release, Aug. 2026. [Online]. Available: https://www.alibabacloud.com/en/press-room/alibaba-unveils-qwen3-8-max

[3] OpenAI, "GPT-5.6: Frontier intelligence that scales with your ambition," OpenAI Product Announcement, Jul. 2026. [Online]. Available: https://openai.com/index/gpt-5-6/

[4] I. Moshkov, D. Hanley, I. Sorokin, S. Toshniwal, C. Henkel, B. Schifferer, W. Du, and I. Gitman, "Aimo-2 winning solution: Building state-of-the-art mathematical reasoning models with openmathreasoning dataset," *arXiv preprint arXiv:2504.16891*, 2025.

[5] Meta Engineering, "Capacity efficiency at meta: How unified ai agents optimize performance at hyperscale," https://engineering.fb.com/2026/04/16/developer-tools/capacity-efficiency-at-meta-how-unified-ai-agents-optimize-performance-at-hyperscale/, Apr. 2026, published April 16, 2026; accessed September 9, 2026.

[6] T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. Fishman, M. Aljubeh, P. Thacker, L. Fauconnet *et al.*, "Gdpval: Evaluating ai model performance on real-world economically valuable tasks," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 24 005–24 040.

[7] L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi *et al.*, "Humanity's last exam," *arXiv preprint arXiv:2501.14249*, 2025.

[8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?" in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 54 107–54 157.

[9] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.

[10] M. Ferrari, P. Tillet, A. Ibrahim, J. Gershenson, and S. Coffey, "How GPT-5.6 fuses frontier intelligence with frontier efficiency," OpenAI Engineering, July 2026, published July 29, 2026; accessed September 3, 2026.

[11] S. Jeong, M. Kim, and T. Kim, "Agentic neural architecture search," *arXiv preprint arXiv:2607.07984*, 2026. [Online]. Available: https://arxiv.org/abs/2607.07984

[12] DeepSeek-AI, A. Liu, A. Mei *et al.*, "Deepseek-v3.2: Pushing the frontier of open large language models," 2025. [Online]. Available: https://arxiv.org/abs/2512.02556

[13] Anthropic, "How we built our multi-agent research system," 2025. [Online]. Available: https://www.anthropic.com/engineering/multi-agent-research-system

[14] J. Schmidhuber, "Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements," *arXiv preprint cs/0309048*, 2003. [Online]. Available: https://arxiv.org/abs/cs/0309048

[15] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai, "Self-taught optimizer (STOP): Recursively self-improving code generation," in *Conference on Language Modeling*, 2024. [Online]. Available: https://arxiv.org/abs/2310.02304

[16] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune, "Darwin gödel machine: open-ended evolution of self-improving agents," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 104 223–104 294.

[17] Z. Shi, B. He, Y. Sang, H. Lu, and B. Dumoulin, "A-evolve-training: Autonomous post-training of a 30b model," *arXiv preprint arXiv:2606.20657*, 2026. [Online]. Available: https://arxiv.org/abs/2606.20657

[18] A. Razzhigaev, A. Gritsaev, A. Kaznacheev, N. Dragunov, R. Yampolskiy, and A. Kuznetsov, "Ouroboros: A self-developing frontier coding agent with reviewed core evolution," 2026. [Online]. Available: https://arxiv.org/abs/2608.08311

[19] X. Yin, X. Wang, L. Pan, L. Lin, X. Wan, and W. Y. Wang, "Gödel agent: A self-referential agent framework for recursive self-improvement," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025. [Online]. Available: https://aclanthology.org/2025.acl-long.1354/

原文第 50 页 | 中文翻译初稿

参考文献

7
参考文献

- [1] Moonshot AI，“Kimi K3：开放前沿智能”，Kimi 研究博客，2026 年 7 月。[在线]。可用：https://www.kimi.com/en/blog/kimi-k3

- [2] 阿里云，“阿里巴巴推出 Qwen3.8-Max：迄今为止规模最大、功能最强大的旗舰机型”，阿里云新闻稿，2026 年 8 月。[在线]。可用：https://www.alibabacloud.com/en/press-room/alibaba-unveils-qwen3-8-max

- [3] OpenAI，“GPT-5.6：随你的雄心而扩展的前沿智能”，OpenAI 产品公告，2026 年 7 月。[在线]。可用：https://openai.com/index/gpt-5-6/

- [4] I. Moshkov、D. Hanley、I. Sorokin、S. Toshniwal、C. Henkel、B. Schifferer、W. Du 和 I. Gitman，“Aimo-2 获胜解决方案：使用 openmathreasoning 数据集构建最先进的数学推理模型”，arXiv 预印本 arXiv:2504.16891，2025 年。

- [5] Meta Engineering，“meta 的容量效率：统一人工智能智能体如何优化超大规模性能”，https://engineering.fb.com/2026/04/16/developer-tools/capacity-efficiency-at-meta-how-unified-ai-agents-optimize-performance-at-hyperscale/，2026 年 4 月，2026 年 4 月 16 日发布；访问日期：2026 年 9 月 9 日。

- [6] T. Patwardhan、R. Dias、E. Proehl、G. Kim、M. Wang、O. Watkins、S. Fishman、M. Aljubeh、P. Thacker、L. Fauconnet et al.，“Gdpval：评估人工智能模型在现实世界中具有经济价值的任务中的性能”，载于_国际学习表示会议_，第 1 卷。 2026 年，2026 年，第 24 005-24 040 页。

- [7] L. Phan、A. Gatti、Z. Han、N. Li、J. Hu、H.Zhang、C.B.C.Zhang、M.Shaaban、J.Ling、S.Shi_et al._，“人类的最后一次考试”，arXiv 预印本 arXiv：2501.14249，2025 年。

- [8] C. E. Jimenez、J. Yang、A. Wettig、S. Yao、K. Pei、O. Press 和 K. Narasimhan，“Swe-bench：语言模型能否解决现实世界的 github 问题？”在_国际学习表征会议_，卷。 2024 年，2024 年，第 54 107-54 157 页。

- [9] S. Yao、N. Shinn、P. Razavi 和 K. Narasimhan，“τ -bench：现实世界领域中工具-智能体-用户交互的基准”，arXiv 预印本 arXiv：2406.12045，2024 年。

- [10] M. Ferrari、P. Tillet、A. Ibrahim、J. Gershenson 和 S. Coffey，“GPT-5.6 如何将前沿智能与前沿效率融合”，OpenAI Engineering，2026 年 7 月，2026 年 7 月 29 日出版；访问日期：2026 年 9 月 3 日。

- [11] S. Jeong、M. Kim 和 T. Kim，“智能体神经架构搜索”，arXiv 预印本 arXiv：2607.07984，2026 年。[在线]。可用：https://arxiv.org/abs/2607.07984

- [12] DeepSeek-AI，A. Liu，A. Mei 等人，“Deepseek-v3.2：推动开放大型语言模型的前沿”，2025 年。[在线]。可用：https://arxiv.org/abs/2512.02556

- [13] Anthropic，“我们如何构建多主体研究系统”，2025 年。[在线]。可用：https://www.anthropic.com/engineering/multi-agent-research-system

- [14] J. Schmidhuber，“哥德尔机：自我参照通用问题解决者可证明最佳自我改进”，arXiv 预印本 cs/0309048，2003 年。[在线]。可用：https://arxiv.org/abs/cs/0309048

- [15] E. Zelikman、E. Lorch、L. Mackey 和 A. T. Kalai，“自学优化器 (STOP)：递归 自我改进（Self-Improving）代码生成”，《语言建模会议》，2024 年。[在线]。可用：https://arxiv.org/abs/2310.02304

- [16] J. 张、S. Hu、C. Lu、R. Lange 和 J. Clune，“达尔文哥德尔机：自我改进（Self-Improving）智能体的开放式进化”，《学习表示国际会议》，第 1 卷。 2026 年，2026 年，第 104 223-104 294 页。

- [17] Z. Shi、B. He、Y. Sang、H. Lu 和 B. Dumoulin，“A-evolve-training：30b 模型的自主后期训练”，arXiv 预印本 arXiv：2606.20657，2026 年。[在线]。可用：https://arxiv.org/abs/2606.20657

- [18] A. Razzhigaev、A. Gritsaev、A. Kaznacheev、N. Dragunov、R. Yampolskiy 和 A. Kuznetsov，“Ouroboros：一种自我开发的前沿编码智能体，其核心演化经过审查”，2026 年。[在线]。可用：https://arxiv.org/abs/2608.08311

- [19] X. Yin、X. Wang、L. Pan、L. Lin、X. Wan 和 W. Y. Wang，“哥德尔智能体：递归自我改进（Recursive Self-Improvement）的自指代智能体框架”，载于《计算语言学协会第 63 届年会论文集》，2025 年。[在线]。可用：https://aclanthology.org/2025.acl-long.1354/

50

[20] J. Wen, L. Qiu, J. Benton, J. H. Kirchner, and J. Leike, "Automated weak-to-strong researcher," Anthropic Alignment Science Blog, 2026. [Online]. Available: https://alignment.anthropic.com/2026/automated-w2s-researcher/

[21] A. Iacob, A. Jovanović, W. F. Shen, D. Burkhardt, M. Kurmanji, N. Tastan, L. Sani, N. A. E. Venanzi, A. Odonnat, Z. Cao, B. Marino, X. Qiu, and N. D. Lane, "The red queen gödel machine: Co-evolving agents and their evaluators," *arXiv preprint arXiv:2606.26294*, 2026. [Online]. Available: https://arxiv.org/abs/2606.26294

[22] G. Penedo, H. Kydlíček, L. B. allal, A. Lozhkov, M. Mitchell, C. Raffel, L. Von Werra, and T. Wolf, "The fineweb datasets: Decanting the web for the finest text data at scale," in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 30 811–30 849. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/370df50ccfdf8bde18f8f9c2d9151bda-Paper-Datasets_and_Benchmarks_Track.pdf

[23] H. Zhang, S. Zhang, K. Li, C. Zhang, Y. Chen, Y. Zhang, L. Bai, and S. Hu, "Self-harness: Harnesses that improve themselves," *arXiv preprint arXiv:2606.09498*, 2026. [Online]. Available: https://arxiv.org/abs/2606.09498

[24] S. team, A. Bolton, A. Lerchner, A. Cordell, A. Moufarek, A. Bolt, A. Lampinen, A. Mitenkova, A. O. Hallingstad, B. Vujatovic, B. Li, C. Lu, D. Wierstra, D. P. Sawyer, D. Slater, D. Reichert, D. Vercelli, D. Hassabis, D. A. Hudson, D. Williams, E. Hirst, F. Pardo, F. Hill, F. Besse, H. Openshaw, H. Chan, H. Soyer, J. X. Wang, J. Clune, J. Agapiou, J. Reid, J. Marino, J. Kim, K. Gregor, K. Sridhar, K. McKinney, L. Kampis, L. M. Zhang, L. Matthey, L. Wang, M. A. Raad, M. Loks-Thompson, M. Engelcke, M. Kecman, M. Jackson, M. Gazeau, O. Purkiss, O. Knagg, P. Stys, P. Mendolicchio, R. Hadsell, R. Ke, R. Faulkner, S. Chakera, S. S. Baveja, S. Legg, S. Kashem, T. Terzi, T. Keck, T. Harley, T. Scholtes, T. Roberts, V. Mnih, Y. Liu, Z. Wang, and Z. Ghahramani, "Sima 2: A generalist embodied agent for virtual worlds," 2025. [Online]. Available: https://arxiv.org/abs/2512.04797

[25] Y. Li, Y. Miao, Y. Shen, and Y. Liu, "Pando: Efficient multimodal ai agents via online skill distillation," *arXiv preprint arXiv:2605.24785*, 2026.

[26] Z. Tao, T.-E. Lin, X. Chen, H. Li, Y. Wu, Y. Li, Z. Jin, F. Huang, D. Tao, and J. Zhou, "A survey on self-evolution of large language models," 2024. [Online]. Available: https://arxiv.org/abs/2404.14387

[27] H.-a. Gao, J. Geng, W. Hua, M. Hu, X. Juan, H. Liu, S. Liu, J. Qiu, X. Qi, Q. Ren, Y. Wu, H. Wang, H. Xiao, Y. Zhou, S. Zhang, J. Zhang, J. Xiang, Y. Fang, Q. Zhao, D. Liu, C. Qian, Z. Wang, M. Hu, H. Wang, Q. Wu, H. Ji, and M. Wang, "A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence," *Transactions on Machine Learning Research*, Jan. 2026. [Online]. Available: https://openreview.net/forum?id=CTr3bovS5F

[28] J. Fang, Y. Peng, X. Zhang, Y. Wang, X. Yi, G. Zhang, Y. Xu, B. Wu, S. Liu, Z. Li, Z. Ren, N. Aletras, X. Wang, H. Zhou, and Z. Meng, "A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems," 2025. [Online]. Available: https://arxiv.org/abs/2508.07407

[29] Z. Ren, Y. Chen, D. Guo, G. Rong, T. Li, R. B. Xiong, Q. Lan, W. Wang, L. Nanbo, Y. Yang, M. Zhuge, and J. Schmidhuber, "Self-improvements in modern agentic systems: A survey," 2026. [Online]. Available: https://arxiv.org/abs/2607.13104

[30] S. Liu, Z. Lin, Y. Zhang, Y. Ren, Y. Wu, Y. Li, Z. Wang, Z. Fu, and J. Ye, "The path to recursive self-improving agents: Foundation, framework, and future directions," *Preprints*, Aug. 2026. [Online]. Available: https://www.preprints.org/manuscript/202608.0051

[31] Q. Zong, J. Liu, J. Shen, Z. Tang, L. Wu, Y. Liu, R. Wang, Z. Wang, W. Wang, C. Qian, X. Chen, and Y. Song, "Co-Evolution in Agentic Systems: Toward Self-Directed Evolution Beyond Human Design," 2026. [Online]. Available: https://arxiv.org/abs/2608.10299

[32] Y. Xu, W. Zhang, Y. Chen, X. Lin, and Y. Zhang, "Self-Evolving Agents as Dynamic Graph Transformation: A Survey and New Perspective," 2026. [Online]. Available: https://arxiv.org/abs/2608.18104

[33] D. Ye, Z. Zhang, H. Wang, and C. Miao, "A Survey on AI for AI: When the Improver Becomes the Improvee," Preprints.org, 2026, preprint. [Online]. Available: https://www.preprints.org/manuscript/202609.0591

[34] D. Li, B. Jiang, L. Huang, A. Beigi, C. Zhao, Z. Tan, A. Bhattacharjee, Y. Jiang, C. Chen, T. Wu, K. Shu, L. Cheng, and H. Liu, "From generation to judgment: Opportunities and challenges of LLM-as-a-judge," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, C. Christodoulopoulos,

原文第 51 页 | 中文翻译初稿

参考文献

7

- [20] J. Wen、L. Qiu、J. Benton、J. H. Kirchner 和 J. Leike，“自动从弱到强的研究人员”，人类排列科学博客，2026 年。[在线]。可用：https://alignment.anthropic.com/2026/automated-w2s-resea rcher/

- [21] A. Iacob、A. Jovanović、W. F. Shen、D. Burkhardt、M. Kurmanji、N. Tastan、L. Sani、N. A. E. Venanzi、A. Odonnat、Z. Cao、B. Marino、X. Qiu 和 N. D. Lane，“红皇后哥德尔机器：共同进化的智能体及其评估者”，arXiv 预印本arXiv:2606.26294，2026。[在线]。可用：https://arxiv.org/abs/2606.26294

- [22] G. Penedo、H. Kydlíček、L. B. allal、A. Lozhkov、M. Mitchell、C. Raffel、L. Von Werra 和 T. Wolf，“精细网络数据集：在规模上解析网络以获取最精细的文本数据”，《神经信息处理系统进展》、A. Globerson、L. Mackey、D. Belgrave、A. Fan、U. Paquet、J. Tomczak 和 C. 张，编辑，卷。 37. Curran Associates, Inc.，2024 年，第 30 811-30 849 页。[在线]。可用：https://proceedings.neurips.cc/p aper_files/paper/2024/file/370df50ccfdf8bde18f8f9c2d9151bda-Paper-Datasets_and_Benchmarks_Track.pdf

- [23] H.Zhang、S.Zhang、K.Li、C.Zhang、Y.Chen、Y.Zhang、L.Bai 和 S.Hu，“Self-运行支撑框架（Harness）：提高自己的 运行支撑框架（Harnesses）”，arXiv 预印本 arXiv：2606.09498，2026 年。[在线]。可用：https://arxiv.org/abs/2606.09498

- [24] S.队、A. Bolton、A. Lerchner、A. Cordell、A. Moufarek、A. Bolt、A. Lampinen、A. Mitenkova、A. O. Hallingstad、B. Vujatovic、B. Li、C. Lu、D. Wierstra、D. P. Sawyer、D. Slater、D. Reichert、D. Vercelli、D. Hassabis、 D. A. Hudson、D. Williams、E. Hirst、F. Pardo、F. Hill、F. Besse、H. Openshaw、H. Chan、H. Soyer、J. X. Wang、J. Clune、J. Agapiou、J. Reid、J. Marino、J. Kim、K. Gregor、K. Sridhar、K. McKinney、L. Kampis、L. M. 张、L. Matthey、L. Wang、M. A. Raad、M. Loks-Thompson、M. Engelcke、M. Kecman、M. Jackson、M. Gazeau、O. Purkiss、O. Knagg、P. Stys、P. Mendolicchio、R. Hadsell、R. Ke、R. Faulkner、S. Chakera、S. S. Baveja、S. Legg、S. Kashem、T. Terzi、T. Keck、T. Harley、T. Scholtes、T. Roberts、V. Mnih、Y. Liu、Z. Wang 和 Z. Ghahramani，“Sima 2：虚拟世界的多面手具身智能体”，2025 年。[在线]。可用：https://arxiv.org/abs/2512.04797

- [25] Y. Li、Y. Miao、Y. Shen 和 Y. Liu，“Pando：通过在线技能蒸馏实现高效多模式人工智能智能体”，arXiv 预印本 arXiv：2605.24785，2026 年。

- [26] Z.陶，T.-E。 Lin，X. Chen，H. Li，Y. Wu，Y. Li，Z. Jin，F. Huang，D.Tao，and J. Zhou，“大语言模型自我进化研究”，2024。[在线]。可用：https://arxiv.org/abs/2404.14387

- [27] H.-a。高，耿，华，胡，X，娟，刘，刘，刘，邱，齐，任，吴，王，肖，周，张，张，向，方，赵，刘，钱，王，胡，王，吴，季，王，“调查自我进化智能体：在通往人工智能的道路上进化什么、何时、如何以及何处”，机器学习研究交易，2026 年 1 月。[在线]。可用：https://openreview.net/forum?id=CTr3bovS5F

- [28] J. Fang、Y. Peng、X. 张、Y. Wang、X. Yi、G. 张、Y. Xu、B. Wu、S. Liu、Z. Li、Z. Ren、N. Aletras、X. Wang、H. Zhou 和 Z

。孟，“自我进化人工智能智能体的全面调查：连接 基础模型（Foundation Models） 和终身智能体系统的新范式”，2025 年。[在线]。可用：https://arxiv.org/abs/2508.07407

- [29] Z. Ren，Y. Chen，D.Guo，G.Rong，T.Li，R.B.Xiong，Q.Lan，W.Wang，L.Nanbo，Y.Yang，M.Zhuge，J.Schmidhuber，“现代智能体系统中的自我改进：一项调查”，2026年。[在线]。可用：https://arxiv.org/abs/2607.13104

- [30] S. Liu、Z. Lin、Y. Zhang、Y. Ren、Y. Wu、Y. Li、Z. Wang、Z. Fu 和 J. Ye，“递归自我改进（Recursive Self-Improving） 智能体之路：基础、框架和未来方向”，预印本，2026 年 8 月。[在线]。可用：https://www.preprints.org/manuscript/202608.0051

- [31] Q. Zong、J. Liu、J. Shen、Z. Tang、L. Wu、Y. Liu、R. Wang、Z. Wang、W. Wang、C. qian、X. Chen 和 Y. Song，“智能体系统中的协同进化：迈向超越人类设计的自我导向进化”，2026 年。[在线]。可用：https://arxiv.org/abs/2608.10299

- [32] Y. Xu、W. Zhang、Y. Chen、X. Lin 和 Y. Zhang，“作为动态图转换的自进化智能体：调查和新视角”，2026 年。[在线]。可用：https://arxiv.org/abs/2608.18104

- [33] D. Ye、Z. Zhang、H. Wang 和 C. Miao，“人工智能的人工智能调查：当改进者成为改进者”，Preprints.org，2026 年，预印本。 [在线的]。可用：https://www.preprints.org/manuscript/202609.0591

- [34] D. Li、B. Jiang、L. Huang、A. Beigi、C. Zhao、Z. Tan、A. Bhattacharjee、Y. Jiang、C. Chen、T. Wu、K. Shu、L. Cheng 和 H. Liu，“从生成到判断：法学硕士作为法官的机遇与挑战”，《2025 年自然语言处理经验方法会议论文集》， C.克里斯托杜洛普洛斯，

51

T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 2757–2791. [Online]. Available: https://aclanthology.org/2025.emnlp-main.138/

[35] A. Yehudai, L. Eden, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, A. Cohan, and M. Shmueli-Scheuer, "A Survey on Evaluation of LLM-based Agents," in *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, 2026, pp. 26 690–26 714. [Online]. Available: https://aclanthology.org/2026.findings-acl.1330/

[36] Z. Shan and F. Shao, "A Survey on Rubric-Guided Reinforcement Learning for Language Models," 2026, arXiv version 2, revised 2026-08-31; Accepted to Findings of EMNLP 2026. [Online]. Available: https://arxiv.org/abs/2608.27505

[37] H. Kim, D. Ryu, X. Yi, J. Yao, J. Lian, M. Huang, S. Duan, J. Bak, and X. Xie, "The Road to Artificial SuperIntelligence: A Comprehensive Survey of Superalignment," 2024, arXiv version 4, revised 2026-06-17. [Online]. Available: https://arxiv.org/abs/2412.16468

[38] P. Slattery, A. K. Saeri, E. A. C. Grundy, J. Graham, M. Noetel, R. Uuk, J. Dao, S. Pour, S. Casper, and N. Thompson, "The AI risk repository: A meta-review, database, and taxonomy of risks from artificial intelligence," 2024, arXiv version 3, revised 2026-05-05. [Online]. Available: https://arxiv.org/abs/2408.12622

[39] L. Pan, M. Saxon, W. Xu, D. Nathani, X. Wang, and W. Y. Wang, "Automatically correcting large language models: Surveying the landscape of diverse automated correction strategies," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 484–506, 2024. [Online]. Available: https://aclanthology.org/2024.tacl-1.27/

[40] V. Venktesh, M. Rathee, and A. Anand, "Trust but Verify! A Survey on Verification Design for Test-time Scaling," 2025, arXiv version 3, revised 2025-09-09. [Online]. Available: https://arxiv.org/abs/2508.16665

[41] S. Niu, G. Chen, Y. Chen, Z. Wen, J. Hu, Z. Deng, D. Chen, S. Zhang, R. Chen, Z. Lian, S. Xu, G. Dai, Y. Zhang, W. Luo, Y. Zhang, M. Tan, and C. Deng, "A Survey on Self-Improving Test-Time Intelligence: Feedback-Driven Adapting, Learning, and Scaling at Inference," 2026, accepted by Machine Intelligence Research. [Online]. Available: https://arxiv.org/abs/2609.01679

[42] L. Long, R. Wang, R. Xiao, J. Zhao, X. Ding, G. Chen, and H. Wang, "On LLMs-driven synthetic data generation, curation, and evaluation: A survey," in *Findings of the Association for Computational Linguistics: ACL 2024*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 11 065–11 082. [Online]. Available: https://aclanthology.org/2024.findings-acl.658/

[43] J. Zheng, C. Shi, X. Cai, Q. Li, D. Zhang, C. Li, D. Yu, and Q. Ma, "Lifelong Learning of Large Language Model based Agents: A Roadmap," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026, arXiv version 2, revised 2026-01-11. [Online]. Available: https://arxiv.org/abs/2501.07278

[44] W.-C. Huang, W. Zhang, Y. Liang, Y. Bei, Y. Chen, T. Feng, X. Pan, Z. Tan, Y. Wang, T. Wei, S. Wu, R. Xu, L. Yang, R. Yang, W. Yang, C.-Y. Yeh, H. Zhang, H. Zhang, S. Zhu, H. P. Zou, W. Zhao, S. Wang, W. Xu, Z. Ke, Z. Hui, D. Li, Y. Wu, L. He, C. Wang, X. Xu, B. Huang, J. Tan, S. Heinecke, H. Wang, C. Xiong, A. A. Metwally, J. Yan, C.-Y. Lee, H. Zeng, Y. Xia, X. Wei, A. Payani, Y. Wang, H. Ma, W. Wang, C. Wang, Y. Zhang, X. E. Wang, Y. Zhang, J. You, H. Tong, X. Luo, X. Liu, Y. Sun, W. Wang, J. McAuley, J. Zou, J. Han, P. S. Yu, and K. Shu, "A Survey of Agent Memory in the Second Half: Towards Self-Evolving and Long-Horizon Agents," *Transactions on Machine Learning Research*, 2026, arXiv version 4, revised 2026-08-04; Survey Certification. [Online]. Available: https://arxiv.org/abs/2602.06052

[45] C. Zhou, H. Chai, W. Chen, Z. Guo, R. Shan, Y. Song, T. Xu, Y. Yang, A. Yu, W. Zhang, C. Zheng, J. Zhu, Z. Zheng, Z. Zhang, X. Lou, C. Zhang, Z. Fu, J. Wang, W. Liu, J. Lin, and W. Zhang, "Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering," 2026. [Online]. Available: https://arxiv.org/abs/2604.08224

[46] K. Ramnath, K. Zhou, S. Guan, S. S. Mishra, X. Qi, Z. Shen, S. Wang, S. Woo, S. Jeoung, Y. Wang, H. Wang, H. Ding, Y. Lu, Z. Xu, Y. Zhou, B. Srinivasan, Q. Yan, Y. Chen, H. Ding, P. Xu, and L. L. Cheong, "A systematic survey of automatic prompt optimization techniques," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 33 078–33 110. [Online]. Available: https://aclanthology.org/2025.emnlp-main.1681/

[47] Y.-A. Lee, G.-T. Yi, M.-Y. Liu, J.-C. Lu, G.-B. Yang, and Y.-N. Chen, "Compound AI systems optimization: A survey of methods, challenges, and future directions," in *Proceedings of the 2025 Conference on Empirical*

原文第 52 页 | 中文翻译初稿

参考文献

7

T. Chakraborty、C. Rose 和 V. Peng，编辑。中国苏州：计算语言学协会，2025 年 11 月，第 2757-2791 页。 [在线的]。可用：https://aclanthology.org/2025.emnlp-main.138/

- [35] A. Yehudai、L. Eden、A. Li、G. Uziel、Y. Zhu、R. Bar-Haim、A. Cohan 和 M. Shmueli-Scheuer，“基于 LLM 的智能体的评估调查”，载于_计算语言学协会的调查结果：ACL 2026_。计算语言学协会，2026 年，第 26 690-26 714 页。[在线]。可用：https://aclanthology.org/2026.findings-acl.1330/

- [36] Z. Shan 和 F. Shao，“Rubric-Guided 强化学习（Reinforcement Learning） for Language Models 的调查”，2026 年，arXiv 第 2 版，修订于 2026-08-31；接受 EMNLP 2026 的调查结果。[在线]。可用：https://arxiv.org/abs/2608.27505

- [37] H. Kim、D. Ryu、X. Yi、J. Yao、J. Lian、M. Huang、S. Duan、J. Bak 和 X. Xie，“人工智能之路：超对齐的综合综述”，2024 年，arXiv 第 4 版，修订于 2026-06-17。 [在线的]。可用：https://arxiv.org/abs/2412.16468

- [38] P. Slattery、A. K. Saeri、E. A. C. Grundy、J. Graham、M. Noetel、R. Uuk、J. Dao、S. Pour、S. Casper 和 N. Thompson，“AI 风险存储库：人工智能风险的元审查、数据库和分类”，2024 年，arXiv 第 3 版，修订于 2026 年 5 月 05 日。 [在线的]。可用：https://arxiv.org/abs/2408.12622

- [39] L. Pan、M. Saxon、W. Xu、D. Nathani、X. Wang 和 W. Y. Wang，“自动校正大型语言模型：调查各种自动校正策略的前景”，计算语言学协会交易，卷。 12，第 484-506 页，2024 年。[在线]。可用：https://aclanthology.org/2024.tacl-1.27/

- [40] V. Venktesh、M. Rathee 和 A. Anand，“信任但验证！测试时间扩展的验证设计调查”，2025 年，arXiv 第 3 版，修订于 2025 年 9 月 9 日。 [在线的]。可用：https://arxiv.org/abs/2508.16665

- [41] S. Niu、G. Chen、Y. Chen、Z. Wen、J. Hu、Z. Deng、D. Chen、S. 张、R. Chen、Z. Lian、S. Xu、G. Dai、Y. 张、W. Luo、Y. 张、M. Tan 和 C. Deng，《自我改进（Self-Improving） 测试时智能调查：反馈驱动的适应、学习和扩展》 Inference，”2026，被机器智能研究接受。 [在线的]。可用：https://arxiv.org/abs/2609.01679

- [42] L. Long、R. Wang、R. Xiao、J. Zhu、X. Ding、G. Chen 和 H. Wang，“关于法学硕士驱动的合成数据生成、管理和评估：一项调查”，载于_计算语言学协会的调查结果：ACL 2024_、L.-W. Ku，A. Martins 和 V. Srikumar，编辑。泰国曼谷：计算语言学协会，2024 年 8 月，第 11 065-11 082 页。[在线]。可用：https://aclanthology.org/2024.findings-acl.658/

- [43] J. Cheng、C. Shi、X. Cai、Q. Li、D. Zhang、C. Li、D. Yu 和 Q. Ma，“基于大型语言模型的智能体的终身学习：路线图”，IEEE 模式分析和机器智能汇刊，2026 年，arXiv 第 2 版，修订于 2026 年 1 月 11 日。 [在线的]。可用：https://arxiv.org/abs/2501.07278

- [44] W.-C。黄、张伟、梁Y、贝、陈、冯、潘、谭、王、魏、吴、徐、杨、杨、杨、C.-Y。叶红、张红、张红、朱书、邹海平、赵文书

。王、徐、柯、惠、李、吴、贺、王、徐、黄、谭、S. Heinecke、王、熊、A. A. Metwally、J. Yan、C.-Y。 Lee, H. Zeng, Y. Xia, X. Wei, A. Payani, Y. Wang, H. Ma, W. Wang, C. Wang, Y.Zhang, X.E. Wang, Y.Zhang, J. You, H. Tong, X. Luo, X. Liu, Y. Sun, W. Wang, J. McAuley, J. Zou, J. Han, P. S. Yu, and K. Shu, “下半场主体记忆调查：走向自我进化和 长时程智能体（Long-Horizon Agents），《机器学习研究汇刊》，2026 年，arXiv 第 4 版，2026-08-04 修订；调查认证。 [在线的]。可用：https://arxiv.org/abs/2602.06052

- [45] C. Zhou、H. Chai、W. Chen、Z.Guo、R. Shan、Y. Song、T. Xu、Y. Yang、A. Yu、W. Zhang、C. Cheng、J. Zhu、Z. Cheng、Z. Zhang、X. Lou、C. Zhang、Z. Fu、J. Wang、W. Liu、J. Lin 和 W. Zhang，《法学硕士智能体人的外化：对记忆、技能、协议和规则的统一审查》 运行支撑框架（Harness） 工程，”2026。[在线]。可用：https://arxiv.org/abs/2604.08224

- [46] K. Ramnath、K. Zhou、S.guan、S.S. Mishra、X. Qi、Z. Shen、S. Wang、S. Woo、S. Jeoung、Y. Wang、H. Wang、H. Ding、Y. Lu、Z. Xu、Y. Zhou、B. Srinivasan、Q. Yan、Y. Chen、H. Ding、P. Xu 和 L. L. Cheong，“自动提示优化的系统调查”技术”，《2025 年自然语言处理经验方法会议论文集》，C. Christodoulopoulos、T. Chakraborty、C. Rose 和 V. Peng，编辑。中国苏州：计算语言学协会，2025 年 11 月，第 33 078-33 110 页。[在线]。可用：https://aclanthology.org/2025.emnlp-main.1681/

- [47] Y.-A。李，G.-T。易，M.-Y。刘，J.-C。卢，G.-B。杨，和Y.-N。 Chen，“复合人工智能系统优化：方法、挑战和未来方向的调查”，载于_2025 年实证会议论文集_

52

Methods in Natural Language Processing, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 28 760–28 775. [Online]. Available: https://aclanthology.org/2025.emnlp-main.1463/

[48] L. Yue, K. R. Bhandari, C.-Y. Ko, D. Patel, S. Lin, N. Zhou, J. Gao, P.-Y. Chen, and S. Pan, "From Static Templates to Dynamic Runtime Graphs: A Survey of Workflow Optimization for LLM Agents," 2026. [Online]. Available: https://arxiv.org/abs/2603.22386

[49] J. Li, Z. Jin, T. Men, Y. Hao, K. Zhu, L. Wang, D. Huang, L. Wang, S. Hua, L. Wang, J. Gao, H. Yuan, R. Xu, K. Liu, and J. Zhao, "Agentic Environment Engineering for Large Language Models: A Survey of Environment Modeling, Synthesis, Evaluation, and Application," 2026. [Online]. Available: https://arxiv.org/abs/2606.12191

[50] T. Ding, A. Nannapaneni, B. Liu, and L. Zhang, "Autonomous Research Agents: A Survey of AI Scientists and the Verification Gap," 2026. [Online]. Available: https://arxiv.org/abs/2608.05179

[51] H. Zhou, H. Hu, T. Luo, Y. Shang, C. Fang, Z. Chen, L. Xiao, and Q. Zhang, "Self-Evolving Coding Agents," 2026, arXiv version 3, revised 2026-08-29. [Online]. Available: https://arxiv.org/abs/2608.03392

[52] C. Zhu, L. Tian, B. Tan, Z. Zhou, Y. Sun, Y. Wang, C. Lv, Y. Wen, Y. He, J. Lin, Y. Chen, C. W. Tan, Q. Wei, L. Zhao, B. Pu, K. Li, Y. Xue, and J. Lin, "The Path to Self-Evolving Clinical Systems: Scaling Medical Agents from Assistance to Autonomy," 2026, arXiv version 2, revised 2026-08-12. [Online]. Available: https://arxiv.org/abs/2607.11175

[53] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, J. Shen, G. Ye, H. Lin, J. Poulos, M. Wang, M. Nezhurina, J. Jitsev, D. Lu, O. M. Mastromichalakis, Z. Xu, Z. Chen, Y. Liu, R. Zhang, L. L. Chen, A. Kashyap, J.-L. Uslu, J. Li, J. Wu, M. Yan, S. Bian, V. Sharma, K. Sun, S. Dillmann, A. Anand, A. Lanpouthakoun, B. Koopah, C. Hu, E. Guha, G. H. S. Dreiman, J. Zhu, K. Krauth, L. Zhong, N. Muennighoff, R. Amanfu, S. Tan, S. Pimpalgaonkar, T. Aggarwal, X. Lin, X. Lan, X. Zhao, Y. Liang, Y. Wang, Z. Wang, C. Zhou, D. Heineman, H. Liu, H. Trivedi, J. Yang, J. Lin, M. Shetty, M. Yang, N. Omi, N. Raoof, S. Li, T. Y. Zhuo, W. Lin, Y. Dai, Y. Wang, W. Chai, S. Zhou, D. Wahdany, Z. She, J. Hu, Z. Dong, Y. Zhu, S. Cui, A. Saiyed, A. Kolbeinsson, J. Hu, C. M. Rytting, R. Marten, Y. Wang, A. Dimakis, A. Konwinski, and L. Schmidt, "Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces," 2026. [Online]. Available: https://arxiv.org/abs/2601.11868

[54] W. Huang, C. Lee, L. Tng, and S. Ge, "Deepswe: Measuring frontier coding agents on original, long-horizon engineering tasks," 2026. [Online]. Available: https://arxiv.org/abs/2607.07946

[55] Z. Wang, T. Shi, J. He, M. Cai, J. Zhang, and D. Song, "Cybergym: Evaluating ai agents' real-world cybersecurity capabilities at scale," 2026. [Online]. Available: https://arxiv.org/abs/2506.02548

[56] S. Lee and D. Brumley, "Exploitbench: A capability ladder benchmark for llm cybersecurity agents," 2026. [Online]. Available: https://arxiv.org/abs/2605.14153

[57] D. Shepard and R. Salimans, "Automationbench," 2026. [Online]. Available: https://arxiv.org/abs/2604.18934

[58] J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese, "Browsecomp: A simple yet challenging benchmark for browsing agents," 2025. [Online]. Available: https://arxiv.org/abs/2504.12516

[59] W. Wang, P. Piękos, L. Nanbo, F. Laakom, Y. Chen, M. Ostaszewski, M. Zhuge, and J. Schmidhuber, "Huxley-g\"odel machine: Human-level coding agent development by an approximation of the optimal self-improving machine," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=T0EiEuhOOL

[60] S. Chen, S. Lin, Y. Shi, H. Lian, X. Gu, L. Yun, D. Chen, L. Cao, J. Liu, N. Xia, and Q. Wang, "Swe-exp: Experience-driven software issue resolution," 2026. [Online]. Available: https://arxiv.org/abs/2507.23361

[61] Z. Sun, Z. Liu, Y. Zang, Y. Cao, X. Dong, T. Wu, D. Lin, and J. Wang, "SEAgent: Self-evolving computer use agent with autonomous learning from experience," in *Forty-third International Conference on Machine Learning*, 2026. [Online]. Available: https://openreview.net/forum?id=AvBgmOGsdQ

[62] Z. Dai, S. He, H. Li, Q. Zhou, J. Li, M. Song, G. Long, H. Si, X. Yao, L. Zhang *et al.*, "Metis: Bridging text and code memory for self-evolving agents," *arXiv preprint arXiv:2606.24151*, 2026.

原文第 53 页 | 中文翻译初稿

参考文献

7

自然语言处理方法，C. Christodoulopoulos、T. Chakraborty、C. Rose 和 V. Peng，编辑。中国苏州：计算语言学协会，2025 年 11 月，第 28 760-28 775 页。[在线]。可用：https://aclanthology.org/2025.emnlp-main.1463/

- [48] L. Yue，K. R. Bhandari，C.-Y。 Ko，D. Patel，S. Lin，N. Zhou，J. Gau，P.-Y。 Chen 和 S. Pan，“从静态模板到动态运行时图：LLM 智能体工作流程优化调查”，2026 年。[在线]。可用：https://arxiv.org/abs/2603.22386

- [49] J. Li、Z. Jin、T. Men、Y.hao、K. Zhu、L. Wang、D. Huang、L. Wang、S. Hua、L. Wang、J. Gau、H. Yuan、R. Xu、K. Liu 和 J. Zhu，“大型语言模型的智能体环境工程：环境建模、综合、评估和应用综述”，2026 年。[在线]。可用：https://arxiv.org/abs/2606.12191

- [50] T. Ding、A. Nannapaneni、B. Liu 和 L. Zhang，“自主研究智能体：人工智能科学家调查和验证差距”，2026 年。[在线]。可用：https://arxiv.org/abs/2608.05179

- [51] H. Zhou、H. Hu、T. Luo、Y. Shang、C. Fang、Z. Chen、L. Xiao 和 Q. Zhang，“自我进化编码智能体”，2026 年，arXiv 第 3 版，修订于 2026 年 8 月 29 日。 [在线的]。可用：https://arxiv.org/abs/2608.03392

- [52] C. Zhu、L. Tian、B. Tan、Z. Zhou、Y. Sun、Y. Wang、C. Lv、Y. Wen、Y. He、J. Lin、Y. Chen、C. W. Tan、Q. Wei、L. Zhu、B. Pu、K. Li、Y. Xu 和 J. Lin，“自我进化临床系统之路：将医疗机构从辅助扩展到自主”，2026 年， arXiv 第 2 版，修订于 2026 年 8 月 12 日。 [在线的]。可用：https://arxiv.org/abs/2607.11175

- [53] M. A. Merrill、A. G. Shaw、N. Carlini、B. Li、H. Raj、I. Bercovich、L. Shi、J. Y. Shin、T. Walshe、E. K. Buchanan、J. Shen、G. Ye、H. Lin、J. Poulos、M. Wang、M. Nezhurina、J. Jitsev、D. Lu、O. M. Mastromichalakis、Z. Xu、Z.陈，Y. 刘，R. 张，L. L. 陈，A. Kashyap，J.-L。 Uslu、J. Li、J. Wu、M. Yan、S. Bian、V. Sharma、K. Sun、S. Dillmann、A. Anand、A. Lanpouthakoun、B. Koopah、C. Hu、E. Guha、G. H. S. Dreiman、J. Zhu、K. Krauth、L.zhong、N. Muennighoff、R. Amanfu、S. Tan、S. Pimpalgaonkar、T. Aggarwal、X. Lin、X. Lan、X. Zhao、Y. Liang、Y. Wang、Z. Wang、C. Zhou、D. Heineman、H. Liu、H. Trivedi、J. Yang、J. Lin、M. Shetty、M. Yang、N. Omi、N. Raoof、S. Li、T. Y. Zhuo、W. Lin、Y. Dai、Y. Wang、W. Chai、S. Zhou、D. Wahdany、Z. She、J. Hu、Z. Dong、Y. Zhu、S. Cui、A. Saiyed、A. Kolbeinsson、J. Hu、C. M. Rytting、R. Marten、Y. Wang、A. Dimakis、A. Konwinski 和 L. Schmidt，“终端工作台：在命令行界面中针对艰巨、现实的任务对智能体进行基准测试”，2026 年。[在线]。可用：https://arxiv.org/abs/2601.11868

- [54] W. Huang、C. Lee、L. Tng 和 S. Ge，“Deepswe：测量原始 长时程（Long-Horizon） 工程任务的前沿编码智能体”，2026 年。[在线]。可用：https://arxiv.org/abs/2607.07946

- [55] Z. Wang、T. Shi、J. He、M. Cai、J. Zhang 和 D. Song，“Cybergym：大规模评估人工智能智能体的现实世界网络安全能力”，2026 年。[在线]。可用：https://arxiv.org/abs/2506.02548

- [56] S. Lee 和 D. Brumley，“Exploitbench：LLM 网络安全智能体的能力阶梯基准”，2026 年。[在线]。可用：https://arxiv.org/abs/2605.14153

- [57] D.谢泼德和R

。 Salimans，“Automationbench”，2026。[在线]。可用：https://arxiv.org/abs/2604.18934

- [58] J. Wei、Z. Sun、S. Papay、S. McKinney、J. Han、I. Fulford、H. W. Chung、A. T. Passos、W. Fedus 和 A. Glaese，“Browsecomp：浏览智能体的简单但具有挑战性的基准”，2025 年。[在线]。可用：https://arxiv.org/abs/2504.12516

- [59] W. Wang、P. Piękos、L. Nanbo、F. Laakom、Y. Chen、M. Ostaszewski、M. Zhuge 和 J. Schmidhuber，“Huxley-g\”odel 机：通过近似最优 自我改进（Self-Improving） 机进行人类级编码智能体开发”，载于_第十四届学习表示国际会议_， 2026。[在线]。可用：https://openreview.net/forum?id=T0EiEuhOOL

- [60] S. Chen、S. Lin、Y. Shi、H. Lian、X. Gu、L. Yun、D. Chen、L. Cao、J. Liu、N. Xia 和 Q. Wang，“Swe-exp：体验驱动的软件问题解决”，2026 年。[在线]。可用：https://arxiv.org/abs/2507.23361

- [61] Z. Sun、Z. Liu、Y. Zang、Y. Cao、X. Dong、T. Wu、D. Lin 和 J. Wang，“SEAgent：具有自主学习经验的自我进化计算机使用智能体”，载于《第四十三届机器学习国际会议》，2026 年。[在线]。可用：https://openreview.net/forum?id=AvBgmOGsdQ

- [62] Z. Dai、S. He、H. Li、Q. Zhou、J. Li、M. Song、G. Long、H. Si、X. Yao、L. Zhang et al.，“Metis：为自我进化智能体桥接文本和代码记忆”，arXiv 预印本 arXiv：2606.24151，2026 年。

53

[63] J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, and Y.-G. Jiang, "Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses," 2026. [Online]. Available: https://arxiv.org/abs/2604.25850

[64] H. Shi, Z. Xu, H. Wang, W. Qin, W. Wang, Y. Wang, Z. Wang, S. Ebrahimi, and H. Wang, "Continual learning of large language models: A comprehensive survey," *ACM Comput. Surv.*, vol. 58, no. 5, Nov. 2025. [Online]. Available: https://doi.org/10.1145/3735633

[65] R. Wang, J. Lehman, J. Clune, and K. O. Stanley, "Paired open-ended trailblazer (poet): Endlessly generating increasingly complex and diverse learning environments and their solutions," 2019. [Online]. Available: https://arxiv.org/abs/1901.01753

[66] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems," in *International Conference on Learning Representations*, 2025.

[67] J. Zhang, J. Xiang, Z. Yu, F. Teng, X.-H. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu, "AFlow: Automating agentic workflow generation," in *International Conference on Learning Representations*, 2025.

[68] OpenAI, "Research engineer / research scientist / ai systems engineer, rsi," https://openai.com/careers/research-engineer-research-scientist-ai-systems-engineer-rsi-san-francisco/, 2026, accessed: 2026-09-08.

[69] Tencent, "Tencent releases and open-sources Tencent Hy4 preview," Tencent, Aug. 2026, published August 28, 2026; accessed September 5, 2026. [Online]. Available: https://www.tencent.com/tencent-releases-and-open-sources-tencent-hy4-preview/

[70] Anthropic Institute, "When ai builds itself," https://www.anthropic.com/institute/recursive-self-improvement, Jun. 2026, accessed: 2026-09-08.

[71] T. Wu, L. Luo, Y.-F. Li, S. Pan, T.-T. Vu, and G. Haffari, "Continual Learning for Large Language Models: A Survey," 2024, arXiv version 2, revised 2024-02-07. [Online]. Available: https://arxiv.org/abs/2402.01364

[72] J. Zheng, S. Qiu, C. Shi, and Q. Ma, "Towards lifelong learning of large language models: A survey," *ACM Comput. Surv.*, vol. 57, no. 8, Mar. 2025. [Online]. Available: https://doi.org/10.1145/3716629

[73] P. Trirat, W. Jeong, and S. J. Hwang, "Automl-agent: A multi-agent LLM framework for full-pipeline automl," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR / OpenReview.net, 2025.

[74] Y. Shang, Y. Li, K. Zhao, L. Ma, J. Liu, F. Xu, and Y. Li, "AgentSquare: Automatic LLM agent search in modular design space," in *International Conference on Learning Representations*, 2025.

[75] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, K. Liu, L. Maksin, T. Patwardhan, L. Weng, and A. Mądry, "MLE-bench: Evaluating machine learning agents on machine learning engineering," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=6s5uXNWGIh

[76] Y. Yamada, R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha, "The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search," 2025. [Online]. Available: https://arxiv.org/abs/2504.08066

[77] G. Starace, O. Jaffe, D. Sherburn, J. Aung, C. J. Shern, L. Maksin, R. Dias, E. Mays, B. Kinsella, W. Thompson, J. Heidecke, A. Glaese, and T. Patwardhan, "Paperbench: evaluating ai's ability to replicate ai research," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. ICML'25. JMLR.org, 2025.

[78] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel, "Promptbreeder: self-referential self-improvement via prompt evolution," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.

[79] T. Li, Y. Wang, Z. Chen, Z. Wang, L. Ma, and G.-J. Qi, "C-evolve: Consensus-based evolution for prompt groups," in *International Conference on Learning Representations*, C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust, Eds., vol. 2026, 2026, pp. 37 510–37 586. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2026/file/3f1351a10a3f904e6f3845a94275a909-Paper-Conference.pdf

[80] J. Chen, Z. Song, J. Liu, S. Zhou, H. Wu, H. Shi, C. Zhou, H. Li, X. Yang, D. Zhu *et al.*, "Decoevo: Score-decoupled co-evolution of solver and rubric-generator skills in text space," *arXiv preprint arXiv:2607.25675*, 2026.

原文第 54 页 | 中文翻译初稿

参考文献

7

- [63] J. Lin、S. Liu、C. Pan、L. Lin、S. Dou、Z. Xi、X. Huang、H. Yan、Z. Han、T. Gui 和 Y.-G。Jiang，“智能体 运行支撑框架（Harness） 工程：可观察性驱动的编码智能体 运行支撑框架（Harnesses） 自动演化”，2026 年。[在线]。可用：https://arxiv.org/abs/2604.25850

- [64] H. Shi、Z. Xu、H. Wang、W.qin、W. Wang、Y. Wang、Z. Wang、S. Ebrahimi 和 H. Wang，“大语言模型的 持续学习（Continual Learning）：综合调查”，ACM Comput。 Surv.，卷。 58，没有。 2025 年 11 月 5 日。[在线]。可用：https://doi.org/10.1145/3735633

- [65] R. Wang、J. Lehman、J. Clune 和 K. O. Stanley，“配对开放式开拓者（诗人）：不断生成日益复杂和多样化的学习环境及其解决方案”，2019 年。[在线]。可用：https://arxiv.org/abs/1901.01753

- [66] S. Hu、C. Lu 和 J. Clune，“智能体系统的自动化设计”，《学习表示国际会议》，2025 年。

- [67] 张建，项建，余子，滕峰，X.-H。 Chen、J. Chen、M. Zhuge、X. Cheng、S. Hong、J. Wang、B. Cheng、B. Liu、Y. Luo 和 C. Wu，“AFlow：自动化智能体工作流生成”，《学习表示国际会议》，2025 年。

- [68] OpenAI，“研究工程师/研究科学家/人工智能系统工程师，rsi”，https://openai.com/careers/research-scientist-ai-systems-engineer-rsi-san-francisco/，2026 年，访问时间：2026-09-08。

- [69] 腾讯，“腾讯发布并开源腾讯 Hy4 预览版”，腾讯，2026 年 8 月，2026 年 8 月 28 日发布；访问日期：2026 年 9 月 5 日。[在线]。获取地址：https://www.tencent.com/tencent-releases-and-open-sources-tencent-hy4-preview/

- [70] 人类研究所，“当人工智能构建自身时”，https://www.anthropic.com/institute/recursive-自我改进（Self-Improvement），2026 年 6 月，访问时间：2026-09-08。

- [71] 吴涛，罗丽，Y.-F。李生.潘,T.-T. Vu 和 G. Haffari，“大型语言模型的 持续学习（Continual Learning）：一项调查”，2024 年，arXiv 第 2 版，修订于 2024 年 2 月 07 日。 [在线的]。可用：https://arxiv.org/abs/2402.01364

- [72] J. Cheng、S. Qiu、C. Shi 和 Q. Ma，“走向大型语言模型的终身学习：一项调查”，ACM Comput。 Surv.，卷。 57、没有。 2025 年 3 月 8 日。[在线]。可用：https://doi.org/10.1145/3716629

- [73] P. Trirat、W. Jeong 和 S. J. Hwang，“Automl-agent：用于全流程 automl 的多智能体 LLM 框架”，载于 ICML，ser。机器学习研究论文集，卷。 267.PMLR/OpenReview.net，2025。

- [74] Y. Shang、Y. Li、K. Zhu、L. Ma、J. Liu、F. Xu 和 Y. Li，“AgentSquare：模块化设计空间中的自动 LLM 智能体搜索”，《学习表示国际会议》，2025 年。

- [75] J. S. Chan、N. Chowdhury、O. Jaffe、J. Aung、D. Sherburn、E. Mays、G. Starace、K. Liu、L. Maksin、T. Patwardhan、L. Weng 和 A. Mądry，“MLE-bench：评估机器学习工程中的机器学习智能体”，《学习表示国际会议》，2025 年。[在线]。可用：https://openreview.net/forum?id=6s5uXNWGIh

- [76] Y. Yamada、R. T. Lange、C. Lu、S. Hu、C. Lu、J. Foerster、J. Clune 和 D. Ha，“人工智能科学家-v2：通过智能体树搜索进行车间级自动化科学发现”，2025 年。[在线]。可用：https://arxiv.org/abs/2504.08066

- [77] G. 斯塔雷斯，O

。 Jaffe、D. Sherburn、J. Aung、C. J. Shern、L. Maksin、R. Dias、E. Mays、B. Kinsella、W. Thompson、J. Heidecke、A. Glaese 和 T. Patwardhan，“Paperbench：评估人工智能复制人工智能研究的能力”，载于《第 42 届国际机器学习会议论文集》，序列号： ICML'25。 JMLR.org，2025 年。

- [78] C. Fernando、D. Banarse、H. Michalewski、S. Osindero 和 T. Rocktäschel，“Promptbreeder：通过即时进化实现自我指涉 自我改进（Self-Improvement）”，《第 41 届国际机器学习会议论文集》，系列。 ICML'24。 JMLR.org，2024 年。

- [79] T. Li、Y. Wang、Z. Chen、Z. Wang、L. Ma 和 G.-J。 Qi，“C-evolve：提示群体基于共识的进化”，《国际学习表征会议》，C. Vondrick、B. Hariharan、C. Raffel、L. Pinto、D. Yang 和 A. Faust，编辑，卷。 2026 年，2026 年，第 37 510-37 586 页。[在线]。可用：https://proceedings.iclr.cc/paper_files/paper/2026/file/3f1351a10a3f904e6f3845a94275a909-Paper-Conference.pdf

- [80] J. Chen, Z. Song, J. Liu, S. Zhou, H. Wu, H. Shi, C. Zhou, H. Li, X. Yang, D. Zhu et al.，“Decoevo：文本空间中解算器和标题生成器技能的分数解耦协同进化”，arXiv 预印本 arXiv:2607.25675，2026。

54

[81] Z. Wu, L. Zhang, T. Wang, R. Zhao, P. Andrews, C. Aloisi, and Y. He, "EDIT: Evidence-diagnosed intervention training for rule-faithful llm grading," *arXiv preprint arXiv:2606.06350*, 2026. [Online]. Available: https://arxiv.org/abs/2606.06350

[82] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab, "GEPA: Reflective prompt evolution can outperform reinforcement learning," in *International Conference on Learning Representations*, 2026.

[83] Y. Choi, D. Kim, J. Baek, and S. J. Hwang, "Multimodal prompt optimization: Why not leverage multiple modalities for mllms," 2026. [Online]. Available: https://arxiv.org/abs/2510.09201

[84] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," 2023. [Online]. Available: https://arxiv.org/abs/2305.16291

[85] S. Ouyang, J. Yan, I. Hsu, Y. Chen, K. Jiang, Z. Wang, R. Han, L. Le, S. Daruki, X. Tang *et al.*, "Reasoningbank: Scaling agent self-evolving with reasoning memory," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 94 327–94 354.

[86] J. Ni, Y. Liu, X. Liu, Y. Sun, M. Zhou, P. Cheng, D. Wang, E. Zhao, X. Jiang, and G. Jiang, "Trace2skill: Distill trajectory-local lessons into transferable agent skills," *arXiv preprint arXiv:2603.25158*, 2026.

[87] H. Lu and C. Xia, "Agent-agnostic end-to-end c/c++ application performance optimization," in *Proceedings of the 2026 International Conference on Supercomputing Workshops*. Association for Computing Machinery, 2026, pp. 65–69. [Online]. Available: https://doi.org/10.1145/3774895.3812199

[88] OpenAI, "Harness engineering: Leveraging codex in an agent-first world," https://openai.com/index/harness-engineering/, Feb. 2026, published February 11, 2026; accessed September 9, 2026.

[89] J. Su, Z. Wu, S. Huang, and W. Feng, "AIPC: Agent-based automation for ai model deployment with qualcomm ai runtime," *arXiv preprint arXiv:2604.14661*, 2026. [Online]. Available: https://arxiv.org/abs/2604.14661

[90] J. Jaber and O. Jaber, "Autokernel: Autonomous gpu kernel optimization via iterative agent-driven search," *arXiv preprint arXiv:2603.21331*, 2026.

[91] Y. Wu, J. Zhang, J. Shi, X. Lei, Q. Gu, Y. Zhang, Z. Wang, C. He, C. Huang, M. Song *et al.*, "Harnessdev: Can llms create and evolve their own agent harness?" *arXiv preprint arXiv:2609.01437*, 2026.

[92] OpenAI and Thrive Holdings, "Building self-improving tax agents with Codex," Engineering case study, 2026. [Online]. Available: https://openai.com/index/building-self-improving-tax-agents-with-codex/

[93] J. Zhang, B. Zhao, W. Yang, J. Foerster, J. Clune, M. Jiang, S. Devlin, and T. Shavrina, "Hyperagents," *arXiv preprint arXiv:2603.19461*, 2026. [Online]. Available: https://arxiv.org/abs/2603.19461

[94] Weco Team, "AIDE^2: The first evidence of recursive self-improvement," *Weco AI Blog*, Jul. 2026, industrial research disclosure; technical report and artifact release announced as forthcoming at the time of writing. [Online]. Available: https://www.weco.ai/blog/first-evidence-of-recursive-self-improvement

[95] Microsoft Research, "SynthLLM: Breaking the ai data wall with scalable synthetic data," https://www.microsoft.com/en-us/research/articles/synthllm-breaking-the-ai-data-wall-with-scalable-synthetic-data/, 2025, accessed September 9, 2026.

[96] Z. Wang, Y. Liang, X. Zhang, Q. Wu, S. Han, A. Bastos, R. Wang, C. Bansal, B. Peng, J. Gao, S. Rajmohan, and H. Yao, "SynthAgent: Adapting web agents with synthetic supervision," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2026, pp. 15 730–15 752. [Online]. Available: https://aclanthology.org/2026.acl-long.716/

[97] NVIDIA, "Nemotron-4 340b technical report," *arXiv preprint arXiv:2406.11704*, 2024. [Online]. Available: https://arxiv.org/abs/2406.11704

[98] H. Lu, Y. Wen, P. Cheng, R. Ding, J. Guo, H. Xu, C. Wang, H. Chen, xiaoxi jiang, and guanjunjiang, "Search self-play: Pushing the frontier of agent capability without supervision," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=ZmGirmNJqE

[99] A. Zhao, Y. Wu, T. Wu, Q. Xu, Y. Yue, M. Lin, S. Wang, Q. Wu, Z. Zheng, and G. Huang, "Absolute zero: Reinforced self-play reasoning with zero data," in *NeurIPS*, 2025.

原文第 55 页 | 中文翻译初稿

参考文献

7

- [81] Z. Wu、L. 张、T. Wang、R. Zhu、P. Andrews、C. Aloisi 和 Y. He，“编辑：忠实于规则的 llm 分级的证据诊断干预训练”，arXiv 预印本 arXiv：2606.06350，2026 年。[在线]。可用：https://arxiv.org/abs/2606.06350

- [82] L. A. Agrawal、S. Tan、D. Soylu、N. Ziems、R. Khare、K. Opsahl-Ong、A. Singhvi、H. Shandilya、M. J. Ryan、M. Jiang、C. Potts、K. Sen、A. G. Dimakis、I. Stoica、D. Klein、M. Zaharia 和 O. Khattab，《GEPA：反思》即时进化可以超越 强化学习（Reinforcement Learning）”，《学习表示国际会议》，2026 年。

- [83] Y. Choi、D. Kim、J. Baek 和 S. J. Hwang，“多模式提示优化：为什么不利用多种模式进行 mllms”，2026 年。[在线]。可用：https://arxiv.org/abs/2510.09201

- [84] G. Wang、Y. Xie、Y. Jiang、A. Mandlekar、C. Xiao、Y. Zhu、L. Fan 和 A. Anandkumar，“Voyager：具有大型语言模型的开放式实体智能体”，2023 年。[在线]。可用：https://arxiv.org/abs/2305.16291

- [85] S. Ouyang, J. Yan, I. Hsu, Y. Chen, K. Jiang, Z. Wang, R. Han, L. Le, S. Daruki, X. Tang et al., “Reasoningbank: Scaling agent self-evolving with Reasoning Memory”，《学习表示国际会议》，卷。2026 年，2026 年，第 94 327-94 354 页。

- [86] J. Ni、Y. Liu、X. Liu、Y. Sun、M. Zhou、P. Cheng、D. Wang、E. Zhu、X. Jiang 和 G. Jiang，“Trace2skill：将轨迹本地课程提炼为可转移的智能体技能”，arXiv 预印本 arXiv：2603.25158，2026 年。

- [87] H. Lu 和 C. Xia，“与智能体无关的端到端 c/c++ 应用程序性能优化”，载于《2026 年超级计算研讨会国际会议论文集》。计算机协会，2026 年，第 65-69 页。 [在线的]。可用：https://doi.org/10.1145/3774895.3812199

- [88] OpenAI，“运行支撑框架（Harness） 工程：在智能体优先的世界中利用 Codex”，https://openai.com/index/运行支撑框架（Harness）-engineering/，2026 年 2 月，2026 年 2 月 11 日发布；访问日期：2026 年 9 月 9 日。

- [89] J. Su、Z. Wu、S. Huang 和 W. Feng，“AIPC：使用高通人工智能运行时进行人工智能模型部署的基于智能体的自动化”，arXiv 预印本 arXiv：2604.14661，2026 年。[在线]。可用：https://arxiv.org/abs/2604.14661

- [90] J. Jaber 和 O. Jaber，“Autokernel：通过迭代智能体驱动搜索实现自主 GPU 内核优化”，arXiv 预印本 arXiv:2603.21331，2026 年。

- [91] Y. Wu、J. 张、J. Shi、X. Lei、Q. Gu、Y. 张、Z. Wang、C. He、C. Huang、M. Song 等人，“Harnessdev：llms 能否创建和发展自己的智能体 运行支撑框架（Harness）？” arXiv 预印本 arXiv:2609.01437，2026 年。

- [92] OpenAI 和 Thrive Holdings，“使用 Codex 构建 自我改进（Self-Improving） 税务智能体”，工程案例研究，2026 年。[在线]。可用：https://openai.com/index/building-自我改进（Self-Improving）

-tax-agents-with-codex/

- [93] J. 张、B. 赵、W. Yang、J. Foerster、J. Clune、M. Jiang、S. Devlin 和 T. Shavrina，“Hyperagents”，arXiv 预印本 arXiv：2603.19461，2026。[在线]。可用：https://arxiv.org/abs/2603.19461

- [94] Weco团队，“AIDE2：递归自我改进（Recursive Self-Improvement）的第一个证据”，Weco AI博客，2026年7月，行业研究披露；在撰写本文时宣布即将发布技术报告和工件。 [在线的]

。可用：https://www.weco.ai/blog/first-evidence-of-recursive-自我改进（Self-Improvement）

- [95] 微软研究院，“SynthLLM：利用可扩展的合成数据打破人工智能数据墙”，https://www.microsoft.com/en-us/research/articles/synthllm-writing-the-ai-data-wall-with-scalable-synthetic-data/，2025 年，2026 年 9 月 9 日访问。

- [96] Z. Wang、Y. Liang、X. Zhang、Q. Wu、S. Han、A. Bastos、R. Wang、C. Bansal、B. Peng、J. Gau、S. Rajmohan 和 H. Yao，“SynthAgent：通过综合监督调整网络智能体”，载于_计算语言学协会第 64 届年会论文集（第一卷：长论文）_。计算语言学协会，2026 年，第 15 730-15 752 页。[在线]。可用：https://aclanthology.org/2026.acl-long.716/

- [97] NVIDIA，“Nemotron-4 340b 技术报告”，arXiv 预印本 arXiv:2406.11704，2024 年。[在线]。可用：https://arxiv.org/abs/2406.11704

- [98] H. Lu, Y. Wen, P. Cheng, R. Ding, J.Guo, H. Xu, C. Wang, H. Chen, xiaoxi Jiang, and guanjun Jiang, “Search selfplay: Pushing the Frontier of Agent Capability withoutsupervised,” in 第十四届学习表征国际会议, 2026. [在线].可用：https://openreview.net/forum?id=ZmGirmNJqE

- [99] A.Zhao、Y.Wu、T.Wu、Q.Xu、Y.Yue、M.Lin、S.Wang、Q.Wu、Z.Zheng 和 G.Huang，“绝对零：零数据强化自我对弈推理”，《NeurIPS》，2025 年。

55

[100] K. Dong and T. Ma, "STP: self-play LLM theorem provers with iterative conjecturing and proving," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR / OpenReview.net, 2025.

[101] A. Wilf, P. Aggarwal, B. Parno, D. Fried, L.-P. Morency, P. P. Liang, and S. Welleck, "Propose, solve, verify: Self-play through formal verification," in *Forty-third International Conference on Machine Learning*, 2026. [Online]. Available: https://openreview.net/forum?id=Vuaq1qXCki

[102] X. Zeng, Y. Chen, L. Liu, C. Luo, Y. Chen, and Zhuangzhuoran, "Teaching llm to be persuasive: Reward-enhanced policy optimization for alignment from heterogeneous rewards," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics: Industry Track*, 2026. [Online]. Available: https://aclanthology.org/2026.acl-industry.34/

[103] C. Huang, W. Yu, X. Wang, H. Zhang, Z. Li, R. Li, J. Huang, H. Mi, and D. Yu, "R-zero: Self-evolving reasoning LLM from zero data," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=96apU6YzSO

[104] Q. Zhang, C. Hu, S. Upasani, B. Ma, F. Hong, V. Kamanuru, J. Rainton, C. Wu, M. Ji, H. Li *et al.*, "Agentic context engineering: Evolving contexts for self-improving language models," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 86 069–86 100.

[105] W. Zhang, X. Zhang, C. Zhang, L. Yang, J. Shang, Z. Wei, H. P. Zou, Z. Huang, Z. Wang, Y. Gao *et al.*, "Personaagent: Bridging memory and action for personalized llm agents," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 26 421–26 439.

[106] F. Shang and Y. Yang, "Hypothesis-driven skill optimization for llm agents," *arXiv preprint arXiv:2606.22330*, 2026.

[107] R. K. Arora, J. Wei, R. Soskin Hicks, P. Bowman, J. Quiñonero-Candela, F. Tsimpourlas, M. Sharman, M. Shah, A. Vallone, A. Beutel, J. Heidecke, and K. Singhal, "Healthbench: Evaluating large language models towards improved human health," *arXiv preprint arXiv:2505.08775*, 2025. [Online]. Available: https://arxiv.org/abs/2505.08775

[108] A. Karpathy, "autoresearch: Ai agents running research on single-gpu nanochat training automatically," GitHub repository, 2026. [Online]. Available: https://github.com/karpathy/autoresearch

[109] Y. Wu, J. Zhang, J. Shi, Y. Zhang, X. Lei, J. Zhou, Z. Wang, Y. Wu, H. Zhou, D. Wang *et al.*, "Aspire: Can models self-evolve from vague goals?" *arXiv preprint arXiv:2608.31111*, 2026.

[110] M. Suzgun, M. Yuksekgonul, F. Bianchi, D. Jurafsky, and J. Zou, "Dynamic cheatsheet: Test-time learning with adaptive memory," in *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 7080–7106.

[111] P. Wang, Z. Ma, Y. Chang, X. Luo, X. Yang, S. Feng, Y. Yang, and D. Li, "Self-evolving embodied agents via skill-harness evolution," 2026. [Online]. Available: https://arxiv.org/abs/2608.11350

[112] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in neural information processing systems*, vol. 36, pp. 46 534–46 594, 2023.

[113] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," 2023. [Online]. Available: https://arxiv.org/abs/2303.11366

[114] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *NeurIPS*, 2023.

[115] P. Banerjee, M. Moshtaghi, and A. Chadha, "APEX-EM: Non-parametric online learning for autonomous agents via structured procedural-episodic experience replay," 2026. [Online]. Available: https://arxiv.org/abs/2603.29093

[116] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "ExpeL: LLM agents are experiential learners," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 632–19 642. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/29936

[117] K. Tsui, "Self-correction bench: Uncovering and addressing the self-correction blind spot in large language models," 2026. [Online]. Available: https://arxiv.org/abs/2507.02778

原文第 56 页 | 中文翻译初稿

参考文献

7

- [100] K. Dong 和 T. Ma，“STP：具有迭代猜想和证明的自我对弈 LLM 定理证明器”，载于 ICML，ser。机器学习研究论文集，卷。 267.PMLR/OpenReview.net，2025。

- [101] A. Wilf、P. Aggarwal、B. Parno、D. Fried、L.-P。 Morency、P.P.Liang 和 S.Welleck，“提出、解决、验证：通过形式验证进行自我对弈”，载于_第四十三届机器学习国际会议_，2026 年。[在线]。可用：https://openreview.net/forum?id=Vuaq1qXCki

- [102] X. Zeng、Y. Chen、L. Liu、C. Luo、Y. Chen 和 Zhuangzhuoran，“Teaching llm to be persuasive: Reward-enhanced policy optimization foralignment from heheterogeneouswards”，载于《计算语言学协会第 64 届年会论文集：行业赛道》，2026 年。[在线]。可用：https://aclanthology.org/2026.acl-industry.34/

- [103] C. Huang、W. Yu、X. Wang、H. Zhang、Z. Li、R. Li、J. Huang、H. Mi 和 D. Yu，“R-zero：从零数据开始的自我进化推理法学硕士”，载于《第十四届学习表征国际会议》，2026 年。[在线]。可用：https://openreview.net/forum?id=96apU6YzSO

- [104] Q. Zhang, C. Hu, S. Upasani, B. Ma, F. Hong, V. Kamanuru, J. Rainton, C. Wu, M. Ji, H. Li et al.，“智能体上下文工程：自我改进（Self-Improving）语言模型的演化上下文”，《学习表示国际会议》，第 1 卷。 2026 年，2026 年，第 86 069-86 100 页。

- [105] W.Zhang、X.Zhang、C.Zhang、L.Yang、J.Shang、Z.Wei、H.P.Zou、Z.Huang、Z.Wang、Y.Gao_et al.，“Personaagent：桥接个性化 llm 智能体的记忆和行动”，载于计算语言学协会的调查结果：ACL 2026，2026 年，第 26 页421-26 439。

- [106] F. Shang 和 Y. Yang，“LLM 智能体的假设驱动技能优化”，arXiv 预印本 arXiv：2606.22330，2026 年。

- [107] R. K. Arora、J. Wei、R. Soskin Hicks、P. Bowman、J. Quiñonero-Candela、F. Tsimpourlas、M. Sharman、M. Shah、A. Vallone、A. Beutel、J. Heidecke 和 K. Singhal，“Healthbench：评估大型语言模型以改善人类健康”，arXiv 预印本arXiv:2505.08775，2025。[在线]。可用：https://arxiv.org/abs/2505.08775

- [108] A. Karpathy，“自动研究：人工智能智能体自动运行单 GPU Nanochat 训练研究”，GitHub 存储库，2026 年。[在线]。可用：https://github.com/karpathy/autoresearch

- [109] Y. Wu, J. Zhang, J. Shi, Y. Zhang, X. Lei, J. Zhou, Z. Wang, Y. Wu, H. Zhou, D. Wang et al.，“Aspire：模型能否从模糊目标中自我进化？” arXiv 预印本 arXiv:2608.31111，2026 年。

- [110] M. Suzgun、M. Yuksekgonul、F. Bianchi、D. Jurafsky 和 J. Zou，“动态备忘单：自适应记忆的测试时学习”，载于_计算语言学协会欧洲分会第 19 届会议论文集（第 1 卷：长论文）_，2026 年，第 110 页。 7080-7106。

- [111] P. Wang、Z. Ma、Y. Chang、X. Luo、X. Yang、S. Feng、Y. Yang 和 D. Li，“通过技能 运行支撑框架（Harness）进化实现自我进化的具体智能体”，2026 年。[在线]。可用：

https://arxiv.org/abs/2608.11350

- [112] A. Madaan、N. Tandon、P. Gupta、S. Hallinan、L. Gau、S. Wiegreffe、U. Alon、N. Dziri、S. Prabhumoye、Y

。 Yang et al. ，“自我细化：具有自我反馈的迭代细化”，神经信息处理系统的进展，卷。 36，第 46 534-46 594 页，2023 年。

- [113] N. Shinn、F. Cassano、E. Berman、A. Gopinath、K. Narasimhan 和 S. Yao，“反射：具有言语强化学习（Reinforcement Learning） 的语言主体”，2023 年。[在线]。可用：https://arxiv.org/abs/2303.11366

- [114] S. Yao、D. Yu、J. Zhu、I. Shafran、T. Griffiths、Y. Cao 和 K. Narasimhan，“思想之树：使用大型语言模型故意解决问题”，载于 NeurIPS，2023 年。

- [115] P. Banerjee、M. Moshtaghi 和 A. Chadha，“APEX-EM：通过结构化程序情景经验重放实现自主智能体的非参数在线学习”，2026 年。[在线]。可用：https://arxiv.org/abs/2603.29093

- [116] A.赵，D.黄，Q.徐，M.林，Y.-J。 Liu 和 G. Huang，“ExpeL：LLM 智能体人是体验式学习者”，载于_AAAI 人工智能会议论文集_，第 1 卷。 38，没有。 2024 年 17 日，第 19 632-19 642 页。[在线]。可用：https://ojs.aaai.org/index.php/AAAI/article/view/29936

- [117] K. Tsui，“自校正工作台：发现并解决大型语言模型中的自校正盲点”，2026 年。[在线]。可用：https://arxiv.org/abs/2507.02778

56

[118] G. Tyen, H. Mansoor, V. Carbune, P. Chen, and T. Mak, "LLMs cannot find reasoning errors, but can correct them given the error location," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 13 894–13 908. [Online]. Available: https://aclanthology.org/2024.findings-acl.826/

[119] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=IkmD3fKBPQ

[120] Y. Li, "Decomposing LLM self-correction: The accuracy-correction paradox and error depth hypothesis," *arXiv preprint arXiv:2601.00828*, 2026. [Online]. Available: https://arxiv.org/abs/2601.00828

[121] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" in *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, Eds., vol. 2024, 2024, pp. 36 545–36 593. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/9ddc141bdbf9d1db510cefff56c586ad-Paper-Conference.pdf

[122] Y. Verma, "Try again, don't look back: Blind resampling outperforms self-repair in small code models," *arXiv preprint arXiv:2607.26117*, 2026. [Online]. Available: https://arxiv.org/abs/2607.26117

[123] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," *Advances in Neural Information Processing Systems*, vol. 36, pp. 21 558–21 572, 2023. [Online]. Available: https://arxiv.org/abs/2305.01210

[124] Google Research, "Achieving 10,000x training data reduction with high-fidelity labels," https://research.google/blog/achieving-10000x-training-data-reduction-with-high-fidelity-labels/, Aug. 2025, published August 7, 2025; accessed September 9, 2026.

[125] M. Abdin, S. Agarwal, A. Awadallah, V. Balachandran, H. Behl, L. Chen, G. de Rosa, S. Gunasekar, M. Javaheripi, N. Joshi, P. Kauffmann, Y. Lara, C. C. T. Mendes, A. Mitra, B. Nushi, D. Papailiopoulos, O. Saarikivi, S. Shah, V. Shrivastava, V. Vineet, Y. Wu, S. Yousefi, and G. Zheng, "Phi-4-reasoning technical report," *arXiv preprint arXiv:2504.21318*, 2025. [Online]. Available: https://arxiv.org/abs/2504.21318

[126] NVIDIA, "NVIDIA NeMo Curator: Data curation for generative ai," https://docs.nvidia.com/nemo/curator/, 2025, accessed September 9, 2026.

[127] D. Chen, Y. Huang, Z. Ma, H. Chen, X. Pan, C. Ge, D. Gao, Y. Xie, Z. Liu, J. Gao, Y. Li, B. Ding, and J. Zhou, "Data-juicer: A one-stop data processing system for large language models," in *Proceedings of the 2024 International Conference on Management of Data*, 2024. [Online]. Available: https://doi.org/10.1145/3626246.3653385

[128] P. Liu, J. Wu, and U. Thakore, "Leveraging agents to debug nccl watchdog timeouts: A technical, repeatable workflow," @Scale: Systems & Reliability, Jun. 2026, published June 25, 2026.

[129] Amazon Web Services, "Agent toolkit for aws," https://aws.amazon.com/products/developer-tools/agent-toolkit-for-aws/, May 2026, announced May 6, 2026; accessed September 9, 2026.

[130] LinkedIn Engineering, "Contextual agent playbooks and tools: How linkedin gave ai coding agents organizational context," LinkedIn Engineering Blog, Jan. 2026, published January 27, 2026.

[131] OpenAI, "Research acceleration: The view inside OpenAI," OpenAI, Sep. 2026. [Online]. Available: https://openai.com/index/research-acceleration-view-inside-openai/

[132] E. Wang and D. Meyerzon, "How we optimized Dash's relevance judge with DSPy," Dropbox.Tech, Mar. 2026, published March 17, 2026; accessed September 5, 2026. [Online]. Available: https://dropbox.tech/machine-learning/optimizing-dropbox-dash-relevance-judge-with-dspy

[133] L. Quintanilla and V. Dibia, "Introducing agent optimizer in Foundry Agent Service," Microsoft Foundry Blog, Jun. 2026, published June 3, 2026; accessed September 5, 2026. [Online]. Available: https://devblogs.microsoft.com/foundry/agent-optimizer-build2026/

[134] D. Rodge, "Post-train NVIDIA Cosmos 3 in one day using agent skills," NVIDIA Technical Blog, Jul. 2026. [Online]. Available: https://developer.nvidia.com/blog/post-train-nvidia-cosmos-3-in-one-day-using-agent-skills/

[135] OpenAI, "GPT-6 Astra system card," OpenAI Deployment Safety Hub, Sep. 2026. [Online]. Available: https://deploymentsafety.openai.com/gpt-6-astra/vision

原文第 57 页 | 中文翻译初稿

参考文献

7

- [118] G. Tyen、H. Mansoor、V. Carbune、P. Chen 和 T. Mak，“LLM 无法发现推理错误，但可以在给定错误位置的情况下纠正它们”，载于_计算语言学协会的调查结果：ACL 2024_，2024 年，第 13 894-13 908 页。[在线]。可用：https://aclanthology.org/2024.findings-acl.826/

- [119] J. Huang、X. Chen、S. Mishra、H. S. Cheng、A. W. Yu、X. Song 和 D. Zhou，“大型语言模型尚不能自我纠正推理”，《国际学习表示会议》，2024 年。[在线]。可用：https://openreview.net/forum?id=IkmD3fKBPQ

- [120] Y. Li，“分解 LLM 自我校正：准确性校正悖论和错误深度假设”，arXiv 预印本 arXiv：2601.00828，2026 年。[在线]。可用：https://arxiv.org/abs/2601.00828

- [121] T.X.Olausson、J.P.Inala、C.Wang、J.Gao 和 A.Solar-Lezama，“自我修复是代码生成的灵丹妙药吗？”在《国际学习表征会议》中，B. Kim、Y. Yue、S. Chaudhuri、K. Fragkiadaki、M. Khan 和 Y. Sun 编辑，卷。 2024 年，2024 年，第 36 545-36 593 页。[在线]。可用：https://proceedings.iclr.cc/paper_files/paper/2024/file/9ddc141bdbf9d1db510cefff56c586ad-Paper-Conference.pdf

- [122] Y. Verma，“再试一次，不要回头：盲重采样在小代码模型中优于自我修复”，arXiv 预印本 arXiv：2607.26117，2026 年。[在线]。可用：https://arxiv.org/abs/2607.26117

- [123] J. Liu，C. S. Xia，Y. Wang，L. Zhang，“ChatGPT 生成的代码真的正确吗？对代码生成的大型语言模型的严格评估”，神经信息处理系统进展，卷。 36，第 21 558-21 572 页，2023 年。[在线]。可用：https://arxiv.org/abs/2305.01210

- [124] 谷歌研究，“通过高保真标签实现 10,000 倍的训练数据缩减”，https://research.google/blog/achieving-10000x-training-data-reduction-with-high-fidelity-labels/，2025 年 8 月，2025 年 8 月 7 日发布；访问日期：2026 年 9 月 9 日。

- [125] M. Abdin、S. Agarwal、A. Awadallah、V. Balachandran、H. Behl、L. Chen、G. de Rosa、S. Gunasekar、M. Javaheripi、N. Joshi、P. Kauffmann、Y. Lara、C. C. T. Mendes、A. Mitra、B. Nushi、D. Papailiopoulos、O. Saarikivi、S. Shah、V. Shrivastava、V. Vineet、Y. Wu、S. Yousefi 和 G. Cheng，“Phi-4 推理技术报告”，arXiv 预印本 arXiv：2504.21318，2025 年。[在线]。可用：https://arxiv.org/abs/2504.21318

- [126] NVIDIA，“NVIDIA NeMo Curator：生成式人工智能的数据管理”，https://docs.nvidia.com/nemo/curator/，2025 年，2026 年 9 月 9 日访问。

- [127] D. Chen、Y. Huang、Z. Ma、H. Chen、X. Pan、C. Ge、D. Gau、Y. Xie、Z. Liu、J. Gau、Y. Li、B. Ding 和 J. Zhou，“Data-juicer：大型语言模型的一站式数据处理系统”，载于《2024 年数据管理国际会议论文集》，2024 年。[在线]。可用：https://doi.org/10.1145/3626246.3653385

- [128] P. Liu、J. Wu 和 U. Thakore，“利用智能体调试 nccl 看门狗超时：技术性、可重复的工作流程”，@Scale：系统与可靠性，2026 年 6 月，2026 年 6 月 25 日发布。

- [129] Amazon Web Services，“aws 智能体工具包”，https://aws.amazon.com/products/developer-tools/agent-toolkit -for-aws/，2026 年 5 月，2026 年 5 月 6 日发布；访问日期：2026 年 9 月 9 日。

- [130] LinkedIn 工程，“上下文智能体手册和工具：linkedin 如何为人工智能编码智能体提供组织上下文”，LinkedIn 工程博客，2026 年 1 月，2026 年 1 月 27 日发布。

- [131] OpenAI，“研究加速：OpenAI 内部的观点”，OpenAI，2026 年 9 月。[在线]。可用：https://openai.com/index/research-acceleration-view-inside-openai/

- [132] E. Wang 和 D. Meyerzon，“我们如何使用 DSPy 优化 Dash 的相关性判断”，Dropbox.Tech，2026 年 3 月，2026 年 3 月 17 日出版；访问日期：2026 年 9 月 5 日。[在线]。可用：https://dropbox.tech/machine-learning/optimizing-dropbox-dash-relevance-judge-with-dspy

- [133] L. Quintanilla 和 V. Dibia，“在 Foundry 智能体服务中引入智能体优化器”，Microsoft Foundry 博客，2026 年 6 月，2026 年 6 月 3 日发布；访问日期：2026 年 9 月 5 日。[在线]。可用：https://devblogs.microsoft.com/foundry/agent-optimizer-build2026/

- [134] D. Rodge，“使用智能体技能在一天内对 NVIDIA Cosmos 3 进行后期训练”，NVIDIA 技术博客，2026 年 7 月。[在线]。可用：https://developer.nvidia.com/blog/post-train-nvidia-cosmos-3-in-one-day-using-agent-skills/

- [135] OpenAI，“GPT-6 Astra 系统卡”，OpenAI 部署安全中心，2026 年 9 月。[在线]。可用：https://deploymentsafety.openai.com/gpt-6-astra/vision

[136] J. Ning, X. Li, J. Zeng, H. Kang, and C. Xiong, "Auto research with specialist agents develops effective and non-trivial training recipes," 2026. [Online]. Available: https://arxiv.org/abs/2605.05724

[137] L. Soldaini, R. Kinney, A. Bhagia, D. Schwenk, D. Atkinson, R. Authur, B. Bogin, K. R. Chandu, J. Dumas, Y. Elazar, V. Hofmann, A. H. Jha, S. Kumar, L. Lucy, X. Lyu, N. Lambert, I. Magnusson, J. Morrison, N. Muennighoff, A. Naik, C. Nam, M. E. Peters, A. Ravichander, K. Richardson, Z. Shen, E. Strubell, N. Subramani, O. Tafjord, P. Walsh, L. Zettlemoyer, N. A. Smith, H. Hajishirzi, I. Beltagy, D. Groeneveld, J. Dodge, and K. Lo, "Dolma: an open corpus of three trillion tokens for language model pretraining research," in *ACL (1)*. Association for Computational Linguistics, 2024, pp. 15 725–15 788.

[138] . AI, :, A. Young, B. Chen, C. Li, C. Huang, G. Zhang, G. Zhang, G. Wang, H. Li, J. Zhu, J. Chen, J. Chang, K. Yu, P. Liu, Q. Liu, S. Yue, S. Yang, S. Yang, W. Xie, W. Huang, X. Hu, X. Ren, X. Niu, P. Nie, Y. Li, Y. Xu, Y. Liu, Y. Wang, Y. Cai, Z. Gu, Z. Liu, and Z. Dai, "Yi: Open foundation models by 01.ai," 2025. [Online]. Available: https://arxiv.org/abs/2403.04652

[139] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, "Qwen3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[140] M. Abdin, J. Aneja, H. Behl, S. Bubeck, R. Eldan, S. Gunasekar, M. Harrison, R. J. Hewett, M. Javaheripi, P. Kauffmann, J. R. Lee, Y. T. Lee, Y. Li, W. Liu, C. C. T. Mendes, A. Nguyen, E. Price, G. de Rosa, O. Saarikivi, A. Salim, S. Shah, X. Wang, R. Ward, Y. Wu, D. Yu, C. Zhang, and Y. Zhang, "Phi-4 technical report," 2024. [Online]. Available: https://arxiv.org/abs/2412.08905

[141] J. Li, A. Fang, G. Smyrnis, M. Ivgi, M. Jordan, S. Y. Gadre, H. Bansal, E. K. Guha, S. S. Keh, K. Arora, S. Garg, R. Xin, N. Muennighoff, R. Heckel, J. Mercat, M. F. Chen, S. Gururangan, M. Wortsman, A. Albalak, Y. Bitton, M. Nezhurina, A. Abbas, C. Hsieh, D. Ghosh, J. Gardner, M. Kilian, H. Zhang, R. Shao, S. M. Pratt, S. Sanyal, G. Ilharco, G. Daras, K. Marathe, A. Gokaslan, J. Zhang, K. R. Chandu, T. Nguyen, I. Vasiljevic, S. M. Kakade, S. Song, S. Sanghavi, F. Faghri, S. Oh, L. Zettlemoyer, K. Lo, A. El-Nouby, H. Pouransari, A. Toshev, S. Wang, D. Groeneveld, L. Soldaini, P. W. Koh, J. Jitsev, T. Kollar, A. Dimakis, Y. Carmon, A. Dave, L. Schmidt, and V. Shankar, "Datacomp-lm: In search of the next generation of training sets for language models," in *NeurIPS*, 2024.

[142] Y. He, C. Huang, Z. Li, J. Huang, and Y. Yang, "Visplay: Self-evolving vision-language models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2026, pp. 26 274–26 284.

[143] Y. Li, J. Yang, Z. Zheng, Z. Hu, Y. Sui, S. Wang, Y. He, and B. Hooi, "Apex: Autonomous policy exploration for self-evolving llm agents," *arXiv preprint arXiv:2605.21240*, 2026.

[144] K. Liang, J. Kruk, S. Qian, X. Yang, S. Bi, Y. Yao, S. Nie, M. Zhang, L. Liu, J. F. Fisac *et al.*, "Learning personalized agents from human feedback," *arXiv preprint arXiv:2602.16173*, 2026.

[145] S. A. Er, D. Ribeiro, Y. Virkar, S. Lakew, A. Kalyanpur, J. Gung, T. Delteil, and A. Gupta, "Memtoolagent: Leveraging memory for tool using agents based on environment and user feedback," *arXiv preprint arXiv:2606.07909*, 2026.

[146] Z. Bai, S. Li, T. Huang, and B. F. Karlsson, "Practice: From experience to expertise in self-evolving embodied agents," *arXiv preprint arXiv:2608.30760*, 2026.

[147] Y. Xiao, Y. Sun, H. Wu, W. Hui, W. Da, Z. Luo, M. Chuan, Y. Hu, W. Li, and C. Jiang, "Pilot in the loop: Live self-improvement for long-horizon agents," *arXiv preprint arXiv:2608.26530*, 2026.

[148] T. Wei, Z. Shi, M. Lin, B. He, Z. Liu, Y. Sang, Y. Bei, X. Ning, J. Zou, T.-W. Li *et al.*, "Evo-harness: Context-to-harness skill compilation for self-evolving agents," *arXiv preprint arXiv:2608.15071*, 2026.

[149] T. Wei, N. Sachdeva, B. Coleman, Z. He, Y. Bei, X. Ning, M. Ai, Y. Li, J. He, E. H. Chi *et al.*, "Evo-memory: Benchmarking llm agent test-time learning with self-evolving memory," *arXiv preprint arXiv:2511.20857*, 2025.

[150] J. Shi, S. Tao, Y. Wu, Z. Wang, J. Zhang, J. Liu, X. Lei, X. Zhang, S. Fang, Z. Tan *et al.*, "S3gym: Can llms turn self-testing and self-judging into self-improvement?" *arXiv preprint arXiv:2608.31100*, 2026.

原文第 58 页 | 中文翻译初稿

参考文献

7

- [136] J. Ning、X. Li、J. Zeng、H. Kang 和 C. Xiong，“与专业智能体的汽车研究开发出有效且重要的训练方案”，2026 年。[在线]。可用：https://arxiv.org/abs/2605.05724

- [137] L. Soldaini、R. Kinney、A. Bhagia、D. Schwenk、D. Atkinson、R. Authur、B. Bogin、K. R. Chandu、J. Dumas、Y. Elazar、V. Hofmann、A. H. Jha、S. Kumar、L. Lucy、X. Lyu、N. Lambert、I. Magnusson、J. Morrison、N. Muennighoff、 A. Naik、C. Nam、M. E. Peters、A. Ravichander、K. Richardson、Z. Shen、E. Strubell、N. Subramani、O. Tafjord、P. Walsh、L. Zettlemoyer、N. A. Smith、H. Hajishirzi、I. Beltagy、D. Groeneveld、J. Dodge 和 K. Lo，《Dolma：包含 3 万亿代币的开放语料库》语言模型预训练研究”，见 ACL (1) 。计算语言学协会，2024 年，第 15 725-15 788 页。

- [138] 。 AI, :, A. Young、B. Chen、C. Li、C. Huang、G. 张、G. 张、G. Wang、H. Li、J. Zhu、J. Chen、J. Chang、K. Yu、P. Liu、Q. Liu、S. Yue、S. Yang、S. Yang、W. Xie、W. Huang、X. Hu、X. Ren、X. Niu、P. Nie、Y. Li、Y. Xu、Y. Liu、Y. Wang、 Y. Cai、Z. Gu、Z. Liu 和 Z. Dai，“Yi：由 01.ai 打开 基础模型（Foundation Models）”，2025 年。[在线]。可用：https://arxiv.org/abs/2403.04652

- [139] A. 杨、A. 李、B. 杨、B. 张、B. 惠、B. 郑、B. 余、C. 高、C. 黄、C. 吕、C. 郑、D. 刘、F. 周、F. 黄、F. 胡、H. 葛、H. 伟、H. 林、J. 唐、J. 杨、J. 涂、J. 张、J. 杨、J. 杨、J. 周、J. 周、J. 林、K.党、包、杨、于丽、邓丽、李明、薛明、李明、张平、王平、朱庆、门、高、刘、罗、李、唐、尹、任、王、张、任、范、苏、张、张、万、刘、王、Z.崔，张，周，邱，“Qwen3技术报告”，2025。[在线]。可用：https://arxiv.org/abs/2505.09388

- [140] M. Abdin、J. Aneja、H. Behl、S. Bubeck、R. Eldan、S. Gunasekar、M. Harrison、R. J. Hewett、M. Javaheripi、P. Kauffmann、J. R. Lee、Y. T. Lee、Y. Li、W. Liu、C. C. T. Mendes、A. Nguyen、E. Price、G. de Rosa、O. Saarikivi、 A. Salim、S. Shah、X. Wang、R. Ward、Y. Wu、D. Yu、C. 张和 Y. 张，“Phi-4 技术报告”，2024 年。[在线]。可用：https://arxiv.org/abs/2412.08905

- [141] J. Li、A. Fang、G. Smyrnis、M. Ivgi、M. Jordan、S. Y. Gadre、H. Bansal、E. K. Guha、S. S. Keh、K. Arora、S. Garg、R. Xin、N. Muennighoff、R. Heckel、J. Mercat、M. F. Chen、S. Gururangan、M. Wortsman、A. Albalak、Y. Bitton、M. Nezhurina、A. Abbas、C. Hsieh、D. Ghosh、J. Gardner、M. Kilian、H. 张、R. Shao、S. M. Pratt、S. Sanyal、G. Ilharco、G. Daras、K. Marathe、A. Gokaslan、J. Zhang、K. R. Chandu、T. Nguyen、I. Vasiljevic、S. M. Kakade、S. Song、S. Sanghavi、F. Faghri、S. Oh、L. Zettlemoyer、K. Lo、A. El-Nouby、H. Pouransari、A. Toshev、S. Wang、D. Groeneveld、L. Soldaini、P. W. Koh、J. Jitsev、T. Kollar、A. Dimakis、Y. Carmon、A. Dave、L. Schmidt 和V. Shankar，“Datacomp-lm：寻找下一代语言模型训练集”，载于 NeurIPS，2024 年。

- [142] Y. He、C. Huang、Z. Li、J. Huang 和 Y. Yang，“Visplay：自进化视觉语言模型”，载于 _IEEE/CVF 计算机视觉和模式识别会议 (CVPR) 会议记录_，2026 年 6 月，第 26 274-26 284 页。

- [143] Y. Li、J. Yang、Z. Cheng、Z. Hu、Y

。 Sui、S. Wang、Y. He 和 B. Hooi，“Apex：自我进化 llm 智能体的自主策略探索”，arXiv 预印本 arXiv：2605.21240，2026 年。

- [144] K.Liang、J.Kruk、S.Qian、X.Yang、S.Bi、Y.Yao、S.Nie、M.Zhang、L.Liu、J.F.Fisac_et al._，“从人类反馈中学习个性化智能体”，arXiv 预印本 arXiv：2602.16173，2026 年。

- [145] S. A. Er、D. Ribeiro、Y. Virkar、S. Lakew、A. Kalyanpur、J. Gung、T. Delteil 和 A. Gupta，“Memtoolagent：根据环境和用户反馈利用智能体的工具内存”，arXiv 预印本 arXiv：2606.07909，2026 年。

- [146] Z. Bai、S. Li、T. Huang 和 B. F. Karlsson，“实践：从经验到自我进化具体智能体的专业知识”，arXiv 预印本 arXiv：2608.30760，2026 年。

- [147] Y. Shaw、Y. Sun、H. Wu、W. Hui、W. Da、Z. Luo、M. Chuan、Y. Hu、W. Li 和 C. Jiang，“循环中的飞行员：为 长时程智能体（Long-Horizon Agents） 直播 自我改进（Self-Improvement）”，arXiv 预印本 arXiv:2608.26530，2026。

- [148] T. Wei，Z. Shi，M. Lin，B. He，Z. Liu，Y. Sang，Y. Bei，X. Ning，J. Zou，T.-W. Li et al.，“Evo-运行支撑框架（Harness）：自我进化智能体的上下文到 运行支撑框架（Harness） 技能编译”，arXiv 预印本 arXiv：2608.15071，2026 年。

- [149] T. Wei, N. Sachdeva, B. Coleman, Z. He, Y. Bei, X. Ning, M. Ai, Y. Li, J. He, E. H. Chi et al.，“Evo-memory：使用自我进化记忆对 llm 智能体测试时学习进行基准测试”，arXiv 预印本 arXiv:2511.20857， 2025 年。

- [150] J. Shi, S. Tao, Y. Wu, Z. Wang, J. Zhang, J. Liu, X. Lei, X. Zhu, S. Fang, Z. Tan et al.，“S3gym：llms能否将自我测试和自我判断变成自我改进（Self-Improvement）？” arXiv 预印本 arXiv:2608.31100，2026 年。

58

[151] M. Lin, J. Wu, Z. Wang, Z. Shi, Y. Sang, B. He, Z. Liu, T. Wei, Z. Wu, Z. Zhang *et al.*, "Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving llm agents," *arXiv preprint arXiv:2605.30621*, 2026.

[152] X. Zhang, Y. Cui, G. Wang, Z. Li, W. Qiu, B. Zhu, and P. He, "Library drift: Diagnosing and fixing a silent failure mode in self-evolving llm skill libraries," *arXiv preprint arXiv:2605.19576*, 2026.

[153] Y. Liu, Z. Su, Y. Zhang, J. Guo, Z. Xie, H. Jing, L. Xie, Q. Zong, Y. Yim, Z. Zhang *et al.*, "Rethinking self-evolving agent skills: Feedback dynamics over multiple rounds," *arXiv preprint arXiv:2608.02636*, 2026.

[154] K. Hambardzumyan, N. Baldwin, E. Toledo, R. Hazra, M. Kuchnik, B. A. Omari, T. S. Foster, A. Protopopov, J.-C. Gagnon-Audet, I. Mediratta, K. Niu, M. Shvartsman, A. Lupidi, A. Audran-Reiss, P. Pathak, T. Shavrina, D. Magka, H. Momand, D. Dunfield, N. Cancedda, P. Stenetorp, C.-J. Wu, J. N. Foerster, Y. Bachrach, and M. Josifoski, "AIRA$_2$: Overcoming bottlenecks in AI research agents," *arXiv preprint arXiv:2603.26499*, 2026. [Online]. Available: https://arxiv.org/abs/2603.26499

[155] Y.-H. Chen, J. Wen, and J. H. Kirchner, "Automated researchers can mitigate well-characterized alignment failures," Anthropic Alignment Science Blog, Aug. 2026, research report; accessed September 6, 2026. [Online]. Available: https://alignment.anthropic.com/2026/automated-alignment-researchers/

[156] S. Jiang, L. Ma, Z. Hong, K. Wang, Z. Lu, T. Wang, S. Chen, J. Zhang, T. Pan, W. Li *et al.*, "Sea-eval: A benchmark for evaluating self-evolving agents beyond episodic assessment," *arXiv preprint arXiv:2604.08988*, 2026.

[157] C. Zheng, C. Xue, B. Liang, J. Yang, and C. Zhang, "Seagym: An evaluation environment for self-evolving llm agents," *arXiv preprint arXiv:2606.17546*, 2026.

[158] Z. Qian, J. Lei, Y. Wang, and N. Cao, "Hypoforge: A self-improving multi-agent framework for automated hypothesis generation and testing via scientific skill learning," 2026. [Online]. Available: https://arxiv.org/abs/2608.25770

[159] J. Lu, Z. Kong, Y. Wang, R. Fu, H. Wan, C. Yang, W. Lou, H. Sun, L. Wang, Y. Jiang, X. Wang, X. Sun, and D. Zhou, "Beyond static tools: Test-time tool evolution for scientific reasoning," 2026. [Online]. Available: https://arxiv.org/abs/2601.07641

[160] Y. Zhang, X. Cheng, T. Liu, Y. Du, and W. Jin, "Drugsage:self-evolving agent experience for efficient state-of-the-art drug discovery," 2026. [Online]. Available: https://arxiv.org/abs/2605.15461

[161] P. Hebbar, Y. Manawat, S. Verboomen, A. Ivanova, S. Palanimalai, K. Bhatia, and V. Baskaran, "SIA: Self improving ai with harness and weight updates," *arXiv preprint arXiv:2605.27276*, 2026. [Online]. Available: https://arxiv.org/abs/2605.27276

[162] X. Liu, Z. Bai, H. Ci, K. Y. Ma, and M. Z. Shou, "World-vla-loop: Closed-loop learning of video world model and vla policy," 2026. [Online]. Available: https://arxiv.org/abs/2602.06508

[163] J. Li, Y. Lai, W. Li, J. Ren, M. Zhang, X. Kang, S. Wang, P. Li, Y.-Q. Zhang, W. Ma, and Y. Liu, "Agent hospital: A simulacrum of hospital with evolvable medical agents," 2025. [Online]. Available: https://arxiv.org/abs/2405.02957

[164] Y. He, J. Liu, Z. Hu, Y. Chen, Y. Liu, Y. Sui, Y. Li, N. Chen, J. Hu, B. Hooi, X. Xu, and J. Bian, "Evoclinician: A self-evolving agent for multi-turn medical diagnosis via test-time evolutionary learning," 2026. [Online]. Available: https://arxiv.org/abs/2601.22964

[165] L. Fan, P. Dai, Z. Deng, H. Wang, X. Gong, Y. Zheng, and Y. Ou, "Evolving medical imaging agents via experience-driven self-skill discovery," 2026. [Online]. Available: https://arxiv.org/abs/2603.05860

[166] Y. Zhu, Z. Wang, Y. Qi, L. Gu, D. Sui, H. Hu, X. Zhang, Z. He, Y. Wang, J. He, L. Ma, and L. Yu, "HealthFlow: Automating Electronic Health Record Analysis via a Strategically Self-Evolving Multi-Agent Framework," *npj Digital Medicine*, vol. 9, no. 1, p. 660, Aug. 2026. [Online]. Available: https://doi.org/10.1038/s41746-026-03097-0

[167] Z. Tang *et al.*, "Workspace-bench 1.0: Benchmarking ai agents on workspace tasks with large-scale file dependencies," 2026. [Online]. Available: https://arxiv.org/abs/2605.03596

[168] Z. Zhu, Q. He, S. Li, Y. Chen, H. Sun, Z. Li, Y. Li, and Z. Liu, "ForgeTrain: Forging production-grade training frameworks via harness-driven AI development," https://github.com/OpenBMB/ForgeTrain, 2026, technical report and GitHub repository. [Online]. Available: https://github.com/OpenBMB/ForgeTrain

原文第 59 页 | 中文翻译初稿

参考文献

7

- [151] M. Lin, J. Wu, Z. Wang, Z. Shi, Y. Sang, B. He, Z. Liu, T. Wei, Z. Wu, Z.Zhang et al.，“运行支撑框架（Harness）更新不是 运行支撑框架（Harness）的好处：解开自我进化 llm 智能体的进化能力”，arXiv 预印本arXiv：2605.30621，2026。

- [152] X. 张、Y. Cui、G. Wang、Z. Li、W. Qiu、B. Zhu 和 P. He，“库漂移：诊断和修复自我进化 llm 技能库中的静默故障模式”，arXiv 预印本 arXiv:2605.19576，2026 年。

- [153] Y. Liu, Z. Su, Y.Zhang, J.Guo, Z.Xie, H.Jing, L.Xie, Q.Zong, Y.Yim, Z.Zhang_et al._，“重新思考自我进化智能体技能：多轮反馈动态”，arXiv 预印本 arXiv:2608.02636，2026。

- [154] K. Hambardzumyan、N. Baldwin、E. Toledo、R. Hazra、M. Kuchnik、B. A. Omari、T. S. Foster、A. Protopopov、J.-C. Gagnon-Audet、I. Mediratta、K. Niu、M. Shvartsman、A. Lupidi、A. Audran-Reiss、P. Pathak、T. Shavrina、D. Magka、H. Momand、D. Dunfield、N. Cancedda、P. Stenetorp、C.-J。 Wu、J. N. Foerster、Y. Bachrach 和 M. Josifoski，“AIRA2：克服人工智能研究智能体的瓶颈”，arXiv 预印本 arXiv：2603.26499，2026 年。[在线]。可用：https://arxiv.org/abs/2603.26499

- [155] Y.-H。 Chen、J. Wen 和 J. H. Kirchner，“自动化研究人员可以减轻特征明确的对齐失败”，人类对齐科学博客，2026 年 8 月，研究报告；访问日期：2026 年 9 月 6 日。[在线]。可用：https://alignment.anthropic.com/2026/automated-alignment-researchers/

- [156] S. Jiang、L. Ma、Z. Hong、K. Wang、Z. Lu、T. Wang、S. Chen、J. Zhu、T. Pan、W. Li et al.，“Sea-eval：评估超越情景评估的自我进化智能体的基准”，arXiv 预印本 arXiv：2604.08988，2026 年。

- [157] C. Cheng、C. Xu、B. Liang、J. Yang 和 C. Zhu，“Seagym：自我进化 llm 智能体的评估环境”，arXiv 预印本 arXiv：2606.17546，2026 年。

- [158] Z.Qian、J.Lei、Y.Wang 和 N.Cao，“Hypoforge：通过科学技能学习自动生成假设和测试的 自我改进（Self-Improving）多智能体框架”，2026 年。[在线]。可用：https://arxiv.org/abs/2608.25770

- [159] J. Lu、Z. Kong、Y. Wang、R. Fu、H. Wan、C. Yang、W. Lou、H. Sun、L. Wang、Y. Jiang、X. Wang、X. Sun 和 D. Zhou，“超越静态工具：科学推理的测试时工具演化”，2026 年。[在线]。可用：https://arxiv.org/abs/2601.07641

- [160] Y. 张、X. Cheng、T. Liu、Y. Du 和 W. Jin，“Drugsage：用于高效最先进药物发现的自我进化智能体体验”，2026 年。[在线]。可用：https://arxiv.org/abs/2605.15461

- [161] P. Hebbar、Y. Manawat、S. Verboomen、A. Ivanova、S. Palanimalai、K. Bhatia 和 V. Baskaran，“SIA：通过 运行支撑框架（Harness）和权重更新自我改进人工智能”，arXiv 预印本 arXiv：2605.27276，2026 年。[在线]。可用：https://arxiv.org/abs/2605.27276

- [162] X. Liu、Z. Bai、H. Ci、K. Y. Ma 和 M. Z. Shou，“World-vla-loop：视频 世界模型（World Model）和 vla 策略的闭环学习”，2026 年。[在线]。可用：https://arxiv.org/abs/2602.06508

- [163] J. Li，Y. Lai，W. Li，J. Ren，M. Zhang，X. Kang，S. Wang，P. Li，Y.-Q.张、W. Ma 和 Y. Liu，“智能体医院：具有可进化医疗智能体的医院模拟”，2025 年。[在线]

。可用：https://arxiv.org/abs/2405.02957

- [164] Y. He、J. Liu、Z. Hu、Y. Chen、Y. Liu、Y. Sui、Y. Li、N. Chen、J. Hu、B. Hooi、X. Xu 和 J. Bian，“Evoclinician：通过测试时进化学习进行多轮医学诊断的自我进化智能体”，2026 年。[在线]。可用：https://arxiv.org/abs/2601.22964

- [165] L. Fan、P. Dai、Z. Deng、H. Wang、X. Kong、Y. Cheng 和 Y. Ou，“通过经验驱动的自我技能发现来发展医学成像剂”，2026 年。[在线]。可用：https://arxiv.org/abs/2603.05860

- [166] Y. Zhu、Z. Wang、Y. Qi、L. Gu、D. Sui、H. Hu、X. Zhang、Z. He、Y. Wang、J. He、L. Ma 和 L. Yu，“HealthFlow：通过战略性自我进化多智能体框架实现电子健康记录分析自动化”，npj Digital Medicine，卷。 9、不。 1，p。 660，2026 年 8 月。[在线]。可用：https://doi.org/10.1038/s41746-026-03097-0

- [167] Z. Tang 等人，“Workspace-bench 1.0：在具有大规模文件依赖性的工作区任务上对人工智能智能体进行基准测试”，2026 年。[在线]。可用：https://arxiv.org/abs/2605.03596

- [168] Z. Zhu、Q. He、S. Li、Y. Chen、H. Sun、Z. Li、Y. Li 和 Z. Liu，“ForgeTrain：通过 运行支撑框架（Harness） 驱动的人工智能开发打造生产级训练框架”，https://github.com/OpenBMB/ForgeTrain，2026 年，技术报告和 GitHub 存储库。 [在线的]。可用：https://github.com/OpenBMB/ForgeTrain

[169] The ForgeStencil Authors, "ForgeStencil: Autonomous agents for stencil optimization and deployment," https://github.com/OpenBMB/ForgeStencil, 2026, gitHub repository.

[170] A. Karpathy, "autoresearch: Autonomous ai research on small language models," https://github.com/karpathy/autoresearch, 2026, gitHub repository.

[171] K. Jordan, J. Bernstein, B. Rappazzo, @fernbear.bsky.social, B. Vlado, Y. Jiacheng, F. Cesista, B. Koszarsky, and @Grad62304977, "modded-nanogpt: Speedrunning the nanogpt baseline," https://github.com/KellerJordan/modded-nanogpt, 2024, gitHub repository. [Online]. Available: https://github.com/KellerJordan/modded-nanogpt

[172] E. Lin, S. Modi, S. K. S. Hari, Q. Huang, Z. Ye, N. Qin, F. Zhou, Y. Zhang, J. Wang, S. Damani, D. Peri, O. Xie, A. Kane, M. Maor, M. Behar, T. Cao, R. Mehta, V. Singh, V. S. Mailthody, T. Chen, Z. Ye, H. Chen, T. Chen, V. Grover, W. Chen, W. Liu, E. Chung, L. Ceze, R. Bringmann, C. Zeller, M. Lightstone, C. Kozyrakis, and H. Shi, "Sol-execbench: Speed-of-light benchmarking for real-world gpu kernels against hardware limits," 2026. [Online]. Available: https://arxiv.org/abs/2603.19173

[173] Y. Lyu, X. Zhang, X. Yi, Y. Zhao, S. Guo, W. Hu, J. Piotrowski, J. Kaliski, J. Urbani, Z. Meng, L. Zhou, and X. Yan, "Evoscientist: Towards multi-agent evolving ai scientists for end-to-end scientific discovery," 2026. [Online]. Available: https://arxiv.org/abs/2603.08127

[174] M. Yuksekgonul, D. Koceja, X. Li, F. Bianchi, J. McCaleb, X. Wang, J. Kautz, Y. Choi, J. Zou, C. Guestrin, and Y. Sun, "Learning to discover at test time," 2026. [Online]. Available: https://arxiv.org/abs/2601.16175

[175] A. Khajeh, X. Lei, W. Ye, Z. Yang, L. Hung, D. Schweigert, and H.-K. Kwon, "A materials discovery framework based on conditional generative models applied to the design of polymer electrolytes," *Digital Discovery*, vol. 4, no. 1, p. 11–20, 2025. [Online]. Available: http://dx.doi.org/10.1039/D4DD00293H

[176] X. Huang, J. Chen, Y. Fei, Z. Li, P. Schwaller, and G. Ceder, "Cascade: Cumulative agentic skill creation through autonomous development and evolution," 2026. [Online]. Available: https://arxiv.org/abs/2512.23880

[177] N. Team, "S1-nexusagent: a self-evolving agent framework for multidisciplinary scientific research," 2026. [Online]. Available: https://arxiv.org/abs/2602.01550

[178] S. Shen, W. Cheng, M. Ma, A. Turcan, M. J. Zhang, and J. Ma, "Skillfoundry: Building self-evolving agent skill libraries from heterogeneous scientific resources," 2026. [Online]. Available: https://arxiv.org/abs/2604.03964

[179] R. Jin, Z. Zhang, M. Wang, and L. Cong, "Stella: Self-evolving llm agent for biomedical research," 2025. [Online]. Available: https://arxiv.org/abs/2507.02004

[180] Z. Guo, J. Wang, F. Ling, W. Wei, X. Yue, Z. Jiang, W. Xu, J.-J. Luo, L. Cheng, Y.-G. Ham, F. Song, P. Gentine, T. Yamagata, B. Fei, W. Zhang, X. Gu, C. Li, Y. Wang, T. Chen, W. Ouyang, B. Zhou, and L. Bai, "A self-evolving ai agent system for climate science," 2025. [Online]. Available: https://arxiv.org/abs/2507.17311

[181] Z. Zhang, Z. Qiu, Y. Wu, S. Li, D. Wang, Y. Liu, Z. Zhou, Y. Hu, Y. Chen, D. An, Y. Wang, Y. Li, Z. Zhong, C. Ou, Z. Wang, F. Tang, J. X. Chen, R. Ma, J. Li, X. Wang, W. Lu, H. Xue, W. Zhang, Z. Wei, R. Ma, Z. Shi, K. Wang, Q. Liu, B. Dong, Y. He, T. Liu, J. Gu, S. Song, Q. Feng, J. Zhang, B. Zhang, L. Tian, L. Bai, Q. Gao, S. Sun, and S. Zheng, "Origene: A self-evolving virtual disease biologist automating therapeutic target discovery," *bioRxiv*, 2026. [Online]. Available: https://www.biorxiv.org/content/early/2026/02/25/2025.06.03.657658

[182] B. Sun, W. Guo, Z. Yu, and L. Yang, "Self-evolving scientific agent designs physically-reasoned whitebox fluid control," 2026. [Online]. Available: https://arxiv.org/abs/2606.08405

[183] A. Qu, H. Zheng, Z. Zhou, Y. Yan, Y. Tang, S. Y. Ong, F. Hong, K. Zhou, C. Jiang, M. Kong, J. Zhu, X. Jiang, S. Li, C. Wu, B. K. H. Low, J. Zhao, and P. P. Liang, "Coral: Towards autonomous multi-agent evolution for open-ended discovery," 2026. [Online]. Available: https://arxiv.org/abs/2604.01658

[184] Y. Du, B. Yu, T. Liu, T. Shen, J. Chen, J. G. Rittig, K. Sun, Y. Zhang, A. Krishnan, Y. Zhang, D. Rosen, R. Pirone, Z. Song, B. Zhou, C. Masschelein, Y. Wang, H. Wang, H. Jia, C. Zhang, H. Zhao, M. Ester, N. Hacohen, T. Head-Gordon, C. P. Gomes, H. Sun, C. Duan, P. Schwaller, and W. Jin, "Accelerating scientific discovery with autonomous goal-evolving agents," 2026. [Online]. Available: https://arxiv.org/abs/2512.21782

[185] A. Zala, J. Cho, H. Lin, J. Yoon, and M. Bansal, "Envgen: Generating and adapting environments via llms for training embodied agents," 2024. [Online]. Available: https://arxiv.org/abs/2403.12014

[186] J. Guo, L. Yang, P. Chen, Q. Xiao, Y. Wang, X. Juan, J. Qiu, K. Shen, and M. Wang, "Genenv: Difficulty-aligned co-evolution between llm agents and environment simulators," 2025. [Online]. Available: https://arxiv.org/abs/2512.19682

原文第 60 页 | 中文翻译初稿

参考文献

7

- [169] ForgeStencil 作者，“ForgeStencil：用于模板优化和部署的自主智能体”，https://github.com/OpenBMB/ForgeStencil，2026，gitHub 存储库。

- [170] A. Karpathy，“autoresearch：小语言模型的自主人工智能研究”，https://github.com/karpathy/autoresearch，2026，gitHub 存储库。

- [171] K. Jordan、J. Bernstein、B. Rappazzo、@fernbear.bsky.social、B. Vlado、Y. Jia Cheng、F. Cesista、B. Koszarsky 和 @Grad62304977，“modded-nanogpt：加速 nanogpt 基线”，https://github.com/KellerJordan/mo dded-nanogpt，2024 年， gitHub 存储库。 [在线的]。可用：https://github.com/KellerJordan/modded-nanogpt

- [172] E. Lin、S. Modi、S. K. S. Hari、Q. Huang、Z. Ye、N. Qing、F. Zhou、Y. Zhang、J. Wang、S. Damani、D. Peri、O. Xie、A. Kane、M. Maor、M. Behar、T. Cao、R. Mehta、V. Singh、V. S. Mailthody、T. Chen、Z. Ye、H. Chen、T. Chen、V. Grover、W. Chen、W. Liu、E. Chung、L. Ceze、R. Bringmann、C. Zeller、M. Lightstone、C. Kozyrakis 和 H. Shi，“Sol-execbench：针对硬件限制的现实世界 GPU 内核的光速基准测试”，2026 年。[在线]。可用：https://arxiv.org/abs/2603.19173

- [173] Y. Lyu、X. Zhang、X. Yi、Y. Zhao、S.Guo、W.Hu、J.Piotrowski、J.Kaliski、J.Urbani、Z.Meng、L.Zhou 和 X.Yan，“Evoscientist：迈向多智能体进化人工智能科学家以实现端到端科学发现”，2026 年。[在线]。可用：https://arxiv.org/abs/2603.08127

- [174] M. Yuksekgonul、D. Koceja、X. Li、F. Bianchi、J. McCaleb、X. Wang、J. Kautz、Y. Choi、J. Zou、C. Guestrin 和 Y. Sun，“在测试时学习发现”，2026 年。[在线]。可用：https://arxiv.org/abs/2601.16175

- [175] A. Khajeh、X. Lei、W. Ye、Z. Yang、L. Hung、D. Schweigert 和 H.-K。 Kwon，“基于应用于聚合物电解质设计的条件生成模型的材料发现框架”，Digital Discovery，卷。 4、没有。 1，p。 2025 年 11-20 日。[在线]。可用：http://dx.doi.org/10.1039/D4DD00293H

- [176] X. Huang、J. Chen、Y. Fei、Z. Li、P. Schwaller 和 G. Ceder，“级联：通过自主发展和进化来创造累积智能体技能”，2026 年。[在线]。可用：https://arxiv.org/abs/2512.23880

- [177] N. Team，“S1-nexusagent：用于多学科科学研究的自我进化智能体框架”，2026 年。[在线]。可用：https://arxiv.org/abs/2602.01550

- [178] S. Shen、W. Cheng、M. Ma、A. Turcan、M. J. Zhang 和 J. Ma，“Skillfoundry：利用异构科学资源构建自我进化智能体技能库”，2026 年。[在线]。可用：https://arxiv.org/abs/2604.03964

- [179] R. Jin、Z. Zhang、M. Wang 和 L. Cong，“Stella：用于生物医学研究的自我进化 llm 智能体”，2025 年。[在线]。可用：https://arxiv.org/abs/2507.02004

- [180] Z.郭，J.王，F.凌，W.魏，X.岳，Z.江，W.徐，J.-J。罗立成，Y.-G。 Ham、F. Song、P. Gentine、T. Yamagata、B. Fei、W. Zhang、X. Gu、C. Li、Y. Wang、T. Chen、W. Ouyang、B. Zhou 和 L. Bai，“气候科学的自我进化人工智能智能体系统”，2025 年。[在线]。可用：https://arxiv.org/abs/2507.17311

- [181] 张子、邱子、吴宇、李世民、王东、刘宇、周子、胡宇、陈宇、安大、王宇、李宇、钟子、欧晨、王正、唐芳、陈建新、马荣杰

。 Li, X. Wang, W. Lu, H.薛, W.Zhang, Z.Wei, R.Ma, Z.Shi, K.Wang, Q.Liu, B.Dong, Y.He, T.Liu, J.Gu, S.Song, Q.Feng, J.Zhang, B.Zhang, L.Tian, L.Bai, Q.Gao, S.Sun, and S.Zheng,“Origene：一个自我进化的虚拟疾病生物学家，自动发现治疗靶点，” bioRxiv，2026。[在线]。可用：https://www.biorxiv.org/content/early/2026/02/25/2025.06.03.657658

- [182] B.Sun、W.Guo、Z.Yu 和 L.Yang，“自我进化科学智能体设计物理合理的白盒流体控制”，2026 年。[在线]。可用：https://arxiv.org/abs/2606.08405

- [183] A. Qu、H. Cheng、Z. Zhou、Y. Yan、Y. Tang、S. Y. Ong、F. Hong、K. Zhou、C. Jiang、M. Kong、J. Zhu、X. Jiang、S. Li、C. Wu、B. K. H. Low、J. Zhao 和 P. P. Liang，“Coral：面向开放式发现的自主多主体进化”，2026 年。[在线]。可用：https://arxiv.org/abs/2604.01658

- [184] Y. Du、B. Yu、T. Liu、T. Shen、J. Chen、J. G. Rittig、K. Sun、Y. Zhang、A. Krishnan、Y. Zhang、D. Rosen、R. Pirone、Z. Song、B. Zhou、C. Masschelein、Y. Wang、H. Wang、H. Jia、C. Zhu、H. Zhao、M. Ester、N. Hacohen、T. Head-Gordon、C. P. Gomes、H. Sun、C. Duan、P. Schwaller 和 W. Jin，“利用自主目标进化智能体加速科学发现”，2026 年。[在线]。可用：https://arxiv.org/abs/2512.21782

- [185] A. Zala、J. Cho、H. Lin、J. Yoon 和 M. Bansal，“Envgen：通过 llms 生成和调整环境以训练具体智能体”，2024 年。[在线]。可用：https://arxiv.org/abs/2403.12014

- [186] J.Guo、L.Yang、P.Chen、Q.Xiao、Y.Wang、X.Juan、J.Qiu、K.Shen 和 M.Wang，“Genev：llm 智能体和环境模拟器之间的难度协调协同进化”，2025 年。[在线]。可用：https://arxiv.org/abs/2512.19682

60

[187] M. Faldor, J. Zhang, A. Cully, and J. Clune, "Omni-epic: Open-endedness via models of human notions of interestingness with environments programmed in code," 2025. [Online]. Available: https://arxiv.org/abs/2405.15568

[188] H. Kang, X. Ye, Y. Liu, S. H. Mantri, L. Mao, J. Fleming, D. Regmi, and L. Qin, "Simworld studio: Automatic environment generation with evolving coding agent for embodied agent learning," 2026. [Online]. Available: https://arxiv.org/abs/2605.09423

[189] G. Tziafas and H. Kasaei, "Lifelong robot library learning: Bootstrapping composable and generalizable skills for embodied control with language models," 2024. [Online]. Available: https://arxiv.org/abs/2406.18746

[190] R. Ju, X. Wang, X. Ding, Y. Yang, H. Wu, S. Jiang, Q. Zhang, H. Wen, X. Li, W. Wang, K. Li, Y. Liu, H. Dai, W. Wang, and T. Cao, "Embodiskill: Skill-aware reflection for self-evolving embodied agents," 2026. [Online]. Available: https://arxiv.org/abs/2605.10332

[191] R. Lu, Y. Wu, E. Kou, L. Fu, W. Xiao, A. Mandlekar, Y. Xu, G. Shi, K. Goldberg, A. Chen, M. Chowdhury, Y. Zhu, L. J. Fan, and G. Wang, "Aspire: Agentic /skills discovery for robotics," 2026. [Online]. Available: https://arxiv.org/abs/2607.00272

[192] W. Xiao, J. Xie, T. Zhang, H. Lin, L. M. Fu, H. Xue, J. Lu, Y. Yang, C. Dai, Z. Wang, J. Wu, G. Wang, S. S. Sastry, K. Goldberg, L. J. Fan, Y. Zhu, and G. Shi, "Enpire: Agentic robot policy self-improvement in the real world," 2026. [Online]. Available: https://arxiv.org/abs/2606.19980

[193] S. K. S. Ghasemipour, A. Wahid, J. Tompson, P. Sanketi, and I. Mordatch, "Self-improving embodied foundation models," 2025. [Online]. Available: https://arxiv.org/abs/2509.15155

[194] A. Sharma, A. M. Ahmed, R. Ahmad, and C. Finn, "Self-improving robots: End-to-end autonomous visuomotor reinforcement learning," 2023. [Online]. Available: https://arxiv.org/abs/2303.01488

[195] V. Giridhar, A. Khandelwal, J. A. Collins, I. Georgiev, and A. Garg, "Beyond imitation: Self-improving robot policies via off-policy q-planning," 2026. [Online]. Available: https://arxiv.org/abs/2608.21204

[196] G. Li, R. Han, C. Li, H. Li, S. Wang, W. Ding, H. Zhang, and C. Xu, "Agentic self-evolutionary replanning for embodied navigation," 2026. [Online]. Available: https://arxiv.org/abs/2603.02772

[197] Y. Guo, T. Lee, L. X. Shi, J. Chen, P. Liang, and C. Finn, "Vlaw: Iterative co-improvement of vision-language-action policy and world model," 2026. [Online]. Available: https://arxiv.org/abs/2602.12063

[198] H. Bi, Z. Zhou, Y. Tang, J. Pang, S. Huang, H. Liu, R. Wang, S. Huang, Y. Wang, Y. Cheng, R. Zhao, Z. Li, H. Tan, X. Liu, J. Wan, J. Liu, M. Zhao, F. Bao, and J. Zhu, "Motus2: A self-evolving general world model for dexterous manipulation," 2026. [Online]. Available: https://arxiv.org/abs/2608.30237

[199] C. Luo, Z. Zeng, M. Jia, Y. Du, and C. Sun, "Self-improving loops for visual robotic planning," 2026. [Online]. Available: https://arxiv.org/abs/2506.06658

[200] M. Robeyns, M. Szummer, and L. Aitchison, "A self-improving coding agent," in *ICLR Workshop on Scaling Self-Improving Foundation Models*, 2025. [Online]. Available: https://arxiv.org/abs/2504.15228

[201] Y. Li, Y. Zhang, X. Zhang, X. Liu, and Y. Liu, "Codeskill: Learning self-evolving skills for coding agents," 2026. [Online]. Available: https://arxiv.org/abs/2605.25430

[202] Y. Hu, Y. Cai, Y. Du, X. Zhu, X. Liu, Z. Yu, Y. Hou, S. Tang, and S. Chen, "Self-evolving multi-agent collaboration networks for software development," 2024. [Online]. Available: https://arxiv.org/abs/2410.16946

[203] Z. Weng, A. Antoniades, D. Nathani, Z. Zhang, X. Pu, and X. E. Wang, "Group-evolving agents: Open-ended self-improvement via experience sharing," 2026. [Online]. Available: https://arxiv.org/abs/2602.04837

[204] Y. Zhang, Y. Dai, J. Tan, L. Yang, R. Mullur, T. Hoang, Z. Hu, J. Zhu, P. Mui, S. Savarese, R. Xu, and Z. Chen, "Darwinx: Evolving agent harnesses through natural selection," 2026. [Online]. Available: https://arxiv.org/abs/2608.07545

[205] T. Fan and C. Huang, "Helix: Model-harness co-evolution for recursive self-improvement," 2026. [Online]. Available: https://arxiv.org/abs/2608.13951

[206] Y. Wei, Z. Sun, E. McMilin, J. Gehring, D. W. Zhang, G. Synnaeve, D. Fried, L. ZHANG, and S. Wang, "Toward training superintelligent software agents through self-play SWE-RL," in *Forty-third International Conference on Machine Learning*, 2026. [Online]. Available: https://openreview.net/forum?id=0ophJB76sC

原文第 61 页 | 中文翻译初稿

参考文献

7

- [187] M. Faldor、J. Zhang、A. Cully 和 J. Clune，“全方位史诗：通过代码编程环境中的人类兴趣概念模型实现开放性”，2025 年。[在线]。可用：https://arxiv.org/abs/2405.15568

- [188] H. Kang、X. Ye、Y. Liu、S. H. Mantri、L. Mao、J. Fleming、D. Regmi 和 L.qin，“Simworld studio：用于具体智能体学习的不断发展的编码智能体的自动环境生成”，2026 年。[在线]。可用：https://arxiv.org/abs/2605.09423

- [189] G. Tziafas 和 H. Kasaei，“终身机器人库学习：利用语言模型引导可组合和可概括的技能以实现具体控制”，2024 年。[在线]。可用：https://arxiv.org/abs/2406.18746

- [190] R. Ju、X. Wang、X. Ding、Y. Yang、H. Wu、S. Jiang、Q. Zhang、H. Wen、X. Li、W. Wang、K. Li、Y. Liu、H. Dai、W. Wang 和 T. Cao，“Embodiskill：自我进化的具体智能体的技能感知反射”，2026 年。[在线]。可用：https://arxiv.org/abs/2605.10332

- [191] R. Lu、Y. Wu、E. Kou、L. Fu、W. Xiao、A. Mandlekar、Y. Xu、G. Shi、K. Goldberg、A. Chen、M. Chowdhury、Y. Zhu、L. J. Fan 和 G. Wang，“Aspire：机器人技术的智能体/技能发现”，2026 年。[在线]。可用：https://arxiv.org/abs/2607.00272

- [192] W.Xiao、J.Xie、T.Zhang、H.Lin、L.M.Fu、H.Xue、J.Lu、Y.Yang、C.Dai、Z.Wang、J.Wu、G.Wang、S.S.Sastry、K.Goldberg、L.J.Fan、Y.Zhu 和 G.Shi，“Enpire：现实世界中的智能体机器人策略 自我改进（Self-Improvement）”， 2026。[在线]。可用：https://arxiv.org/abs/2606.19980

- [193] S.K.S. Ghasemipour、A. Wahid、J. Tompson、P. Sanketi 和 I. Mordatch，“自我改进（Self-Improving）体现了 基础模型（Foundation Models）”，2025 年。[在线]。可用：https://arxiv.org/abs/2509.15155

- [194] A. Sharma、A. M. Ahmed、R. Ahmad 和 C. Finn，“自我改进（Self-Improving）机器人：端到端自主视觉电机 强化学习（Reinforcement Learning）”，2023 年。[在线]。可用：https://arxiv.org/abs/2303.01488

- [195] V. Giridhar、A. Khandelwal、J. A. Collins、I. Georgiev 和 A. Garg，“超越模仿：通过离策略 q 规划实现 自我改进（Self-Improving）机器人策略”，2026 年。[在线]。可用：https://arxiv.org/abs/2608.21204

- [196] G. Li、R. Han、C. Li、H. Li、S. Wang、W. Ding、H. Zhang 和 C. Xu，“实体导航的智能体自我进化重新规划”，2026 年。[在线]。可用：https://arxiv.org/abs/2603.02772

- [197] Y.Guo、T.Lee、L.X.Shi、J.Chen、P.Liang 和 C.Finn，“Vlaw：视觉语言动作（Vision-Language-Action）策略和 世界模型（World Model）的迭代共同改进”，2026 年。[在线]。可用：https://arxiv.org/abs/2602.12063

- [198] H. Bi, Z. Zhou, Y. Tang, J. Pang, S. Huang, H. Liu, R. Wang, S. Huang, Y. Wang, Y. Cheng, R. Zhao, Z. Li, H. Tan, X. Liu, J. Wan, J. Liu, M. Zhao, F. Bao, and J. Zhu，“Motus2：用于灵巧操作的自我进化通用 世界模型（World Model），” 2026。[在线]。可用：https://arxiv.org/abs/2608.30237

- [199] C. Luo、Z. Zeng、M. Jia、Y. Du 和 C. Sun，“用于视觉机器人规划的 自我改进（Self-Improving） 循环”，2026 年。[在线]。可用：https://arxiv.org/abs/2506.06658

- [200] M. Robeyns、M. Szummer 和 L. Aitchison，“A 自我改进（Self-Improving） 编码智能体”，载于 _ICLR 缩放 自我改进（Self-Improving） 基础模型（Foundation Models）_ 研讨会，2025 年。[在线]。可用：https://arxiv.org/abs/2504.15228

- [201] 李Y，张Y，张X，X

。 Liu 和 Y. Liu，“Codeskill：学习编码智能体的自我进化技能”，2026 年。[在线]。可用：https://arxiv.org/abs/2605.25430

- [202] Y. Hu、Y. Cai、Y. Du、X. Zhu、X. Liu、Z. Yu、Y. Hou、S. Tang 和 S. Chen，“用于软件开发的自进化多智能体协作网络”，2024 年。[在线]。可用：https://arxiv.org/abs/2410.16946

- [203] Z. Weng、A. Antoniades、D. Nathani、Z. Zhang、X. Pu 和 X. E. Wang，“群体进化智能体：通过经验共享实现开放式 自我改进（Self-Improvement）”，2026 年。[在线]。可用：https://arxiv.org/abs/2602.04837

- [204] Y. Zhang、Y. Dai、J. Tan、L. Yang、R. Mullur、T. Hoang、Z. Hu、J. Zhu、P. Mui、S. Savarese、R. Xu 和 Z. Chen，“Darwinx：通过自然选择进化媒介 运行支撑框架（Harnesses）”，2026 年。[在线]。可用：https://arxiv.org/abs/2608.07545

- [205] T. Fan 和 C. Huang，“Helix：递归自我改进（Recursive Self-Improvement） 的 Model-运行支撑框架（Harness） 协同进化”，2026 年。[在线]。可用：https://arxiv.org/abs/2608.13951

- [206] Y. Wei、Z. Sun、E. McMilin、J. Gehring、D. W. Zhang、G. Synnaeve、D. Fried、L. ZHANG 和 S. Wang，“通过自我对弈 SWE-RL 训练超级智能软件智能体”，载于《第四十三届机器学习国际会议》，2026 年。[在线]。可用：https://openreview.net/forum?id=0ophJB76sC

61

[207] C. Xiao, Z. Jiao, S. Wang, W. Wang, B. Zhao, H. Wei, L. Zhang, and L. Qu, "Socratic-swe: Self-evolving coding agents via trace-derived agent skills," 2026. [Online]. Available: https://arxiv.org/abs/2606.07412

[208] K. Lan, B. Jin, Z. Zhu, S. Chen, S. Zhang, K. Q. Zhu, and M. Wu, "Depression diagnosis dialogue simulation: Self-improving psychiatrist with tertiary memory," 2024. [Online]. Available: https://arxiv.org/abs/2409.15084

[209] M. Almansoori, K. Kumar, and H. Cholakkal, "MedAgentSim: Self-Evolving Multi-Agent Simulations for Realistic Clinical Interactions," in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2025*, ser. Lecture Notes in Computer Science, vol. 15968. Springer Nature Switzerland, 2025, pp. 362–372.

[210] R. Ren, Y. Wang, Y. Liang, L. Luo, J. Liu, H. Wang, C. Feng, Y. Zhang, C. Miao, J.-R. Wen, and W. X. Zhao, "Emulating clinician cognition via self-evolving deep clinical research," 2026. [Online]. Available: https://arxiv.org/abs/2603.10677

[211] X. Han, Y. Fan, S. Zhao, H. Wang, and B. Qin, "Gsem: Graph-based self-evolving memory for experience augmented clinical reasoning," 2026. [Online]. Available: https://arxiv.org/abs/2603.22096

[212] W. Shen, B. Jian, J. Li, C. Liu, J. Moll, X. Hu, D. Rueckert, H. B. Li, and J. Pan, "Evo-medagent: Beyond one-shot diagnosis with agents that remember, reflect, and improve," 2026. [Online]. Available: https://arxiv.org/abs/2604.14475

[213] Q. Liu, Z. Hu, T. Huang, Y. Niu, X. Zhang, S. Ma, C. Lin, G. K. Huat, H. E. Kwon, F. Gao, X. Sun, Z. Ying, and G. Qiang, "EvoMDT: A Self-Evolving Multi-Agent System for Structured Clinical Decision-Making in Multi-Cancer," *npj Digital Medicine*, vol. 9, no. 1, p. 124, Jan. 2026. [Online]. Available: https://doi.org/10.1038/s41746-025-02304-8

[214] Y. Yang, J. Li, Q. Pan, J. Zhou, K. Chen, Q. Chen, J. Zhao, N. Zhou, X. Li, and L. He, "Psychagent: An experience-driven lifelong learning agent for self-evolving psychological counselor," 2026. [Online]. Available: https://arxiv.org/abs/2604.00931

[215] H. Sun, W. Li, Y. Zhang, Z. Lin, F. Zhang, K. Chen, X. He, Y. Li, M. Liu, L. Liu, and Y. Jiang, "Experience makes skillful: Enabling generalizable medical agent reasoning via self-evolving skill memory," 2026. [Online]. Available: https://arxiv.org/abs/2606.09365

[216] S. Li, J. Xu, T. Bao, Y. Liu, Y. Liu, Y. Liu, L. Wang, W. Lei, S. Wang, Y. Xu, Y. Cui, J. Yao, S. Koga, and Z. Huang, "A co-evolving agentic ai system for medical imaging analysis," 2025. [Online]. Available: https://arxiv.org/abs/2509.20279

[217] Z. Du, L. Zheng, R. Hu, Y. Xu, X. Li, Y. Sun, W. Chen, J. Wu, H. Cai, and H. Ying, "Llms can simulate standardized patients via agent coevolution," in *ACL (1)*. Association for Computational Linguistics, 2025, pp. 17 278–17 306.

原文第 62 页 | 中文翻译初稿

参考文献

7

- [207] C.Xiao、Z.Jiao、S.Wang、W.Wang、B.Zhao、H.Wei、L.Zhang 和 L.Qu，“Socratic-swe：通过跟踪衍生智能体技能实现自我进化编码智能体”，2026 年。[在线]。可用：https://arxiv.org/abs/2606.07412

- [208]兰凯、金本、朱子、陈世、张世、朱国庆、吴明，“抑郁症诊断对话模拟：自我改进（Self-Improving）具有三级记忆的精神科医生”，2024年。[在线]。可用：https://arxiv.org/abs/2409.15084

- [209] M. Almansoori、K. Kumar 和 H. Cholakkal，“MedAgentSim：用于现实临床交互的自我进化多智能体模拟”，载于_医学图像计算和计算机辅助干预 - MICCAI 2025_，系列。计算机科学讲义，卷。 15968。施普林格自然瑞士，2025 年，第 362-372 页。

- [210] 任瑞、王玉、梁玉、罗丽、刘建、王红、冯春、张玉、苗春、J.-R。 Wen 和 W.X.Zhao，“通过自我进化的深度临床研究模拟临床医生认知”，2026 年。[在线]。可用：https://arxiv.org/abs/2603.10677

- [211] X. Han、Y. Fan、S. Zhao、H. Wang 和 B.qin，“Gsem：用于经验增强临床推理的基于图的自我进化记忆”，2026 年。[在线]。可用：https://arxiv.org/abs/2603.22096

- [212] W. Shen、B.jian、J. Li、C. Liu、J. Moll、X. Hu、D. Rueckert、H. B. Li 和 J. Pan，“Evo-medagent：超越具有记忆、反映和改进功能的一次性诊断”，2026 年。[在线]。可用：https://arxiv.org/abs/2604.14475

- [213] Q. Liu、Z. Hu、T. Huang、Y. Niu、X. Zhang、S. Ma、C. Lin、G. K. Huat、H. E. Kwon、F. Gau、X. Sun、Z. Ying 和 G. Gang，“EvoMDT：用于多种癌症结构化临床决策的自我进化多智能体系统”，npj 数字医学，卷。 9、不。 1，p。 124，2026 年 1 月。[在线]。可用：https://doi.org/10.1038/s41746-025-02304-8

- [214] Y. Yang、J. Li、Q. Pan、J. Zhou、K. Chen、Q. Chen、J. Zhao、N. Zhou、X. Li 和 L. He，“Psychagent：一种经验驱动的自我进化心理咨询师终身学习智能体”，2026 年。[在线]。可用：https://arxiv.org/abs/2604.00931

- [215] H. Sun、W. Li、Y.Zhang、Z.Lin、F.Zhang、K.Chen、X.He、Y.Li、M.Liu、L.Liu 和 Y.Jiang，“经验使人熟练：通过自我进化技能记忆实现通用医疗智能体推理”，2026 年。[在线]。可用：https://arxiv.org/abs/2606.09365

- [216] S. Li、J. Xu、T. Bao、Y. Liu、Y. Liu、Y. Liu、L. Wang、W. Lei、S. Wang、Y. Xu、Y. Cui、J. Yao、S. Koga 和 Z. Huang，“用于医学影像分析的共同进化智能体人工智能系统”，2025 年。[在线]。可用：https://arxiv.org/abs/2509.20279

- [217] Z. Du、L. Cheng、R. Hu、Y. Xu、X. Li、Y. Sun、W. Chen、J. Wu、H. Cai 和 H. Ying，“Llms 可以通过智能体协同进化模拟标准化患者”，参见_ACL (1)_。计算语言学协会，2025 年，第 17 278-17 306 页。

62

Appendix A: RSI Landscape

Figure 18 summarizes the surveyed papers by autonomy level and improvement target.

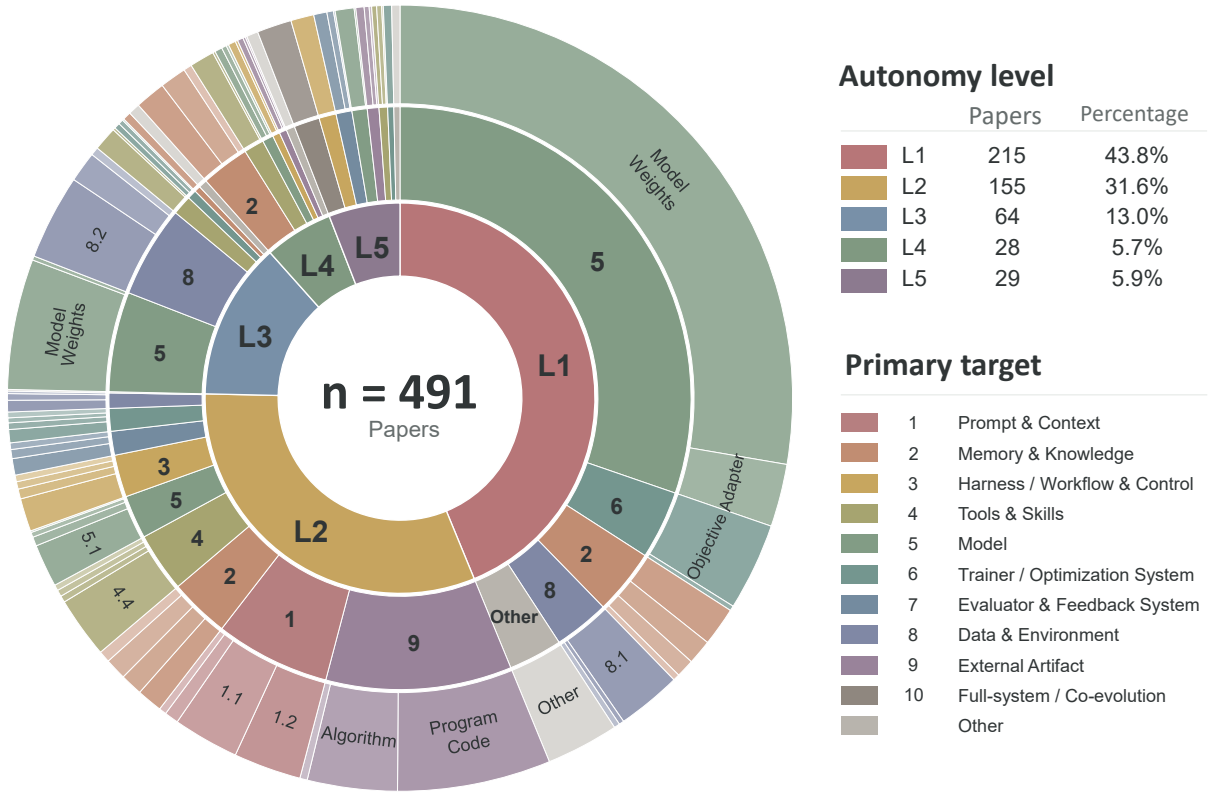


Figure 18 RSI Landscape: Autonomy Levels and Improvement Targets. Distribution of 491 surveyed papers. The inner, middle, and outer rings represent autonomy levels (L1–L5), primary improvement targets, and sub-targets, respectively. Autonomy-level percentages are based on paper counts. Papers associated with multiple improvement targets contribute fractional weights to the target categories.

Table 12 details the primary targets and sub-targets represented in Figure 18.

Table 12 Taxonomy of Improvement Targets in Recursive Self-Improvement Systems. The taxonomy decomposes the improvement targets represented in Fig. 18 into primary targets and sub-targets. For each sub-target, the table specifies the concrete system component subject to modification.

Primary Target	Sub-target	Modified Component
1. Prompt & Context	1.1 Instruction	System prompts, role specifications, behavioral rules, constraints, and high-level instructions
	1.2 Task Prompt / Template	Task descriptions, problem formulations, prompt templates, and reusable task specifications
	1.3 Reasoning Prompt / Protocol	Explicit reasoning instructions and protocols, including chain-of-thought, reflection, critique, and debate
	1.4 Demonstrations / Exemplars	Few-shot examples, demonstrations, successful trajectories, and worked solutions included in context
	1.5 Context Composition	Context selection, ordering, compression, summarization, and allocation of the available context budget

Continued on next page

原文第 63 页 | 中文翻译初稿

附录 A：RSI 实施技术
附录 A：RSI 格局

图 18 按自主水平和改进目标总结了调查的论文。

自主水平
论文百分比
L1 215 43.8%
L2 155 31.6%
2
L3 64 13.0%
L5 L4 28 5.7%
L4 5
8 L5 29 5.9%
L3
5
L1 主要目标
n = 491 篇论文 1 提示和上下文
2 记忆和知识
3
3 运行支撑框架（Harness）/ 工作流程和控制
5 6 4 工具和技能
4 L2 5 模型
2
2 6 训练师/优化系统
8
7 评估器和反馈系统
1 其他
9 8 数据和环境
9 外部神器
10全系/协同进化
其他

图 18 RSI 格局：自主水平和改进目标。分发了 491 篇调查论文。内环、中环、外环分别代表自治级别（L1-L5）、主要改进目标和子目标。自主级别百分比基于纸张数量。与多个改进目标相关的论文为目标类别贡献了分数权重。

表 12 详细介绍了图 18 中所示的主要目标和子目标。

表 12 递归自我改进（Recursive Self-Improvement）系统中的改进目标分类。该分类法将图 18 中表示的改进目标分解为主要目标和子目标。对于每个子目标，该表指定了需要修改的具体系统组件。

主要目标	子目标	修改的组件
	1.1 说明
1.2 任务提示/模板	系统提示、角色规范、行为规则、
约束和高级说明
任务描述、问题表述、提示模板和可重复使用的任务规范
1. Prompt & Con-	1.3 推理 Prompt / Proto-	显式推理指令和协议，包括
文本	col	思想链、反思、批评和辩论
	1.4 演示/Exem-
plars	少量示例、演示、成功的轨迹-
以及上下文中包含的可行解决方案
	1.5 上下文组合	上下文选择、排序、压缩、摘要
以及可用上下文预算的分配

下页继续

Table 12 continued

Primary Target	Sub-target	Modified Component
2. Memory & Knowledge	2.1 Episodic / Experience Memory	Records of prior interactions, execution trajectories, successful and failed attempts, and task-specific reflections
	2.2 Semantic / Knowledge Memory	Persistent factual, conceptual, and domain knowledge, including structured or external knowledge stores
	2.3 Procedural Memory	Reusable strategies, experience-derived rules, heuristics, procedures, and playbooks
	2.4 Memory Representation / Organization	Memory schemas, hierarchical structures, summaries, graphs, indexes, and vector representations
	2.5 Memory Operations	Policies governing memory writing, retrieval, updating, consolidation, prioritization, and forgetting
3. Harness / Workflow & Control	3.1 Workflow / Graph	Agent graphs, pipelines, nodes, edges, and execution dependencies between computational stages
	3.2 Planning / Decomposition	Task decomposition, planning modules, subgoal structures, and execution order
	3.3 Routing / Scheduling	Model routing, agent routing, tool routing, execution scheduling, and computational resource allocation
	3.4 Verification / Reflection Loop	Control loops for checking and revising intermediate or final outputs, including critic–revise, verify–retry, and reflection procedures
	3.5 Multi-agent Structure / Protocol	Agent population, role assignment, interaction topology, coordination mechanisms, and communication protocols
	3.6 Harness Implementation / Scaffold Code	Source code implementing the agent scaffold, orchestration logic, workflow controller, and runtime coordination mechanisms
4. Tools & Skills	4.1 Tool Set / Inventory	The collection of tools, APIs, external services, and callable capabilities available to the system
	4.2 Tool Definition / Interface	Tool descriptions, function signatures, schemas, API wrappers, and input–output specifications
	4.3 Tool Implementation	Executable code and internal logic implementing tools or callable external functions
	4.4 Skill / Macro Library	Reusable skills, macros, subroutines, procedures, and higher-level behavioral modules
	4.5 Skill Composition	Rules and structures for combining lower-level skills into higher-level procedures or capabilities
5. Model	5.1 Model Weights	Trainable parameters of the backbone model
	5.2 Adapter / Auxiliary Module	Parameters of LoRA modules, adapters, memory modules, and other trainable auxiliary components
	5.3 Architecture	Network architecture, module composition, connectivity patterns, and computational structure
	5.4 Inference Policy / Configuration	Persistent decoding strategies, test-time policies, inference configurations, and runtime model settings

Continued on next page

原文第 64 页 | 中文翻译初稿

附录 A：RSI 实施技术

	表 1	2 续
主要目标	子目标	修改的组件
	2.1 情景/经验
记忆
2.2 语义/知识
记忆	先前交互的记录、执行轨迹、成功和失败的尝试以及特定于任务的反思
持久的事实、概念和领域知识，
包括结构化或外部知识存储
2.内存与
知识	2.3 程序性内存
2.4 内存表示/
组织
2.5 内存操作	可重用策略、经验衍生规则、启发式、
程序和剧本
内存模式、层次结构、摘要、
图形、索引和向量表示
管理内存写入、检索、更新、
巩固、优先排序和遗忘
	3.1 工作流/图
3.2 规划/分解	智能体图、管道、节点、边和执行
计算阶段之间的依赖关系
任务分解、规划模块、子目标结构和执行顺序
	3.3 路由/调度	模型路由、智能体路由、工具路由、执行
调度和计算资源分配
3. 运行支撑框架（Harness）/ 工作
流程与控制	3.4 验证/反思
循环	用于检查和修改中间或
最终输出的控制循环，包括批评-修订、验证-重试和
反思程序
	3.5 多智能体结构 /
协议
3.6 运行支撑框架（Harness）实现 /
脚手架代码	智能体群体、角色分配、交互拓扑、
协调机制和通信协议
实现智能体脚手架、编排逻辑、工作流控制器和运行时协调
机制的源代码
	4.1 工具集/清单	系统可用的工具、API、外部服务和
可调用功能的集合

||4.2 工具定义/接口|工具描述、函数签名、模式、API
包装器和输入输出规范|
|4.工具与技能|4.3 工具实现|可执行代码和内部逻辑实现工具或
可调用的外部函数|
||4.4 技能/宏库|可重复使用的技能、宏、子例程、过程和
更高级别的行为模块|
||4.5 技能组合|将较低级别的技能组合成
较高级别的程序或能力的规则和结构|
||5.1 模型权重
5.2 适配器/辅助模块|主干模型的可训练参数
LoRA 模块、适配器、内存模块和其他可训练辅助组件的参数|
|5.模型|5.3 架构|网络架构、模块组成、连接
模式和计算结构|
||5.4 推理策略/配置|持久解码策略、测试时策略、推理配置和运行时模型设置|

下页继续

Table 12 continued

Primary Target	Sub-target	Modified Component
6. Trainer / Optimization System	6.1 Training Objective	Loss functions, training rewards, learning objectives, and optimization criteria
	6.2 Optimizer / Update Rule	Optimizers, parameter-update algorithms, learning-rate policies, and associated hyperparameters
	6.3 Training Schedule / Pipeline	Ordering, configuration, and interaction of training stages such as fine-tuning, reinforcement learning, and distillation
	6.4 Data Selection / Curriculum	Training-sample selection rules, data mixtures, difficulty schedules, and curriculum policies
	6.5 Search / Meta-optimization Procedure	Evolutionary, search, selection, and meta-optimization procedures used to generate and select candidate improvements
7. Evaluator & Feedback System	7.1 Evaluator / Judge	Evaluators, critics, graders, judge models, and related components for assessing candidate outputs or system variants
	7.2 Reward / Fitness Function	Reward models, fitness functions, utility functions, preference models, and scoring rules
	7.3 Verifier	Correctness verifiers, test generators, proof checkers, consistency checks, and validation mechanisms
	7.4 Feedback / Credit Assignment	Mechanisms that transform observed outcomes into localized, aggregated, or temporally assigned improvement signals
8. Data & Environment	8.1 Training / Experience Data	Training datasets, replay buffers, experience pools, interaction trajectories, and other data used for subsequent learning
	8.2 Task / Curriculum Generator	Components that generate tasks, problems, challenges, or training episodes for subsequent improvement cycles
	8.3 Environment / Simulator	Environment dynamics, simulators, interaction rules, task worlds, and structures governing system–environment interaction
	8.4 World Model	Learned models used to represent, predict, generate, or simulate environmental states and dynamics
9. External Artifact	9.1 Program / Solution Code	Task-level programs, patches, or solution code produced by the system, excluding code implementing the agent itself
	9.2 Algorithm	Algorithms, kernels, mathematical procedures, symbolic methods, and computational techniques
	9.3 Scientific / Design Artifact	Scientific hypotheses, experimental designs, architectures, circuits, engineering designs, and related research artifacts
10. Full-system / Co-evolution	10.1 Multi-target Harness	Multiple harness-level components jointly modified within the same improvement process, such as prompts, memory, workflows, and tools
	10.2 Model–Harness Co-evolution	Model parameters and surrounding harness or scaffold components jointly modified across improvement cycles
	10.3 Improvement-loop / Meta-RSI	The mechanism that generates, evaluates, selects, and applies candidate changes across successive improvement cycles

原文第 65 页 | 中文翻译初稿

附录 A：RSI 实施技术

	表 12	续
主要目标	子目标	修改的组件
	6.1 训练目标
6.2 优化器/更新规则	损失函数、训练奖励、学习目标和
优化标准
优化器、参数更新算法、学习率
策略和相关超参数
6.训练师/优化系统	6.3 训练计划/
管道
6.4 数据选择/课程-
lum
6.5 搜索/元优化
程序	训练的排序、配置和交互
阶段，例如微调、强化学习（Reinforcement Learning）和
蒸馏
训练样本选择规则、数据混合物、难度
时间表和课程策略
进化、搜索、选择和元优化
用于生成和选择候选改进的程序
	7.1 评估者/评判者	评估者、评论家、评分者、评判模型以及用于评估候选输出或系统变体的相关
组件
7.评估器和反馈系统	7.2 奖励/适应度函数
7.3 验证器
7.4 反馈/信用分配-
ment	奖励模型、适应度函数、效用函数、偏好模型和评分规则
正确性验证器、测试生成器、证明检查器、
一致性检查和验证机制
将观察到的结果转换为局部
化的机制，
聚合的或临时分配的改进
信号
	8.1 训练/经验数据	训练数据集、重播缓冲区、经验池、交互
动作轨迹以及用于后续
学习的其他数据
8.数据与环境-
ment|8.2 任务/课程生成器-
tor
8.3 环境/模拟器

8.4 世界模型（World Model）|为后续改进周期生成任务、问题、挑战
或训练片段的组件
环境动态、模拟器、交互
规则、任务世界和管理系统的结构 -
环境交互
学习模型用于表示、预测、生成或
模拟环境状态和动态|
||9.1 程序/解决方案代码|系统
生成的任务级程序、补丁或解决方案代码，不包括实现智能体
本身的代码|
|9.外部工件|9.2 算法
9.3 科学/设计工件|算法、内核、数学过程、符号
方法和计算技术
科学假设、实验设计、架构、电路、工程设计和相关研究
工件|
||10.1 多目标运行支撑框架（Harness）|在同一个改进流程中联合修改多个运行支撑框架（Harness）级别的组件，例如提示、
内存、工作流程和工具|
|10.全系统/
协同进化|10.2模型-运行支撑框架（Harness）协同
进化
10.3改进循环/元-
RSI||
模型参数和周围的运行支撑框架（Harness）或支架
在改进周期中联合修改的组件
生成、评估、选择和
在连续的过程中应用候选更改的机制改进
周期|

Appendix B: Industry Landscape

Table 13 organizes the surveyed industrial systems by company archetype and improvement target.

Company Archetypes in the Emerging RSI / Self-Improvement Landscape

One row per product or representative work; 72 distinct companies/teams; public-source snapshot: September 2026

Organization principle: company archetype, not geography. The RSI tag is a compact mapping to the B0–L5 scheme and is not a claim that the company itself uses that label. 🌐 *= clickable primary source.*

Table 13 RSI / self-improvement company landscape organized by company archetype.

Company	Product / representative work	Sub-scenario	Improvement target / artifact	AI-controlled part	RSI relation	🌐
A. Frontier foundation-model & general-agent labs Broad model/platform labs; RSI appears as one capability frontier rather than the sole company thesis.						
OpenAI	Research acceleration / automated research intern	AI-for-AI research	research code; experiments; evals	code; experiments; candidate integration	L2	🌐
	GPT-Red	self-play robustness	red-team policy; adversarial data	attack; defend; train; evaluate	L2	🌐
	Self-improving tax agents	production adaptation	agent code; prompts; eval set	mine failures; patch; evaluate	L4 cand.	🌐
	Harness Engineering	agent-first software R&D	repo; CI; agent instructions	code; test; PR; repair	L1-L2 adj.	🌐
	Symphony	agent orchestration	task state; repo; workflows	schedule; execute; handoff	L1 adj.	🌐
	AgentKit / prompt optimizer / RFT	agent optimization stack	prompts; graders; weights	optimize; grade; fine-tune	L1-L2	🌐
	Deep Research	autonomous research	task-local evidence	search; browse; synthesize	B0	🌐
Anthropic	When AI builds itself	RSI roadmap	AI-development process	code; experiments; future successor design	L5 target	🌐
	Automated Weak-to-Strong Researcher	automated alignment research	hypotheses; code; experiment logs	propose; train; evaluate; share	L2	🌐
	Tool optimization with Claude	tool self-optimization	tool specs; implementations	analyze traces; rewrite; evaluate	L2	🌐
	Harness design for long-running apps	autonomous software R&D	harness; evaluator; app code	plan; generate; evaluate; iterate	L2 adj.	🌐
	Effective long-running agent harnesses	cross-context persistence	progress files; git state	initialize; code; handoff	L1 adj.	🌐
	Agent Skills	persistent skill substrate	skills; scripts; resources	discover; load; reuse	L1 enabler	🌐
	Multi-agent Research	autonomous research	task-local findings	plan; spawn; search; synthesize	B0	🌐
	Managed Agents	long-horizon agent infrastructure	stable interface; harness	run; resume; supervise	L1 enabler	🌐
	Parallel Claude compiler project	autonomous software engineering	shared codebase; tests	decompose; code; test; coordinate	B0-L1 adj.	🌐
Google DeepMind	AlphaEvolve	task-specific program search	algorithms; kernels; system code	generate; mutate; evaluate; select	B0	🌐
	AI Co-Scientist	scientific hypothesis search	hypotheses; research proposals	generate; debate; rank; refine	L2 adj.	🌐
	AlphaChip	AI-for-hardware co-design	chip layouts; design policy	place; score; learn; transfer	L2 adj.	🌐
NVIDIA	Eureka	reward / policy design	reward code; robot policies	reward synthesis; simulation; policy training	L2	🌐
	ASPIRE	reward / policy design	reward code; robot policies	reward synthesis; simulation; policy training	L2	🌐
Microsoft	Agent Lightning	agent reinforcement learning	policy weights; experience traces	collect; credit; train; evaluate	L2	🌐

continued on next page

原文第 66 页 | 中文翻译初稿

附录 B：行业格局

表 13 按公司原型和改进目标组织了调查的工业系统。
新兴 RSI / 自我改进（Self-Improvement）景观中的公司原型

每个产品或代表性作品占一行； 72 个不同的公司/团队；公共源快照：2026 年 9 月

组织原则：公司原型，而非地域。 RSI 标签是 B0-L5 方案的紧凑映射，并不是公司本身使用该标签的声明。 ⧉ = 可点击的主要来源。

表 13 按公司原型组织的 RSI / 自我改进（Self-Improvement）公司景观。

公司	产品/代表作品	子场景	改进目标/工件	AI控制部分	RSI关系	⧉
A.前沿基础
-模型&总智能体实验室
广义模型
/平台实验室； RSI 似乎
rs 是一种能力前沿，而不是
而不是唯一的公司论点。
OpenAI	研究加速/自动化
研究实习生	AI-for-AI 研究	研究代码；实验；评估	代码；实验；候选整合	L2	⧉
	GPT-Red	自我对战鲁棒性	红队策略；对抗性
数据	攻击；保卫;火车;评估	L2	⧉
	自我改进（Self-Improving）税务智能体	生产适配	智能体代码；提示；评估集	地雷故障；修补;评估	L4 cand。	⧉
	运行支撑框架（Harness） 工程	智能体优先软件
研发	repo; CI;智能体指令	代码；测试;公关；修复	L1-L2 调整	⧉
	Symphony	智能体编排	任务状态；回购协议；工作流程	时间表；执行;切换	L1 adj.	⧉
	AgentKit / 提示优化器 / RFT	智能体优化堆栈	提示；评分员；权重	优化；年级;微调	L1-L2	⧉
	深度研究	自主研究	任务局部证据	搜索；浏览;合成	B0	⧉
人择	当人工智能自我构建时	RSI路线图	人工智能开发流程	代码；实验；未来的成功
设计	r
L5目标	⧉
	自动从弱到强重新
搜索器	自动对齐
研究	假设；代码;实验
日志	建议；火车;评价;分享	L2	⧉
	与 Claude 一起进行工具优化	工具自我优化	工具规格；实施	分析痕迹；改写;评估	L2	⧉
	运行支撑框架（Harness） 专为长时间运行的应用程序而设计	自主软件

研发|运行支撑框架（Harness）；评估员；应用程序代码|计划；产生;评价;迭代|L2 adj.|  |
||有效的长时间运行智能体工具|跨上下文持久性|进度文件； git 状态|初始化；代码;切换|L1 adj.|  |
||智能体技能|持久的技能基础|技能；脚本；资源|发现；加载;重用|L1 推动者|
|
||多主体研究|自主研究|任务局部发现|计划；产卵；搜索;合成|B0|  |
||托管智能体|长时程（Long-Horizon） 智能体基础设施|稳定的接口； 运行支撑框架（Harness）|运行；恢复;监督|L1 推动者||
||并行 Claude 编译器项目|自主软件
工程|共享代码库；测试|分解；代码;测试;坐标|B0-L1 调节|  |
|Google DeepMind|AlphaEvolve|特定任务程序
搜索|算法；内核；系统代码|生成；变异；评价;选择|B0||
||人工智能联合科学家|科学假设
搜索|假设；研究提案|生成；辩论;秩;精炼|L2 adj.|  |
||AlphaChip|人工智能硬件协同
设计|芯片布局；设计策略|地点；分数;学习

;转移|L2 adj.|  |
|NVIDIA|Eureka|奖励/策略设计|奖励代码；机器人策略|奖励合成；模拟;警察-
冰冷训练|L2|  |
||ASPIRE|奖励/策略设计|奖励代码；机器人策略|奖励合成；模拟;警察-
冰冷训练|L2||
|微软|智能体闪电|智能体强化
学习|策略权重；经历痕迹|收集；信用;火车;评估|L2|  |

接下页

Table 13 – continued

Company	Product / representative work	Sub-scenario	Improvement target / artifact	AI-controlled part	RSI relation	
	Agent Lightning v1.0	harnessed agentic RL	harness traces; policy weights	interact; retokenize; train; benchmark	L2	
	SkillOpt	skill optimization	skill files; instructions	edit; evaluate; optimize	L2	
	ReVeal	self-verifying code agents	code; tests; verifier policy	generate; verify; revise; scale	L2	
	Universal Verifier / auto-research	agent verification R&D	rubrics; verifier; eval pipeline	design; test; compare; refine	L2 adj.	
Meta	Self-Taught Evaluator	evaluator self-training	judge model; synthetic preferences	generate; judge; train; iterate	L2	
	HyperAgents	meta-agent self-modification	task agent; meta agent; program	solve; self-edit; evaluate; archive	L5 cand.	
Alibaba / Qwen	Qwen-Agent	agent training / synthetic environments	training data; environments; post-training	simulation; data synthesis; tool use; RL	L1-L2	
	AgentWorld	agent training / synthetic environments	training data; environments; post-training	simulation; data synthesis; tool use; RL	L1-L2	
	Qwen-Scope	agent training / synthetic environments	training data; environments; post-training	simulation; data synthesis; tool use; RL	L1-L2	
DeepSeek	R1	reasoning self-bootstrapping	reasoning policy; verifier	self-generated reasoning; RL; verifier iteration	L2	
	Math-V2	reasoning self-bootstrapping	reasoning policy; verifier	self-generated reasoning; RL; verifier iteration	L2	
Tencent AI Lab	R-Zero	self-play reasoning	tasks; pseudo-labels; policy weights	challenge generation; solve; vote; RL	L2-L3	
ByteDance Seed	Seed-Thinking	model / agent post-training	data; reward; policy weights	data filtering; reward verification; RL	L1-L2	
	Seed1.5-VL	model / agent post-training	data; reward; policy weights	data filtering; reward verification; RL	L1-L2	
MiniMax	M2	persistent cloud agents	memory; skills; agent policy	tool use; long-run execution; skill reuse	L1-L2	
	MaxHermes	persistent cloud agents	memory; skills; agent policy	tool use; long-run execution; skill reuse	L1-L2	
	MaxClaw	persistent cloud agents	memory; skills; agent policy	tool use; long-run execution; skill reuse	L1-L2	
Moonshot AI	Kimi K3	long-horizon agents	agent orchestration; tool policy	planning; tool use; swarm coordination	L1-L2 adj.	
	Agent Swarm	long-horizon agents	agent orchestration; tool policy	planning; tool use; swarm coordination	L1-L2 adj.	
Zhipu AI / Z.AI	AutoGLM	computer-use agent training	policy weights; virtual-phone environments	perception; planning; action; RL	L2	
	AgentRL	computer-use agent training	policy weights; virtual-phone environments	perception; planning; action; RL	L2	
Deep Cogito	Cogito v2	reasoning post-training	reasoning policy; weights	search; distill; iterative alignment	L1-L2	
	IDA	reasoning post-training	reasoning policy; weights	search; distill; iterative alignment	L1-L2	
Poolside	Model Factory	automated model R&D	synthetic data; RL; architecture	eval; code-exec RL; ablations; data mix	L1-L2	
Thinking Machines Lab	Tinker Agent RL	model customization / agent RL	policy weights; tool-use policy	RL; LLM-judge grading; tool discovery	L1-L2	
	Inkling	model customization / agent RL	policy weights; tool-use policy	RL; LLM-judge grading; tool discovery	L1-L2	
Nous Research	Hermes Agent	persistent agents	skills; memory; tool gateway	memory; skill reuse; tool orchestration	L1-L2	

continued on next page

原文第 67 页 | 中文翻译初稿

表 13 - 续

公司	产品/代表作品	子场景	改进目标/工件	AI控制部分	RSI关系	
	闪电特工 v1.0	利用智能体 RL	运行支撑框架（Harness）痕迹；策略权重	互动；重新标记；火车;基准-标记	L2	
	SkillOpt	技能优化	技能文件；说明	编辑；评价;优化	L2
	ReVeal	自验证代码
智能体	代码；测试；验证者策略	生成；核实;修订;规模	L2
	通用验证器/自动研究	智能体验证研发	细则；验证者；评估管道	设计；测试;比较;精炼	L2 adj.
元	自学评估器	评估器自我训练	判断模型；综合偏好-
生成；法官;火车;迭代	L2	
	HyperAgents	元智能体自我修改	任务智能体；元智能体；编程	求解；自我编辑；评价;存档	L5 cand。	
阿里巴巴/Qwen	Qwen-Agent	智能体训练/合成环境	训练数据；环境；
训练后	模拟；数据综合；工具
使用； RL	L1-L2	
	AgentWorld	智能体训练/合成环境	训练数据；环境；
训练后	模拟；数据综合；工具
使用； RL	L1-L2	
	Qwen-Scope	智能体训练/合成环境	训练数据；环境；
训练后	模拟；数据综合；工具
使用； RL	L1-L2	
DeepSeek	R1	推理自引导	推理策略；验证者	自生成推理； RL； veri-
更进一步的迭代	L2	
	Math-V2	推理自我	推理策略；验证者	自生成推理； RL；真实-	L2	
		引导		更快速的迭代		
腾讯AI实验室	R-Zero	自玩推理	任务；伪标签；策略
权重	挑战生成；解决;投票；
RL	L2-L3	
字节跳动Seed	Seed-Thinking	模型/智能体后训练	数据；报酬;策略权重	数据过滤；奖励验证；
RL	L1-L2
	Seed1.5-VL	模型/智能体训练后	数据；报酬;策略权重	数据过滤；奖励验证；
RL	L1-L2	
MiniMax	M2	持久云智能体	内存；技能；智能体策略	工具使用；长期执行；技能
重用	L1-L2	
	MaxHermes	持久云智能体	内存；技能；智能体策略	工具使用；长期执行；技能

重用|L1-L2|&#xfffd;|
||MaxClaw|持久云智能体|内存；技能；智能体策略|工具使用；长期执行；技能
重用|L1-L2|&#xfffd;|
|Moonshot AI|Kimi K3|长时程智能体（Long-Horizon Agents）|智能体编排；工具策略|规划；工具的使用；群体协调
tion|-
L1-L2 adj.|&#xfffd;|
||智能体群|长时程智能体（Long-Horizon Agents）|智能体编排；工具策略|规划；工具的使用；群体协调
tion|-
L1-L2 adj.|&#xfffd;|
|智普AI / Z.AI|AutoGLM|计算机使用智能体
训练|策略权重；虚拟电话
环境|感知；规划;行动; RL|L2|&#xfffd;|
||AgentRL|计算机使用智能体
训练|策略权重；虚拟电话
环境|感知；规划;行动; RL|L2|&#xfffd;|
|Deep Cogito|Cogito v2|推理后训练|推理策略；权重|搜索；蒸馏;迭代对齐|L1-L2|&#xfffd;
|
||IDA|推理训练后|推理策略；权重|搜索；蒸馏;迭代对齐|L1-L2|&#xfffd;|
|池畔|模型工厂|自动化模型研发|综合数据； RL；架构|评估；代码执行 RL；消融

;数据
混合|L1-L2|&#xfffd;|
|Thinking Machines L|ab Tinker Agent RL|模型定制/
agent RL|策略权重；工具使用策略|RL； LLM-法官分级；工具发现-
发现|L1-L2|&#xfffd;|
||Inkling|模型定制/
智能体RL|策略权重；工具使用策略|RL； LLM-法官分级；工具发现-
发现|L1-L2|&#xfffd;|
|Nous Research|赫尔墨斯特工|持久特工|技能；记忆;工具网关|内存；技能重用；工具编排
tion|-
L1-L2|&#xfffd;|
||||||下页继续|

Table 13 – continued

Company	Product / representative work	Sub-scenario	Improvement target / artifact	AI-controlled part	RSI relation	
	skills / memory	persistent agents	skills; memory; tool gateway	memory; skill reuse; tool orchestration	L1-L2	
B. RSI-native / AI4AI companies Self-improvement, AI-for-AI, self-evolution, or recursive improvement is central to the company/research thesis.						
Ricursive Intelligence	AI-chip co-design	AI systems / chips	EDA designs; compute stack	design search; verify; iterate	L2; L5 vision	
Recursive	Automated AI Research	model-training research	training recipes; kernels; code	ideas; experiments; branch merge	L2	
Imbue	Catalyst	research search	recipes; code; hypotheses	population search; experiments; interpretation	L2-L3	
	Darwinian Evolver	research search	recipes; code; hypotheses	population search; experiments; interpretation	L2-L3	
Weco AI	AIDE2	meta-improvement of researcher	research harness; improver code	rewrite improver; eval; inheritance	L5 cand.	
Sakana AI	Darwin Godel Machine	self-editing agents / AI research	agent source; research pipeline	self-modify; benchmark; archive; experiments	L5 cand.	
	AI Scientist	self-editing agents / AI research	agent source; research pipeline	self-modify; benchmark; archive; experiments	L5 cand.	
Evolvent AI	Org self-evolving agents	software-agent evolution	skills; memory; code; environments	tasks; feedback; refactor; skill updates	L2-L3	
	RSIBench	software-agent evolution	skills; memory; code; environments	tasks; feedback; refactor; skill updates	L2-L3	
	Terrarium	software-agent evolution	skills; memory; code; environments	tasks; feedback; refactor; skill updates	L2-L3	
MetaCircle	ComfyResearch	AI4AI / autoresearch	training workflows; research hypotheses	experiment compose; code; hypothesis; scoring	L2; L5 vision	
	OPHIS	AI4AI / autoresearch	training workflows; research hypotheses	experiment compose; code; hypothesis; scoring	L2; L5 vision	
Frontis AI / Xianyuan	OpenRSI	AI4AI / self-improving agents	skills; memory; harness; weights	experience; update search; eval; post-training	L2-L3	
	Frontis-MA1	AI4AI / self-improving agents	skills; memory; harness; weights	experience; update search; eval; post-training	L2-L3	
EvoMap	Evolver	shared code evolution	genes / capsules; code assets	generate; test; publish; reuse; adapt	L2-L3	
	GEP	shared code evolution	genes / capsules; code assets	generate; test; publish; reuse; adapt	L2-L3	
	GeneBench	shared code evolution	genes / capsules; code assets	generate; test; publish; reuse; adapt	L2-L3	
Endless Frontier	BigBang-v1	AI-research data generation	synthetic programs; training data; critic	generate; execute; critique; meta-critique	L2-L3	
Mirendil	Self-accelerating AI	AI R&D automation	model code; experiments; research stack	experiment generation; execution; iteration	L2	
	AI Scientist	AI R&D automation	model code; experiments; research stack	experiment generation; execution; iteration	L2	
Chaoyan Intelligence*	TUMIX	self-evolving research models	research policy; tool-use policy	questioning; experiments; code; self-check	L2-L3	
	R1-Code-Interpreter	self-evolving research models	research policy; tool-use policy	questioning; experiments; code; self-check	L2-L3	
	AI Scientist	self-evolving research models	research policy; tool-use policy	questioning; experiments; code; self-check	L2-L3	
Theseus	Argus	long-horizon self-evolving agents	experience; memory; environment setup	plan; code; review; experience writeback	L2-L3	
	SetupX	long-horizon self-evolving agents	experience; memory; environment setup	plan; code; review; experience writeback	L3	

continued on next page

原文第 68 页 | 中文翻译初稿

表 13 - 续

公司	产品/代表作品	子场景	改进目标/工件	AI控制部分	RSI关系
	技能/记忆	持久性智能体	技能；记忆;工具网关	内存；技能重用；工具编排
tion	a-
L1-L2
B。 RSI-native / AI4AI 公司
自我改进（Self-Improvement）、AI-for	-AI、自我进化或递归改进是公司/研究论文的核心。		
Ricursive Intelligence
递归
Imbue	AI芯片协同设计
自动化AI研究
Catalyst	AI系统/芯片
模型训练研究
研究搜索	EDA设计；计算堆栈
训练方法；内核；代码
食谱；代码;假设	设计搜索；核实;迭代
想法；实验；分支合并
群体搜索；实验；
解释	L2； L5视力
L2
L2-L3
达尔文进化论	研究搜索	食谱；代码;假设	总体搜索；实验；
解释	L2-L3
Weco AI	AIDE2	研究员的元改进	研究运行支撑框架（Harness）;改进代码	重写改进器；评估；继承	L5
cand。
Sakana AI	达尔文哥德尔机	自编辑智能体/AI
研究	智能体来源；研究管线	自我修改；基准;存档；
实验	L5 cand。
	AI科学家	自编辑智能体/AI
研究	智能体来源；研究管线	自我修改；基准;存档；
实验	L5 cand。
进化人工智能	组织自我进化智能体	软件智能体进化	技能；记忆;代码;环境	任务；反馈;重构；技能
更新	L2-L3
	RSIBench	软件智能体进化	技能；记忆;代码;环境	任务；反馈;重构；技能
更新	L2-L3
	玻璃容器	软件智能体进化	技能；记忆;代码;环境	任务；反馈;重构；技能

更新|L2-L3|&#65533;|
|MetaCircle|ComfyResearch|AI4AI / autoresearch|训练工作流程；研究
假设|实验撰写；代码;假设；得分|h-
L2； L5视觉|&#65533;|
||OPHIS|AI4AI / 自动研究|训练工作流程；研究
假设|实验撰写；代码;假设；得分|h-
L2； L5视觉|&#65533;|
|Frontis AI / 先源|OpenRSI|AI4AI / 自我改进（Self-Improving）
智能体|技能；记忆; 运行支撑框架（Harness）；体重|经验；更新搜索； eval;
训练后|L2-L3|&#65533;|
||Frontis-MA1|AI4AI / 自我改进（Self-Improving）
智能体|技能；记忆; 运行支撑框架（Harness）；体重|经验；更新搜索； eval;
训练后|L2-L3|&#65533;|
|EvoMap|Evolver|共享代码进化|基因/胶囊；代码资产|生成；测试;发布;重用;
适应|L2-L3|&#65533;|
||GEP|共享代码进化|基因/胶囊；代码资产|生成；测试;发布;重用;
适应|L2-L3|&#65533;|
||GeneBench|共享代码进化|基因/胶囊；代码资产|生成；测试;发布;重用;
适应|L2-L3|&#65533;|
|无尽前沿|BigBang-v1|人工智能研究数据生成
tion|-
综合程序；训练
数据；批评|生成；执行;批判;元-
批评|L2-L3|&#65533;|
|Mirendil|自加速AI|AI研发自动化|模型代码；实验；重新
搜索堆栈|实验生成；执行;
迭代|L2|&#65533;|
||AI科学家|AI研发自动化|模型代码；实验；重新
搜索堆栈|实验生成

;执行;
迭代|L2|&#65533;|
|超研智能*|TUMIX|自我演化研究
模型|研究策略；工具使用策略|质疑；实验；代码;
自检|L2-L3|&#65533;|
||R1-Code-Interpreter|自我演化研究
模型|研究策略；工具使用策略|质疑；实验；代码;
自检|L2-L3|&#65533;|
||AI科学家|自我进化研究
模型|研究策略；工具使用策略|质疑；实验；代码;
自检|L2-L3|&#65533;|
|忒修斯|阿古斯|长时程（Long-Horizon） 自我进化特工|经验；记忆;环境设置|计划；代码;审查;经验
回写|L2-L3|&#65533;

|
||SetupX|长时程（Long-Horizon） 自我进化智能体|经验；记忆;环境设置|计划；代码;审查;经验回写|L3|  |

接下页

Table 13 – continued

Company	Product / representative work	Sub-scenario	Improvement target / artifact	AI-controlled part	RSI relation	
Adaption Labs	AutoScientist	AI-research automation	training recipes; datasets; code	research plan; experiments; selection	L2	
	Forge	AI-research automation	training recipes; datasets; code	research plan; experiments; selection	L2	
C. Autonomous R&D & scientific-discovery companies	Primary product is automated research/discovery; RSI relevance comes from automating experiment and knowledge-production loops.					
Periodic Labs	Autonomous laboratory	AI for science	hypotheses; experiment data; models	experiment design; lab run; learning	L3 cand.	
FutureHouse	BixBench	AI-scientist evaluation	research tasks; benchmark frontier	bioinformatics workflows; open-ended eval	B0-L2 adj.	
Edison Scientific	Kosmos	autonomous science	hypotheses; code; scientific artifacts	literature; experiments; synthesis	L2-L3	
Axiom Math	Putnam 2025	formal mathematics	proofs; verifier traces	conjecture; proof search; formal verification	L2	
	IMO 2026	formal mathematics	proofs; verifier traces	conjecture; proof search; formal verification	L2	
Harmonic	Aristotle	theorem proving	formal proofs	translate; prove; verify	L1-L2	
Core Automation	AI systems-research stack	automated systems research	systems code; research hypotheses	design; code; benchmark; iterate	L2 cand.	
Discovery Loop	Autonomous discovery loop	automated experimentation	protocols; findings	hypothesis; experiment; analysis; next-step	L3 cand.	
Lila Sciences	Autonomous Science platform	scientific discovery	hypotheses; experiments; post-training data	design; lab execution; real-time learning	L3 cand.	
Karpathy / autoresearch	autoresearch	narrow automated research	training code; configs	edit; train; measure; keep	L2	
Prime Intellect	Autonomous research	model R&D	training recipe; optimizer; code	hypothesis; GPU runs; ablation	L2	
	Speedrun Frontier	model R&D	training recipe; optimizer; code	hypothesis; GPU runs; ablation	L2	
Analemma	FARS	autonomous research	proposals; code; logs; papers	topic; experiment; analysis; writing	L2-L3	
Novix	AutoAgent	AI research agent	research workflow; eval harness	idea; tool use; experiments; reporting	L2	
	OpenHarness	AI research agent	research workflow; eval harness	idea; tool use; experiments; reporting	L2	
	AI-Researcher	AI research agent	research workflow; eval harness	idea; tool use; experiments; reporting	L2	
UniPat	UniScientist	research / evaluation agents	research traces; benchmarks	experiments; coding; multi-agent eval	L1-L2	
	UniSwarm	research / evaluation agents	research traces; benchmarks	experiments; coding; multi-agent eval	L1-L2	
	UniMath	research / evaluation agents	research traces; benchmarks	experiments; coding; multi-agent eval	L1-L2	
Kai Chen / venture TBD*	Intern-S1	AI for science models	scientific model; training data	model training; scientific reasoning	adj.; entity TBD	
D. Agent optimization, evaluation & learning infrastructure	Infrastructure for feedback, skills, RL, evaluation, simulators, training data, or production-agent improvement.					
Warp	Skill optimization loop	coding-agent skills	skills; instructions; examples	feedback mining; skill rewrite; eval	L2	
Factory	Signals	production coding agents	agent behavior; product code	session mining; failure clusters; patch PRs	L4	
	Software Factory	production coding agents	agent behavior; product code	session mining; failure clusters; patch PRs	L4	
LangChain	LangSmith self-improving evaluators	evaluator alignment	judge prompts; evaluator model	feedback ingest; judge update; eval	L1-L2	
Replit	Agent 3	software engineering	application code; tests	build; test; repair; long runs	L1-L2	

continued on next page

原文第 69 页 | 中文翻译初稿

		表	13 - 续			
公司	产品/代表作品	子场景	改进目标/工件	AI控制部分	RSI关系	�
适应实验室	AutoScientist	人工智能研究自动化	训练食谱；数据集；代码	研究计划；实验；选择-
tion	L2	�
	Forge	人工智能研究自动化	训练食谱；数据集；代码	研究计划；实验；选择-
tion	L2	�
C.自主研发
&科学发现公司
主要
产品是自动化研究
rch/discovery； RSI 相关性
来自自动化实验和 kn
owledge-producti
循环。
定期实验室	自主实验室	科学人工智能	假设；实验数据；
模型	实验设计；实验室运行；学习	L3 cand。	�
FutureHouse	BixBench	AI-科学家评估	研究任务；生物信息学工作流程的基准；开放式评估	B0-L2 adj.	�
爱迪生科学	宇宙	自主科学	假设；代码;科学艺术-
事实	文献；实验；合成	L2-L3	�
公理数学	Putnam 2025	形式数学	证明；验证者痕迹	猜想；证据检索；正式
验证	L2	�
	IMO 2026	正式数学	证明；验证者痕迹	我 猜想；证据检索；正式
验证	L2	�
调和	亚里士多德	定理证明	形式证明	翻译;证明;验证	L1-L2	�
核心自动化	人工智能系统-研究堆栈	自动化系统
研究	系统代码；研究假设-
ses	设计；代码;基准;迭代	L2 cand。	�
发现循环	自主发现循环	自动化实验
tion	-
协议；研究结果	假设；实验;分析；
下一步	L3 cand。	�
Lila Sciences	自主科学平台	科学发现	假设；实验；训练后数据	设计；实验室执行；实时
学习	L3 cand。	�
Karpathy / autore-
搜索	autoresearch	狭义自动化重新
搜索	训练代码；配置	编辑；火车;措施;保持	L2	�
原始智力	自主研究	模型研发	训练配方；优化器；代码	假设； GPU运行；消融	L2	�

||极速边境|模型研发|训练配方；优化器；代码|假设； GPU运行；消融|L2|&#65533;|
|Analemma|FARS|自主研究|提案；代码;日志；论文|主题；实验;分析;写作|L2-L3|&#65533;|
|Novix|AutoAgent|AI 研究智能体|研究工作流程；评估 运行支撑框架（Harness）|想法；工具的使用；实验；报告
ing|-
L2|&#65533;|
||OpenHarness|AI 研究智能体|研究工作流程；评估 运行支撑框架（Harness）|想法；工具的使用；实验；报告
ing|-
L2|&#65533;|
||AI-研究员|AI研究智能体|研究工作流程；评估 运行支撑框架（Harness）|想法；工具的使用；实验；报告
ing|-
L2|&#65533;|
|UniPat|UniScientist|研究/评估
智能体|研究痕迹；基准|实验；编码;多智能体
eval|L1-L2|&#65533;|
||UniSwarm|研究/评估
智能体|研究痕迹；基准|实验；编码;多智能体
eval|L1-L2|&#65533;|
||UniMath|研究/评估
智能体|研究痕迹；基准|实验；编码;多智能体
eval|L1-L2|&#65533;|
|陈凯 / 风险投资
待定*|实习生-S1|科学模型的人工智能|科学模型；训练数据|模型训练；科学推理|形容词;实体
待定|&#65533;|
|D

。智能体优化|、评估和学习基础设施
反馈、技能、RL、评估、模拟器、训练数据或生产智能体改进的基础设施。||
|Warp|技能优化循环|编码智能体技能|技能；指示;例子|反馈挖掘；技能重写；评估|L2|&#65533;|
|工厂|信号|生产编码
智能体|智能体行为；产品代码|会话挖矿；故障集群；
补丁 PR|L4|&#65533;|
||软件工厂|生产编码
智能体|智能体行为；产品代码|会话挖矿；故障集群；
补丁 PR|L4|&#65533;|
LangChain	LangSmith 自我改进（Self-Improving） 评估器	评估器对齐	判断提示；评估者模型	反馈摄取；法官更新；评估	L1-L2	&#65533;
Replit	Agent 3	软件工程	应用程序代码；测试	构建；测试;维修;长跑	L1-L2	&#65533;
					下页继续

Table 13 – continued

Company	Product / representative work	Sub-scenario	Improvement target / artifact	AI-controlled part	RSI relation	
MorphMind	Caliper	agent calibration / org memory	agent skills; shared experience	measure; route; reuse experience	L1-L2 adj.	
DatologyAI	BeyondWeb	data optimization	training corpus; synthetic data	curate; filter; synthesize; benchmark	L1-L2	
	DatBench	data optimization	training corpus; synthetic data	curate; filter; synthesize; benchmark	L1-L2	
Ineffable Intelligence	RL infrastructure	RL / experience infrastructure	training environments; trajectories	rollouts; reward; distributed RL	L1-L3 enabler	
Goodfire	Ember	interpretability-driven training	feature activations; reward signals	feature discovery; feedback; RL	L1-L2	
	RLFR	interpretability-driven training	feature activations; reward signals	feature discovery; feedback; RL	L1-L2	
Patronus AI	Generative Simulators	evaluation / simulation	simulators; eval suites	scenario generation; grading; failure analysis	L2-L3 enabler	
	Percival	evaluation / simulation	simulators; eval suites	scenario generation; grading; failure analysis	L2-L3 enabler	
Braintrust	Loop	production feedback / eval	prompts; evals; datasets	trace ingest; scoring; prompt optimization	L1-L2	
	Autoevals	production feedback / eval	prompts; evals; datasets	trace ingest; scoring; prompt optimization	L1-L2	
Mechanize	RL environments	experience / eval infrastructure	RL tasks; graders; environments	task design; grading; training signal	L3 enabler	
	GBA Eval	experience / eval infrastructure	RL tasks; graders; environments	task design; grading; training signal	L3 enabler	
Kando AI	Early company signal	undisclosed / early-stage	undisclosed	undisclosed	unverified	
Entropy Order*	Data-expert platform	high-quality training data	expert data; eval assets	data production; benchmarking	L1-L3 enabler	
Compounding Intelligence / CORAL*	CORAL	multi-agent research	shared notes; skills; logs	parallel experiments; knowledge sharing; eval	L2-L3	
Naive.ai*	Research-lab signal	early RSI lab	undisclosed	undisclosed	RSI claim; unverified	
E. Embodied, world-model & continual-adaptation companies	Persistent adaptation is tied to world models, robotics, environments, or test-time/continual learning.					
AI2 Robotics	FiS-VLA	embodied policy learning	VLA policy; action data	data; training; planning; control	L2-L3 adj.	
	Video2Act	embodied policy learning	VLA policy; action data	data; training; planning; control	L2-L3 adj.	
X Square Robot	WALL	world models / skill acquisition	world model; skills; action policy	video skill capture; world prediction; control	L2-L3	
	HOST	world models / skill acquisition	world model; skills; action policy	video skill capture; world prediction; control	L2-L3	
Galaxea AI	G0 / G0.5	VLA R&D platform	VLA model; data; deployment stack	training; eval; real-robot deployment	L2 adj.	
	GForge	VLA R&D platform	VLA model; data; deployment stack	training; eval; real-robot deployment	L2 adj.	
TARS Robotics	AWE	tactile world models	tactile world model; manipulation policy	multimodal sensing; prediction; control	L2-L3 adj.	
	TacForeSight	tactile world models	tactile world model; manipulation policy	multimodal sensing; prediction; control	L2-L3 adj.	

continued on next page

原文第 70 页 | 中文翻译初稿

表 13 - 续

公司	产品/代表作品	子场景	改进目标/工件	AI控制部分	RSI关系	
MorphMind	Caliper	智能体校准/组织
内存	智能体技能；分享经验	措施；路线;重用经验	L1-L2 adj.	
DataologyAI	BeyondWeb	数据优化	训练语料库；综合数据	策划；筛选;合成;基准-
标记	L1-L2	
	DatBench	数据优化	训练语料库；综合数据	策划；筛选;合成;基准-
标记|L1-L2||
不可言喻的智能	强化学习基础设施	强化学习/体验基础设施-
结构|训练环境；轨迹-
ries|展开轨迹（Rollouts）；报酬;分布式 RL|L1-L3 使能器||
Goodfire	Ember	可解释性驱动
训练|功能激活；奖励信号-
nals	特征发现；反馈; RL	L1-L2	
	RLFR	可解释性驱动
训练|特征激活；奖励信号-
nals	特征发现；反馈; RL	L1-L2	
Patronus AI	生成模拟器	评估/模拟	模拟器；评估套件	场景生成；等级;故障分析	L2-L3 推动因素	
	Percival	评估/模拟	模拟器；评估套件	场景生成；等级;故障分析	L2-L3 推动者	
Braintrust	循环	生产反馈/
评估	提示；评估；数据集	跟踪摄取；得分；及时优化	L1-L2	
	自动评估	生产反馈/
评估|提示；评估；数据集|跟踪摄取；得分；及时优化|L1-L2||
机械化	强化学习环境	体验/评估基础设施-
结构|强化学习任务；评分员；环境|任务设计；等级;训练
信号|L3 启动器| 
	GBA 评估	经验/评估基础设施-
结构|RL 任务；评分员；环境|任务设计；等级;训练
信号|L3 启动器||
Kando AI	早期公司信号	未公开/早期
阶段	未公开	未公开	未验证	
熵阶*	数据专家平台	高质量训练
数据|专家数据；评估资产|数据生产；基准测试|L1-L3 推动者||
复合智能-
gence / CORAL*|CORAL|多智能体研究|共享笔记；技能；日志|并行实验；知识
分享；评估	L2-L3	
Naive.ai*	研究实验室信号	早期 RSI 实验室	未公开	未公开	RSI 声明；联合国-	
E。体现的、世界移动

|德尔和持续适应公司
|持续适应与世界模型（World Models）、机器人、环境
|ment或测试时/持续学习|验证
g.||相关。
|AI2 Robotics|FiS-VLA|具体策略学习|VLA 策略；动作数据|数据；训练;规划;控制|L2-L3 调节|  |
||Video2Act|体现策略学习|VLA 策略；动作数据|数据；训练;规划;控制|L2-L3 调节|  |
|X方形机器人|WALL|世界模型（World Models） / 技能
获取|世界模型（World Model）;技能；行动方针|视频技能捕捉；世界预测-
tion；控制|L2-L3|  |
||HOST|世界模型（World Models） / 技能
获取|世界模型（World Model）;技能；行动方针|视频技能捕捉；世界预测-
tion；控制|L2-L3|  |
|Galaxea AI|G0 / G0.5|VLA研发平台|VLA模型；数据;部署
堆栈|训练；评估；真实机器人部署-
ment|L2 adj.||
||GForge|VLA研发平台|VLA模型；数据;部署
堆栈|训练；评估；真实机器人部署-
ment|L2 adj.||
|TARS Robotics|AWE|触觉 世界模型（World Models）|触觉 世界模型（World Model）

;操纵策略|多模态传感；预测;
控制|L2-L3 adj.|  |
||TacForeSight|触觉 世界模型（World Models）|触觉 世界模型（World Model）;操纵策略|多模态传感；预测;
控制|L2-L3 adj.|  |

接下页

Table 13 – continued

Company	Product / representative work	Sub-scenario	Improvement target / artifact	AI-controlled part	RSI relation	
Synapx Dynamics	SYNWorld	embodied data / model stack	data; world model; post-training	data synthesis; training; eval; control	L2-L3	
	OctoMind	embodied data / model stack	data; world model; post-training	data synthesis; training; eval; control	L2-L3	
	OctoSense	embodied data / model stack	data; world model; post-training	data synthesis; training; eval; control	L2-L3	
MirrOS	Code as Worlds	world models / test-time adaptation	world code; fast weights	environment modeling; eval; test-time update	L1-L2	
	Spatial-TTT	world models / test-time adaptation	world code; fast weights	environment modeling; eval; test-time update	L1-L2	
Wuya Zhiyuan*	SHINE	continual / test-time learning	LoRA; skill vectors; weights	context adaptation; skill transfer	L1-L2	
	LIFT	continual / test-time learning	LoRA; skill vectors; weights	context adaptation; skill transfer	L1-L2	
	PaST	continual / test-time learning	LoRA; skill vectors; weights	context adaptation; skill transfer	L1-L2	
Singularity Escape / Nexus*	Nexus	collaborative self-evolving agents	shared state; org knowledge; harness	collaboration; experience capture; harness adaptation	L3-L4 cand.	
F. Persistent-memory & personal-AI companies	Cross-task memory, identity, or reusable skills are the main substrate of adaptation.					
Engram	Knowledge Cartridges	persistent enterprise memory	knowledge modules; memory	extract; store; retrieve; adapt	L1-L2 adj.	
Lemon AI / Hexdo	LemonAI Evolving	local persistent agents	workspace; memory; experience library	web / files / code; persistence; reuse	L1-L2	
EverMind	EverOS	agent memory / OS	long-term memory; skills	retrieve; skill forge; active tasks; eval	L2-L3	
	Raven	agent memory / OS	long-term memory; skills	retrieve; skill forge; active tasks; eval	L2-L3	
	EverMemOS	agent memory / OS	long-term memory; skills	retrieve; skill forge; active tasks; eval	L2-L3	
Mindverse	Second Me	personal memory / adaptation	identity model; memory; LoRA	memory modeling; personalization; retrieval	L1-L2	
	Me.bot	personal memory / adaptation	identity model; memory; LoRA	memory modeling; personalization; retrieval	L1-L2	

Reading guide. B0 denotes task-local autonomy only. L1–L5 follow the review's increasing autonomy scale. "adj." marks RSI-adjacent infrastructure or behavior; "cand." marks a plausible but not fully demonstrated level; "target" marks an explicit future objective rather than a demonstrated current capability. Asterisks indicate entries whose company identity or public technical boundary still requires stronger verification.

原文第 71 页 | 中文翻译初稿

表 13 - 续

公司	产品/代表作品	子场景	改进目标/工件	AI控制部分	RSI关系	
Synapx Dynamics	SYNWorld	体现数据/模型
堆栈|数据； 世界模型（World Model）；训练后|数据综合；训练;评估；控制-
控制|L2-L3|
	OctoMind	体现数据/模型
堆栈|数据； 世界模型（World Model）；训练后|数据综合；训练;评估；控制-
控制|L2-L3|
	OctoSense	体现数据/模型
堆栈|数据； 世界模型（World Model）；训练后|数据综合；训练;评估；控制-
控制|L2-L3||
MirrOS	世界代码	世界模型（World Models）/ 测试-
时间适应|世界代码；快速权重|环境建模；评估；测试-
时间更新	L1-L2	
	空间-TTT	世界模型（World Models）/ 测试-
时间适应|世界代码；快速权重|环境建模；评估；测试-
时间更新	L1-L2	
无雅致远*	SHINE	持续/测试时
学习	LoRA；技能向量；权重	上下文适应；技能转移	L1-L2	
	LIFT	持续/测试时
学习	LoRA；技能向量；权重	上下文适应；技能转移	L1-L2	
	PaST	持续/测试时
学习	LoRA；技能向量；权重	上下文适应；技能转移	L1-L2	
奇点逃脱 /
Nexus*|Nexus|协作自我
进化智能体|共享状态；组织知识；
运行支撑框架（Harness）|协作；经验捕捉；
运行支撑框架（Harness）适配	L3-L4 cand。	
F。持久记忆和个人人工智能公司
跨任务|记忆、身份或可重复使用的技能是适应的主要基础。|||
印记	知识盒	持久性企业
内存|知识模块；记忆|摘录；店铺;取回;适应|L1-L2 adj.||
Lemon AI / Hexdo	LemonAI Evolving	本地持久智能体	工作空间；记忆;经验
库|网页/文件/代码；持久性；
重用|L1-L2||
EverMind	EverOS	智能体内存/操作系统	长期内存；技能	检索；技能锻造；活动任务；
eval|L2-L3||

||Raven|智能体内存/操作系统|长期记忆；技能|检索；技能锻造；活动任务；
eval|L2-L3|□|
||EverMemOS|智能体内存/操作系统|长期内存；技能|检索；技能锻造；活动任务；
eval|L2-L3|□|
|Mindverse|第二个我|个人记忆/
适应|身份模型；记忆; LoRA|内存建模；个性化
检索|;
L1-L2|□|
||Me.bot|个人记忆/
适应|身份模型；记忆; LoRA|内存建模；个性化
检索|;
L1-L2|□|

阅读指南。 B0 仅表示任务局部自治。 L1-L5 遵循审查不断增加的自主权规模。 “形容词。”标记与 RSI 相邻的基础设施或行为； “坎。”标志着一个看似合理但未充分证明的水平； “目标”标志着明确的未来目标，而不是已证明的当前能力。星号表示其公司身份或公共技术边界仍需要更强验证的条目。

Appendix C: RSI Across Applications

Having classified systems from B0 to L5 according to the improvement functions placed under AI control, we now examine how RSI is applied across different scenarios. As illustrated in Fig. 9, applications differ in both what they improve and the feedback required to validate and retain an update. We organize the evidence into four application regimes, including AI for science (S1), embodied intelligence (S2), software engineering (S3), and healthcare (S4). These scenarios are not mutually exclusive, as a healthcare AI scientist may fall under both S1 and S4.

Each subsection follows a common structure. We first characterize the typical AI workflow in the target scenario, identify how an RSI loop can emerge from its tasks and available feedback, and review the challenges of the RSI for a specific scenario. We then group representative systems according to the components or workflows they improve, and compare where they intervene in the RSI loop, what they update, and how these improvements are retained. Finally, we assess the gap between current research and the full RSI loop envisioned for each scenario, highlighting the strength of existing evidence, scenario-specific risks, and the key barriers to greater improvement.

C.1 S1: RSI for Science

Science is a particularly consequential setting for RSI, as scientific progress depends not only on solving isolated problems, but also on improving the mechanisms by which future questions, hypotheses, and experiments are generated. Scientific RSI therefore differs from automated discovery that merely searches for a better molecule, equation, or hypothesis. A scientific RSI loop starts from a research challenge, generates candidate hypotheses and experiments, evaluates them through computation or interaction with the physical world, and uses the resulting evidence or falsification to diagnose the AI-for-science (AI4Sci) system.

RSI for science is mainly challenged by three characteristics. First, scientific discovery is open-ended. Neither the space of relevant hypotheses nor the set of necessary tools and experiments is known in advance, and costly experiments make exploration itself part of the improvement problem. Second, scientific feedback is often sparse, delayed, and non-identifying. For example, a failed experiment may indicate an incorrect hypothesis, but it may also result from an inappropriate model, an invalid protocol, or an unreliable instrument. Third, scientific validity is conditional on assumptions, domains, and measurement procedures. An update that succeeds in one setting cannot be inherited safely without recording its provenance, scope of validity, and uncertainty. These characteristics make scientific RSI simultaneously a problem of evidence acquisition, capability evolution, and epistemic control. Accordingly, we organize existing work around three components that may evolve, including the scientific hypothesis module, the experimental agent, and the reflection system or improver.

C.1.1 Evolving Scientific Hypothesis Modules

The hypothesis module determines which explanations or candidate methods are proposed and prioritized. Its evolution requires feedback to change the mechanism used for future hypothesis generation, rather than merely selecting a better hypothesis for the current problem. HypoForge [158] continually distills scientific experience into reusable procedural skills for hypothesis generation, experimental design, and execution. It updates generation skills from discriminator critiques when direct empirical feedback is unavailable, and refines testing skills from real execution outcomes and attributed failures. EvoScientist [173] instead maintains an ideation memory of promising and failed research directions, which is retrieved in later tasks to adjust subsequent proposals. This produces persistent memory-based adaptation, although the underlying models, memory schema, and update procedure remain fixed.

A more direct approach updates the candidate generator itself. TTT-Discover [174] applies reinforcement learning at test time, allowing scientific feedback to modify model weights while solving a new problem. The self-improvable polymer discovery framework [175] similarly adds simulation-evaluated candidates to the training data and retrains its generative model. The evolution of these systems remain task- or domain-specific and are produced by fixed training and selection procedures. Current evidence therefore supports bounded hypothesis-level improvement, not a general mechanism that becomes progressively better at formulating scientific hypotheses across domains.

原文第 72 页 | 中文翻译初稿

附录 A：RSI 实施技术
附录 C：跨应用程序的 RSI

根据人工智能控制下的改进功能将系统分类为 B0 到 L5，现在我们研究 RSI 如何在不同场景中应用。如图 9 所示，应用程序在改进内容以及验证和保留更新所需的反馈方面都有所不同。我们将证据分为四种应用模式，包括科学人工智能 (S1)、具身智能（Embodied Intelligence）(S2)、软件工程 (S3) 和医疗保健 (S4)。这些场景并不相互排斥，因为医疗保健人工智能科学家可能同时属于 S1 和 S4。

每个小节都遵循一个共同的结构。我们首先描述目标场景中的典型人工智能工作流程，确定 RSI 循环如何从其任务和可用反馈中出现，并回顾特定场景的 RSI 挑战。然后，我们根据代表性系统改进的组件或工作流程对它们进行分组，并比较它们在 RSI 循环中的干预位置、更新内容以及如何保留这些改进。最后，我们评估了当前研究与每种情景设想的完整 RSI 循环之间的差距，强调了现有证据的强度、特定情景的风险以及取得更大改进的关键障碍。

C.1 S1：科学 RSI

科学对于 RSI 来说是一个特别重要的背景，因为科学进步不仅取决于解决孤立的问题，还取决于改进产生未来问题、假设和实验的机制。因此，科学 RSI 不同于仅仅寻找更好的分子、方程或假设的自动发现。科学 RSI 循环从研究挑战开始，生成候选假设和实验，通过计算或与物理世界的交互对其进行评估，并使用所得证据或证伪来诊断人工智能科学 (AI4Sci) 系统。

科学 RSI 主要受到三个特征的挑战。首先，科学发现是开放式的。相关假设的空间和必要的工具和实验都是事先未知的，而昂贵的实验使探索本身成为改进问题的一部分。其次，科学反馈往往稀疏、延迟且不确定。例如，失败的实验可能表明假设不正确，但也可能是由不合适的模型、无效的方案或不可靠的仪器造成的。第三，科学有效性取决于假设、领域和测量程序。在一种设置中成功的更新如果不记录其出处、有效性范围和不确定性，就无法安全地继承。这些特征使得科学 RSI 同时成为证据获取、能力演化和认知控制的问题。因此，我们围绕可能演变的三个组件组织现有的工作，包括科学假设模块、实验智能体和反射系统或改进器。

C.1.1 不断发展的科学假设模块

假设模块确定提出哪些解释或候选方法并确定优先级

。它的演变需要反馈来改变用于未来假设生成的机制，而不仅仅是为当前问题选择更好的假设。HypoForge [158] 不断将科学经验提炼成可重复使用的程序技能，用于假设生成、实验设计和执行。当无法获得直接的经验反馈时，它会根据判别器的批评更新生成技能，并根据实际执行结果和归因失败来完善测试技能。 EvoScientist [173] 相反，保留了有希望和失败的研究方向的构思记忆，在以后的任务中检索该记忆以调整后续的建议。尽管底层模型、内存模式和更新过程保持固定，但这会产生持久的基于内存的适应。

更直接的方法更新候选生成器本身。 TTT-Discover [174] 在测试时应用 强化学习（Reinforcement Learning），允许科学反馈来修改模型权重，同时解决新问题。可自我改进的聚合物发现框架[175]同样将模拟评估的候选者添加到训练数据中，并重新训练其生成模型。这些系统的发展仍然是特定于任务或领域的，并且是通过固定的训练和选择程序产生的。因此，目前的证据支持有限的假设水平改进，而不是一种在跨领域制定科学假设方面逐渐变得更好的通用机制。

C.1.2 Evolving Experimental Agents

An experimental agent turns a hypothesis into an executable procedure by selecting tools, writing code, or operating laboratory equipment. Its evolution occurs when execution feedback is converted into reusable actions. Test-Time Tool Evolution [159] synthesizes, verifies, and reuses executable scientific tools during inference, whose SciEvo benchmark contains 1,590 tasks supported by 925 evolved tools. CASCADE [176] creates, repairs, and accumulates executable skills for chemistry and materials science, while S1-NexusAgent [177] distills complete research trajectories into reusable Scientific Skills. SkillFoundry [178] builds and continually maintains a library of reusable scientific skills by mining heterogeneous resources such as papers, repositories, documentation, and APIs, compiling them into executable skill packages, and validating, merging, or pruning these skills based on execution and downstream utility. These methods make the agent's action space persistent and extensible, although their tool-generation and admission rules remain externally specified.

Other systems evolve broader experimental strategies and workflows. DrugSAGE [160] stores verified modeling procedures, training strategies, and recurring error fixes in a cross-task memory. Experience collected from 16 tasks improves performance on 17 held-out tasks, providing direct evidence that inherited experience changes behavior on new problems. STELLA [179] updates reasoning templates and expands a Tool Ocean for biomedical research, EarthLink [180] retains successful scripts and analytical workflows for climate science, and OriGene [181] refines thinking templates, tool compositions, and analytical protocols using human and experimental feedback. In physical simulation, the self-evolving fluid-control agent [182] jointly revises its agent definition and the white-box controller it produces. This case also illustrates an important evaluation boundary, where improvements to the experimental agent must be measured separately from improvements to the scientific artifact, such as the controller.

Across these systems, the strongest evidence comes from validated reuse on later tasks rather than growth in the size of a tool or memory library. Reliable experimental-agent evolution therefore requires explicit skill preconditions, execution tests, provenance, versioning, and rollback. Without these controls, a locally successful workflow may be inherited outside the conditions under which it was validated.

C.1.3 Evolving Reflection Systems and Improvers

The reflection system interprets evidence and identifies the cause of failure, whereas the improver decides what component to modify, how to modify it, and whether the update should be retained. A fixed critic that revises the current output executes a predefined improvement procedure, while an evolving improver must instead update its own diagnostic, intervention, or promotion policy from the outcomes of earlier improvements.

SIA [161] provides the broadest component-level update loop among current AI4Sci systems. Its Feedback-Agent uses task outcomes to choose between modifying the agent scaffold, tools, retry and search logic, and low-rank adapter (LoRA) weights with different strategies. CORAL [183] replaces fixed evolutionary-search heuristics with long-running asynchronous agents that autonomously inspect prior attempts, decide when to test or invoke evaluators, and externalize discoveries as shared attempts, notes, and reusable skills, with heartbeat-triggered reflection and redirection sustaining long-horizon exploration. SAGA [184] extends the editable surface to the objective layer. Under a fixed high-level scientific goal, it diagnoses failure modes in optimized candidate populations, proposes or reweights concrete objectives, and implements them as executable scoring functions that steer subsequent search.

C.1.4 Toward Recursively Improving Science Agents

Taken together, these lines of work point toward a broader vision of scientific RSI. Its ultimate form is not merely a system that produces better hypotheses or experimental results, but one that becomes progressively better at conducting science with original innovation. It should jointly improve how hypotheses are generated, experiments are executed, and evidence is interpreted, such that each research cycle produces both scientific findings and reusable improvements to future discovery.

Current AI4Sci systems realize this vision only partially. B0-style output refinement is already widespread, while L1 persistent updates have been demonstrated through model retraining and memory accumulation [173–175]. L2 represents the strongest current frontier, with systems autonomously selecting or applying updates

原文第 73 页 | 中文翻译初稿

附录 A：RSI 实施技术

C.1.2 不断进化的实验智能体

实验智能体通过选择工具、编写代码或操作实验室设备将假设转化为可执行程序。当执行反馈转换为可重用的操作时，它就会发生演变。 Test-Time Tool Evolution [159] 在推理过程中综合、验证和重用可执行的科学工具，其 SciEvo 基准包含由 925 个进化工具支持的 1,590 个任务。 CASCADE [176] 创建、修复和积累化学和材料科学的可执行技能，而 S1-NexusAgent [177] 将完整的研究轨迹提炼成可重复使用的科学技能。 SkillFoundry [178] 通过挖掘论文、存储库、文档和 API 等异构资源，将它们编译成可执行的技能包，并根据执行和下游实用程序验证、合并或修剪这些技能，构建并持续维护可重用的科学技能库。这些方法使智能体的动作空间持久且可扩展，尽管它们的工具生成和准入规则仍然是外部指定的。

其他系统发展了更广泛的实验策略和工作流程。 DrugSAGE [160] 在跨任务内存中存储经过验证的建模过程、训练策略和重复出现的错误修复。从 16 项任务中收集的经验提高了 17 项保留任务的绩效，提供了继承经验改变新问题行为的直接证据。 STELLA [179] 更新了推理模板并扩展了生物医学研究的工具海洋，EarthLink [180] 保留了气候科学的成功脚本和分析工作流程，OriGene [181] 使用人类和实验反馈完善了思维模板、工具组合和分析协议。在物理模拟中，自进化流体控制智能体[182]共同修改其智能体定义及其产生的白盒控制器。这个案例还说明了一个重要的评估边界，其中实验智能体的改进必须与科学工件（例如控制器）的改进分开衡量。

在这些系统中，最有力的证据来自于后续任务中经过验证的重用，而不是工具或内存库大小的增长。因此，可靠的实验智能体进化需要明确的技能前提条件、执行测试、出处、版本控制和回滚。如果没有这些控制，本地成功的工作流程可能会在其验证条件之外被继承。

C.1.3 不断发展的反射系统和改进器

反思系统解释证据并识别失败原因，而改进者决定修改哪些组件、如何修改以及是否应保留更新。修改当前输出的固定批评者执行预定义的改进程序，而不断发展的改进者必须根据早期改进的结果更新自己的诊断、干预或促销策略。

SIA [161] 提供当前 AI4Sci 系统中最广泛的组件级更新循环

。其 FeedbackAgent 使用任务结果在修改智能体支架、工具、重试和搜索逻辑以及具有不同策略的低等级适配器 (LoRA) 权重之间进行选择。 CORAL [183] 用长期运行的异步智能体取代了固定的进化搜索启发式，这些智能体可以自主检查先前的尝试，决定何时测试或调用评估器，并将发现具体化为共享尝试、注释和可重用技能，并通过心跳触发的反射和重定向来维持 长时程（Long-Horizon） 探索。 SAGA [184]将可编辑表面扩展到目标层。在固定的高级科学目标下，它诊断优化候选群体中的故障模式，提出或重新衡量具体目标，并将它们实现为指导后续搜索的可执行评分函数。

C.1.4 递归改进科学智能体

总而言之，这些工作为科学 RSI 指明了更广阔的视野。它的最终形式不仅是一个能够产生更好的假设或实验结果的系统，而且是一个通过原始创新逐渐更好地进行科学的系统。它应该共同改进假设的生成、实验的执行和证据的解释，以便每个研究周期都能产生科学发现和对未来发现的可重复使用的改进。

目前的 AI4Sci 系统仅部分实现了这一愿景。 B0 式的输出细化已经很普遍，而 L1 持久更新已经通过模型再训练和内存积累得到了证明[173-175]。 L2 代表当前最强的前沿，系统自主选择或应用更新

to model weights, tools, skills, workflows, and scaffolds [159, 161, 176]. A few systems exhibit aspects of L3 through adaptive exploration and cross-task experience reuse [160, 183], whereas end-to-end L4 environment adaptation and L5 evolution of the improver itself remain largely unexplored.

How far are we from True RSI for Science? Progress toward higher-level scientific RSI will mainly require more reliable attribution of experimental outcomes, more selective transfer of improvements across tasks, and stronger evaluation of whether successive system updates genuinely improve future research. Equally important is maintaining provenance, reproducibility, and stable scientific and safety constraints as increasingly powerful components of the improvement process become adaptive.

C.2 S2: RSI for Embodied Intelligence

For embodied intelligence, an agent's actions produce observable consequences in a physical or simulated environment, which provides a natural setting for RSI. However, three main challenges remain. First, experience is endogenous, as the current policy determines which states the agent reaches and therefore what evidence is available for further improvement. Second, capability is distributed across the sensorimotor system, including perception, planning, control, and the physical embodiment, making failures difficult to attribute to a single component. Third, physical trials cannot be freely reproduced, reversed, or reset, so candidate updates must be evaluated under safety, hardware, and resource constraints.

Embodied RSI must therefore jointly address experience generation, cross-component credit assignment, and safe inheritance. Its interaction loop begins with environments and curricula that provide learning tasks, where the agent harness organizes relevant capabilities, the policy executes actions, and world models and evaluators interpret the resulting outcomes. Validated feedback is then written back to these components, allowing the updated system to participate in the next round of interaction. Accordingly, we review existing work on evolving environments and curricula, agent harnesses, policies and action models, and world models and evaluators.

C.2.1 Evolving Environments and Curricula

An embodied RSI loop begins by determining what the agent will experience and learn from. POET [65] addresses the limitation of fixed, human-designed curricula by co-evolving a population of environment–agent pairs. It mutates environments, admits challenges that are neither trivial nor unsolvable through a minimal criterion, optimizes the corresponding policies, and periodically transfers policies across environments to reuse behavioral stepping stones. EnvGen [185] targets the high cost of directly deploying large language models as embodied agents and the inability of static curricula to address individual weaknesses. It instead employs an LLM as a curriculum designer, where the LLM generates simulator configurations, a smaller RL agent trains in them, and skill-level performance is returned to the LLM to produce environments targeting the agent's remaining weaknesses. GenEnv [186] further addresses the mismatch between static training data and an improving learner by parameterizing the environment simulator as a trainable curriculum policy. Its α-Curriculum Reward favors tasks near the learner's current capability frontier, after which the learner is optimized with GRPO and the simulator is updated through reward-weighted regression.

Whereas these methods adapt tasks within an existing simulator, OMNI-EPIC [187] addresses the restricted search space imposed by manually predefined environment distributions. It uses LLMs to generate executable environment and reward code, while conditioning subsequent proposals on an archive of prior tasks and the current agent's learning progress to seek challenges that are both learnable and novel. SimWorld Studio [188] tackles a further obstacle, where generated three-dimensional content is often static, unverifiable, and unsuitable for embodied training. Its SimCoder agent writes executable Unreal Engine code, revises environments using compilation errors, physics checks, and vision-language-model critiques, and stores successful tools and skills for later generation. Learner performance then guides it toward progressively harder Gym-compatible worlds. Collectively, these methods convert learner performance into a mechanism for evolving the distribution and the generator of future embodied experience.

原文第 74 页 | 中文翻译初稿

附录 A：RSI 实施技术

对权重、工具、技能、工作流程和支架进行建模 [159, 161, 176]。一些系统通过自适应探索和跨任务经验重用来展示 L3 的各个方面 [160, 183]，而改进器本身的端到端 L4 环境适应和 L5 演化在很大程度上仍未被探索。

我们距离真正的 RSI 科学还有多远？迈向更高水平的科学 RSI 的进展主要需要对实验结果进行更可靠的归因，更有选择性地跨任务改进转移，以及对连续的系统更新是否真正改善未来的研究进行更强有力的评估。同样重要的是，随着改进过程中日益强大的组成部分变得适应性强，保持来源、可重复性以及稳定的科学和安全约束。

C.2 S2：具身智能（Embodied Intelligence） 的 RSI

对于 具身智能（Embodied Intelligence），智能体的行为会在物理或模拟环境中产生可观察的后果，这为 RSI 提供了自然的设置。然而，仍然存在三个主要挑战。首先，经验是内生的，因为当前的策略决定了智能体达到的状态，从而决定了有哪些证据可用于进一步改进。其次，能力分布在整个感觉运动系统中，包括感知、计划、控制和物理体现，使得故障很难归因于单个组件。第三，物理试验不能自由复制、逆转或重置，因此候选更新必须在安全、硬件和资源限制下进行评估。

因此，体现的 RSI 必须共同解决经验生成、跨组件 贡献归因（Credit Assignment） 和安全继承问题。其交互循环从提供学习任务的环境和课程开始，其中智能体 运行支撑框架（Harness） 组织相关功能，策略执行操作，世界模型（World Models） 和评估者解释结果。然后，经过验证的反馈会写回这些组件，从而允许更新后的系统参与下一轮交互。因此，我们回顾了有关不断变化的环境和课程、智能体 运行支撑框架（Harnesses）、策略和行动模型以及 世界模型（World Models） 和评估者的现有工作。

C.2.1 不断变化的环境和课程

体现的 RSI 循环首先确定智能体将经历什么并从中学习。 POET [65]通过共同进化环境-主体对来解决固定的、人为设计的课程的局限性。它改变环境，通过最低标准承认既不平凡也不无法解决的挑战，优化相应的策略，并定期跨环境转移策略以重用行为垫脚石。 EnvGen [185]针对的是直接部署大型语言模型作为具体智能体的高成本以及静态课程无法解决个人弱点的问题。相反，它采用 LLM 作为课程设计者，LLM 生成模拟器配置，较小的 RL 智能体在其中进行训练，并且技能水平表现返回给 LLM，以生成针对智能体剩余弱点的环境

。 GenEnv [186]通过将环境模拟器参数化为可训练的课程策略，进一步解决了静态训练数据和改进学习器之间的不匹配问题。它的_α_-课程奖励有利于学习者当前能力前沿附近的任务，之后使用 GRPO 优化学习者，并通过奖励加权回归更新模拟器。

虽然这些方法适应现有模拟器中的任务，但 OMNI-EPIC [187] 解决了手动预定义的环境分布所施加的受限搜索空间。它使用 LLM 生成可执行环境和奖励代码，同时根据先前任务的存档和当前智能体的学习进度来调整后续建议，以寻求既可学习又新颖的挑战。 SimWorld Studio [188] 解决了另一个障碍，即生成的三维内容通常是静态的、无法验证的且不适合具体训练。它的 SimCoder 智能体编写可执行的虚幻引擎代码，使用编译错误、物理检查和视觉语言模型批评来修改环境，并为下一代存储成功的工具和技能。然后，学习者的表现会引导其走向难度逐渐加大、与健身房兼容的世界。总的来说，这些方法将学习者的表现转化为一种进化分布和未来体验体验生成器的机制。

C.2.2 Evolving Skills, Memory, and Agent Harnesses

Given a task, the agent harness determines how memories, skills, tools, and execution rules are assembled into an actionable system. Voyager [84] writes successful Minecraft behaviors into an executable skill library, while LRLL [189] uses self-guided exploration and a wake–sleep process to grow and reorganize a library of composable robot programs. EmbodiSkill [190] further distinguishes between failures caused by an incorrect skill and failures caused by the agent not following valid guidance, updating a skill only when the trajectory provides relevant evidence.

More recent systems expand the update target from individual skills to the surrounding agent architecture. SHAPER [111] jointly evolves reusable skills and the context-code harness through environment rollouts, while ASPIRE [191] diagnoses robot execution failures, validates candidate repairs, and retains successful fixes as transferable skills. ENPIRE [192] goes further by allowing coding agents to revise robot policies, training procedures, and supporting infrastructure through repeated real-world experiments.

These systems approach system-level evolution as experience changes not only what the agent knows, but also how its capabilities are selected, composed, and improved. The resulting harness ultimately shapes how the policy converts observations and instructions into actions.

C.2.3 Evolving Policies and Action Models

With the task and execution context in place, the policy determines how the agent acts in the environment. Self-Improving Embodied Foundation Models [193] derive reward and success signals from a pretrained model, allowing robots to practice tasks and improve their policies with limited human supervision. MEDAL++ [194] similarly supports autonomous practice by learning both how to complete and undo a task, reducing the need for manual environment resets. Other methods update more targeted components. Q-Planning [195] keeps a large behavior-cloning policy fixed but continually trains a smaller value function from both successful and failed deployment trajectories, while SERP [196] adjusts its action model from recent navigation failures before replanning. In each case, interaction produced by the current policy becomes training evidence for its successor.

However, the update algorithm, reward definition, and permitted model components remain specified in advance, making these systems primarily cases of policy-level self-improvement. Moreover, before such experience can be safely inherited, the system must predict and evaluate what its actions actually caused.

C.2.4 Evolving World Models and Evaluators

Once the policy acts, world models and evaluators turn the resulting interaction into predictions, outcome judgments, and learning signals. VLAW [197] uses real-robot rollouts to improve an action-conditioned video world model, which then generates additional synthetic experience for training the vision-language-action policy. World-VLA-Loop [162] makes this relation iterative, where policy failures refine the world model, and the refined model provides a more reliable environment for the next round of policy optimization. Motus2 [198] integrates policy, simulation, and evaluation functions into a shared model, using failed and suboptimal interactions to improve its dynamics and value predictions. SAIL [199] follows a related loop in which a video planner executes its own generated plans and is fine-tuned on the resulting trajectories.

These methods improve not only the policy but also the mechanisms that predict consequences and produce future training signals. Their feedback can then update the policy, harness, or curriculum used in the next round, closing the embodied RSI loop. Nevertheless, their objectives and update procedures remain externally fixed, so they instantiate bounded model–policy co-evolution rather than unrestricted recursive improvement.

C.2.5 Toward Recursively Improving Embodied Agents

Taken together, these lines of work suggest that embodied RSI should improve not only task performance but also the process through which an agent acquires and validates new capabilities. An environment generator should propose tasks near the agent's capability frontier; the harness should assemble relevant memories, skills, tools, and models; the policy should interact with the environment; and world models and evaluators

原文第 75 页 | 中文翻译初稿

附录 A：RSI 实施技术

C.2.2 进化技能、记忆和智能体 运行支撑框架（Harnesses）

给定任务，智能体 运行支撑框架（Harness） 确定如何将记忆、技能、工具和执行规则组装成可操作的系统。 Voyager [84] 将成功的 Minecraft 行为写入可执行技能库，而 LRLL [189] 使用自我引导探索和唤醒睡眠过程来增长和重组可组合机器人程序库。 EmbodiSkill [190]进一步区分了由不正确的技能引起的失败和由于智能体未遵循有效指导而导致的失败，仅当轨迹提供相关证据时才更新技能。

最近的系统将更新目标从个人技能扩展到周围的智能体架构。 SHAPER [111] 通过环境 展开轨迹（Rollouts） 共同发展可重用技能和上下文代码 运行支撑框架（Harness），而 ASPIRE [191] 诊断机器人执行故障，验证候选修复，并将成功修复保留为可转移技能。 ENPIRE [192] 更进一步，允许编码智能体通过重复的现实世界实验来修改机器人策略、训练程序和支持基础设施。

这些系统接近系统级演进，因为经验不仅改变了智能体所知道的内容，还改变了其能力的选择、组合和改进方式。由此产生的 运行支撑框架（Harness） 最终决定了策略如何将观察和指令转化为行动。

C.2.3 不断发展的策略和行动模式

任务和执行上下文就位后，策略将确定智能体在环境中的行为方式。 自我改进（Self-Improving） 体现 基础模型（Foundation Models） [193] 从预训练模型中获取奖励和成功信号，允许机器人在有限的人类监督下练习任务并改进其策略。 MEDAL++ [194] 同样通过学习如何完成和撤消任务来支持自主实践，从而减少手动环境重置的需要。其他方法更新更有针对性的组件。 Q-Planning [195] 保持大型行为克隆策略固定，但不断从成功和失败的部署轨迹中训练较小的价值函数，而 SERP [196] 在重新规划之前根据最近的导航失败调整其操作模型。在每种情况下，当前策略产生的互动都会成为其继任者的训练证据。

然而，更新算法、奖励定义和允许的模型组件仍然是预先指定的，使得这些系统主要是策略级 自我改进（Self-Improvement） 的情况。此外，在安全地继承此类经验之前，系统必须预测并评估其行为实际造成的后果。

C.2.4 不断发展的 世界模型（World Models） 和评估器

一旦策略生效，世界模型（World Models） 和评估者就会将由此产生的交互转化为预测、结果判断和学习信号。 VLAW [197] 使用真实机器人 展开轨迹（Rollouts） 来改进动作条件视频 世界模型（World Model），然后生成额外的合成经验来训练 视觉语言动作（Vision-Language-Action） 策略。

World-VLA-Loop[162]使这种关系迭代，其中策略失败细化了世界模型（World Model），细化后的模型为下一轮策略优化提供了更可靠的环境

。 Motus2 [198]将策略、模拟和评估功能集成到一个共享模型中，使用失败和次优的交互来改进其动态和价值预测。 SAIL [199]遵循一个相关的循环，其中视频规划器执行其自己生成的计划并根据结果轨迹进行微调。

这些方法不仅改进了策略，还改进了预测后果和产生未来训练信号的机制。然后，他们的反馈可以更新下一轮使用的策略、运行支撑框架（Harness） 或课程，从而关闭具体的 RSI 循环。尽管如此，他们的目标和更新程序仍然是外部固定的，因此他们实例化了有界模型-策略协同进化，而不是无限制的递归改进。

C.2.5 递归改进具体智能体

总而言之，这些工作表明，体现的 RSI 不仅应该提高任务绩效，还应该提高智能体获取和验证新能力的过程。环境生成器应该提出靠近智能体能力边界的任务； 运行支撑框架（Harness）应该组装相关的记忆、

技能、工具和模型；策略应与环境相互作用；和 世界模型（World Models） 和评估者

should interpret the resulting outcomes. Verified improvements should then be inherited by the policy, world model, or harness, allowing each interaction cycle to improve both current behavior and future learning.

Current systems realize this loop only partially. B0-style within-episode reflection and replanning are already widespread. L1 persistent improvement has been demonstrated through policy and model updates based on embodied experience [193, 194, 199]. L2 represents a major current frontier, with systems selecting and applying updates to skills, prompts, control programs, and agent harnesses under fixed evaluation procedures [111, 190, 191]. Some systems exhibit L3 capabilities by generating experience according to the learner's state and retaining validated skills across tasks [84, 189]. L4 agent–environment co-evolution has also emerged, primarily in simulation [65, 185, 186, 188]. ENPIRE [192] further approaches bounded L5 by allowing coding agents to revise policies, training procedures, and supporting infrastructure. However, its environment interfaces, evaluators, and safety constraints remain externally specified.

How far are we from True RSI for Embodied Intelligence? True embodied RSI requires agents to actively acquire informative experience, attribute outcomes across the full sensorimotor system, and validate updates under physical conditions that cannot be freely reset. Improvements must remain effective across tasks, environments, and embodiments, with explicit provenance, uncertainty, and rollback. The system must also learn to improve its own harness and evaluation process without weakening fixed safety constraints. Current methods demonstrate these abilities separately, but have not yet integrated them into a safe, persistent, and autonomous real-world improvement loop.

C.3 S3: RSI for Software Engineering

Software engineering provides an unusually concrete setting for studying RSI because both the product and the agent that develops it are represented as executable code. A typical loop begins when a coding agent attempts an issue, observes repository-grounded feedback such as test failures, runtime traces, or code review, and uses this evidence to modify a persistent part of itself. The modified component may be the agent's skills, workflow, or implementation, and accepted changes are reused in later tasks.

RSI for software engineering then faces three fundamental challenges. First, the system must continually generate software tasks that target its current capability gaps while remaining both verifiable and valuable for future learning. Second, improvement for software engineering must assess both whether a modification is effective now and whether it increases the system's capacity for further improvement. A change may raise immediate benchmark performance yet cause subsequent search to stagnate, whereas a temporarily weaker variant may produce stronger descendants. Third, coordinating the evolution of multiple components while preserving the outer contract (e.g., evaluation criteria, improvement objectives) remains a challenge. Accordingly, we organize existing work around three evolving components, including agent implementations and harnesses, software-engineering experience and collaboration, and the improvement process itself.

C.3.1 Evolving Coding-Agent Implementations and Harnesses

The most direct form of software-engineering RSI treats the coding agent itself as an editable software project. SICA [200] allows an agent to inspect previous versions and benchmark results, modify its own Python codebase, and use the selected version in the next improvement round. Because the same system acts both as the software developer and as the object being developed, changes to file-editing tools, subagents, or context management can improve both ordinary coding and subsequent self-modification. However, its benchmark, utility function, and sandbox remain externally fixed.

More recent work narrows self-modification to the agent *harness* (e.g., middleware, verification rules, and control logic). Self-Harness [23] uses execution traces to identify recurring weaknesses, asks the same underlying model to propose small changes to its current harness, and retains a proposal only after regression testing. Agentic Harness Engineering [63] exposes harness components as separately editable and revertible files, distills long trajectories into structured evidence, and checks whether each proposed change produces its predicted effect. Its separate evolver role makes the loop less directly self-referential than Self-Harness, but the accepted harness still persists across later executions. Ouroboros [18] extends this idea to deployment: reviewed changes to the agent's tools, context assembly, prompts, and core implementation become the

原文第 76 页 | 中文翻译初稿

附录 A：RSI 实施技术

应解释由此产生的结果。然后，经过验证的改进应由策略 世界模型（World Model） 或 运行支撑框架（Harness） 继承，从而允许每个交互周期改进当前行为和未来的学习。

当前的系统仅部分地实现了该循环。 B0 式的情节内反思和重新规划已经很普遍。 L1 的持续改进已通过基于具体经验的策略和模型更新得到证明[193,194,199]。 L2 代表当前的主要前沿，系统在固定评估程序下选择和应用技能、提示、控制程序和智能体 运行支撑框架（Harnesses） 的更新 [111,190,191]。一些系统通过根据学习者的状态生成经验并保留跨任务的经过验证的技能来展示 L3 能力 [84, 189]。 L4 主体-环境协同进化也出现了，主要是在模拟中[65,185,186,188]。 ENPIRE [192] 通过允许编码智能体修改策略、训练程序和支持基础设施，进一步接近有界 L5。然而，它的环境接口、评估器和安全约束仍然是外部指定的。

我们距离 具身智能（Embodied Intelligence） 的真实 RSI 有多远？真正的体现 RSI 要求智能体主动获取信息丰富的经验，在整个感觉运动系统中归因结果，并在无法自由重置的物理条件下验证更新。改进必须在任务、环境和实施例中保持有效，并具有明确的来源、不确定性和回滚。系统还必须学会在不削弱固定安全约束的情况下改进自己的运行支撑框架（Harness）和评估流程。目前的方法分别展示了这些能力，但尚未将它们集成到安全、持久和自主的现实世界改进循环中。

C.3 S3：软件工程的 RSI

软件工程为研究 RSI 提供了一个异常具体的环境，因为产品和开发它的智能体都表示为可执行代码。当编码智能体尝试解决问题、观察基于存储库的反馈（例如测试失败、运行时跟踪或代码审查）并使用此证据来修改其自身的持久部分时，典型的循环就开始了。修改的组件可能是智能体的技能、工作流程或实施，并且接受的更改将在以后的任务中重复使用。

软件工程的 RSI 面临三个基本挑战。首先，系统必须不断生成针对其当前能力差距的软件任务，同时保持可验证性和对未来学习的价值。其次，软件工程的改进必须评估修改现在是否有效以及是否增加了系统进一步改进的能力。更改可能会立即提高基准性能，但会导致后续搜索停滞，而暂时较弱的变体可能会产生更强的后代。第三，在保留外部契约（例如评估标准、改进目标）的同时协调多个组件的演变仍然是一个挑战

。因此，我们围绕三个不断发展的组件组织现有的工作，包括智能体实现和 运行支撑框架（Harnesses）、软件工程经验和协作以及改进过程本身。

C.3.1 不断发展的编码智能体实现和 运行支撑框架（Harnesses）

软件工程 RSI 最直接的形式将编码智能体本身视为可编辑的软件项目。 SICA [200] 允许智能体检查以前的版本和基准测试结果，修改自己的 Python 代码库，并在下一轮改进中使用所选版本。由于同一系统既充当软件开发人员又充当正在开发的对象，因此对文件编辑工具、子智能体或上下文管理的更改可以改进普通编码和随后的自我修改。然而，它的基准、效用函数和沙箱仍然是外部固定的。

最近的工作将自我修改范围缩小到智能体_运行支撑框架（Harness）_（例如，中间件、验证规则和控制逻辑）。 Self-运行支撑框架（Harness） [23] 使用执行跟踪来识别重复出现的弱点，要求相同的底层模型对其当前的 运行支撑框架（Harness） 提出微小的更改，并且仅在回归测试后保留提案。 Agentic 运行支撑框架（Harness） Engineering [63] 将 运行支撑框架（Harness） 组件公开为可单独编辑和可恢复的文件，将长轨迹提炼为结构化证据，并检查每个提议的更改是否产生其预测效果。其单独的演化者角色使得

循环不像 Self-运行支撑框架（Harness） 那样直接自引用，但接受的 运行支撑框架（Harness） 在以后的执行中仍然存在。 Ouroboros [18] 将这个想法扩展到部署：对智能体工具、上下文组装、提示和核心实现进行审查的更改成为

runtime used for later work. Together, these systems show how version control, regression tests, and rollback can turn agent self-modification into a persistent and auditable process.

C.3.2 Evolving Software-Engineering Experience, Skills, and Collaboration

A lighter form of self-evolution changes how a coding agent uses prior development experience without rewriting its full implementation. SWE-Exp [60] extracts both successful and failed issue-resolution trajectories into an experience bank and retrieves relevant lessons for later repository tasks. CODESKILL [201] goes beyond storing cases, where it learns a management policy that extracts procedural skills at multiple levels, updates or removes them as new trajectories arrive, and maintains a compact skill bank for future agents. In both cases, the persistent output is reusable development knowledge which supports cross-task adaptation. However, their fixed extraction and update procedures do not themselves become better through use.

Self-evolution can also change the organization of a software team. EvoMAC [202] represents a multi-agent coding workflow as a network of role prompts and communication links, then uses test feedback and textual back-propagation to revise both the agents and their connections. This improves the workflow used to construct the current program, but the reported updates occur during test time for each task.

C.3.3 Evolving the Improvement Process

The strongest systems do not follow a single sequence of accepted edits; they also search over alternative evolutionary paths. Darwin Gödel Machine (DGM) [16] maintains an archive of coding-agent variants, selects previous agents as parents, lets them modify their own implementations, and evaluates their descendants on executable coding tasks. Keeping multiple lineages prevents one harmful edit from permanently blocking progress and allows temporarily weak variants to become useful stepping stones. Huxley–Gödel Machine (HGM) [59] further argues that current benchmark performance is not the same as the ability to produce better descendants. It therefore estimates *metaproductivity* from an agent's descendant lineage and allocates evaluation effort toward agents with stronger long-term improvement potential.

Population-based methods broaden what can be inherited across lineages. Group-Evolving Agents [203] allows several parent agents to share successful modifications and failure experience when producing a new group, so useful discoveries are not isolated within one branch. DarwinX [204] evolves populations of complete harnesses around a frozen model and admits variants only when they extend task coverage without regressing previously solved cases. HELIX [205] proposes a further model–harness loop in which harness evolution improves current execution and generates verified trajectories for updating the model, after which the harness is rebuilt for the changed model.

C.3.4 Toward Recursively Improving Software-Engineering Agents

Taken together, these works suggest a software-engineering RSI loop of *Development Challenge* → *Agent-Level Modification* → *Repository-Grounded Trial* → *Regression-Aware Selection* → *Versioned Inheritance*. Repository feedback triggers a persistent agent update, which is evaluated on current and held-out tasks before being inherited as a traceable and reversible version for future development and self-improvement.

Current systems realize this loop only partially. B0 task-local refinement is widespread, while L1 persistent updates appear in cross-task experience and skill learning [60, 201]. L2 is the strongest established level, with agents selecting changes to harnesses, tools, or workflows under fixed objectives and evaluators [23, 63, 204]. L3 is emerging through learner-conditioned task generation in Self-play SWE-RL and Socratic-SWE [206, 207]. L4 environment adaptation remains largely absent. SICA and DGM exhibit bounded L5 characteristics by modifying code that shapes later self-improvement [16, 200], while HGM introduces lineage-based meta-selection without adapting the selection rule itself [59]. Full L5 improvement of the software-development improver has not yet been demonstrated.

原文第 77 页 | 中文翻译初稿

附录 A：RSI 实施技术

运行时用于以后的工作。这些系统共同展示了版本控制、回归测试和回滚如何将智能体自我修改转变为持久且可审核的过程。

C.3.2 不断发展的软件工程经验、技能和协作

一种较轻的自我进化形式改变了编码智能体如何使用先前的开发经验，而无需重写其完整实现。SWE-Exp [60] 将成功和失败的问题解决轨迹提取到经验库中，并为以后的存储库任务检索相关经验教训。CODESKILL [201] 不仅仅存储案例，它还学习管理策略，提取多个级别的程序技能，在新轨迹到达时更新或删除它们，并为未来的智能体维护一个紧凑的技能库。在这两种情况下，持久输出都是可重用的开发知识，支持跨任务适应。然而，它们的固定提取和更新过程本身并不会通过使用而变得更好。

自我进化还可以改变软件团队的组织。 EvoMAC [202] 将多智能体编码工作流程表示为角色提示和通信链接的网络，然后使用测试反馈和文本反向传播来修改智能体及其连接。这改进了用于构建当前程序的工作流程，但报告的更新发生在每个任务的测试期间。

C.3.3 改进改进过程

最强大的系统不会遵循单一的已接受编辑序列；他们还寻找替代的进化路径。达尔文哥德尔机（DGM）[16]维护编码智能体变体的档案，选择以前的智能体作为父代，让它们修改自己的实现，并评估其后代的可执行编码任务。保留多个谱系可以防止一种有害的编辑永久阻碍进展，并允许暂时较弱的变体成为有用的垫脚石。赫胥黎-哥德尔机（HGM）[59]进一步认为，当前的基准性能与产生更好后代的能力并不相同。因此，它根据智能体的后裔谱系来估计“元生产力”，并将评估工作分配给具有更强长期改进潜力的智能体。

基于群体的方法拓宽了跨谱系的遗传范围。 Group-Evolving Agents [203] 允许多个父智能体在生成新组时共享成功的修改和失败的经验，因此有用的发现不会孤立在一个分支内。 DarwinX [204]围绕冻结模型进化出完整的 运行支撑框架（Harnesses） 群体，并且仅当它们扩展任务覆盖范围而不回归先前解决的案例时才允许变体。 HELIX [205]提出了进一步的模型--运行支撑框架（Harness）循环，其中运行支撑框架（Harness）进化改进了当前执行并生成用于更新模型的经过验证的轨迹，之后为改变的模型重建运行支撑框架（Harness）。

C.3.4 递归改进软件工程智能体

总而言之，这些工作提出了一个软件工程 RSI 循环：开发挑战 → 智能体级别修改 → 基于存储库的试验 → 回归感知选择 → 版本化继承

。存储库反馈会触发持久智能体更新，该更新会根据当前和保留的任务进行评估，然后作为可跟踪和可逆的版本继承以供未来开发和 自我改进（Self-Improvement） 使用。

当前的系统仅部分地实现了该循环。 B0 任务局部细化很普遍，而 L1 持续更新出现在跨任务经验和技能学习中 [60, 201]。 L2 是最强的既定级别，智能体在固定目标和评估者下选择对 运行支撑框架（Harnesses）、工具或工作流程进行更改 [23,63,204]。 L3 是通过自我游戏 SWE-RL 和 Socratic-SWE [206, 207] 中的学习者条件任务生成而出现的。 L4 环境适应仍然基本上不存在。 SICA 和 DGM 通过修改后来形成 自我改进（Self-Improvement） [16, 200] 的代码来表现出有界的 L5 特征，而 HGM 则引入了基于谱系的元选择，而不调整选择规则本身 [59]。软件开发改进器的完整 L5 改进尚未得到证实。

How far are we from True RSI for Software Engineering? True software-engineering RSI requires robust specifications, attributable improvements, and reliable transfer across repositories and software versions. As test generation and improvement strategies become adaptive, user intent, safety constraints, auditability, and rollback must remain externally protected.

C.4 S4: RSI for Healthcare

As one of the most prominent scenarios for AI application, healthcare spans tasks like diagnosis, medical image analysis, and treatment planning. A typical RSI loop begins with an agent performing a clinical task and receiving verifiable feedback from diagnostic results, clinician corrections, or clinical guidelines. The agent then persistently updates its components (e.g., clinical knowledge, reasoning strategies, workflows) with improvements retained in memory, skill libraries, or model parameters for reuse across subsequent cases.

RSI for healthcare is mainly challenged by three characteristics. First, healthcare does not permit unrestricted trial and error, as diagnostic and treatment actions may harm patients and therefore require expert, ethical, and institutional oversight. Second, clinical feedback is often delayed, heterogeneous, and confounded, making it difficult to attribute patient outcomes to a particular agent decision or update. Third, clinical improvements are population- and institution-dependent, so an update validated in one setting cannot be safely inherited without recording its provenance, scope, and uncertainty. These characteristics make healthcare RSI a problem of safe feedback acquisition, reliable credit assignment, and governed capability inheritance. Accordingly, we organize existing work around three components that may evolve: clinical memory and knowledge, clinical reasoning strategies, and healthcare tools and workflows.

C.4.1 Evolving Clinical Memory and Knowledge

The most common form of RSI in healthcare evolves the agent's clinical memory, where the feedback from completed cases is converted into persistent knowledge that can influence decisions for subsequent patients. Early systems primarily retain case-level experience. MedAgent-Zero in Agent Hospital [163] stores successful treatment cases and reflects on failed cases to derive reusable diagnostic rules, allowing a doctor agent with frozen model weights to retrieve these experiences in later simulated encounters. Similarly, Agent Mental Clinic [208] compares the psychiatrist agent's diagnosis with labeled outcomes through a supervisor module and writes the resulting diagnostic experience into a tiered memory architecture. MedAgentSim [209] further combines medical records with experience records, enabling diagnostic agents to reuse knowledge accumulated from earlier simulated doctor–patient interactions. In these systems, the output of one encounter becomes part of the persistent context from which the agent reasons about future cases.

More recent methods evolve the structure and reliability of remembered knowledge rather than simply accumulating raw trajectories. DxEvolve [210] distills diagnostic encounters into *diagnostic cognition primitives*, which encode reusable patterns for information acquisition and diagnostic reasoning and can be retrieved in new cases. GSEM [211] organizes experience as a dual-layer graph and uses subsequent feedback to recalibrate node quality and inter-experience edge weights, thereby changing both what is remembered and which memories are considered applicable. Evo-MedAgent [212] maintains complementary stores for retrospective clinical episodes, procedural heuristics, and tool reliability. After each chest-radiography case, reflection updates these stores so that later cases inherit both clinical lessons and revised trust in diagnostic tools. Nevertheless, most evidence remains based on simulated or retrospective cases with externally supplied labels, and the validation of evolution in real-world cases is still lacking.

C.4.2 Evolving Clinical Reasoning Strategies

Beyond evolving clinical facts, healthcare agents can evolve the strategies that determine how evidence is acquired, interpreted, and integrated. EvoClinician [164] instantiates this process through a *Diagnose–Grade–Evolve* loop, where an Actor sequentially asks questions and orders examinations, a Process Grader evaluates each action according to its clinical yield and resource cost, and an Evolver uses this feedback to revise the Actor's prompt and memory before the next case. The inherited evolution object is thus a diagnostic policy, for example, which symptom to clarify or which examination to prioritize.

原文第 78 页 | 中文翻译初稿

附录 A：RSI 实施技术

我们距离真正的软件工程 RSI 有多远？真正的软件工程 RSI 需要强大的规范、可归因的改进以及跨存储库和软件版本的可靠传输。随着测试生成和改进策略变得适应性强，用户意图、安全约束、可审计性和回滚必须保持外部保护。

C.4 S4：医疗保健领域的 RSI

作为人工智能应用最突出的场景之一，医疗保健涵盖了诊断、医学图像分析和治疗计划等任务。典型的 RSI 循环始于智能体执行临床任务并接收来自诊断结果、临床医生更正或临床指南的可验证反馈。然后，智能体持续更新其组件（例如，临床知识、推理策略、工作流程），并通过保留在内存、技能库或模型参数中的改进来在后续案例中重用。

医疗保健领域的 RSI 主要面临三个特征的挑战。首先，医疗保健不允许无限制的试验和错误，因为诊断和治疗行为可能会伤害患者，因此需要专家、道德和机构的监督。其次，临床反馈通常是延迟的、异质的和混乱的，使得很难将患者的结果归因于特定的智能体决策或更新。第三，临床改进取决于人群和机构，因此在一种环境中验证的更新如果不记录其来源、范围和不确定性，就无法安全地继承。这些特征使医疗保健 RSI 成为安全反馈获取、可靠 贡献归因（Credit Assignment）和受控能力继承的问题。因此，我们围绕可能发展的三个组成部分组织现有的工作：临床记忆和知识、临床推理策略以及医疗保健工具和工作流程。

C.4.1 不断发展的临床记忆和知识

医疗保健领域最常见的 RSI 形式会发展智能体人的临床记忆，其中已完成病例的反馈会转化为持久的知识，从而影响后续患者的决策。早期的系统主要保留案例级别的经验。 Agent Hospital [163] 中的 MedAgent-Zero 存储成功的治疗案例并反思失败的案例以得出可重复使用的诊断规则，从而允许具有冻结模型权重的医生智能体在以后的模拟遭遇中检索这些经验。类似地，特工心理诊所[208]通过主管模块将精神科医生的诊断与标记结果进行比较，并将所得的诊断经验写入分层内存架构中。 MedAgentSim [209]进一步将医疗记录与经验记录结合起来，使诊断智能体能够重用从早期模拟医患互动中积累的知识。在这些系统中，一次遭遇的输出成为持久上下文的一部分，智能体可以从中推断出未来的情况。

最近的方法发展了记忆知识的结构和可靠性，而不是简单地积累原始轨迹

。 DxEvolve [210] 将诊断遭遇提炼为_诊断认知原语_，它编码用于信息获取和诊断推理的可重用模式，并且可以在新病例中检索。 GSEM [211]将经验组织为双层图，并使用后续反馈来重新校准节点质量和经验间边缘权重，从而改变所记住的内容以及哪些记忆被认为是适用的。 Evo-MedAgent [212] 维护回顾性临床事件、程序启发和工具可靠性的补充存储。在每个胸部X光检查病例之后，反射都会更新这些存储，以便后来的病例继承临床经验并修正对诊断工具的信任。然而，大多数证据仍然基于带有外部提供标签的模拟或回顾性案例，并且仍然缺乏对现实案例中进化的验证。

C.4.2 不断发展的临床推理策略

除了不断发展的临床事实之外，医疗保健人员还可以发展确定如何获取、解释和整合证据的策略。EvoClinician [164]通过_Diagnose-Grade-Evolve_循环实例化这个过程，其中Actor依次提出问题并命令检查，Process Grader根据其临床产量和资源成本评估每个动作，Evolver使用此反馈在下一个案例之前修改Actor的提示和记忆。因此，继承的进化对象是一种诊断策略，例如，要澄清哪些症状或优先考虑哪些检查。

Reasoning evolution can also occur at multi-agent coordination level. EvoMDT [213] decomposes oncology decision-making among role-specialized agents and uses expert assessments and outcome-oriented signals to update prompts, consensus weights, and retrieval scope. Thus, feedback can change not only an individual agent's reasoning but also how specialist opinions are weighted and reconciled in later cases. In psychological counseling, PsychAgent [214] extracts practice-grounded skills from historical counseling trajectories, adds them to an evolving skill repertoire, and further internalizes selected skills through rejection fine-tuning.

C.4.3 Evolving Healthcare Tools and Workflows

A third line of work evolves the agent's action repertoire by turning successful executions into reusable tools, skills, or clinical workflows. MACRO [165] begins with a fixed collection of medical-imaging tools but identifies recurring multi-step patterns in verified execution trajectories, synthesizes them into composite tools, and registers these composites as new high-level actions for subsequent imaging tasks. The resulting agent therefore changes what it can directly invoke, rather than merely retrieving an earlier solution. SkeMex [215] follows a related approach at the level of procedural skills: it distills informative clinical trajectories into general, task-specific, and action-level skills, estimates their context-dependent utility from environmental feedback, and manages them through a *Read–Write–Assess–Govern* lifecycle that can promote, merge, update, or remove entries. In both cases, inheritance operates through an evolving action space whose components are selected according to their demonstrated utility in later tasks.

Other systems evolve larger analytical workflows and their supporting components. TissueLab [216] allows domain experts to inspect intermediate medical-imaging results and provide corrections that guide active learning, classifier refinement, and subsequent workflow construction across pathology, radiology, and spatial-omics tasks. HealthFlow [166] converts completed EHR analyses into persistent safeguards, reusable workflows, dataset-specific anchors, and code snippets; an evaluator first identifies execution or methodological defects, after which a reflector validates, updates, or retires experience before it is retrieved for later analyses. These systems move healthcare RSI from selecting predefined tools to constructing reusable procedures. However, because technical success or case-specific clinician feedback does not guarantee safety across patient populations, evolved workflows must be validated and reversible before being inherited across cases.

C.4.4 Toward Recursively Improving Healthcare Agents

The ultimate form of RSI for healthcare is not merely a system that produces better diagnoses, recommendations, or analytical outputs, but one that becomes progressively better at performing healthcare tasks while preserving clinical validity and safety. Each episode should produce both a task outcome and a validated improvement to the agent's memory, reasoning strategy, or action repertoire, such that the resulting capability can be safely reused and further evaluated in subsequent cases.

B0-style within-case reflection and answer refinement are already widespread, but they do not persistently alter future behavior. L1 persistent inheritance has been demonstrated through case memories, diagnostic rules, and reusable clinical knowledge [208–210]. L2 represents the strongest current frontier: systems can diagnose failures and autonomously select or apply updates to prompts, memories, reasoning policies, tools, workflows, and coordination mechanisms while their objectives, evaluators, and deployment gates remain externally fixed [164–166, 211–213, 215]. A few systems exhibit L3-like elements by using current uncertainty or accumulated experience to shape subsequent evidence acquisition, simulated encounters, or expert annotation [163, 216]; however, their patient populations, task distributions, and admission criteria remain largely externally specified. EvoPatient further explores L4-style adaptation by co-evolving simulated patients and doctors [217], but the simulator's clinical realism and evaluation criteria remain externally determined. End-to-end L4 adaptation from real longitudinal outcomes and L5 evolution of the clinical improver itself remain largely unexplored.

How far are we from True RSI for Healthcare? Progress toward higher-level healthcare RSI will mainly require reliable attribution of longitudinal patient outcomes, safe acquisition of improvement feedback without unrestricted clinical exploration, and selective transfer of updates across populations and institutions. Equally important is a governed capability-commitment process in which every persistent update is clinically validated, prospectively monitored, traceable to its supporting evidence, and reversible when its assumptions or safety guarantees no longer hold.

原文第 79 页 | 中文翻译初稿

附录 A：RSI 实施技术

推理进化也可以发生在多智能体协调层面。 EvoMDT [213]分解角色专业智能体之间的肿瘤学决策，并使用专家评估和以结果为导向的信号来更新提示、共识权重和检索范围。因此，反馈不仅可以改变个体智能体人的推理，还可以改变在以后的案例中专家意见的权衡和协调方式。在心理咨询中，PsychAgent [214]从历史咨询轨迹中提取基于实践的技能，将它们添加到不断发展的技能库中，并通过拒绝微调进一步内化所选技能。

C.4.3 不断发展的医疗保健工具和工作流程

第三条工作线通过将成功的执行转化为可重用的工具、技能或临床工作流程来发展智能体的行动库。 MACRO [165]从一组固定的医学成像工具开始，但在经过验证的执行轨迹中识别重复出现的多步骤模式，将它们合成为复合工具，并将这些复合材料注册为后续成像任务的新的高级操作。因此，生成的智能体会更改它可以直接调用的内容，而不仅仅是检索早期的解决方案。 SkeMex [215] 在程序技能层面遵循相关方法：它将信息丰富的临床轨迹提炼为一般技能、特定任务技能和行动技能，从环境反馈中估计其上下文相关效用，并通过“读-写-评估-治理”生命周期来管理它们，该生命周期可以促进、合并、更新或删除条目。在这两种情况下，继承都是通过不断发展的动作空间来运作的，其组件是根据其在后续任务中所展示的效用来选择的。

其他系统发展了更大的分析工作流程及其支持组件。 TissueLab [216] 允许领域专家检查中间医学成像结果并提供纠正，以指导主动学习、分类器细化以及跨病理学、放射学和空间组学任务的后续工作流程构建。 HealthFlow [166] 将完成的 EHR 分析转换为持久的保障措施、可重用的工作流程、数据集特定的锚点和代码片段；评估者首先识别执行或方法上的缺陷，然后反思者验证、更新或废弃经验，然后再检索经验以供以后分析。这些系统将医疗保健 RSI 从选择预定义工具转变为构建可重复使用的程序。然而，由于技术成功或特定病例的临床医生反馈并不能保证患者群体的安全，因此在跨病例继承之前，必须对改进的工作流程进行验证和可逆。

C.4.4 递归改进医疗保健智能体

用于医疗保健的 RSI 的最终形式不仅是一个能够产生更好的诊断、建议或分析输出的系统，而且是一个在保持临床有效性和安全性的同时逐渐更好地执行医疗保健任务的系统

。每个情节都应该产生任务结果和对智能体记忆、推理策略或动作库的经过验证的改进，以便可以在后续案例中安全地重用并进一步评估所产生的能力。

B0 式的案例内反思和答案细化已经很普遍，但它们不会持续改变未来的行为。 L1持续遗传已通过病例记忆、诊断规则和可重复使用的临床知识得到证明[208-210]。 L2 代表当前最强大的前沿：系统可以诊断故障并自主选择或应用提示、记忆、推理策略、工具、工作流程和协调机制的更新，同时其目标、评估器和部署门保持外部固定[164-166、211-213、215]。一些系统通过使用当前的不确定性或积累的经验来塑造后续的证据获取、模拟遭遇或专家注释，从而表现出类似 L3 的元素 [163, 216]；然而，他们的患者群体、任务分配和入院标准很大程度上仍然是外部指定的。 EvoPatient 通过共同进化模拟患者和医生进一步探索 L4 式适应[217]，但模拟器的临床现实性和评估标准仍然由外部确定。真实纵向结果的端到端 L4 适应和临床改进剂本身的 L5 进化在很大程度上仍未被探索。

我们距离真正的医疗保健 RSI 有多远？迈向更高水平的医疗保健 RSI 的进展主要需要对纵向患者结果进行可靠的归因，在没有不受限制的临床探索的情况下安全地获取改进反馈，以及在人群和机构之间选择性地传输更新。同样重要的是受管理的能力承诺流程，其中每个持久更新都经过临床验证、前瞻性监控、可

追溯至其支持证据，并且当其假设或安全保证不再成立时可逆转。